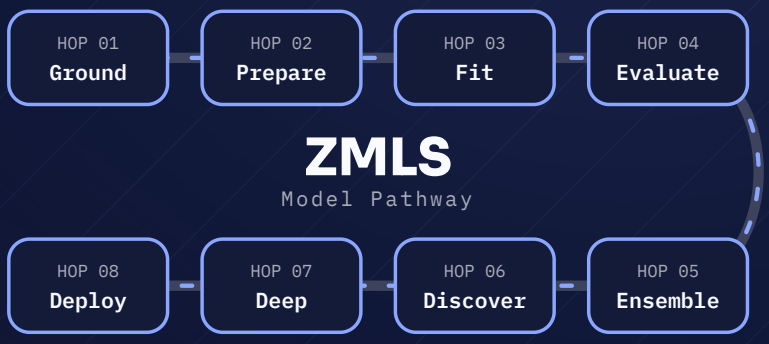
ZMLS

ZyeemZia™ Machine Learning Specialization

From first principles to models in production.



COURSE BOOK · WORKBOOK · EXAM PREPARATION

8 MODULES · 38 LESSONS · 60 EXAM QUESTIONS

COPYRIGHT © ZYEEMZIA

BEFORE YOU BEGIN

How to use this book

A practical, professional machine learning specialization: the maths that matters, problem framing, data preparation, linear models and gradient descent, honest evaluation, trees and boosting, unsupervised learning, recommenders, reinforcement learning, deep learning, transformers and foundation models, and the MLOps and responsible-ML practice needed to run models in the real world.

01
Learn
Work through 38 lessons. Each ends with a knowledge check.

02
Practise
Fill in the templates, answer the scenarios and case studies, review the glossary.

03
Certify
Take the 40-question official exam at zmls.zyeemzia.com, graded on the ZyeemZia server. Pass mark 80%.

04
Get certified
Download your certificate online. Anyone can verify its credential ID at zmls.zyeemzia.com/#verify. 90%+ and the capstone earn Distinction.

Eight-week study plan

WHEN	MODULE	DO THIS	DONE
Weeks 1–2	m1 Ground	Frame your capstone problem; maths drills on vectors, gradients and Bayes	☐
Week 3	m2 Prepare	Profile and clean the data, build point-in-time features	☐
Weeks 4–5	m3 Fit	Baseline, linear and logistic models, regularisation tuning	☐
Week 6	m4 Evaluate	Evaluation plan, thresholds, calibration and leakage review	☐
Week 7	m5 Ensemble	Random forest and gradient boosting with explanations	☐
Weeks 8–9	m6 Discover	Clustering, PCA, anomaly detection or a recommender component	☐
Weeks 10–11	m7 Deep	Neural network, CNN or transformer experiment	☐
Week 12	m8 Deploy	Deployment and monitoring plan, model card, practice and certification exam	☐

Learner name .....

Start date ..... Target exam date .....

ZMLS is an independent certification programme created and issued by ZyeemZia. It is not affiliated with, endorsed by, sponsored by or equivalent to any course, specialization or certification from DeepLearning.AI, Stanford University, Coursera, Google, AWS (Amazon Web Services), Microsoft or any other company, university or certifying body. All product names and trademarks belong to their respective owners. Code examples use widely available open-source Python libraries for teaching purposes.

Copyright © ZyeemZia

CONTENTS

What you will learn

01 Ground — Foundations: Framing and the Maths of Machine Learning

1. What Machine Learning Is and How Projects Flow · 22 min
2. Framing Machine Learning Problems · 24 min
3. Vectors, Matrices and Similarity · 24 min
4. Derivatives, Gradients and the Chain Rule · 24 min
5. Probability and Statistics for Machine Learning · 26 min

02 Prepare — Data Preparation and Feature Engineering

1. Data Sourcing, Labelling and Quality · 24 min
2. Cleaning Data: Missing Values, Outliers and Duplicates · 24 min
3. Encoding Categories and Scaling Numbers · 22 min
4. Feature Engineering · 26 min
5. Pipelines, Imbalance and Feature Selection · 24 min

03 Fit — Supervised Learning: Linear Models and Optimisation

1. Linear Regression and Least Squares · 26 min
2. Gradient Descent in Practice · 24 min
3. Logistic Regression and Classification · 26 min
4. Regularisation: Ridge, Lasso and Elastic Net · 24 min
5. Other Classic Learners: k-NN, Naive Bayes and SVMs · 24 min

04 Evaluate — Evaluation: Measuring Models Honestly

1. Splits and Cross-Validation · 24 min
2. Regression Metrics · 20 min
3. Classification Metrics and Thresholds · 26 min
4. Bias, Variance and Tuning · 24 min
5. Leakage and Experiment Hygiene · 26 min

05 Ensemble — Trees and Ensembles: The Workhorses of Tabular Data

1. Decision Trees · 24 min
2. Bagging and Random Forests · 24 min
3. Gradient Boosting · 26 min
4. Interpreting and Choosing Tabular Models · 26 min

06 Discover — Beyond Labels: Unsupervised Learning, Recommendation and Reinforcement

1. Clustering · 26 min
2. PCA and Dimensionality Reduction · 24 min
3. Anomaly Detection · 24 min
4. Recommender Systems · 26 min
5. Reinforcement Learning: An Introduction · 26 min

07 Deep — Neural Networks and Deep Learning

1. Neurons, Layers and Activations · 24 min
2. Training Deep Networks · 26 min
3. Convolutional Neural Networks · 24 min
4. Sequence Models and Embeddings · 24 min
5. Transformers, Attention and Foundation Models · 34 min

08 Deploy — MLOps, Monitoring and Responsible ML

1. From Notebook to Production · 26 min
2. Monitoring and Drift · 26 min
3. MLOps: Versioning, Pipelines and Automation · 26 min
4. Responsible Machine Learning · 28 min

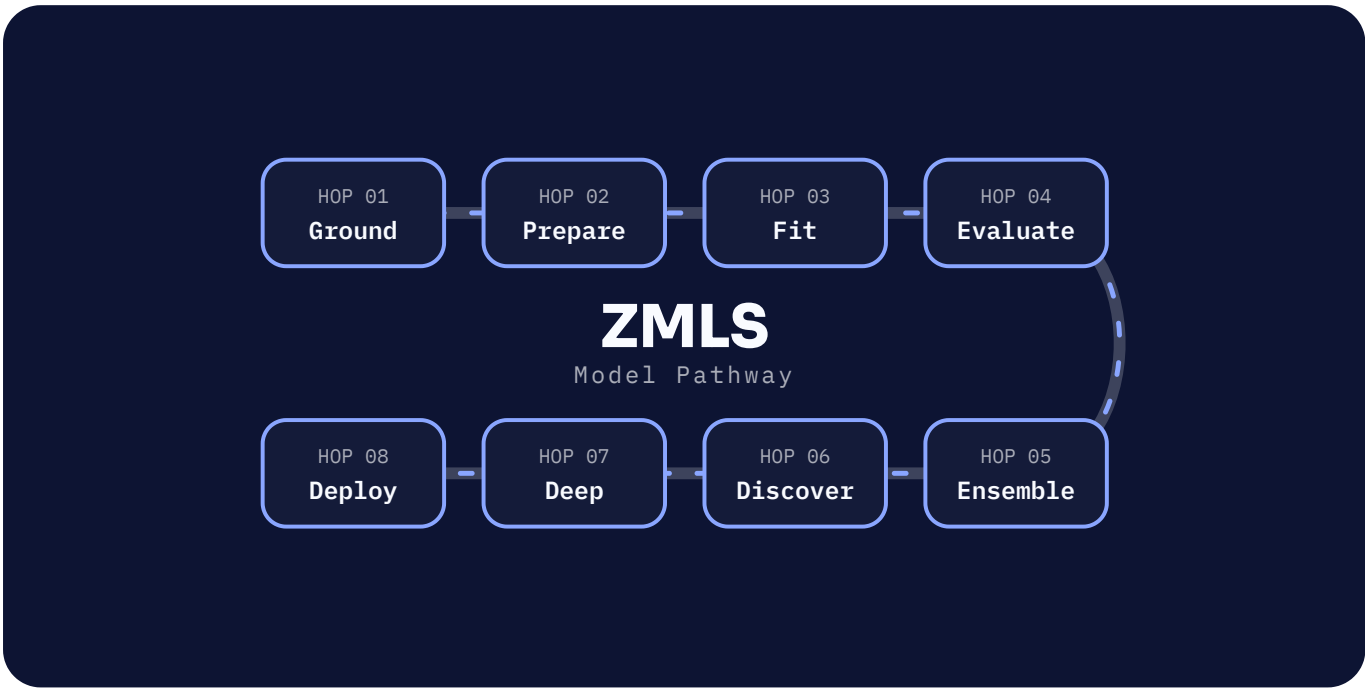
+ Glossary · Capstone guide · Practice exam & answers · Certificate · Notes

Copyright © ZyeemZia

THE FRAMEWORK

The ZMLS Model Pathway

Follow one model along eight stages: ground it in framing and maths, prepare its data, fit it, evaluate it honestly, strengthen it with ensembles, discover structure beyond labels, go deep with neural networks and deploy it responsibly.



01 **Ground**
Ground every model in a clear problem and the linear algebra, calculus and probability it relies on

02 **Prepare**
Turn raw, messy data into clean, leak-free features a model can learn from

03 **Fit**
Fit linear and logistic models with gradient descent, control them with regularisation and know the classic alternatives

04 **Evaluate**
Split, cross-validate and measure with the right metrics, then guard against overfitting and leakage

05 **Ensemble**
Grow decision trees, combine them in random forests and gradient boosting, and explain what they learn

06 **Discover**
Find structure without labels, recommend what people need and learn from rewards

07 **Deep**
Build intuition for neural networks, CNNs, sequence models and transformers, and train them well

08 **Deploy**
Ship models safely, keep them healthy in production and govern them responsibly

Copyright © ZyeemZia

01

MODULE 1 · GROUND

Foundations: Framing and the Maths of Machine Learning

Ground every model in a clear problem and the linear algebra, calculus and probability it relies on

BY THE END OF THIS MODULE YOU WILL BE ABLE TO

- Explain what machine learning is, its main families and the end-to-end workflow
- Frame a business problem as an ML task with a target, timing, metric and baseline
- Compute dot products, norms, cosine similarity and matrix–vector products
- Compute gradients, apply the chain rule and run gradient descent steps by hand
- Apply Bayes' rule and descriptive statistics to reason about data and model outputs

LESSONS

1. What Machine Learning Is and How Projects Flow · 22 min
2. Framing Machine Learning Problems · 24 min
3. Vectors, Matrices and Similarity · 24 min
4. Derivatives, Gradients and the Chain Rule · 24 min
5. Probability and Statistics for Machine Learning · 26 min

MODULE 1 · GROUND · LESSON 1 · 22 MIN

What Machine Learning Is and How Projects Flow

From hand-written rules to learned patterns, and the workflow every ML project follows

- Explain what machine learning is and how it differs from rule-based software
- Distinguish supervised, unsupervised, self-supervised and reinforcement learning
- Describe the end-to-end ML workflow and where most effort really goes
- Name the eight stages of the ZMLS Model Pathway

Most software follows rules that a developer wrote down: if the balance is below zero, block the transfer. **Machine learning (ML)** turns that around. Instead of writing the rules, you show the computer many examples and let an algorithm find a rule that fits them. The result, a **model**, is then used to make predictions on new cases it has never seen.

> DEFINITION · MACHINE LEARNING
> Machine learning is the practice of building systems that improve at a task by learning patterns from data, rather than by following rules written explicitly for every case. A trained model is a function that maps inputs (features) to outputs (predictions), with parameters estimated from examples.

Rule-based software versus machine learning

Rule-based software

- A person writes the logic
- Inputs + rules → outputs
- Transparent and predictable
- Breaks when cases are too many or too fuzzy to list
- Example: VAT calculation

Machine learning

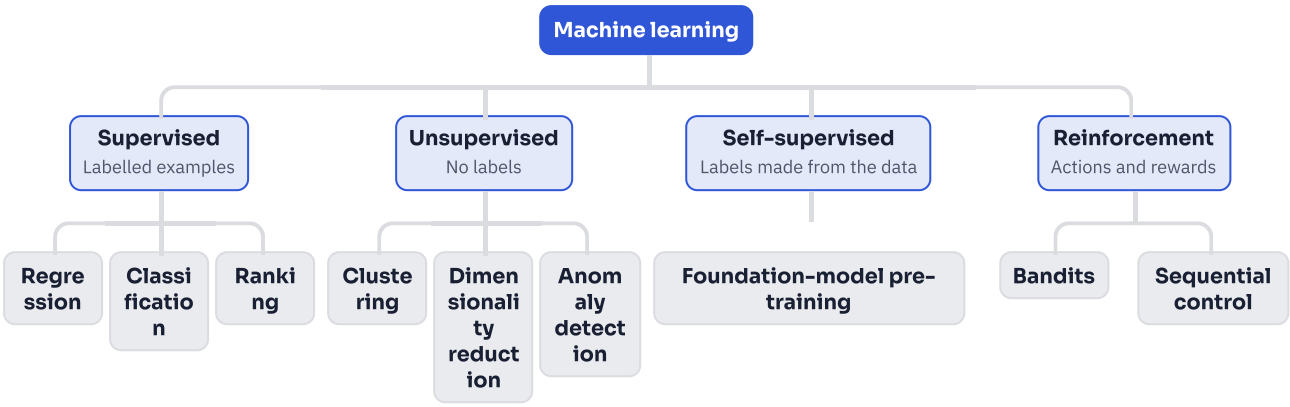
- An algorithm estimates the logic from examples
- Inputs + known outputs → learned rules (the model)
- Handles messy, high-dimensional patterns
- Only as good as its data; can fail silently
- Example: spotting fraudulent wallet transfers

The main families of machine learning

FAMILY	WHAT THE DATA LOOKS LIKE	TYPICAL TASKS	EXAMPLE
Supervised learning	Inputs with known answers (labels)	Regression, classification	Predict a garment order's delivery delay in days
Unsupervised learning	Inputs only, no labels	Clustering, dimensionality reduction, anomaly detection	Group mobile money agents by transaction behaviour
Self-supervised learning	Labels created from the data itself	Pre-training language, image and multimodal foundation models	Predict the masked word in a Bangla sentence
Reinforcement learning	Actions, states and delayed rewards	Sequential decisions, control	Adjust air-conditioning set points in a hospital to cut energy use

Copyright © ZyeemZia

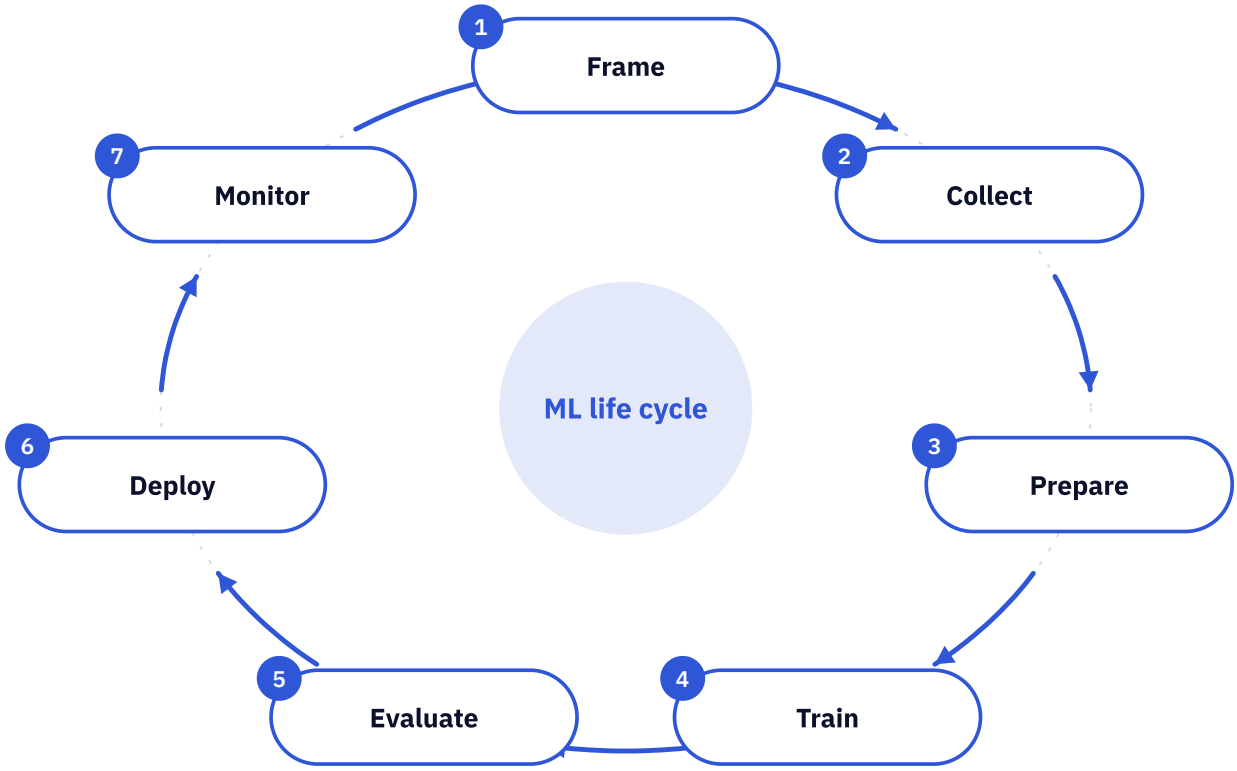
A map of machine learning families



The families differ in the feedback the data provides, not in the algorithms alone. Many modern systems combine them: a model pre-trained with self-supervision is then fine-tuned with labels.

Copyright © ZyeemZia

The machine learning life cycle



1. **Frame** — Decision, target, metric, baseline
2. **Collect** — Find and label the data
3. **Prepare** — Clean, split, engineer features
4. **Train** — Fit candidate models
5. **Evaluate** — Honest test on unseen data
6. **Deploy** — Serve predictions to users
7. **Monitor** — Watch quality and drift, retrain

Monitoring closes the loop: drift, new data or a changed decision sends the team back to framing or data. A model in production is never finished.

INSIGHT · WHERE THE TIME ACTUALLY GOES

In real projects, framing the problem and preparing data usually take far more time than choosing an algorithm. A simple model on clean, well-understood data almost always beats a sophisticated model on data nobody has checked.

Copyright © ZyeemZia

The ZMLS Model Pathway: eight stages

1
Ground
Frame the problem and learn the maths that every model relies on.

2
Prepare
Turn raw data into clean, leak-free features.

3
Fit
Train linear and logistic models with gradient descent and regularisation.

4
Evaluate
Measure honestly with splits, cross-validation and the right metrics.

5
Ensemble
Use trees, forests and boosting, the workhorses of tabular data.

6
Discover
Find structure without labels; recommend and learn from rewards.

7
Deep
Build neural networks: CNNs, sequence models and transformers.

8
Deploy
Ship, monitor and govern models responsibly in production.

Your first model in a few lines

Python notebook

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=1, stratify=y)
model = LogisticRegression(max_iter=1000).fit(X_tr, y_tr)
print(f"test accuracy: {model.score(X_te, y_te):.3f}")
# output: test accuracy: 0.978
```

The whole workflow in miniature: load data, hold some back, fit on the rest, then score on the held-back part. Real projects add framing, cleaning, validation and monitoring around these lines.

SCENARIO · DECIDE · IS ML THE RIGHT TOOL?

A university registrar in Chattogram wants to automatically flag students who have not paid fees by the deadline. The rule is fixed by policy and the payment data is complete. A vendor proposes an ML model. What should you recommend?

- (A) A clustering model to group students by payment habits first
- (B) An ML classifier, because it can learn the pattern from past payments
- (C) A deep neural network, because more capacity gives higher accuracy
- (D) A simple rule or database query, because the logic is known exactly

Circle your choice. Answer and feedback: module answer key.

Copyright © ZyeemZia

Signs that machine learning fits the problem

☐ The rules are too many, too fuzzy or too changeable to write by hand

☐ There is a pattern to learn, and enough examples that show it

☐ Labels or feedback exist, or can be created at reasonable cost

☐ Some prediction errors are tolerable and can be managed

☐ The decision is repeated often enough to justify building and running a model

☐ Someone owns the decision and will act on the predictions

REFLECT

Think of one repeated decision in your organisation. Is it rule-based today? What examples, labels and feedback would you need for a model to help, and what would a wrong prediction cost?

Key terms

Machine learning
Building systems that improve at a task by learning patterns from data instead of following hand-written rules.

Feature
An input variable used by a model, such as age, transaction amount or pixel intensity.

Supervised learning
Learning a mapping from inputs to known outputs using labelled examples.

Self-supervised learning
Learning from labels generated automatically from the data itself, such as predicting a hidden word.

Model
A function with learned parameters that maps input features to a prediction.

Label
The known answer attached to a training example in supervised learning, also called the target.

Unsupervised learning
Finding structure, groups or unusual points in data that has no labels.

Reinforcement learning
Learning a policy of actions by trial and error to maximise long-term reward.

Key takeaways

- A model is a function learned from examples; use ML when the rules are too many, too fuzzy or too changeable to write by hand.
- Supervised, unsupervised, self-supervised and reinforcement learning differ in the feedback the data gives.
- The life cycle is a loop: frame, collect, prepare, train, evaluate, deploy, monitor and return.
- Framing and data work, not algorithm choice, usually decide success.
- When a rule is known exactly and is stable, a rule beats a model.

KNOWLEDGE CHECK

Copyright © ZyeemZia

1. What does a trained machine learning model fundamentally represent?

   (A) A function with parameters estimated from examples that maps inputs to predictions

   (B) A database that stores every training example and searches them one by one at prediction time

   (C) A set of rules written by domain experts and stored as configuration

   (D) A visual dashboard summarising historical performance of the business

2. A hospital wants to group patients by similar admission patterns without any predefined categories. Which family of learning fits?

   (A) Supervised regression

   (B) Reinforcement learning

   (C) Unsupervised learning

   (D) Supervised classification

3. Which stage of the ML workflow usually consumes the most effort in practice?

   (A) Choosing between algorithms of similar accuracy

   (B) Framing the problem and preparing the data

   (C) Writing the final project presentation

   (D) Tuning the learning rate of the final model

4. Pre-training a language model by predicting masked words in sentences is an example of which approach?

   (A) Reinforcement learning

   (B) Unsupervised k-means clustering

   (C) Rule-based programming

   (D) Self-supervised learning

5. In the first-model example, why is part of the data held back with a train/test split?

   (A) To make training faster by using fewer rows

   (B) To remove outliers and duplicate rows from the training data automatically before fitting

   (C) To allow the model to memorise the test answers

   (D) To estimate how the model performs on cases it has not seen during training

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 1 · GROUND · LESSON 2 · 24 MIN

Framing Machine Learning Problems

Turn a business question into a prediction target, a metric and a baseline

- Translate a business decision into a well-defined ML task
- Define the target, the unit of prediction and the moment of prediction
- Choose a success metric linked to business value
- Set a baseline that any model must beat

Most failed ML projects fail before any model is trained. The team predicts the wrong thing, at the wrong time, for a decision nobody will change. **Framing** is the discipline of pinning down exactly what will be predicted, for whom, when, and how the prediction will change an action.

Seven questions that frame an ML problem

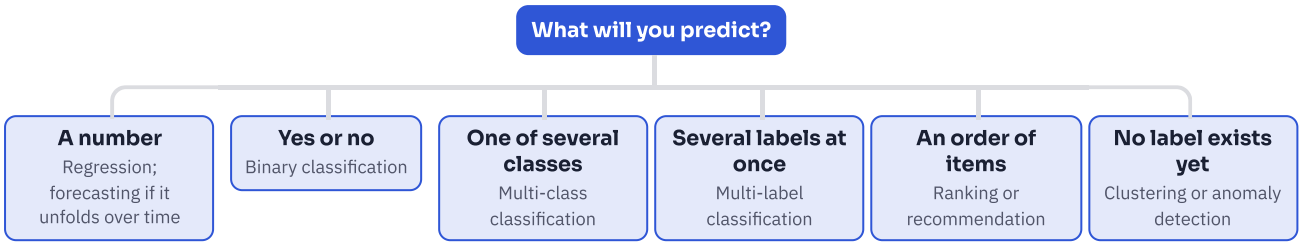
1. **Decision** Which action will change because of the prediction? If no action changes, stop here.
2. **Target** What exactly is predicted? Write the label definition precisely, for example 'no wallet transaction in the next 30 days'.
3. **Unit** One prediction per what: customer, transaction, machine, day, or customer-day?
4. **Timing** When is the prediction made, and what data is genuinely available at that moment?
5. **Task type** Regression, binary or multi-class classification, ranking, forecasting or clustering.
6. **Metric** How will success be measured offline, and which business measure should move?
7. **Baseline** What does a simple rule or the current process achieve today?

From business question to ML task

BUSINESS QUESTION	ML TASK	TARGET	USEFUL METRIC
Which wallet users will stop using the app?	Binary classification	No transaction in next 30 days (yes/no)	Recall at a fixed contact budget, lift
How many shirts will each line produce tomorrow?	Regression / forecasting	Units produced per line per day	Mean absolute error in units
Which loan applicants are likely to default?	Binary classification with probabilities	Default within 12 months	AUC, calibration, expected loss
Which products should we show this shopper?	Ranking / recommendation	Click or purchase on shown items	Precision@k, NDCG, revenue per session
Which emergency cases need urgent review?	Multi-class classification	Triage category assigned by a senior clinician	Recall on the most urgent class

Copyright © ZyeemZia

Which task type fits?



Start from the shape of the answer the decision needs. The task type then narrows the sensible losses, metrics and baselines.

WATCH OUT · BEWARE PROXY TARGETS

We often cannot measure what we care about, so we predict a **proxy**. 'Past spending on healthcare' is a poor proxy for 'need for care' if some groups have historically had less access. A model will optimise the proxy faithfully, including its biases. Always write down how the proxy differs from the real goal.

Putting a value on predictions

MEASURE	FORMULA	WHAT IT TELLS YOU
Net value of acting on a model	`Value = TP × benefit − (TP + FP) × cost_action − FN × cost_missed`	TP, FP and FN are counts of true positives, false positives and false negatives at the chosen threshold. Every contacted case (TP + FP) costs money to act on.
Worked example	`TP = 300, FP = 700, FN = 200; benefit = 2,000 BDT, action cost = 100 BDT, missed cost = 0`	Value = 300 × 2,000 − 1,000 × 100 = 600,000 − 100,000 = 500,000 BDT for one campaign.
Lift	`Lift = precision in targeted group ÷ base rate`	If 30% of contacted users would churn but the base rate is 10%, lift = 0.30 ÷ 0.10 = 3: targeting is three times better than random.

Copyright © ZyeemZia

Where the value comes from: the four outcomes of a churn campaign

CUSTOMER ACTUALLY CHURNS (NO → YES) →

False negative (FN)

- Churner not contacted
- Lost value: cost_missed (0 BDT in the example)

True positive (TP)

- Churner contacted and saved
- Net +1,900 BDT each (2,000 – 100)

True negative (TN)

- Loyal customer left alone
- No cost

False positive (FP)

- Loyal customer contacted
- −100 BDT each

MODEL FLAGS THE CUSTOMER (NO → YES) →

Only the top-right cell creates value, and every flagged customer costs an action. Moving the threshold shifts customers between cells, which is why the metric must reflect costs and capacity.

Two layers of success

Model metric (offline)

- Measured on held-out data
- Accuracy, AUC, MAE, F1, NDCG
- Fast to compute, good for comparing models
- Can improve while business value stays flat

Business metric (online)

- Measured after deployment
- Retention, revenue, cost saved, lives helped
- Slow, noisy, needs an experiment or careful comparison
- The reason the project exists

Baselines you should always compute

- **Majority class**: predict the most common label for everyone; accuracy equals the base rate.
- **Mean or median**: for regression, predict the training mean (for squared error) or median (for absolute error).
- **Last value / seasonal naive**: for forecasting, predict yesterday's value or last week's same day.
- **Simple rule**: the heuristic the business already uses, such as 'no login for 14 days'.
- A model that cannot clearly beat these baselines is not ready, however complex it is.

TEMPLATE

ML problem framing canvas

Agree the problem definition with the decision owner before collecting data or training anything.

Copyright © ZyeemZia

DECISION AND OWNER	PREDICTION TARGET (EXACT DEFINITION)	UNIT AND TIMING	DATA AVAILABLE AT PREDICTION TIME	OFFLINE METRIC AND TARGET	BUSINESS METRIC	BASELINE
Retention team (Head of CRM) chooses whom to call	*No wallet transaction in the 30 days after the scoring date*	*Active customer, scored every Monday*	*Last 90 days of transactions, KYC tier, device data*	*Recall ≥ 0.55 at top 5% contacted*	*30-day retention of contacted users*	*Rule: no login for 14 days (recall 0.31)*

CASE STUDY · FRAMING CHURN FOR A MOBILE FINANCIAL SERVICE

A Dhaka-based mobile money provider wants to 'use AI to reduce churn'. The data team has three years of transactions. Marketing can call 20,000 customers a month and offer a small cashback. An analyst has already trained a model that predicts whether a customer will close their account, reaching 99% accuracy. Fewer than 1% of customers formally close accounts; many more simply go silent.

1. What is wrong with the existing target and the 99% figure?

2. How would you define a better target, unit and timing?

3. Which metric and baseline would you use, given the call capacity?

Model answer: module answer key.

SCENARIO · DECIDE · CHOOSING THE PREDICTION MOMENT

A garment factory in Gazipur wants to predict whether each export order will ship late. One proposed feature is 'number of days the order spent in final inspection'. Predictions are needed when the order is accepted. What should you do with this feature?

- (A) Keep it, because it is strongly correlated with lateness
- (B) Exclude it, because it is not known at the moment the prediction is made
- (C) Replace missing values of it with the average
- (D) Keep it but scale it to the range 0 to 1

Circle your choice. Answer and feedback: module answer key.

Copyright © ZyeemZia

REFLECT

Pick an ML idea from your own work. Write the exact target definition in one sentence, including the time window. Who will act on the prediction, and what will they do differently?

Key terms

Problem framing
Defining the decision, target, unit, timing, metric and baseline before building a model.

Proxy target
A measurable stand-in for the real goal; it can carry bias or diverge from what truly matters.

Prediction time
The moment a prediction is made; only data available at that moment may be used as features.

Lift
How much more often the target occurs in a model-selected group than in the population as a whole.

Target variable
The quantity or category a supervised model is trained to predict.

Unit of prediction
The entity and grain for which one prediction is made, such as customer-week or transaction.

Baseline
A simple reference result, such as the majority class or the current rule, that a model must beat.

Base rate
The overall proportion of positive cases in the population.

Key takeaways

- Frame first: decision, target, unit, timing, metric and baseline, agreed with the decision owner.
- Use only data that exists at the moment the prediction is made.
- Write down how any proxy target differs from the real goal, including who it may under-serve.
- Link model metrics to value: every flagged case costs an action, and only true positives pay back.
- A model that cannot clearly beat a simple baseline is not ready.

KNOWLEDGE CHECK

6. A model predicts loan default with 97% accuracy, and 3% of past loans defaulted. What is the most important first check?

   (A) Increase the number of features to raise accuracy further

   (B) Compare it with the baseline of predicting 'no default' for everyone

   (C) Deploy it quickly because 97% is above the usual target

   (D) Retrain it on a larger network to reach 99%

7. Which question should be answered FIRST when framing an ML project?

   (A) Which algorithm has given the best accuracy on similar problems elsewhere?

   (B) How many GPUs will the training need?

   (C) Which decision or action will change because of the prediction?

   (D) Which programming language should the team use?

Copyright © ZyeemZia

8. A retention campaign contacts 500 customers, of whom 200 are true churners who are saved. Each saved customer is worth 1,500 BDT and every contact costs 100 BDT. What is the net value?

   (A) 250,000 BDT
   (B) 300,000 BDT
   (C) 270,000 BDT
   (D) 200,000 BDT

9. A team predicts 'healthcare spending' as a stand-in for 'healthcare need'. What is the main risk?

   (A) Spending data is always too small and too noisy to train any reliable model at all
   (B) The proxy can encode unequal access, so the model may under-serve some groups
   (C) Regression cannot be used for healthcare questions
   (D) The model will be too slow to serve predictions

10. For a daily demand forecast, which is a sensible baseline?

    (A) Predict zero demand every day
    (B) Use a deep neural network as the baseline
    (C) Predict the largest value ever recorded
    (D) Predict the value from the same weekday last week

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 1 · GROUND · LESSON 3 · 24 MIN

Vectors, Matrices and Similarity

The linear algebra that sits inside every dataset, model and embedding

- Represent examples as vectors and datasets as matrices
- Compute dot products, norms, distances and cosine similarity by hand
- Read matrix shapes and explain what a matrix–vector product does in a model
- Use NumPy for basic vector and matrix operations

To a model, every example is a list of numbers. A customer becomes `[age, monthly_txns, avg_amount, days_since_login]`; an image becomes thousands of pixel values; a sentence becomes an **embedding** of a few hundred to a few thousand numbers. Linear algebra is the language for working with these lists efficiently.

> DEFINITION · VECTOR AND MATRIX
> A **vector** is an ordered list of numbers, such as x = (3, 4). A **matrix** is a rectangular grid of numbers. A dataset with n examples and d features is usually stored as an n × d matrix X, one row per example and one column per feature.

Core vector operations

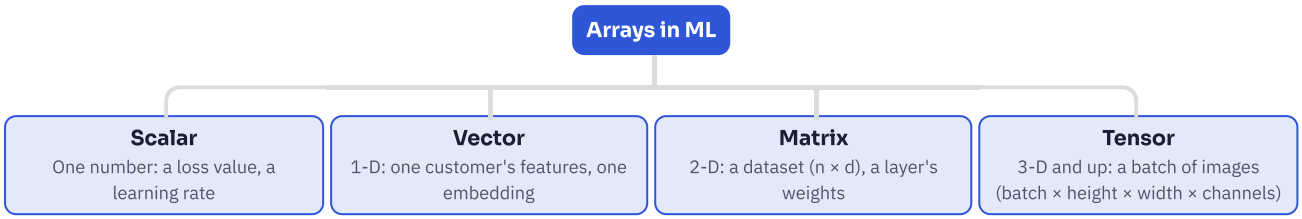
MEASURE	FORMULA	WHAT IT TELLS YOU
Dot product	$a \cdot b = a_1b_1 + a_2b_2 + \dots + a_d b_d$	Multiplies matching entries and adds them. A linear model's raw score is a dot product: w · x + b.
Euclidean norm (length)	$\|x\| = \sqrt{x_1^2 + x_2^2 + \dots + x_d^2}$	For x = (3, 4): ‖x‖ = √(9 + 16) = √25 = 5.
Euclidean distance	$d(a, b) = \|a - b\|$	For a = (1, 2) and b = (4, 6): √(3² + 4²) = 5.
Cosine similarity	$\cos\theta = (a \cdot b) \div (\|a\|\,\|b\|)$	Compares direction, not length. Ranges from −1 (opposite) to 1 (same direction).
Worked cosine	$a = (1, 2, 2)$, $b = (2, 0, 1)$: $a \cdot b = 4$, $\|a\| = 3$, $\|b\| = \sqrt{5} \approx 2.236$	cos θ = 4 ÷ (3 × 2.236) ≈ 0.596, a moderate similarity.

Shapes you will meet constantly

OBJECT	SHAPE	MEANING
Data matrix X	n × d	n examples (rows), d features (columns)
Weight vector w	d × 1	One weight per feature in a linear model
Predictions ŷ = Xw + b	n × 1	One score per example
Weight matrix W in a neural layer	d_in × d_out	Maps d_in inputs to d_out outputs
Embedding table	vocabulary size × embedding size	One learned vector per word, item or user

Copyright © ZyeemZia

From scalars to tensors



Every object a model touches is an array of some rank. Most shape errors in practice come from mixing up which axis holds examples and which holds features.

Matrix–vector product

MEASURE	FORMULA	WHAT IT TELLS YOU
Definition	$(Xw)_i = \Sigma_j X_{ij} w_j$	Each output entry is the dot product of one row of X with w.
Worked example	`X = [[1, 2], [3, 4]], w = (0.5, −1)`	Row 1: 1 × 0.5 + 2 × (−1) = −1.5; row 2: 3 × 0.5 + 4 × (−1) = −2.5; so Xw = (−1.5, −2.5).
Shape rule	`(n × d) times (d × k) gives (n × k)`	Inner dimensions must match; the outer ones give the result's shape.

Vectors and matrices in NumPy

Python notebook

```
import numpy as np
a, b = np.array([1, 2, 2]), np.array([2, 0, 1])
print(a @ b, np.linalg.norm(a))
# output: 4 3.0
cos = a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
print(round(cos, 3))
# output: 0.596
X = np.array([[1, 2], [3, 4]]); w = np.array([0.5, -1])
print(X @ w, X.shape)
# output: [-1.5 -2.5] (2, 2)
```

The @ operator performs dot products and matrix multiplication. Vectorised operations like these are far faster than Python loops on large data.

Euclidean distance or cosine similarity?

Euclidean distance

- Sensitive to magnitude and scale
- Natural for physical measurements and scaled tabular features
- Used by k-means and k-nearest neighbours by default
- Needs features on comparable scales

Cosine similarity

- Ignores length, compares direction
- Natural for text, embeddings and counts
- Common in semantic search and recommenders
- Two documents of different length can still match well

Copyright © ZyeemZia

PRO TIP · SCALE BEFORE YOU MEASURE DISTANCE

If one feature is 'monthly income in taka' (tens of thousands) and another is 'number of children' (0 to 5), Euclidean distance is dominated by income. Standardise features first, or the 'nearest' neighbours are really just 'similar income'.

Why linear algebra matters in ML

- Linear and logistic regression compute w · x + b for every example.
- A neural network layer is a matrix multiplication followed by a non-linear function.
- Embeddings place words, products or users as vectors; similar items have high cosine similarity.
- PCA finds new directions (vectors) that capture the most variance in a matrix.
- GPUs are fast at ML because they are fast at matrix multiplication.
- Vector search systems store millions of embeddings and use approximate nearest-neighbour indexes to return the most similar ones in milliseconds.

SCENARIO · DECIDE · SIMILARITY FOR DOCUMENT SEARCH

A Dhaka law firm builds search over thousands of contracts of very different lengths. Each document is turned into an embedding vector. Long contracts produce vectors with larger norms. Which similarity measure should the search use to compare a query with documents?

(A) Cosine similarity, because it compares direction and ignores vector length

(B) Euclidean distance on the raw vectors

(C) Count of shared words only

(D) Raw dot product, because larger values mean more relevant documents

Circle your choice. Answer and feedback: module answer key.

REFLECT

Choose a dataset you know. Write one example as a feature vector. Which features have very different scales, and how would that affect a distance-based method?

Key terms

Vector
An ordered list of numbers representing one example, a set of weights or an embedding.

Matrix
A rectangular array of numbers; a dataset is commonly an n × d matrix.

Dot product
The sum of products of matching entries of two vectors; the core of linear model scores.

Norm
The length of a vector; the Euclidean (L2) norm is the square root of the sum of squares.

Copyright © ZyeemZia

Cosine similarity
The dot product of two vectors divided by the product of their norms; measures directional similarity.

Euclidean distance
The straight-line distance between two points: the norm of their difference.

Embedding
A learned dense vector that represents an item such as a word, product or user so that similar items are close.

Vectorisation
Writing computations as whole-array operations rather than loops, for speed and clarity.

Key takeaways

- Examples are vectors and datasets are n × d matrices; a linear score is a dot product w · x + b.
- In a matrix product the inner dimensions must match: (n × d)(d × k) gives (n × k).
- Cosine similarity compares direction and suits embeddings; Euclidean distance compares position and needs scaled features.
- Fast matrix arithmetic on GPUs is what makes modern ML practical.

KNOWLEDGE CHECK

11. What is the Euclidean norm of x = (6, 8)?

(A) 10 (B) 14
(C) 100 (D) 48

12. X has shape 500 × 12 and w has shape 12 × 1. What is the shape of Xw?

(A) 12 × 12 (B) 12 × 500
(C) 500 × 1 (D) 500 × 12

13. Two vectors point in exactly opposite directions. What is their cosine similarity?

(A) 0 (B) 1
(C) −1 (D) It depends on their lengths

14. Why should features usually be standardised before using k-nearest neighbours?

(A) Otherwise features with large numeric ranges dominate the distance
(B) Standardisation adds new information to the data
(C) k-nearest neighbours cannot read negative numbers
(D) It converts categorical columns into numbers automatically, so no encoding is needed

15. What is the dot product of a = (2, −1, 3) and b = (1, 4, 2)?

(A) 8 (B) −4
(C) 12 (D) 4

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 1 · GROUND · LESSON 4 · 24 MIN

Derivatives, Gradients and the Chain Rule

How models know which way to adjust their parameters

- Explain a derivative as a rate of change and compute simple derivatives
- Compute a gradient of a function of several variables
- Apply the chain rule to a composed function
- Carry out gradient descent steps by hand and check a gradient numerically

Training a model means finding parameters that make a **loss** (the error) as small as possible. Calculus tells us, for each parameter, whether increasing it would raise or lower the loss and by how much. That information is the **gradient**, and almost every modern training method is built on it.

DEFINITION · DERIVATIVE AND GRADIENT

The **derivative** $f'(x)$ is the slope of f at x: how fast f changes when x changes a little. For a function of several parameters, the **gradient** ∇f is the vector of partial derivatives, one per parameter. It points in the direction of steepest increase; moving against it reduces the function fastest.

Derivatives you need

MEASURE	FORMULA	WHAT IT TELLS YOU
Power rule	$d/dx\ x^n = n\ x^{n-1}$	$d/dx\ x^2 = 2x$; at x = 3 the slope is 6.
Constant multiple and sum	$d/dx\ [a \cdot f + g] = a \cdot f' + g'$	Derivatives can be taken term by term.
Exponential and log	$d/dx\ e^x = e^x$; $d/dx\ \ln x = 1/x$	These appear in logistic regression and cross-entropy loss.
Sigmoid	$\sigma(z) = 1 \div (1 + e^{-z})$; $\sigma'(z) = \sigma(z)(1 - \sigma(z))$	At z = 0, σ = 0.5 and σ′ = 0.25, the steepest point of the curve.
Chain rule	$d/dx\ f(g(x)) = f'(g(x)) \cdot g'(x)$	For $(3x + 1)^2$: $2(3x + 1) \times 3 = 6(3x + 1)$. Backpropagation is the chain rule applied layer by layer.

A gradient by hand

MEASURE	FORMULA	WHAT IT TELLS YOU
Function	$f(w_1, w_2) = w_1^2 + 3w_2^2$	A simple bowl-shaped loss with two parameters.
Partial derivatives	$\partial f/\partial w_1 = 2w_1$; $\partial f/\partial w_2 = 6w_2$	Treat the other parameter as a constant when differentiating.
Gradient at (1, 2)	$\nabla f = (2 \times 1, 6 \times 2) = (2, 12)$	The loss rises much faster along w_2, so w_2 gets the larger correction.

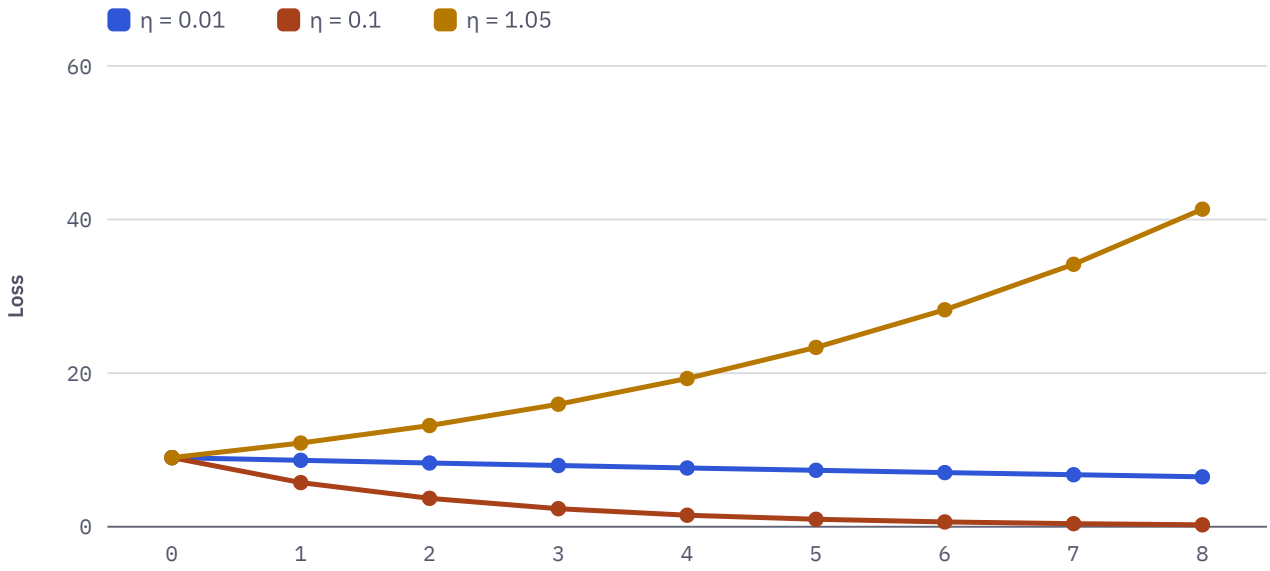
Gradient descent by hand: minimise $f(w) = (w - 3)^2$

1. **Derivative** $f'(w) = 2(w - 3)$. Start at $w_0 = 0$ with learning rate $\eta = 0.1$.
2. **Step 1** Gradient $= 2(0 - 3) = -6$. Update: $w_1 = 0 - 0.1 \times (-6) = 0.6$.
3. **Step 2** Gradient $= 2(0.6 - 3) = -4.8$. Update: $w_2 = 0.6 + 0.48 = 1.08$.

Copyright © ZyeemZia

4. **Step 3** Gradient = 2(1.08 − 3) = −3.84. Update: $w_3 = 1.08 + 0.384 = 1.464$.
5. **Pattern** Each step closes 20% of the remaining gap to the minimum at w = 3, so progress slows as the slope flattens.

Loss per step on f(w) = (w − 3)² for three learning rates



Computed exactly from $w_0 = 0$ (x-axis: step number). With η = 0.1 the loss falls by 36% per step, with η = 0.01 it crawls, and with η = 1.05 every step overshoots further so the loss grows.

Checking a derivative numerically

Python notebook

```
def f(w): return (w - 3) ** 2
def grad(w): return 2 * (w - 3)
h = 1e-5
numeric = (f(0.6 + h) - f(0.6 - h)) / (2 * h)
print(round(numeric, 4), grad(0.6))
# output: -4.8 -4.8
w = 0.0
for _ in range(50): w -= 0.1 * grad(w)
print(round(w, 4))
# output: 3.0
```

A central-difference check confirms that a hand-written gradient is correct. Fifty steps of gradient descent bring w to the minimum at 3.

What the learning rate does

Too small

- Tiny steps
- Training is slow
- May stop early far from the minimum

About right

- Loss falls steadily
- Converges in reasonable time
- Often found by trying values on a log scale

Too large

- Overshoots the minimum
- Loss oscillates or explodes
- On (w − 3)², any η > 1 diverges

Copyright © ZyeemZia

INSIGHT · WHY THE CHAIN RULE MATTERS SO MUCH

A neural network is a long chain of functions: multiply, add, apply an activation, repeat. The chain rule lets us compute the gradient of the final loss with respect to every weight by passing derivatives backwards through the chain. That procedure is **backpropagation**, and automatic differentiation libraries do it for you.

SCENARIO · DECIDE · DIVERGING TRAINING LOSS

A data scientist trains a regression model with gradient descent. The loss goes 120, 340, 1,900, 25,000 and then shows as infinity. The code and data look correct. What is the most likely fix?

- (A) Add more features so the model has more capacity
- (B) Train for more epochs so it eventually settles
- (C) Reduce the learning rate, for example by a factor of 10
- (D) Increase the learning rate so it escapes the bad region

Circle your choice. Answer and feedback: module answer key.

REFLECT

In your own words, explain to a colleague why moving against the gradient reduces the loss. Use a hill-walking analogy and say what the learning rate corresponds to.

Key terms

Derivative
The instantaneous rate of change of a function with respect to its input; the slope of the curve.

Gradient
The vector of partial derivatives of a function; it points in the direction of steepest increase.

Loss function
A function that measures how wrong a model's predictions are; training tries to minimise it.

Gradient check
Comparing an analytic gradient with a numerical finite-difference estimate to catch bugs.

Partial derivative
The derivative of a multi-variable function with respect to one variable, holding the others fixed.

Chain rule
The rule for differentiating composed functions: multiply the outer derivative by the inner derivative.

Learning rate
The step size used when updating parameters in the direction opposite to the gradient.

Sigmoid
The S-shaped function $1 \div (1 + e^{-z})$ that maps any real number into the interval (0, 1).

Key takeaways

- The gradient is the vector of partial derivatives; it points uphill, so training steps against it.
- Gradient descent updates $w \leftarrow w - \eta \times \text{gradient}$; progress slows as the slope flattens near the minimum.
- The learning rate decides everything: too small crawls, too large oscillates or diverges.
- Backpropagation is the chain rule applied layer by layer, and libraries compute it automatically.
- A numerical gradient check confirms a hand-written gradient.

Copyright © ZyeemZia

KNOWLEDGE CHECK

16. What is the derivative of $f(x) = 4x^3$?

(A) $4x^2$ (B) $12x^2$
(C) $12x^3$ (D) $3x^4$

17. For $f(w_1, w_2) = 2w_1^2 + w_2^2$, what is the gradient at (1, 3)?

(A) (4, 6) (B) (2, 9)
(C) (4, 3) (D) (2, 6)

18. Minimising $f(w) = (w - 5)^2$ from $w = 1$ with learning rate 0.25, what is w after one step?

(A) 2 (B) 1.5
(C) 3 (D) 5

19. Which statement about the sigmoid derivative is correct?

(A) It equals 1 everywhere, so gradients never shrink
(B) It equals $\sigma(z)(1 - \sigma(z))$ and peaks at 0.25 when $z = 0$
(C) It is largest when z is very large and positive, so big scores learn fastest
(D) It is undefined at $z = 0$

20. What does backpropagation fundamentally rely on?

(A) Random search over weight values
(B) Sorting the training data by loss
(C) Closed-form matrix inversion for every layer
(D) The chain rule applied through the layers of a network

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 1 · GROUND · LESSON 5 · 26 MIN

Probability and Statistics for Machine Learning

Uncertainty, Bayes' rule and the distributions behind data and models

- Use conditional probability and Bayes' rule, including base rates
- Compute and interpret mean, variance, standard deviation and percentiles
- Recognise common distributions and where they appear in ML
- Distinguish correlation from causation in model results

Data is noisy and the future is uncertain, so ML is fundamentally about probability. Classifiers output probabilities, losses are often negative log-likelihoods, and evaluation is a statistical estimate. A few core ideas cover most of what a practitioner needs.

Probability essentials

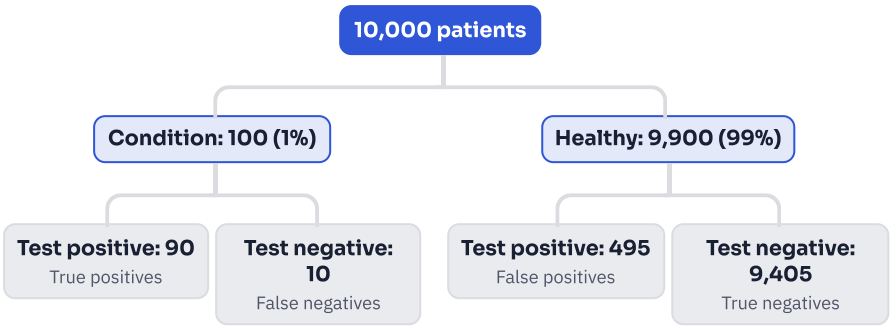
MEASURE	FORMULA	WHAT IT TELLS YOU
Conditional probability	`P(A | B) = P(A and B) ÷ P(B)`	The probability of A given that B is known to have happened.
Bayes' rule	`P(A | B) = P(B | A) × P(A) ÷ P(B)`	Turns 'how likely is the evidence if A is true' into 'how likely is A given the evidence'.
Total probability	`P(B) = P(B | A)P(A) + P(B | not A)P(not A)`	Used for the denominator of Bayes' rule.
Expectation	`E[X] = Σ x · P(x)`	The long-run average value. A model's expected cost uses the same idea.

Worked Bayes example: a screening test

1. **Facts** A condition affects 1% of patients. The test detects 90% of true cases (sensitivity) and wrongly flags 5% of healthy patients.
2. **True positives** P(+ and condition) = 0.90 × 0.01 = 0.009.
3. **False positives** P(+ and healthy) = 0.05 × 0.99 = 0.0495.
4. **All positives** P(+) = 0.009 + 0.0495 = 0.0585.
5. **Answer** P(condition | +) = 0.009 ÷ 0.0585 ≈ 0.154. Only about 15% of positive results are real cases, because the condition is rare.

Copyright © ZyeemZia

The same screening example in natural frequencies

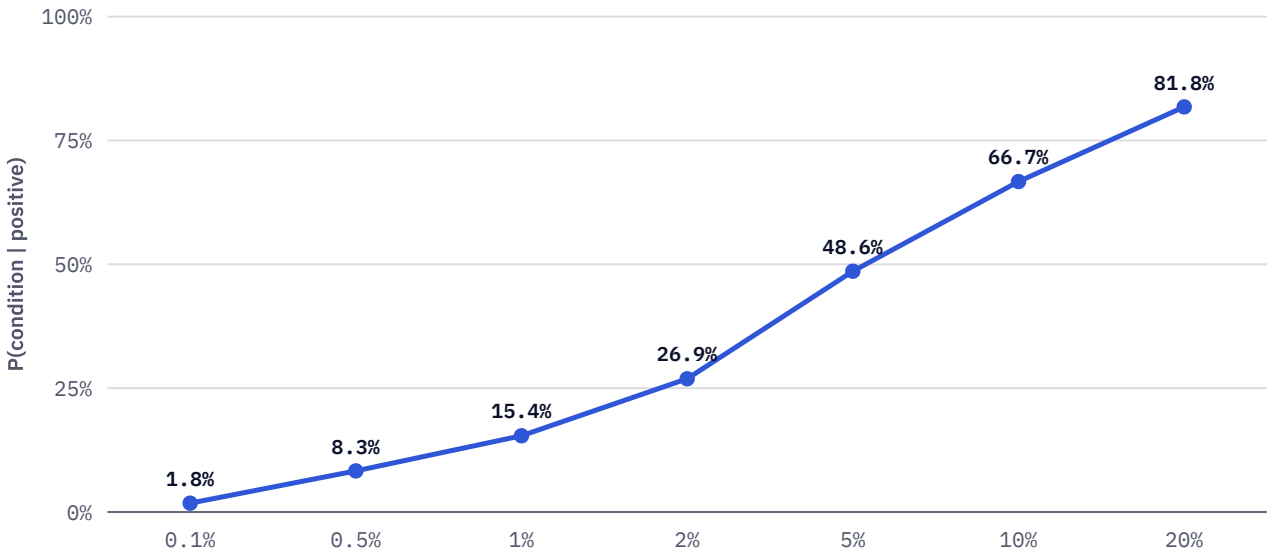


Of 585 positive results, only 90 are real cases: 90 ÷ 585 ≈ 15%. Counting people instead of multiplying probabilities makes the base-rate effect easy to see.

WATCH OUT · THE BASE-RATE TRAP

The same arithmetic applies to fraud alerts, intrusion detection and defect detection. When positives are rare, even an accurate detector produces mostly false alarms. Always check the base rate before judging a model's precision.

Share of positive results that are real, as prevalence changes



Computed with Bayes' rule for a test with 90% sensitivity and a 5% false positive rate (x-axis: prevalence). The test never changes; only the base rate does, and precision moves from under 2% to over 80%.

Describing data

MEASURE	FORMULA	WHAT IT TELLS YOU
Mean	$\bar{x} = (1/n)\ \Sigma\ x_i$	For 4, 8, 6, 2: $\bar{x} = 20 \div 4 = 5$.
Variance (sample)	$s^2 = \Sigma(x_i - \bar{x})^2 \div (n - 1)$	Squared deviations 1, 9, 1, 9 sum to 20; $s^2 = 20 \div 3 \approx 6.67$.

Copyright © ZyeemZia

MEASURE	FORMULA	WHAT IT TELLS YOU
Standard deviation	$s = \sqrt{s^2}$	√6.67 ≈ 2.58, in the same units as the data.
z-score	$z = (x - \bar{x}) \div s$	For x = 8: (8 − 5) ÷ 2.58 ≈ 1.16 standard deviations above the mean.

Distributions you will meet

DISTRIBUTION	DESCRIBES	WHERE IT APPEARS IN ML
Bernoulli	One yes/no trial with probability p	Binary labels; logistic regression's output
Binomial	Number of successes in n trials	Count of conversions in a campaign
Normal (Gaussian)	Symmetric bell curve with mean μ and spread σ	Measurement noise; linear regression assumptions; weight initialisation
Poisson	Counts of events in a fixed interval	Calls per hour at a hospital helpdesk; defects per batch
Uniform	All values in a range equally likely	Random search over hyperparameters
Long-tailed (e.g. log-normal)	Many small values, a few huge ones	Transaction amounts, incomes, page views

Summary statistics and Bayes in Python

Python notebook

```
import numpy as np
x = np.array([4, 8, 6, 2])
print(x.mean(), round(x.std(ddof=1), 2), np.percentile(x, 50))
# output: 5.0 2.58 5.0
prior, sens, fpr = 0.01, 0.90, 0.05
posterior = sens * prior / (sens * prior + fpr * (1 - prior))
print(round(posterior, 3))
# output: 0.154
```

ddof=1 gives the sample standard deviation (dividing by n − 1). NumPy's default, ddof=0, divides by n.

Correlation is not causation

What a model learns

- Associations present in historical data
- Feature X helps predict Y
- Good for forecasting and ranking

What a decision often needs

- The effect of changing X on Y
- Requires experiments or causal methods
- Needed for pricing, policy or treatment choices

EXAMPLE · ICE CREAM AND DROWNING

Across a year, ice-cream sales and drowning incidents rise together, because both increase in hot weather. A model can use ice-cream sales to predict drownings, but banning ice cream would not save swimmers. Before acting on a feature's importance, ask whether it causes the outcome or merely travels with it.

Copyright © ZyeemZia

SCENARIO · DECIDE · INTERPRETING A FRAUD ALERT RATE

A payments company deploys a fraud detector that catches 95% of fraud and flags 2% of legitimate transactions. Fraud is 1 in 1,000 transactions. The operations manager expects most flagged transactions to be fraud. What should you tell her?

(A) About 95% of flags will be legitimate, so plan review capacity and customer friction accordingly

(B) The detector is useless and should be switched off immediately

(C) About 50% of flags will be fraud, because errors balance out

(D) About 95% of flags will be real fraud, matching the detection rate

Circle your choice. Answer and feedback: module answer key.

REFLECT

Find one statistic you report regularly (an average, a rate or a percentage). Is the distribution skewed? Would the median or a percentile tell a more honest story?

Key terms

Conditional probability
The probability of an event given that another event has occurred, written P(A | B).

Bayes' rule
A formula that updates a prior probability using evidence: P(A | B) = P(B | A)P(A) ÷ P(B).

Prior
The probability of a hypothesis before seeing the current evidence; in classification, often the base rate.

Posterior
The updated probability of a hypothesis after taking the evidence into account.

Variance
The average squared deviation from the mean; a measure of spread.

Standard deviation
The square root of the variance, in the original units of the data.

Normal distribution
The symmetric bell-shaped distribution defined by a mean and a standard deviation.

Percentile
The value below which a given percentage of observations fall; the 50th percentile is the median.

Key takeaways

- Bayes' rule turns P(evidence | cause) into P(cause | evidence); the base rate matters as much as the test.
- When positives are rare, even a good detector produces mostly false alarms.
- Describe skewed data with the median and percentiles, not just the mean.
- Models learn associations; acting on a feature as a cause needs experiments or causal methods.

KNOWLEDGE CHECK

Copyright © ZyeemZia

21. A disease affects 2% of people. A test has 80% sensitivity and a 10% false positive rate. Roughly what is P(disease | positive)?

(A) About 80%
(B) About 2%
(C) About 50%
(D) About 14%

22. What is the sample variance of 1, 3, 5?

(A) 2.67
(B) 4
(C) 3
(D) 2

23. Which distribution best describes the number of calls a hospital helpdesk receives per hour?

(A) Bernoulli
(B) Uniform
(C) Poisson
(D) Normal with negative values

24. A feature strongly predicts loan default. What can you safely conclude?

(A) It is associated with default in this data, but that does not show it causes default
(B) Changing the feature for a customer will change their default risk by the same amount
(C) It must be removed because correlated features are unfair
(D) The model has proven a causal mechanism

25. Transaction amounts are heavily right-skewed. Which summary best describes a typical transaction?

(A) The mean
(B) The median
(C) The maximum
(D) The variance

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 1 · GROUND

Answer key

Knowledge checks

1. A A model is a learned function. Some methods keep training data (such as nearest neighbours), but the general idea is parameters estimated from data, not hand-written rules or reports.

2. C There are no labels, and the aim is to find structure, which is unsupervised learning such as clustering.

3. B Data work and framing dominate real projects; algorithm choice is usually a smaller part of the effort.

4. D The labels (the hidden words) come from the data itself, which defines self-supervised learning.

5. D The held-back test set gives an honest estimate of performance on new data; scoring on training data would be optimistic.

6. B Predicting 'no default' for everyone also scores 97%. Without beating the majority-class baseline, the model may have learned nothing useful.

7. C If no action changes, the model has no value. Algorithm and tooling choices come later.

8. A Value = 200 × 1,500 − 500 × 100 = 300,000 − 50,000 = 250,000 BDT. Every contact costs money, including the 200 true positives.

9. B Proxy targets can diverge from the real goal and carry historical bias into the model's decisions.

10. D A seasonal naive forecast is a simple, strong baseline for daily data with weekly patterns.

11. A $\sqrt{(36 + 64)} = \sqrt{100} = 10$.

12. C Inner dimensions (12) match and the result takes the outer dimensions: 500 × 1.

13. C Opposite directions give cos 180° = −1 regardless of length.

14. A Distance sums squared differences, so large-scale features swamp the rest unless scales are made comparable.

15. D $2 \times 1 + (-1) \times 4 + 3 \times 2 = 2 - 4 + 6 = 4$.

16. B Power rule: $4 \times 3 \times x^2 = 12x^2$.

17. A $\partial f/\partial w_1 = 4w_1 = 4$ and $\partial f/\partial w_2 = 2w_2 = 6$.

18. C Gradient = 2(1 − 5) = −8; w = 1 − 0.25 × (−8) = 3.

19. B $\sigma' = \sigma(1 - \sigma)$, maximised at σ = 0.5, giving 0.25; it shrinks towards 0 for large |z|.

20. D Backpropagation propagates derivatives backwards using the chain rule.

21. D 0.8 × 0.02 = 0.016; 0.1 × 0.98 = 0.098; 0.016 ÷ 0.114 ≈ 0.14.

22. B Mean 3; squared deviations 4, 0, 4 sum to 8; 8 ÷ (3 − 1) = 4.

23. C Poisson models counts of independent events in a fixed interval.

24. A Predictive association is not causation; causal claims need experiments or causal analysis.

25. B In skewed data, a few huge values pull the mean upwards; the median is more representative.

Scenarios & case studies

Scenario · Is ML the right tool?
D is best. Correct. When the rule is known, stable and complete, ML adds cost, opacity and failure modes without adding value.

Case study · Framing churn for a mobile financial service
Formal closure is rare and is not how most customers leave, so the target misses the real behaviour; 99% accuracy is roughly what predicting 'never closes' would score. A better target is inactivity, for example no transaction in the 30 days after a weekly scoring date, per active customer, using only data up to that date. Because only 20,000 calls a month are possible, measure recall or precision within the top-ranked 20,000 (or lift at that cut-off), compare against a simple inactivity rule, and plan an A/B test to confirm that calls actually improve retention.

Scenario · Choosing the prediction moment
B is best. Correct. Inspection time is only known near the end of production. Using it would leak future information and give a model that looks excellent offline but cannot work at order acceptance.

Scenario · Similarity for document search
A is best. Correct. Cosine similarity removes the effect of vector magnitude, so a short and a long contract about the same topic can still match closely.

Copyright © ZyeemZia

Scenario · Diverging training loss

C is best. Correct. A loss that grows with each step is the classic sign of overshooting: steps are too large for the curvature of the loss.

Scenario · Interpreting a fraud alert rate

A is best. Correct. Per 100,000 transactions: 95 frauds caught and about 1,998 false alarms, so precision is roughly 95 ÷ 2,093 ≈ 4.5%.

Copyright © ZyeemZia

02

MODULE 2 · PREPARE

Data Preparation and Feature Engineering

Turn raw, messy data into clean, leak-free features a model can learn from

BY THE END OF THIS MODULE YOU WILL BE ABLE TO

- Assess data sources, label quality and representativeness before modelling
- Clean missing values, outliers and duplicates without destroying signal or leaking information
- Encode categorical features and scale numeric ones appropriately for each model family
- Engineer point-in-time features from dates, behaviour, ratios and text
- Build pipelines that handle imbalance and feature selection safely

LESSONS

1. Data Sourcing, Labelling and Quality · 24 min
2. Cleaning Data: Missing Values, Outliers and Duplicates · 24 min
3. Encoding Categories and Scaling Numbers · 22 min
4. Feature Engineering · 26 min
5. Pipelines, Imbalance and Feature Selection · 24 min

MODULE 2 · PREPARE · LESSON 1 · 24 MIN

Data Sourcing, Labelling and Quality

Where training data comes from, how labels are made and how to know if they can be trusted

- Identify common data sources and the risks each brings
- Design a labelling process with clear guidelines and quality checks
- Measure label agreement with Cohen's kappa
- Profile a dataset for the main dimensions of data quality

A model can never be better than the data it learns from. Before any cleaning or feature work, a practitioner asks three questions: **where did this data come from, how were the labels made, and does it represent the cases the model will face?** Problems found here are cheap to fix; problems found after deployment are not.

Common data sources and their typical risks

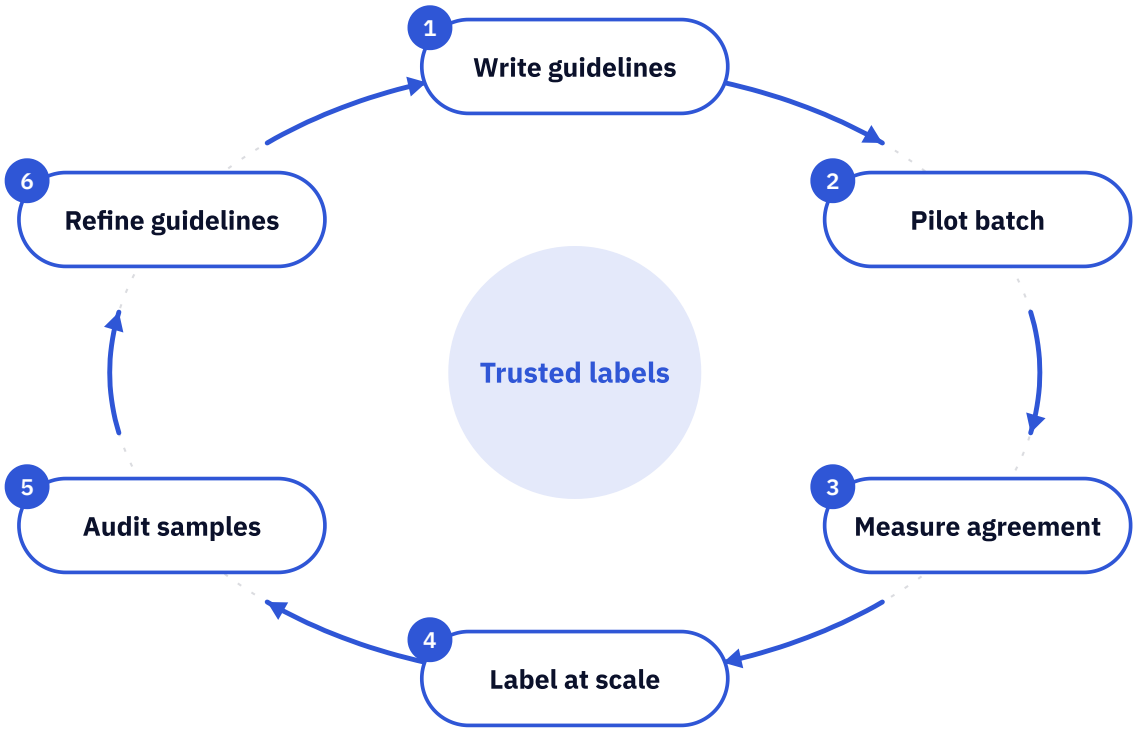
SOURCE	EXAMPLES	TYPICAL RISKS
Operational systems	Core banking, ERP, hospital records, point-of-sale	Fields reused for other purposes, silent schema changes, missing history
Event and log data	App clicks, wallet transactions, machine sensors	Duplicates, clock skew, bot traffic, huge volume
Human labelling	Annotated X-rays, tagged complaints, reviewed fraud cases	Inconsistent guidelines, fatigue, labeller bias
Surveys and forms	Customer feedback, field surveys in rural districts	Self-selection, missing answers, leading questions
External and open data	Weather, census, public datasets	Licence limits, different definitions, stale updates
Synthetic data	Simulated transactions, generated text	May not reflect real-world edge cases or noise

Building a reliable labelling process

1. **Write guidelines** Define each label with examples, counter-examples and rules for edge cases.
2. **Pilot** Have several labellers label the same small batch, then discuss disagreements.
3. **Measure agreement** Compute agreement such as Cohen's kappa; refine guidelines until it is acceptable.
4. **Label at scale** Use trained labellers, with a share of items labelled twice for ongoing checks.
5. **Audit** A senior reviewer samples labels regularly; track error types and labeller drift.
6. **Version** Store labels with guideline version, labeller and date so you can trace changes.

Copyright © ZyeemZia

The labelling quality loop



1. **Write guidelines** — Definitions, examples, counter-examples, edge-case rules
2. **Pilot batch** — Several labellers label the same small sample
3. **Measure agreement** — Cohen's kappa or a similar chance-corrected measure
4. **Label at scale** — Record guideline version, labeller and date
5. **Audit samples** — Re-label a random share and review disagreements
6. **Refine guidelines** — Fix ambiguous rules, then continue

Labelling is a loop, not a one-off task. Each audit feeds back into the guidelines, and every guideline change is versioned so labels stay traceable.

Measuring label agreement

MEASURE	FORMULA	WHAT IT TELLS YOU
Observed agreement	`p_o = items where both labellers agree ÷ total items`	Raw agreement can look high just by chance when one label dominates.
Chance agreement (two labels)	`p_e = p₁(yes) × p₂(yes) + p₁(no) × p₂(no)`	Probability the two labellers would agree if they labelled at random with their own rates.
Cohen's kappa	`κ = (p_o − p_e) ÷ (1 − p_e)`	1 means perfect agreement, 0 means no better than chance.

Copyright © ZyeemZia

MEASURE	FORMULA	WHAT IT TELLS YOU
Worked example	`p_o = 0.85; each labeller says 'urgent' 50% of the time, so p_e = 0.25 + 0.25 = 0.5`	κ = (0.85 − 0.5) ÷ (1 − 0.5) = 0.70, usually read as substantial agreement.

Six dimensions of data quality

Q1
Completeness
Are required values present? What share of each column is missing?

Q2
Validity
Do values follow rules and formats, such as dates that exist and ages above zero?

Q3
Accuracy
Do values match reality? Spot-check against a trusted source.

Q4
Consistency
Do the same facts agree across tables and systems?

Q5
Timeliness
Is the data fresh enough, and are timestamps reliable?

Q6
Uniqueness
Are records duplicated, for example the same transaction logged twice?

A first data profile with pandas

Python notebook

```
import pandas as pd
df = pd.DataFrame({'age': [34, None, 29, 41, 29], 'district': ['Dhaka', 'Sylhet', None, 'Dhaka',
None],
                   'amount': [1200, 560, 560, 98000, 560]})
print(df.isna().sum().to_dict())
# output: {'age': 1, 'district': 2, 'amount': 0}
print(df.duplicated().sum(), df['amount'].median())
# output: 1 560.0
print(df['district'].fillna('Unknown').value_counts().sort_index().to_dict())
# output: {'Dhaka': 2, 'Sylhet': 1, 'Unknown': 2}
```

Count missing values per column, exact duplicate rows and category frequencies before anything else. The 98,000 amount also deserves a question: is it real or a data entry error?

WATCH OUT · REPRESENTATIVENESS BEATS VOLUME

A million rows from Dhaka users tell you little about rural users in Rangpur. Check that the training data covers the populations, devices, seasons and conditions the model will meet. Missing groups lead to models that work well on average but fail for exactly the people who were left out.

WATCH OUT · SYNTHETIC AND MODEL-GENERATED DATA

Synthetic records from simulators or generative models, and labels produced by large language models, can fill gaps, protect privacy or bootstrap a project. They also copy the biases and blind spots of whatever produced them, and a generator that memorised real records can leak them. Tag synthetic rows clearly, check a sample against careful human labels, never judge a model only on synthetic data, and keep the final test set real data from the population the model will serve.

Copyright © ZyeemZia

SCENARIO · DECIDE · A LABELLING DISAGREEMENT

Two nurses label 400 triage notes as 'urgent' or 'routine'. They agree on 88% of notes, but kappa is only 0.35 because nearly all notes are routine. The project lead wants to start labelling 50,000 notes. What should happen next?

- (A) Train the model first and let it decide the ambiguous cases
- (B) Use only one nurse to avoid disagreements
- (C) Review the disagreements, sharpen the definition of 'urgent', and re-pilot before scaling
- (D) Start labelling now, since 88% agreement is high

Circle your choice. Answer and feedback: module answer key.

Data readiness checklist

- ☐ The source, owner and refresh process of each table are documented
- ☐ The label definition is written down and matches the problem framing
- ☐ Label agreement has been measured on a pilot sample
- ☐ Missing values, duplicates and invalid values have been profiled
- ☐ The data covers the populations and conditions the model will serve
- ☐ Personal data is minimised and access is approved
- ☐ Licences and consent allow the intended use

REFLECT

Think of a dataset you rely on. Who created the labels or key fields, under what instructions, and how would you find out whether they are consistent?

Key terms

Data provenance
A record of where data came from and how it was transformed on the way to the model.

Inter-annotator agreement
The degree to which independent labellers assign the same labels to the same items.

Data profiling
Systematic summary of a dataset's columns, types, ranges, missing values and duplicates.

Label noise
Errors or inconsistencies in the target labels of a training set.

Annotation guidelines
Written rules and examples that tell labellers how to assign each label consistently.

Cohen's kappa
An agreement statistic for two labellers that corrects for agreement expected by chance.

Representativeness
How well a dataset reflects the population and conditions where the model will be used.

Copyright © ZyeemZia

Key takeaways

- Ask where the data came from, how labels were made and whether it represents the cases the model will face.
- Measure label agreement with a chance-corrected statistic such as kappa before labelling at scale.
- Coverage of the deployment population matters more than raw volume.
- Synthetic or model-generated data needs tagging, human spot checks and a real test set.
- Keep provenance for every label: guideline version, labeller and date.

KNOWLEDGE CHECK

1. Two labellers agree on 90% of items and chance agreement is 0.6. What is Cohen's kappa?
   - (A) 0.75
   - (B) 0.90
   - (C) 0.30
   - (D) 0.50
2. Why can raw percentage agreement be misleading?
   - (A) It ignores the number of labellers' working hours
   - (B) It can only be computed for numeric labels
   - (C) When one label dominates, labellers agree often by chance alone
   - (D) It always underestimates true agreement, because labellers are usually stricter than needed
3. Which data quality dimension does 'the same transaction appears twice in the log' violate?
   - (A) Timeliness
   - (B) Uniqueness
   - (C) Validity
   - (D) Completeness
4. A credit model is trained only on applicants who were approved in the past. What is the main concern?
   - (A) The dataset will be too large to process
   - (B) Approved applicants never have missing values, so imputation will not be needed
   - (C) Labels for approved applicants are always noisy
   - (D) The data does not represent rejected applicants the model will now score
5. What should be stored alongside each label for traceability?
   - (A) Only the final model's prediction
   - (B) A screenshot of the labelling tool for every item
   - (C) The labeller's salary band
   - (D) The guideline version, labeller and date

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 2 · PREPARE · LESSON 2 · 24 MIN

Cleaning Data: Missing Values, Outliers and Duplicates

Decide carefully what to fix, what to flag and what to leave alone

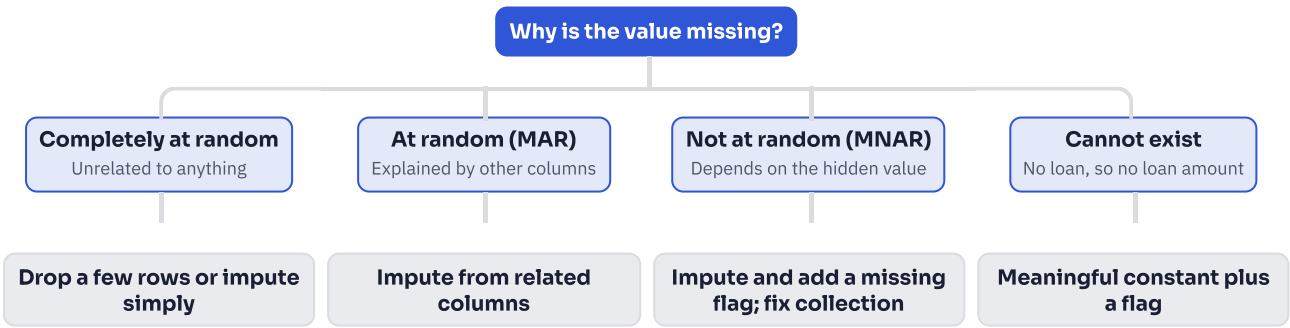
- Diagnose why values are missing and choose a suitable strategy
- Detect outliers with the IQR rule and decide how to treat them
- Remove duplicates and fix invalid values safely
- Fit cleaning steps on training data only to avoid leakage

Cleaning is not about making data look tidy. It is about making it **truthful and usable** without destroying signal. A missing value can be informative, an outlier can be the fraud you want to catch, and an over-eager fix can leak test information into training.

Why values go missing

MECHANISM	MEANING	EXAMPLE	IMPLICATION
Missing completely at random (MCAR)	Missingness unrelated to anything	A sensor drops readings randomly due to a loose cable	Simple imputation is reasonably safe
Missing at random (MAR)	Missingness depends on other observed columns	Older patients skip an online form field more often	Impute using related columns
Missing not at random (MNAR)	Missingness depends on the missing value itself	High earners decline to state income	Missingness carries signal; add an indicator and model it

Choosing a missing-value strategy



The reason for missingness, not the share missing, decides the fix. Whatever you choose, fit the imputer on training data only.

Options for missing values

Drop

- Drop rows: only if few rows and missingness is random
- Drop columns: if mostly empty and not useful
- Risk: bias and lost data

Impute

- Median for skewed numbers, mean for symmetric
- Most frequent or 'Unknown' for categories
- Model-based (k-NN, iterative) for related columns
- Always fit on training data only

Flag

- Add a binary 'was_missing' column
- Lets the model learn if missingness matters
- Cheap and often powerful
- Tree models can also handle missing values natively

Copyright © ZyeemZia

Detecting outliers

MEASURE	FORMULA	WHAT IT TELLS YOU
Interquartile range	`IQR = Q3 − Q1`	The spread of the middle 50% of the data.
Tukey fences	`lower = Q1 − 1.5 × IQR; upper = Q3 + 1.5 × IQR`	Points outside the fences are flagged as potential outliers.
Worked example	`Q1 = 400, Q3 = 1,000 BDT, so IQR = 600`	Fences are 400 − 900 = −500 and 1,000 + 900 = 1,900 BDT. A 98,000 BDT transfer is flagged for review.
z-score rule	`|z| = |x − mean| ÷ std > 3`	Works for roughly normal data; unreliable for skewed data because the outliers inflate the std themselves.

Deciding what to do with an outlier

01 Detect
IQR, z-score or domain limits

→

02 Investigate
Error, rare-but-real, or the target itself?

→

03 Fix errors
Correct or remove impossible values

→

04 Keep real extremes
Cap, log-transform or use robust models

→

05 Document
Record the rule and how many rows it touched

EXAMPLE · WHEN THE OUTLIER IS THE POINT

In a fraud model for a mobile financial service, unusually large transfers at 3 a.m. from a new device are exactly the pattern of interest. Deleting outliers would delete the fraud. In contrast, an age of 212 years in a hospital record is an entry error and should be corrected or set to missing.

Imputation without leakage

Python notebook

```
import numpy as np
from sklearn.impute import SimpleImputer
X_train = np.array([[1.0], [2.0], [np.nan], [100.0]])
X_test = np.array([[np.nan], [3.0]])
imp = SimpleImputer(strategy='median', add_indicator=True).fit(X_train)
print(imp.transform(X_test).tolist())
# output: [[2.0, 1.0], [3.0, 0.0]]
```

The median (2.0) is learned from the training rows only and then applied to the test rows. add_indicator=True appends a 'was missing' flag column.

Other cleaning steps that matter

- **Duplicates**: remove exact duplicates; for near-duplicates (same customer, different spelling) use matching rules.
- **Invalid values**: enforce ranges and formats; convert impossible values to missing rather than guessing.
- **Inconsistent categories**: unify 'Dhaka', 'dhaka ' and 'DHK' with a mapping table.
- **Units and time zones**: convert everything to one unit and one time standard before joining.
- **Keep raw data**: clean into a new version so every step can be reproduced and reversed.

Copyright © ZyeemZia

SCENARIO · DECIDE · IMPUTING BEFORE SPLITTING

An analyst fills missing blood pressure values with the mean of the full dataset, then splits into training and test sets. Validation results look slightly better than expected. What is the issue?

(A) Mean imputation is never allowed in medical data

(B) There is no issue because imputation does not use the labels

(C) The test set should be imputed with zeros instead

(D) The imputation used test-set statistics, a mild form of leakage; fit the imputer on training data only

Circle your choice. Answer and feedback: module answer key.

TEMPLATE

Cleaning log

Record every cleaning rule so the pipeline is reproducible and reviewers can judge its effect.

STEP	COLUMN	RULE	ROWS AFFECTED	REASON	FITTED ON
C-03	*monthly_income*	*Impute median; add was_missing flag*	*1,842 (6.1%)*	*Missing more often for self-employed (MAR)*	*Training split only*
C-04	*age*	*Values < 0 or > 110 set to missing*	*37*	*Data entry errors*	*Rule, not fitted*

REFLECT

Look at one column in your data with missing values. What is the most likely reason they are missing, and would a 'was missing' flag carry useful signal?

Key terms

Imputation
Filling missing values with estimates such as the median, the most frequent value or a model prediction.

Missing indicator
A binary feature recording whether a value was originally missing.

Copyright © ZyeemZia

MCAR
Missing completely at random: missingness unrelated to any data, observed or not.

Outlier
An observation far from most others; it may be an error or a genuine rare event.

Winsorising
Capping extreme values at chosen percentiles instead of removing them.

MNAR
Missing not at random: missingness depends on the unobserved value itself.

Interquartile range (IQR)
The difference between the 75th and 25th percentiles; a robust measure of spread.

Key takeaways

- Clean to make data truthful, not tidy: missing values and outliers can carry signal.
- Diagnose why values are missing (MCAR, MAR, MNAR or structural) before choosing a fix.
- Investigate outliers before acting; the extreme case may be the fraud you want to catch.
- Fit imputers and other cleaning statistics on the training split only, and log every rule.

KNOWLEDGE CHECK

6. Q1 = 20 and Q3 = 50. What is the upper Tukey fence?
   - (A) 80
   - (B) 95
   - (C) 75
   - (D) 65

7. High-income respondents often skip the income question. What type of missingness is this?
   - (A) Missing completely at random
   - (B) Missing at random
   - (C) Missing not at random
   - (D) Not missing at all

8. Where should an imputer's statistics, such as the median, be computed?
   - (A) On the training split only
   - (B) On the full dataset before splitting
   - (C) On the test split only
   - (D) On a random half of the test split

9. In a fraud dataset, several transfers are 50 times larger than typical. What is the BEST first step?
   - (A) Delete them because they distort the mean
   - (B) Investigate whether they are errors or genuine rare events before deciding
   - (C) Replace them with the median amount so the model is not distorted by extremes
   - (D) Multiply all amounts by a constant

10. Why add a missing-indicator column alongside imputation?
    - (A) To make the dataset smaller
    - (B) Because models cannot read imputed values unless a flag tells them which ones changed
    - (C) To remove the need for a test set
    - (D) So the model can learn whether the fact of missingness predicts the target

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 2 · PREPARE · LESSON 3 · 22 MIN

Encoding Categories and Scaling Numbers

Turn strings into numbers and put features on comparable scales

- Choose between one-hot, ordinal and target encoding
- Apply standardisation, min–max scaling and log transforms correctly
- Know which models need scaling and which do not
- Handle unseen and rare categories safely

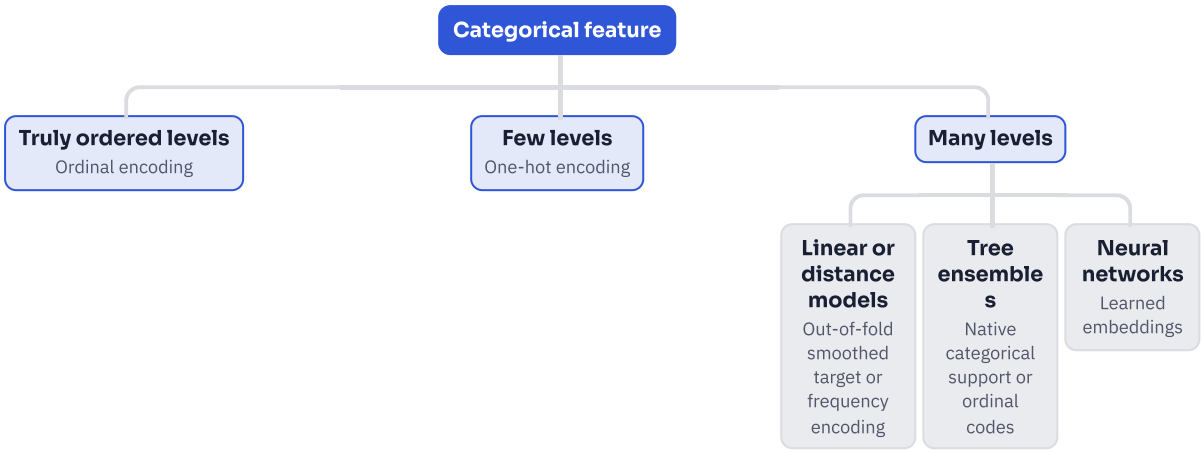
Most algorithms work on numbers, and many assume those numbers are on similar scales. **Encoding** converts categories such as district or device type into numbers; **scaling** puts numeric features on comparable ranges. Both look mechanical, but the wrong choice can mislead a model or leak information.

Encoding options for categorical features

METHOD	HOW IT WORKS	GOOD FOR	WATCH OUT FOR
One-hot	One binary column per category	Nominal categories with few levels (payment method)	Many columns for high-cardinality features
Ordinal	Map ordered levels to integers	Truly ordered categories (education level, risk grade)	Implies order and equal spacing; wrong for nominal data in linear models
Target (mean) encoding	Replace category with a smoothed mean of the target	High-cardinality features (merchant ID, upazila)	Leakage unless computed out-of-fold
Frequency encoding	Replace category with its count or share	High-cardinality features where popularity matters	Different categories can share the same value
Hashing	Hash categories into a fixed number of columns	Huge, changing vocabularies	Collisions; hard to interpret
Learned embeddings	Dense vectors learned by a neural network	Very high cardinality with lots of data	Needs a neural model and enough data

Copyright © ZyeemZia

Choosing an encoding



Order and cardinality drive the choice, and the model family refines it. In every branch, decide in advance how unseen categories are handled in production.

Scaling and encoding formulas

MEASURE	FORMULA	WHAT IT TELLS YOU
Standardisation	`z = (x − μ) ÷ σ`	Mean 0, standard deviation 1. With μ = 50,000 and σ = 20,000 BDT, an income of 90,000 becomes 2.0.
Min–max scaling	`x′ = (x − min) ÷ (max − min)`	Maps to [0, 1]. With min 0 and max 200, the value 50 becomes 0.25. Sensitive to outliers.
Log transform	`x′ = log(1 + x)`	Compresses long right tails such as transaction amounts; log1p handles zeros.
Smoothed target encoding	`enc = (n × mean_cat + m × mean_global) ÷ (n + m)`	With n = 4 rows, category mean 0.5, global mean 0.1 and m = 10: (2 + 1) ÷ 14 ≈ 0.214. Rare categories shrink towards the global mean.

Does the model need scaled features?

Scaling matters

- Linear and logistic regression with regularisation
- k-nearest neighbours and k-means
- Support vector machines
- Neural networks
- PCA

Scaling barely matters

- Decision trees
- Random forests
- Gradient-boosted trees
- Rule-based models

Copyright © ZyeemZia

One-hot encoding and scaling together

Python notebook

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
df = pd.DataFrame({'channel': ['app', 'agent', 'app', 'ussd'], 'amount': [100.0, 300.0, 500.0,
700.0]})
ct = ColumnTransformer([('cat', OneHotEncoder(handle_unknown='ignore'), ['channel']),
                        ('num', StandardScaler(), ['amount'])], sparse_threshold=0)
out = ct.fit_transform(df)
print(ct.get_feature_names_out().tolist())
# output: ['cat__channel_agent', 'cat__channel_app', 'cat__channel_ussd', 'num__amount']
print(out[0].round(3).tolist())
# output: [0.0, 1.0, 0.0, -1.342]
print(ct.transform(pd.DataFrame({'channel': ['bank'], 'amount': [400.0]})).tolist())
# output: [[0.0, 0.0, 0.0, 0.0]]
```

handle_unknown='ignore' turns an unseen category ('bank') into all zeros instead of crashing in production. StandardScaler uses the population standard deviation (about 223.6), so 100 becomes (100 − 400) ÷ 223.6 ≈ −1.342.

WATCH OUT · TARGET ENCODING LEAKS UNLESS DONE OUT-OF-FOLD

If a category's target mean is computed on all training rows, each row's own label contributes to its feature, and the model learns to trust a value that will not behave the same way on new data. Compute target encodings within cross-validation folds (each row encoded using other folds only) and add smoothing for rare categories.

Practical rules

- Fit encoders and scalers on training data only; apply the fitted object to validation, test and production data.
- Group rare categories (for example under 1% of rows) into 'Other' to reduce noise.
- Decide in advance what happens to unseen categories in production.
- Use ordinal encoding for nominal categories only with tree models, where the arbitrary order does little harm.
- Keep the fitted transformers with the model; they are part of it.

SCENARIO · DECIDE · ENCODING 480 UPAZILAS

A microfinance lender in Bangladesh wants to use the borrower's upazila (about 480 sub-districts) in a logistic regression default model. Some upazilas have only a handful of loans. Which approach is most suitable?

(A) Plain target encoding using each upazila's default rate on all training rows

(B) Smoothed target encoding computed out-of-fold, with rare upazilas shrunk towards the global rate

(C) Drop the feature because it has too many categories

(D) Ordinal encoding with upazilas numbered alphabetically

Circle your choice. Answer and feedback: module answer key.

Copyright © ZyeemZia

REFLECT

List the categorical columns in a dataset you know. For each, note its number of levels, whether it is ordered, and which encoding you would use with a linear model and with a tree model.

Key terms

One-hot encoding
Representing a categorical value as a set of binary columns, one per category.

Target encoding
Replacing a category with a statistic of the target, such as its smoothed mean, for that category.

Standardisation
Rescaling a feature to mean 0 and standard deviation 1.

Log transform
Applying a logarithm to compress large values and reduce right skew.

Ordinal encoding
Mapping ordered categories to integers that preserve their order.

High cardinality
A categorical feature with a very large number of distinct values.

Min–max scaling
Rescaling a feature linearly into a fixed range, usually 0 to 1.

Key takeaways

- One-hot suits nominal features with few levels; ordinal encoding only suits truly ordered levels.
- High-cardinality features need out-of-fold target encoding with smoothing, frequency encoding or embeddings.
- Scale features for distance-, gradient- and penalty-based models; trees barely care.
- Fit encoders and scalers on training data and ship them with the model.

KNOWLEDGE CHECK

11. A feature has mean 40 and standard deviation 8. What is the standardised value of 52?

(A) 1.5
(B) 12
(C) 0.67
(D) −1.5

12. Which model is least affected by whether features are scaled?

(A) k-nearest neighbours
(B) A neural network
(C) A random forest
(D) Regularised logistic regression

13. Why is plain target encoding on all training rows risky?

(A) It produces too many columns
(B) It only works for numeric features
(C) Each row's own label influences its feature value, leaking the target
(D) It cannot represent more than ten categories

Copyright © ZyeemZia

14. A category has 2 rows with target mean 1.0; global mean is 0.2; smoothing m = 8. What is the smoothed encoding?

(A) 0.36
(B) 1.00
(C) 0.20
(D) 0.60

15. What does handle_unknown='ignore' in a one-hot encoder achieve?

(A) Unknown categories are silently dropped from the training set before the encoder is fitted
(B) Missing values are imputed with the mode
(C) Rare categories are merged automatically
(D) Unseen categories in production become all-zero columns instead of causing an error

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 2 · PREPARE · LESSON 4 · 26 MIN

Feature Engineering

Create inputs that make the pattern easy for a model to see

- Engineer features from dates, aggregates, ratios and text
- Use domain knowledge to design meaningful features
- Build time-windowed aggregates without looking into the future
- Judge when feature engineering beats a more complex model

A model can only use what it is given. **Feature engineering** reshapes raw data into inputs that expose the pattern: a ratio instead of two raw numbers, a weekday instead of a timestamp, a 30-day transaction count instead of a raw log. On tabular problems, good features often matter more than the choice of algorithm.

Feature patterns that work again and again

PATTERN	RAW DATA	ENGINEERED FEATURE	WHY IT HELPS
Date parts	Transaction timestamp	Hour, weekday, month, is_holiday, days to Eid	Exposes daily, weekly and festival cycles
Recency, frequency, monetary	Transaction log	Days since last txn, txns in 30 days, total amount in 30 days	Summarises behaviour per customer
Ratios	Debt, income	Debt-to-income ratio	Puts values in context; linear models cannot multiply or divide inputs themselves
Differences and trends	Monthly usage	This month – average of previous 3 months	Captures change, often more predictive than level
Interactions	Age, product type	Age × product indicator	Lets linear models express combined effects
Text signals	Complaint text	Length, keyword flags, TF-IDF or embeddings	Turns unstructured text into numbers
Geography	Latitude, longitude	Distance to nearest branch or hospital	Encodes access and travel effort

Feature formulas worth knowing

MEASURE	FORMULA	WHAT IT TELLS YOU
Cyclical encoding	`hour_sin = sin(2π × hour ÷ 24); hour_cos = cos(2π × hour ÷ 24)`	Places 23:00 next to 00:00. At hour 6: sin = 1, cos = 0.
Ratio	`debt_to_income = monthly_debt ÷ monthly_income`	25,000 ÷ 100,000 = 0.25. Guard against division by zero.
TF-IDF weight	`tfidf(t, d) = tf(t, d) × log(N ÷ df(t))`	A word in 10 of 1,000 documents gets idf = ln(100) ≈ 4.6; a word in every document gets 0.
Polynomial term	x^2, $x_1 \times x_2$	Adds curvature or interaction to a linear model.

Copyright © ZyeemZia

WATCH OUT · POINT-IN-TIME CORRECTNESS

Every aggregate must be computed using only data available **before** the prediction time. A '30-day transaction count' for a scoring date of 1 March must use 30 January to 28 February, never data from March. Building features from a table that has already been updated with later events is one of the most common causes of leakage.

Point-in-time windows for a scoring date of 1 March 2026

30 JAN – 28 FEB
Feature window
Only events in these 30 days feed the aggregates.

1 MAR
Cut-off and scoring
Features are frozen and the customer is scored.

5 MAR
Late-arriving record
A 27 Feb event loaded on 5 Mar was not available at scoring; use load time if production would have missed it.

31 MAR
Label known
The outcome window (1–30 Mar) closes and the target can be recorded.

Features look backwards from the cut-off and labels look forwards from it. Rebuilding features from a table that already contains later or late-loaded events quietly breaks this separation.

Point-in-time aggregates in pandas

Python notebook

```
import pandas as pd
tx = pd.DataFrame({'cust': ['A', 'A', 'A', 'B'], 'amount': [500, 700, 900, 300],
                   'date': pd.to_datetime(['2026-02-01', '2026-02-20', '2026-03-05', '2026-02-
25'])})
cutoff = pd.Timestamp('2026-03-01')
window = tx[(tx.date < cutoff) & (tx.date >= cutoff - pd.Timedelta(days=30))]
feat = window.groupby('cust').amount.agg(['count', 'sum'])
print(feat.to_dict('index'))
# output: {'A': {'count': 2, 'sum': 1200}, 'B': {'count': 1, 'sum': 300}}
```

The 5 March transaction is excluded because it happened after the cut-off. The 30-day window starts on 30 January, so the 1 February transaction is included.

A feature engineering loop

1. **Start from hypotheses** Ask domain experts what drives the outcome; write each idea as a candidate feature.
2. **Check availability** Confirm each feature can be computed at prediction time in production.
3. **Build and validate** Compute the feature, look at its distribution and its relationship with the target.
4. **Measure value** Compare cross-validated performance with and without the feature group.
5. **Keep it simple** Drop features that add little; every feature has a maintenance and monitoring cost.

EXAMPLE · DOMAIN KNOWLEDGE WINS

A power plant in Chattogram wanted to predict turbine faults. Raw vibration readings gave a weak model. An engineer suggested features for the change in vibration over the last hour and the temperature difference between inlet and outlet. With these two features, a simple gradient-boosted model detected most faults hours earlier than before.

Copyright © ZyeemZia

CASE STUDY · FEATURES FOR A HOSPITAL READMISSION MODEL

A private hospital in Dhaka wants to predict, at discharge, which patients will be readmitted within 30 days. The first model uses age, sex, ward and length of stay and reaches an AUC of 0.64. The data warehouse also holds diagnosis codes, laboratory results with timestamps, medication lists, previous admissions and follow-up appointment bookings. One analyst proposes adding 'number of outpatient visits in the 30 days after discharge' because it is highly predictive.

1. Which proposed feature must be rejected, and why?

2. Suggest four engineered features likely to add signal at discharge time.

3. How would you test whether the new features genuinely help?

Model answer: module answer key.

SCENARIO · DECIDE · CHOOSING BETWEEN EFFORT OPTIONS

A team's churn model has plateaued. One proposal is to move from gradient boosting to a large neural network. Another is to spend two weeks building behavioural features such as recency, trend in monthly usage and failed transaction counts. Data is tabular, 200,000 customers. What is the better next step?

- (A) Switch to the large neural network first, since deep learning is more powerful
- (B) Build the behavioural features and measure their effect with cross-validation
- (C) Remove all existing features and start again from raw columns
- (D) Collect five more years of history before doing anything

Circle your choice. Answer and feedback: module answer key.

REFLECT

For a prediction you care about, write down three features an expert in that domain would expect to matter but that are not raw columns in your data. How would you compute each at prediction time?

Key terms

Feature engineering
Creating new input variables from raw data to make patterns easier for a model to learn.

RFM features
Recency, frequency and monetary features summarising customer behaviour over a window.

Copyright © ZyeemZia

Cyclical encoding
Representing periodic values such as hour or month with sine and cosine so the ends of the cycle are close.

TF-IDF
A text weighting that increases with a word's frequency in a document and decreases with its frequency across documents.

Aggregation window
The time span, such as the last 30 days, over which a feature is summarised.

Interaction feature
A feature that combines two or more features, such as their product, to capture joint effects.

Point-in-time correctness
Computing features only from data that existed at the moment of prediction.

Key takeaways

- Good features expose the pattern: ratios, recency–frequency–monetary summaries, cyclical time and domain signals.
- Every aggregate must use only data available before the prediction time, including late-arriving data.
- Domain experts are the best source of feature ideas.
- Keep a feature group only if cross-validated performance improves with it.

KNOWLEDGE CHECK

16. Why encode hour of day with sine and cosine?

(A) To reduce the number of rows
(B) So that 23:00 and 00:00 are represented as close together
(C) Because most models cannot read integer values such as 23 or 0 directly
(D) To remove the need for a test set

17. A word appears in 100 of 10,000 documents. What is its idf using natural log?

(A) About 4.6
(B) About 2.0
(C) 100
(D) About 0.01

18. For a scoring date of 1 June, which transactions may feed a '7-day count' feature?

(A) Transactions from 1 June to 7 June
(B) All transactions in June
(C) Transactions from 25 May up to 31 May
(D) Transactions from 25 May to 7 June inclusive

19. Why can a debt-to-income ratio help a logistic regression more than debt and income separately?

(A) Ratios always have lower variance
(B) A linear model cannot form the ratio from the two raw inputs on its own
(C) Logistic regression only accepts one input
(D) Raw values are always leaked

20. What is the most reliable way to judge whether a new feature group is worth keeping?

(A) Check whether the feature names sound relevant to the business experts on the team
(B) Count how many features the model has
(C) Look at training accuracy only
(D) Compare cross-validated performance with and without the feature group

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 2 · PREPARE · LESSON 5 · 24 MIN

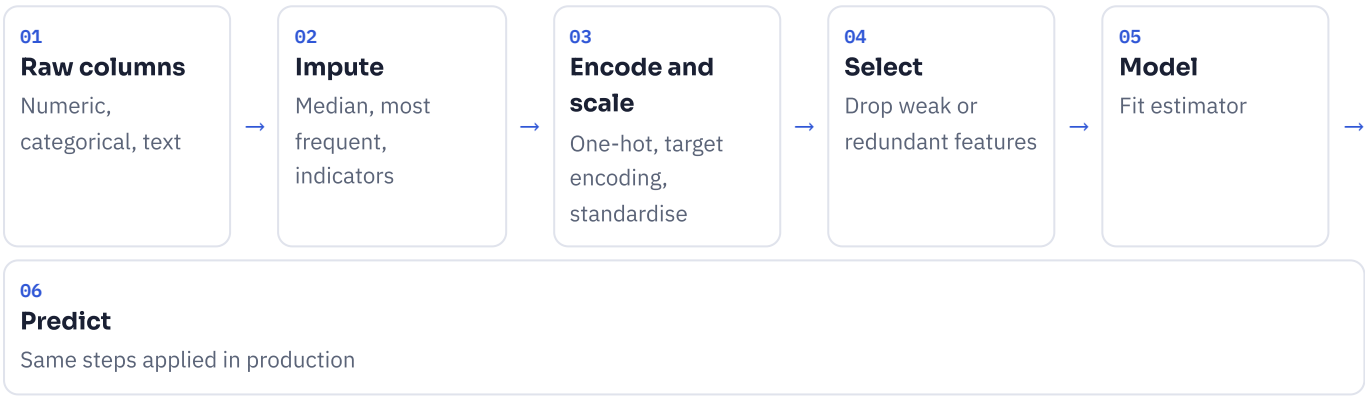
Pipelines, Imbalance and Feature Selection

Package preparation safely, handle rare classes and keep only features that earn their place

- Build a preprocessing and model pipeline that prevents leakage
- Choose strategies for imbalanced classes, including class weights and resampling
- Apply filter, wrapper and embedded feature selection methods
- Explain why resampling must only touch training data

By now the preparation steps are many: impute, encode, scale, engineer, select. Doing them by hand in a notebook invites mistakes, especially leakage. A **pipeline** chains every step with the model so that one `fit` learns everything from training data and one `predict` applies it consistently everywhere.

A typical tabular pipeline



A leak-free pipeline with cross-validation

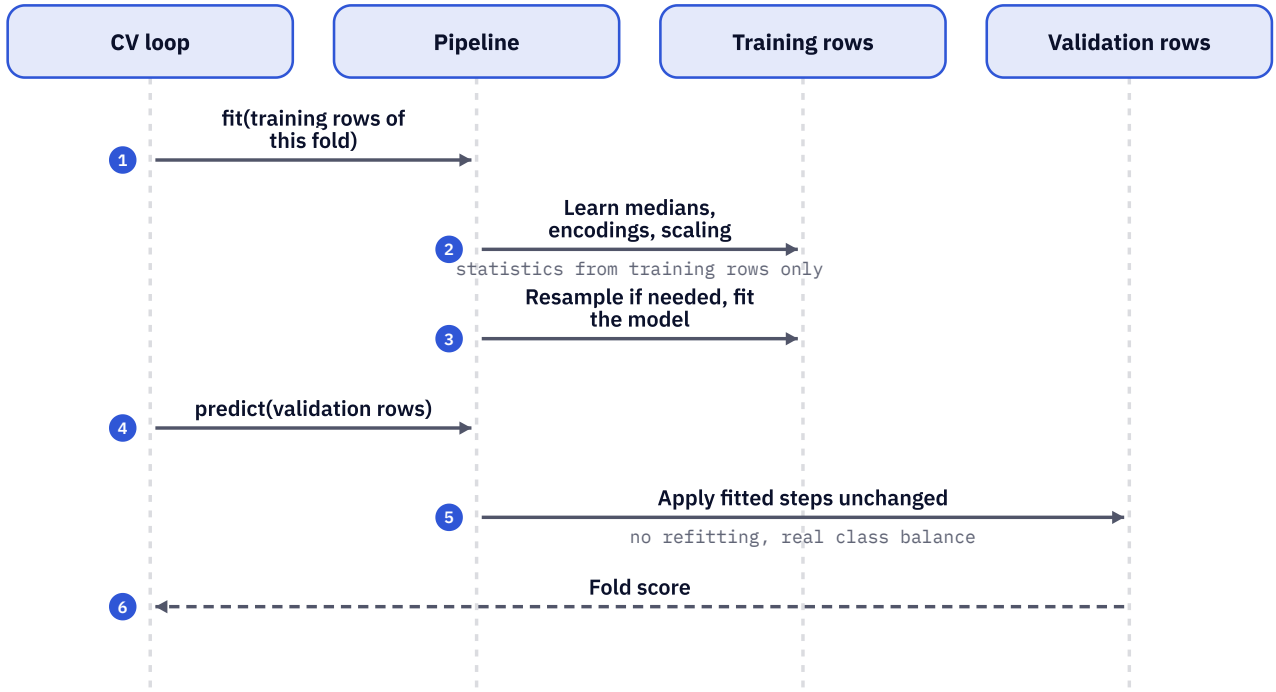
Python notebook

```
from sklearn.datasets import make_classification
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
X, y = make_classification(n_samples=600, n_features=10, weights=[0.9], random_state=7)
pipe = make_pipeline(StandardScaler(), LogisticRegression(class_weight='balanced'))
scores = cross_val_score(pipe, X, y, cv=5, scoring='roc_auc')
print(round(scores.mean(), 3))
# output: 0.872
```

The scaler is refitted inside each fold on that fold's training part only, so no validation information leaks. class_weight='balanced' up-weights the rare positive class (about 10% of rows).

Copyright © ZyeemZia

What happens inside one cross-validation fold



The pipeline refits every step on each fold's training rows and only applies them to the validation rows. That is what keeps imputation, encoding, resampling and selection from leaking.

Imbalance arithmetic

MEASURE	FORMULA	WHAT IT TELLS YOU
Balanced class weight	`w_c = n ÷ (k × n_c)`	n rows, k classes, n_c rows in class c. With 1,000 rows, 900 negatives and 100 positives: w_neg = 1,000 ÷ 1,800 ≈ 0.56, w_pos = 1,000 ÷ 200 = 5.
Effect	`Each positive counts about 9 times as much as a negative in the loss`	5 ÷ 0.56 ≈ 9, matching the 9:1 ratio of the classes.
Imbalance ratio	`IR = n_majority ÷ n_minority`	900 ÷ 100 = 9. Above about 10, plain accuracy becomes almost meaningless.

Ways to handle imbalanced classes

Change the loss

- Class weights or cost-sensitive learning
- No data is duplicated or thrown away
- Usually the first thing to try

Change the data

- Random undersampling of the majority
- Random oversampling or synthetic samples (e.g. SMOTE) of the minority
- Apply to training folds only, never to validation or test

Change the decision

- Keep probabilities, move the threshold
- Choose the threshold from costs or capacity
- Evaluate with precision, recall, PR-AUC

Copyright © ZyeemZia

WATCH OUT · NEVER RESAMPLE BEFORE SPLITTING

If you oversample the minority class and then split, copies of the same minority example land in both training and test sets. The model 'recognises' them and test scores become wildly optimistic. Resample inside the training fold only, and evaluate on data with the real class balance.

Feature selection approaches

APPROACH	HOW IT WORKS	EXAMPLES	TRADE-OFF
Filter	Score each feature independently of a model	Correlation, mutual information, chi-square, variance threshold	Fast, but ignores interactions
Wrapper	Search feature subsets by training models	Recursive feature elimination, forward selection	Can find good subsets, but costly and prone to overfitting the validation set
Embedded	Selection happens during training	L1 (lasso) regularisation, tree-based importance	Efficient; tied to one model type

Why fewer features can be better

- Less overfitting when data is limited relative to the number of features.
- Faster training and prediction, lower serving cost.
- Fewer data pipelines to maintain and monitor for drift.
- Easier explanations for users, auditors and regulators.
- But: never select features using the test set, or the selection itself leaks.

SCENARIO · DECIDE · SUSPICIOUSLY GOOD RESULTS

A student oversampled the fraud class with synthetic examples until it was 50% of the data, then ran a train/test split. Recall on the test set is 0.99 and precision 0.98. In production, precision drops to 0.08. What went wrong?

(A) The fraud labels in production must be wrong

(B) The model needed more synthetic samples to generalise

(C) Resampling before splitting put near-copies of training examples in the test set and changed its class balance

(D) Precision always falls in production by about this amount

Circle your choice. Answer and feedback: module answer key.

Pipeline hygiene checklist

☐ All preprocessing is inside the pipeline and fitted on training data only

☐ Resampling, if used, is applied only within training folds

☐ Feature selection is done inside cross-validation, not on the full dataset

☐ Validation and test sets keep the real-world class balance

☐ The fitted pipeline (not just the model) is saved and versioned

☐ The same pipeline object is used for batch scoring and online serving

Copyright © ZyeemZia

REFLECT

Does your current workflow have any preparation step done outside the model pipeline? What could leak, and how would you move it inside?

Key terms

Pipeline
A chained sequence of preprocessing steps and a model that is fitted and applied as one object.

Class weight
A multiplier on the loss for examples of a class, used to make rare classes count more.

Undersampling
Removing majority-class examples from the training data to balance classes.

Feature selection
Choosing a subset of features to improve generalisation, speed or interpretability.

Class imbalance
A situation where one class is much rarer than another in the target.

Oversampling
Duplicating or synthesising minority-class examples in the training data.

SMOTE
A technique that creates synthetic minority examples by interpolating between neighbouring minority points.

Key takeaways

- Put every preparation step and the model in one pipeline so fit learns only from training data.
- Handle imbalance with class weights first; resample only inside training folds.
- Evaluate on data with the real class balance and with metrics other than accuracy.
- Select features inside cross-validation, never on the full dataset.
- Save and version the whole fitted pipeline, not just the model.

KNOWLEDGE CHECK

21. With 2,000 rows, 1,800 negatives and 200 positives, what is the balanced class weight for positives?

(A) 10
(B) 0.56
(C) 9
(D) 5

22. When should oversampling be applied?

(A) To the full dataset before the split
(B) Only to the training portion, inside each training fold
(C) To the test set only
(D) To both training and test sets equally, so that their class balance matches

23. Which is an embedded feature selection method?

(A) Pearson correlation with the target
(B) Exhaustive forward selection
(C) L1 (lasso) regularisation
(D) Variance threshold

Copyright © ZyeemZia

24. What is the main benefit of wrapping preprocessing and the model in one pipeline?

(A) Preprocessing is learned from training data in each fold and applied identically everywhere

(B) It makes models more accurate in every case

(C) It removes the need for any validation data, because the pipeline checks its own performance

(D) It automatically chooses the best algorithm

25. Why is plain accuracy a weak metric on a dataset with 1% positives?

(A) Accuracy cannot be computed for binary labels

(B) Predicting the majority class for everyone already scores 99%

(C) Accuracy always rewards false positives

(D) Accuracy is only defined for regression

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 2 · PREPARE

Answer key

Knowledge checks

1. A κ = (0.9 − 0.6) ÷ (1 − 0.6) = 0.3 ÷ 0.4 = 0.75.

2. C With a dominant class, two careless labellers would still agree on most items; kappa corrects for this.

3. B Duplicate records break uniqueness and can bias both training and evaluation.

4. D Outcomes are only known for approved loans, so the training data is not representative of all applicants, a form of selection bias.

5. D Versioned guidelines, labeller identity and timestamps let you trace and fix label problems later.

6. B IQR = 30; upper fence = 50 + 1.5 × 30 = 95.

7. C Missingness depends on the value that is missing, which is MNAR.

8. A Learning statistics from validation or test data leaks information about them into training.

9. B Extreme values may be exactly the fraud signal. Investigate first, then fix errors and keep genuine extremes.

10. D Missingness often carries signal, especially when it is not random.

11. A (52 − 40) ÷ 8 = 1.5.

12. C Tree splits depend on the order of values, not their scale.

13. C Computing encodings out-of-fold prevents a row's label from shaping its own feature.

14. A (2 × 1.0 + 8 × 0.2) ÷ (2 + 8) = 3.6 ÷ 10 = 0.36.

15. D It makes the encoder robust to categories that did not appear during training.

16. B Cyclical encoding preserves the circular nature of time; raw 23 and 0 look far apart.

17. A ln(10,000 ÷ 100) = ln(100) ≈ 4.6.

18. C Only data before the prediction time may be used; the window ends just before the scoring date.

19. B Linear models add weighted inputs; they cannot divide one input by another, so the ratio must be supplied.

20. D A controlled comparison on validation data measures the real contribution of the features.

21. D w = n ÷ (k × n_c) = 2,000 ÷ (2 × 200) = 5.

22. B Resampling before the split contaminates the evaluation with copies and an unrealistic class balance.

23. C Lasso drives some coefficients to exactly zero during training, so selection is part of fitting.

24. A Pipelines prevent leakage and keep training and serving transformations consistent.

25. B With heavy imbalance, accuracy hides whether the model finds any positives at all.

Scenarios & case studies

Scenario · A labelling disagreement

C is best. Correct. Low kappa with high raw agreement means the labellers mostly agree on easy routine cases and disagree on the rare urgent ones, which are the cases that matter.

Scenario · Imputing before splitting

D is best. Correct. Any statistic learned from data, including means for imputation, must come from the training split and be applied to validation and test data.

Scenario · Encoding 480 upazilas

B is best. Correct. This captures each area's risk in one column, avoids hundreds of sparse columns and controls leakage and noise for small groups.

Case study · Features for a hospital readmission model

Outpatient visits after discharge happen after the prediction moment, so the feature leaks the future and must be rejected. Useful point-in-time features include the number of admissions in the previous 12 months, days since the last admission, count of chronic conditions from diagnosis codes, whether the last laboratory values before discharge were abnormal, number of medications at discharge, and whether a follow-up appointment was booked before the patient left. Test them by comparing cross-validated AUC (and calibration) with and without each feature group using identical folds, with all aggregates computed only from data timestamped before discharge, then confirm on a later time period held out as a test set.

Copyright © ZyeemZia

Scenario · Choosing between effort options

B is best. Correct. On tabular data, well-designed features usually add more than a change of algorithm, and the gain can be measured directly.

Scenario · Suspiciously good results

C is best. Correct. The test set no longer reflected reality: synthetic neighbours of training points were easy to recognise, and a 50% fraud rate inflates precision compared with the real, very low rate.

Copyright © ZyeemZia

03

MODULE 3 · FIT

Supervised Learning: Linear Models and Optimisation

Fit linear and logistic models with gradient descent, control them with regularisation and know the classic alternatives

BY THE END OF THIS MODULE YOU WILL BE ABLE TO

- Fit and interpret linear regression, including R^2 and residual checks
- Train models with gradient descent and diagnose training from loss curves
- Build logistic regression classifiers and interpret probabilities and odds ratios
- Apply ridge, lasso and elastic net and tune their strength with cross-validation
- Recognise when k-NN, naive Bayes or SVMs are sensible choices

LESSONS

1. Linear Regression and Least Squares · 26 min
2. Gradient Descent in Practice · 24 min
3. Logistic Regression and Classification · 26 min
4. Regularisation: Ridge, Lasso and Elastic Net · 24 min
5. Other Classic Learners: k-NN, Naive Bayes and SVMs · 24 min

MODULE 3 · FIT · LESSON 1 · 26 MIN

Linear Regression and Least Squares

The simplest useful model: fit a straight line, read its coefficients and check its assumptions

- Write the linear regression model and its mean squared error loss
- Fit a one-feature line by hand and compute R^2
- Interpret coefficients carefully, including with correlated features
- Check residuals for the main assumptions

Linear regression predicts a number as a weighted sum of features plus an intercept. It is fast, transparent and surprisingly strong when features are well engineered. It is also the foundation for logistic regression, regularisation and even the layers of a neural network.

The model and its loss

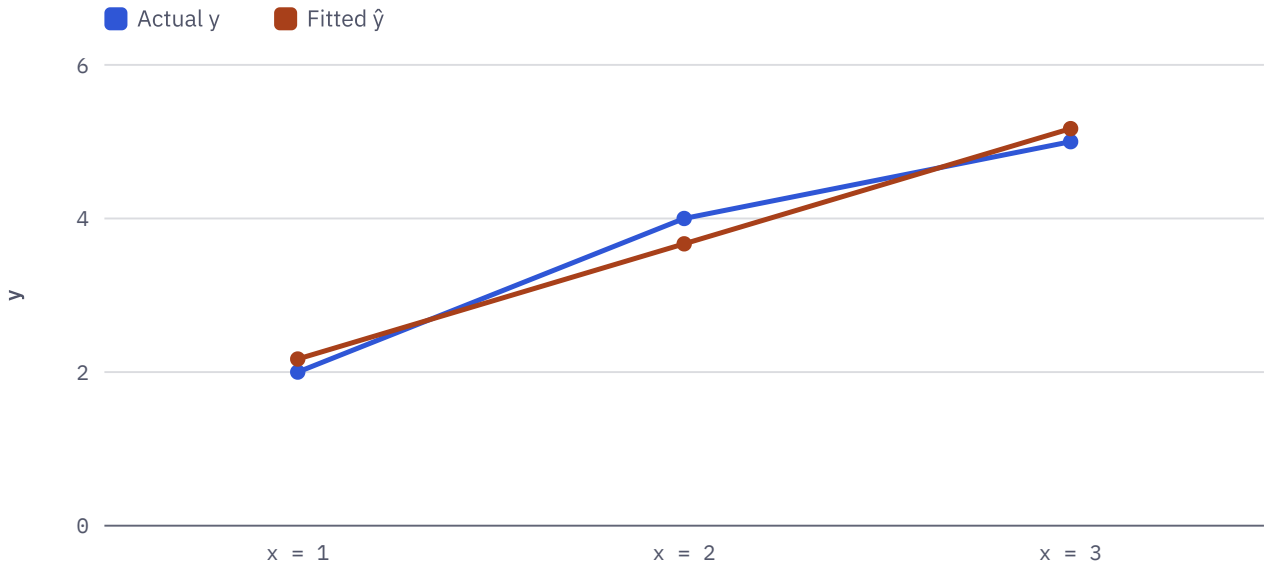
MEASURE	FORMULA	WHAT IT TELLS YOU
Prediction	$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_dx_d + b = w \cdot x + b$	Each weight says how much the prediction changes when its feature rises by one unit, holding the others fixed.
Mean squared error	$MSE = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2$	Squaring punishes large errors heavily and makes the loss smooth and convex.
Normal equation	$w = (X^TX)^{-1}\ X^T\ y$	Closed-form least-squares solution (with a column of ones for the intercept). Exact, but expensive for very many features.
Coefficient of determination	$R^2 = 1 - SS_{res} \div SS_{tot}$	Share of the target's variance explained. 1 is perfect; 0 is no better than predicting the mean.

Fitting a line by hand: points (1, 2), (2, 4), (3, 5)

1. **Means** $\bar{x} = 2$, $\bar{y} = 11 \div 3 \approx 3.667$.
2. **Slope** $w = \Sigma(x - \bar{x})(y - \bar{y}) \div \Sigma(x - \bar{x})^2 = (1.667 + 0 + 1.333) \div 2 = 1.5$.
3. **Intercept** $b = \bar{y} - w\,\bar{x} = 3.667 - 3 = 0.667$, so $\hat{y} = 1.5x + 0.667$.
4. **Residuals** Predictions 2.167, 3.667, 5.167 give residuals −0.167, 0.333, −0.167; $SS_{res} \approx 0.167$.
5. **R^2** $SS_{tot} = 2.778 + 0.111 + 1.778 \approx 4.667$, so $R^2 = 1 - 0.167 \div 4.667 \approx 0.964$.

Copyright © ZyeemZia

The three points and the least-squares line ŷ = 1.5x + 0.667



Residuals are −0.17, +0.33 and −0.17: they sum to zero, as least squares with an intercept guarantees. The line passes through the point of means (2, 3.667).

The same fit in code

Python notebook

```
import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([[1], [2], [3]]); y = np.array([2, 4, 5])
lr = LinearRegression().fit(X, y)
print(round(lr.coef_[0], 3), round(lr.intercept_, 3), round(lr.score(X, y), 3))
# output: 1.5 0.667 0.964
Xb = np.c_[np.ones(3), X]
print(np.linalg.solve(Xb.T @ Xb, Xb.T @ y).round(3).tolist())
# output: [0.667, 1.5]
```

The normal equation gives the same intercept and slope. Solving the linear system is more stable than computing the matrix inverse explicitly.

Assumptions and how to check them

ASSUMPTION	WHAT IT MEANS	HOW TO CHECK	IF VIOLATED
Linearity	The target changes linearly with each feature	Plot residuals against predictions and features	Add transforms, polynomial terms or use trees
Independent errors	One example's error does not predict another's	Check residuals over time or by group	Use time-series methods or grouped models
Constant variance	Errors have similar spread everywhere	Residual spread should not fan out	Log-transform the target or use weighted loss
Few extreme points	No single point dominates the fit	Look for high-leverage points	Investigate, use robust loss such as Huber

Copyright © ZyeemZia

ASSUMPTION	WHAT IT MEANS	HOW TO CHECK	IF VIOLATED
Low multicollinearity	Features are not near-copies of each other	Correlation matrix, variance inflation factor	Drop or combine features; use ridge regularisation

WATCH OUT · COEFFICIENTS ARE NOT CAUSAL EFFECTS

A coefficient of 1,200 BDT on 'has smartphone' in a wallet-spend model means that, among otherwise similar customers in this data, smartphone owners spend 1,200 BDT more on average. It does not mean giving someone a smartphone raises spending by 1,200 BDT. With correlated features, coefficients can also swing wildly or flip sign between samples.

When linear regression shines and when it struggles

Shines

- Relationships are roughly additive and linear
- You need transparent, auditable coefficients
- Data is small or moderate
- As a strong, fast baseline

Struggles

- Strong non-linear effects and interactions
- Many correlated features without regularisation
- Heavy outliers in the target
- Targets that are counts or bounded (consider other models)

SCENARIO · DECIDE · A PUZZLING COEFFICIENT

A model predicting electricity use in Dhaka apartments includes both 'floor area in square feet' and 'number of rooms', which are strongly correlated. The coefficient on rooms is negative, and the facilities manager says 'more rooms cannot reduce electricity use'. What is the best response?

(A) Remove the negative sign by forcing all coefficients to be positive

(B) Conclude that adding rooms reduces electricity use

(C) Discard linear regression and switch to a neural network

(D) Explain that with correlated features the coefficient reflects rooms at fixed floor area, and consider ridge or combining the features

Circle your choice. Answer and feedback: module answer key.

REFLECT

Choose a numeric outcome in your work, such as delivery time or monthly sales. Write a linear model with three features. What sign do you expect for each coefficient, and what could make the fitted sign differ?

Key terms

Linear regression
A model that predicts a numeric target as a weighted sum of features plus an intercept.

Least squares
Fitting parameters by minimising the sum of squared differences between predictions and targets.

Copyright © ZyeemZia

Residual
The difference between an observed value and the model's prediction for it.

Normal equation
The closed-form least-squares solution $w = (X^TX)^{-1}X^Ty$.

Intercept
The model's prediction when all features are zero; the bias term b.

R^2 (coefficient of determination)
The proportion of variance in the target explained by the model relative to predicting the mean.

Multicollinearity
Strong correlation among features that makes individual coefficients unstable and hard to interpret.

Key takeaways

- Linear regression predicts $w \cdot x + b$ and is fitted by minimising squared error.
- Each weight is an association holding other features fixed, not a causal effect.
- R^2 compares the model with predicting the mean; check residual plots for the assumptions.
- Correlated features make individual coefficients unstable even when predictions are fine.

KNOWLEDGE CHECK

1. A model is $\hat{y} = 3x + 2$. What is the residual for the point $x = 4$, $y = 15$?
   - (A) 1
   - (B) −1
   - (C) 14
   - (D) 3
2. SS_res = 20 and SS_tot = 80. What is R^2?
   - (A) 0.25
   - (B) 4
   - (C) 0.75
   - (D) 0.80
3. Residuals fan out as predictions grow larger. Which assumption is violated?
   - (A) Independence of features
   - (B) Constant variance of errors
   - (C) Linearity of the intercept term
   - (D) Normality of the features
4. Why can coefficients become unstable when two features are highly correlated?
   - (A) Correlated features always have missing values
   - (B) Least squares cannot be computed at all when two features are present
   - (C) The intercept becomes undefined
   - (D) The model cannot tell which of the two features deserves the credit
5. Which statement about R^2 is correct?
   - (A) R^2 can never be negative on any dataset
   - (B) R^2 of 0.9 proves the model is causal
   - (C) R^2 measures classification accuracy
   - (D) An R^2 of 0 means the model does no better than predicting the mean

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 3 · FIT · LESSON 2 · 24 MIN

Gradient Descent in Practice

How models are actually trained when there is no closed-form answer

- Derive the gradient of MSE for a linear model
- Run gradient descent steps and explain batch, stochastic and mini-batch variants
- Compare common optimisers and learning-rate schedules
- Diagnose training problems from loss curves

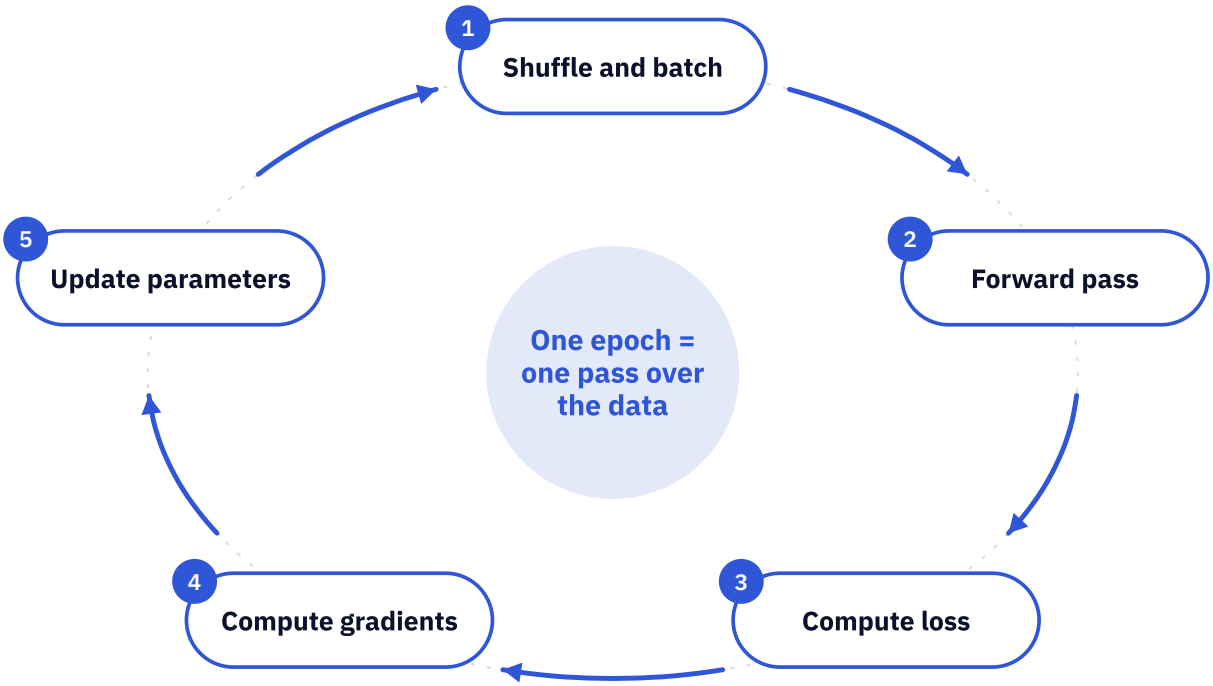
The normal equation works for linear regression, but most models (logistic regression, neural networks, boosted trees' leaf updates) have no tidy closed form. **Gradient descent** is the general method: compute how the loss changes with each parameter, take a small step downhill, repeat.

Gradient descent for linear regression

MEASURE	FORMULA	WHAT IT TELLS YOU
Update rule	$\theta \leftarrow \theta - \eta \nabla L(\theta)$	θ is any parameter, η the learning rate, ∇L the gradient of the loss.
Gradient for a weight	$\partial MSE/\partial w = (2/n) \Sigma (\hat{y}_i - y_i) x_i$	Errors are weighted by their feature values and averaged.
Gradient for the intercept	$\partial MSE/\partial b = (2/n) \Sigma (\hat{y}_i - y_i)$	The average error, doubled.
Worked first step	Data (1, 2), (2, 4), (3, 5); w = b = 0; η = 0.1	Errors −2, −4, −5. ∂w = (2/3)(−25) ≈ −16.67, ∂b = (2/3)(−11) ≈ −7.33. New w ≈ 1.667, b ≈ 0.733.

Copyright © ZyeemZia

The mini-batch training loop



1. **Shuffle and batch** — Draw the next mini-batch of, say, 32–256 rows
2. **Forward pass** — Compute predictions for the batch
3. **Compute loss** — Average error on the batch
4. **Compute gradients** — Analytically or by backpropagation
5. **Update parameters** — Optimiser step with the current learning rate

Each turn of the loop is one step; after every epoch, log training and validation loss to see whether to continue, adjust the learning rate or stop.

Gradient descent from scratch

Python notebook

```
import numpy as np
x = np.array([1., 2., 3.]); y = np.array([2., 4., 5.])
w, b, lr = 0.0, 0.0, 0.1
for step in range(2000):
    err = (w * x + b) - y
    w -= lr * 2 * np.mean(err * x)
    b -= lr * 2 * np.mean(err)
    if step == 0: print(round(w, 3), round(b, 3))
print(round(w, 3), round(b, 3))
# output: 1.667 0.733
# output: 1.5 0.667
```

After one step the values match the hand calculation; after 2,000 steps they reach the least-squares solution found in the previous lesson.

Copyright © ZyeemZia

Batch, stochastic and mini-batch

Batch

- Gradient from all n examples each step
- Smooth, accurate steps
- Slow and memory-heavy on large data

Stochastic (SGD)

- Gradient from one example per step
- Very noisy but cheap steps
- Noise can help escape poor regions

Mini-batch

- Gradient from a batch, e.g. 32–1,024 examples
- Balances noise and speed; fits GPU hardware
- The default in modern deep learning

Common optimisers

OPTIMISER	IDEA	WHEN USED
SGD	Plain gradient steps	Simple models; with care, strong results in vision
SGD with momentum	Keeps a running average of past gradients to smooth and speed up progress	Deep networks, ravine-shaped losses
RMSProp	Scales each parameter's step by recent gradient size	Recurrent networks, noisy problems
Adam / AdamW	Momentum plus per-parameter scaling; AdamW decouples weight decay	Default for most neural networks and transformers

Learning-rate practice

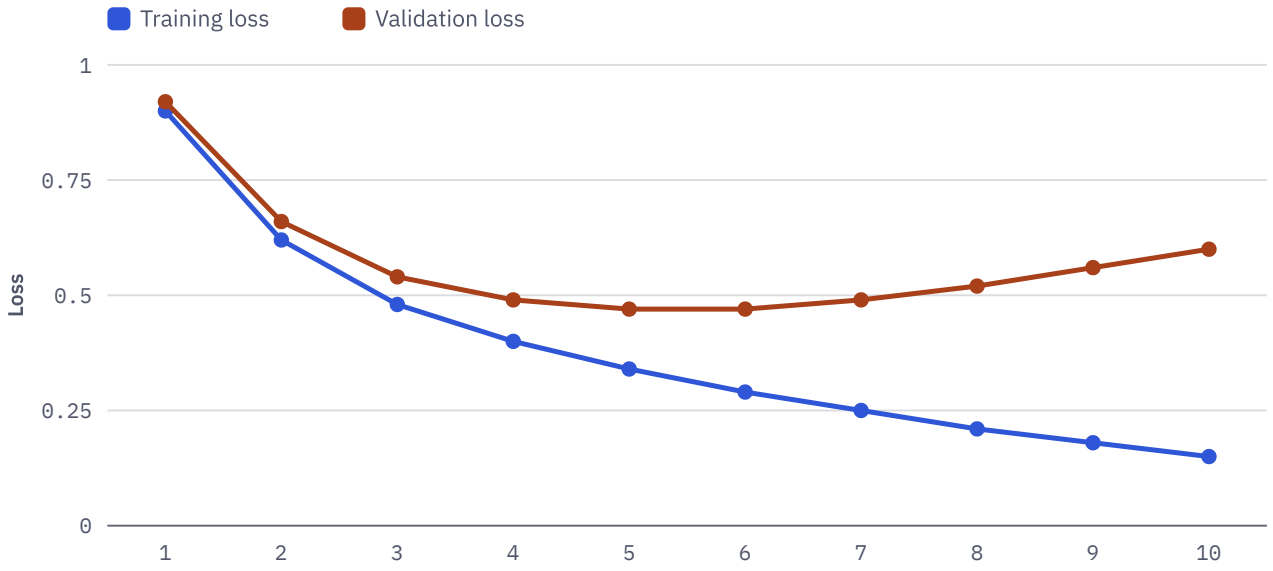
- Search on a log scale, for example 1e-4, 3e-4, 1e-3, 3e-3, 1e-2.
- A short **warm-up** followed by gradual decay (step, cosine) is common for deep networks.
- Standardise features first: unscaled features create stretched loss surfaces and slow, zig-zagging descent.
- Shuffle data every epoch so mini-batches are representative.
- Track training and validation loss every epoch; save the best checkpoint.

Reading loss curves

WHAT YOU SEE	LIKELY CAUSE	WHAT TO TRY
Loss explodes or becomes NaN	Learning rate too high, bad input values	Lower the learning rate, check for NaNs and scale inputs
Loss decreases very slowly	Learning rate too low, unscaled features	Raise the learning rate, standardise features
Loss plateaus early at a high value	Model too simple or bug in data	Check labels and features, add capacity
Training loss falls, validation loss rises	Overfitting	Regularise, early-stop, get more data
Loss jumps around without trend	Batch too small or learning rate too high	Larger batches, lower learning rate

Copyright © ZyeemZia

The overfitting signature in loss curves (illustrative)



Illustrative numbers (x-axis: epoch). Training loss keeps falling, but validation loss bottoms out around epochs 5–6 and then rises: keep that checkpoint and add regularisation rather than training longer.

SCENARIO · DECIDE · SLOW CONVERGENCE

A team trains logistic regression with gradient descent on features including 'annual turnover in taka' (up to 10^9) and 'number of employees' (up to 500). Training needs thousands of epochs and still oscillates. What should they try first?

- (A) Standardise the features before training
- (B) Use batch size 1 for more updates
- (C) Remove the employee count feature
- (D) Switch to a much deeper model

Circle your choice. Answer and feedback: module answer key.

Training debugging checklist

- ☐ Can the model overfit a tiny batch of 10–50 examples? If not, look for a bug
- ☐ Inputs are scaled and contain no NaN or infinite values
- ☐ Labels are aligned with inputs after shuffling and joins
- ☐ Learning rate has been checked on a log-scale sweep
- ☐ Training and validation loss are both plotted per epoch
- ☐ Random seeds are fixed for reproducible comparisons

Key terms

Gradient descent
An iterative method that updates parameters in the direction opposite to the gradient of the loss.

Epoch
One full pass through the training data.

Copyright © ZyeemZia

Mini-batch
A small subset of training examples used to compute one gradient update.

Momentum
An optimiser technique that accumulates past gradients to smooth and accelerate updates.

Learning-rate schedule
A planned change of the learning rate during training, such as warm-up then decay.

Stochastic gradient descent
Gradient descent using one or a few examples per update, giving noisy but cheap steps.

Adam
An adaptive optimiser combining momentum with per-parameter step sizes based on recent gradient magnitudes.

Key takeaways

- Gradient descent repeats: predict, measure loss, compute gradients, step downhill.
- Mini-batches are the default: cheap, slightly noisy steps that suit modern hardware.
- AdamW with a tuned learning rate, warm-up and decay is the usual starting point for deep networks.
- Read curves together: a widening gap between training and validation loss means overfitting.
- Scale inputs and check that the model can overfit a tiny batch before long runs.

KNOWLEDGE CHECK

6. For one example with x = 2, y = 5 and current prediction $\hat{y} = 3$, what is the MSE gradient with respect to w (n = 1)?

   (A) 8 (B) −8
   (C) −4 (D) −2

7. Which variant is the usual default for training neural networks?

   (A) Full-batch gradient descent (B) Exhaustive grid search over weights
   (C) Mini-batch gradient descent (D) The normal equation

8. Training loss falls steadily while validation loss starts rising. What is happening?

   (A) The model is starting to overfit the training data (B) The learning rate is too low for the validation data
   (C) The model is underfitting (D) The data has no labels

9. Why does standardising features speed up gradient descent?

   (A) It reduces the number of training examples (B) It makes the loss surface more evenly shaped, so one learning rate suits all directions
   (C) It removes the need to choose a learning rate, because every feature then moves equally fast (D) It converts regression into classification

10. A quick sanity test before a long training run is to check whether the model can…

    (A) Score 100% on the test set (B) Train without any loss function
    (C) Run one epoch with a learning rate of zero to check speed (D) Overfit a tiny subset of the training data

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 3 · FIT · LESSON 3 · 26 MIN

Logistic Regression and Classification

Turn a linear score into a probability and train it with log loss

- Explain the sigmoid link and interpret probabilities and log-odds
- Compute log loss for individual predictions
- Interpret coefficients as odds ratios
- Extend to multiple classes with softmax

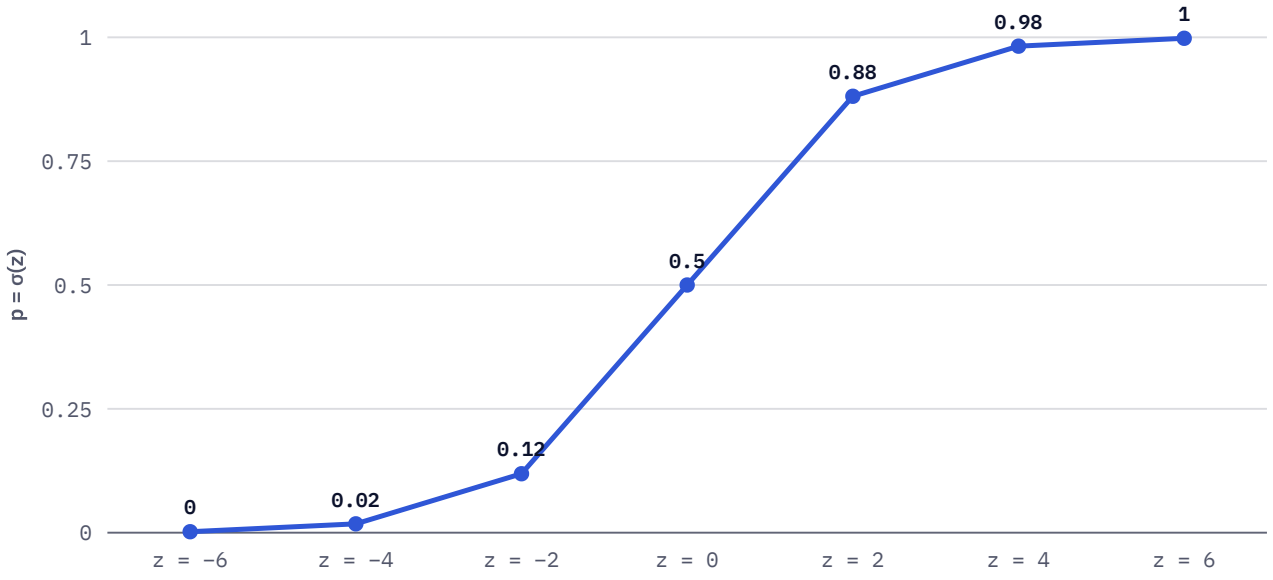
Many business questions are yes/no: will this customer default, is this transaction fraud, does this X-ray show pneumonia? **Logistic regression** takes the same linear score as linear regression and squeezes it into a probability between 0 and 1. It remains one of the most used models in banking, insurance and healthcare because it is fast, well calibrated and explainable.

From score to probability

MEASURE	FORMULA	WHAT IT TELLS YOU
Linear score (log-odds)	`z = w · x + b`	z can be any real number; it is the log of the odds of the positive class.
Sigmoid	`p = σ(z) = 1 ÷ (1 + e⁻ᶻ)`	For z = 0.8: p = 1 ÷ (1 + 0.449) ≈ 0.690.
Odds	`odds = p ÷ (1 − p)`	p = 0.8 gives odds of 4, read as 4 to 1.
Odds ratio of a coefficient	`OR = e^w`	w = 0.4 gives e^0.4 ≈ 1.49: each unit increase in the feature multiplies the odds by about 1.49.

Copyright © ZyeemZia

The sigmoid turns any score into a probability



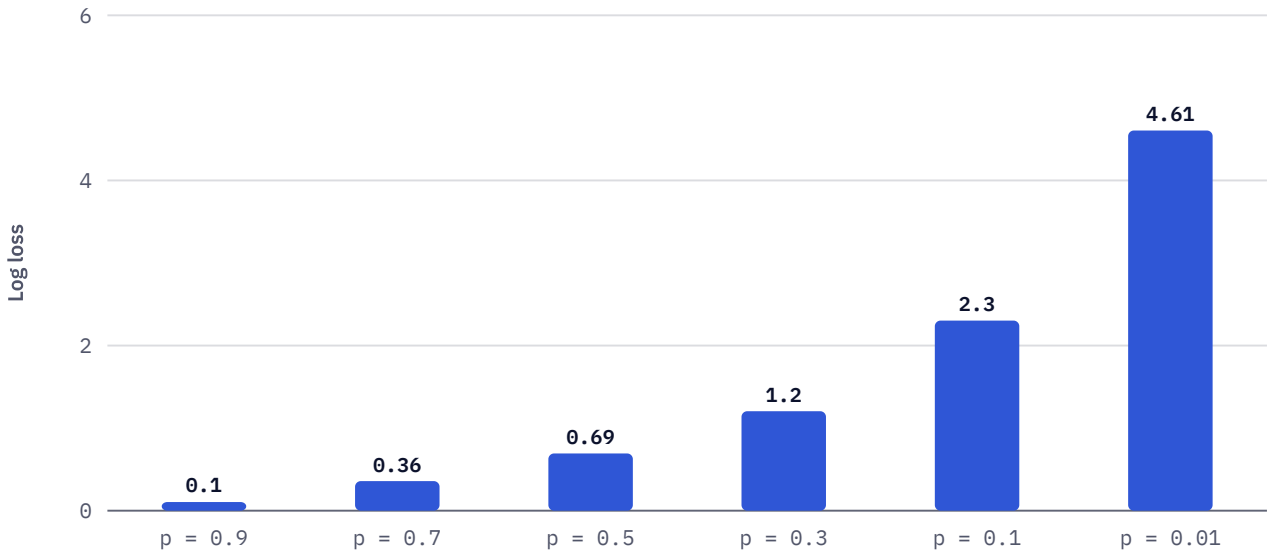
Computed values. The curve is steepest at z = 0, where p = 0.5, and flattens towards 0 and 1, so large positive or negative scores change the probability very little.

Log loss (binary cross-entropy)

MEASURE	FORMULA	WHAT IT TELLS YOU
Definition	`L = −[y ln p + (1 − y) ln(1 − p)]`	Averaged over examples. It heavily punishes confident wrong answers.
Correct and confident	`y = 1, p = 0.9: L = −ln 0.9 ≈ 0.105`	Small loss.
Hesitant	`y = 1, p = 0.69: L = −ln 0.69 ≈ 0.371`	Moderate loss.
Confident and wrong	`y = 0, p = 0.95: L = −ln 0.05 ≈ 3.0`	Huge loss, which is why log loss rewards honest probabilities.
Gradient	`∂L/∂w = (p − y) x`	The same neat 'error times input' form as linear regression.

Copyright © ZyeemZia

Log loss for a true positive (y = 1) at different predicted probabilities



Computed values. Being confidently wrong is punished hardest: predicting 0.01 for a real positive costs 44 times more than predicting 0.9.

Multi-class: softmax

MEASURE	FORMULA	WHAT IT TELLS YOU
Softmax	$p_k = e^{z_k} \div \Sigma_j e^{z_j}$	Turns K scores into K probabilities that sum to 1.
Worked example	$z = (2, 1, 0)$: $e^2 \approx 7.389$, $e^1 \approx 2.718$, $e^0 = 1$; sum ≈ 11.107	p ≈ (0.665, 0.245, 0.090).
Cross-entropy	$L = -\ln p_{true}$	If the true class is the first one, L = −ln 0.665 ≈ 0.408.

Copyright © ZyeemZia

Sigmoid, log loss and a fitted classifier

Python notebook

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import log_loss
print(round(1 / (1 + np.exp(-0.8)), 3), round(-np.log(0.69), 3))
# output: 0.69 0.371
X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)
clf = make_pipeline(StandardScaler(), LogisticRegression(C=1.0)).fit(X_tr, y_tr)
p = clf.predict_proba(X_te)[:, 1]
print(round(log_loss(y_te, p), 3), round(clf.score(X_te, y_te), 3))
# output: 0.068 0.986
```

predict_proba returns probabilities; score reports accuracy at the default 0.5 threshold. Report both: probabilities matter for ranking and risk decisions.

PRO TIP · THE THRESHOLD IS A BUSINESS DECISION

A model outputs probabilities; the cut-off that turns them into actions is chosen from costs and capacity. A bank may decline loans above 0.30 default probability, while a hospital may send any patient above 0.10 for a second review. The default 0.5 is rarely the right choice for imbalanced problems.

CASE STUDY · A TRANSPARENT CREDIT SCORECARD

A non-bank lender in Bangladesh serving small shopkeepers must explain every declined application to the regulator and the customer. The data science team has 40,000 past loans with 8% defaults. A gradient-boosted model scores AUC 0.81; a logistic regression on carefully binned features scores 0.78. The credit committee must choose a model for launch.

1. What are the arguments for launching with logistic regression?
2. How should the decision threshold be set?
3. How could the team still benefit from the stronger model?

Model answer: module answer key.

Copyright © ZyeemZia

SCENARIO · DECIDE · INTERPRETING A COEFFICIENT

In a logistic regression for loan default, the coefficient on 'number of late payments in the last year' is 0.7. A manager asks what this means. Which explanation is correct?

(A) Each late payment adds 0.7 percentage points to the default probability

(B) Each late payment raises the default probability by 70%

(C) The model is 70% sure that late payers default

(D) Each additional late payment multiplies the odds of default by about e^0.7 ≈ 2, other features held fixed

Circle your choice. Answer and feedback: module answer key.

REFLECT

Think of a yes/no decision in your organisation. What would a false positive and a false negative cost? Where would you set the probability threshold, and who should agree to it?

Key terms

Logistic regression
A linear classifier that maps a weighted sum of features through the sigmoid to give a probability.

Odds ratio
The factor by which the odds change for a one-unit increase in a feature: e raised to its coefficient.

Softmax
A function that converts a vector of scores into probabilities that sum to one.

Calibration
How well predicted probabilities match observed frequencies, for example 30% predicted means about 30% happen.

Log-odds (logit)
The natural log of p ÷ (1 – p); logistic regression models it as a linear function of features.

Log loss
Binary cross-entropy; the average negative log probability assigned to the true class.

Decision threshold
The probability cut-off above which a case is assigned to the positive class.

Key takeaways

- Logistic regression maps a linear score through the sigmoid to a probability; softmax does the same for several classes.
- A coefficient multiplies the odds by e^w for each one-unit increase in its feature.
- Log loss rewards honest probabilities and heavily punishes confident mistakes.
- Choose the decision threshold from costs and capacity, not by default at 0.5.

KNOWLEDGE CHECK

11. What is σ(0)?

(A) 0.5
(B) 0
(C) 1
(D) 0.25

Copyright © ZyeemZia

12. A coefficient is ln 3 ≈ 1.099. What happens to the odds when the feature increases by one?

(A) They increase by 1.099
(B) They increase by 3%
(C) They triple
(D) They are unchanged

13. Which prediction gives the larger log loss for a true label of 0?

(A) p = 0.1
(B) p = 0.3
(C) p = 0.9
(D) p = 0.5

14. Softmax scores are (0, 0, 0) for three classes. What are the probabilities?

(A) 1/3 each
(B) 0 each
(C) 1, 0, 0
(D) 0.5, 0.25, 0.25

15. Why is 0.5 often a poor threshold for a fraud model?

(A) Probabilities above 0.5 are impossible for rare classes
(B) Logistic regression cannot output values below 0.5
(C) Thresholds must always equal the base rate
(D) Costs and class balance usually make a different cut-off more valuable

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 3 · FIT · LESSON 4 · 24 MIN

Regularisation: Ridge, Lasso and Elastic Net

Control complexity so models generalise instead of memorising

- Explain overfitting and how penalties reduce it
- Compare L1 and L2 penalties and their effects on coefficients
- Tune the regularisation strength with cross-validation
- Relate λ, alpha and C across common libraries

A model with many features can fit the training data almost perfectly, including its noise. **Regularisation** adds a penalty for large weights to the loss, trading a little training accuracy for much better behaviour on new data. It is the single most useful tool for linear models with many features.

Penalised losses

MEASURE	FORMULA	WHAT IT TELLS YOU
Ridge (L2)	$\text{Loss} = \text{MSE} + \lambda \Sigma w_j^2$	Shrinks all weights smoothly towards zero; handles correlated features well.
Lasso (L1)	$\text{Loss} = \text{MSE} + \lambda \Sigma \lvert w_j \rvert$	Can set weights exactly to zero, performing feature selection.
Elastic net	$\text{Loss} = \text{MSE} + \lambda [\alpha \Sigma \lvert w_j \rvert + (1 - \alpha) \Sigma w_j^2]$	Mixes both; selects groups of correlated features more stably than lasso alone (α here is the mixing ratio).
Ridge with one feature, no intercept	$w = \Sigma x_i y_i \div (\Sigma x_i^2 + \lambda)$	With Σxy = 20 and Σx² = 10: least squares gives w = 2; λ = 10 gives w = 20 ÷ 20 = 1.
Library naming	λ is 'alpha' in many regression APIs; logistic regression often uses $C = 1 \div \lambda$	Larger alpha means stronger penalty; larger C means weaker penalty.

L1 versus L2

L1 (lasso)

- Many coefficients become exactly zero
- Built-in feature selection
- Unstable choice among correlated features
- Good when few features truly matter

L2 (ridge)

- All coefficients shrink, few reach zero
- Spreads weight across correlated features
- Stable and smooth
- Good when many features each matter a little

Copyright © ZyeemZia

Ridge and lasso compared

Python notebook

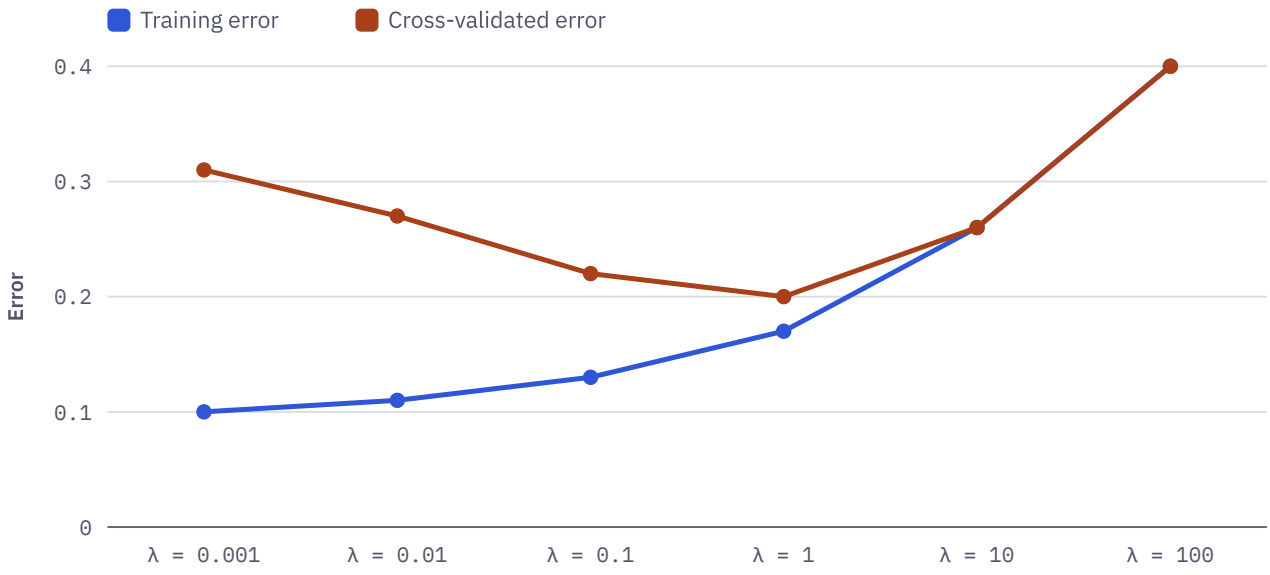
```
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import cross_val_score
X, y = make_regression(n_samples=60, n_features=30, n_informative=5, noise=10, random_state=3)
for m in [LinearRegression(), Ridge(alpha=10.0), Lasso(alpha=1.0)]:
    r2 = cross_val_score(m, X, y, cv=5, scoring='r2').mean()
    nz = (abs(m.fit(X, y).coef_) > 1e-6).sum()
    print(type(m).__name__, round(r2, 3), nz)
# output: LinearRegression 0.97 30
# output: Ridge 0.758 30
# output: Lasso 0.988 10
```

Only 5 of 30 features carry signal and there are just 60 rows. Lasso zeroes out 20 coefficients and gives the best cross-validated R^2; ridge keeps all 30 and, at this strength, over-shrinks the real effects. The penalty strength always needs tuning.

Tuning regularisation strength

1. **Scale features** Penalties compare weights directly, so features must be on similar scales.
2. **Search a log grid** Try λ (alpha) values such as 0.001, 0.01, 0.1, 1, 10, 100.
3. **Cross-validate** For each value, average the validation score over folds.
4. **Pick and simplify** Choose the best value, or the simplest model within one standard error of the best.
5. **Refit and test once** Refit on all training data and evaluate once on the untouched test set.

Training and cross-validated error as λ grows (illustrative)



Illustrative numbers on a log-scaled grid. A weak penalty overfits (large gap), a strong one underfits (both errors high); pick λ near the cross-validated minimum, or the simplest model within one standard error of it.

Copyright © ZyeemZia

INSIGHT · REGULARISATION AS A PRIOR BELIEF

A penalty encodes a belief that small weights are more likely than large ones. Ridge corresponds to assuming weights follow a normal distribution centred on zero; lasso to a sharper, Laplace-shaped belief that many weights are exactly or nearly zero. More data lets the evidence override the prior, which is why regularisation matters most for small datasets.

SCENARIO · DECIDE · MANY CORRELATED FEATURES

A demand model for a supermarket chain has 300 features, many of them overlapping (sales last week, last 2 weeks, last 4 weeks, by store and by region). Lasso keeps a different small set of features each time the data is resampled, which worries the business. What is the best adjustment?

- (A) Use elastic net or ridge so correlated features share weight more stably
- (B) Remove regularisation entirely
- (C) Select features using the test set to make them stable
- (D) Increase the lasso penalty until only one feature remains

Circle your choice. Answer and feedback: module answer key.

TEMPLATE **Regularisation tuning log**

Record each regularisation experiment so the chosen strength is justified and reproducible.

RUN	MODEL	PENALTY AND STRENGTH	CV METRIC (MEAN ± SD)	NON-ZERO FEATURES	NOTES
R-07	*Logistic regression*	*L1, C = 0.1*	*AUC 0.812 ± 0.011*	*23 of 140*	*Chosen: within 1 SE of best, far simpler*
R-08	*Logistic regression*	*L2, C = 1.0*	*AUC 0.818 ± 0.010*	*140 of 140*	*Best score, harder to explain*

REFLECT

Look at a model you have built or used. Was its regularisation strength tuned or left at a default? Which features would you expect lasso to drop, and would the business be comfortable with that?

Copyright © ZyeemZia

Key terms

Overfitting
When a model learns noise in the training data and performs worse on new data.

Ridge regression
Linear regression with an L2 penalty on the sum of squared weights.

Elastic net
A penalty combining L1 and L2 terms.

Weight decay
Shrinking weights a little at every update; equivalent to L2 regularisation for plain gradient descent.

Regularisation
Adding a penalty or constraint during training to reduce model complexity and overfitting.

Lasso
Linear regression with an L1 penalty on the sum of absolute weights; can zero out features.

Hyperparameter
A setting chosen before training, such as the regularisation strength, rather than learned from data.

Key takeaways

- Regularisation adds a penalty on weight size to trade a little training fit for better generalisation.
- L1 (lasso) zeroes weights and selects features; L2 (ridge) shrinks all weights and handles correlation well.
- Elastic net combines both and is more stable with groups of correlated features.
- Scale features first and tune the strength by cross-validation on a log grid.

KNOWLEDGE CHECK

16. Which penalty can set coefficients exactly to zero?

(A) L2 (ridge)
(B) L1 (lasso)
(C) No penalty
(D) Dropout

17. With $\Sigma xy = 30$ and $\Sigma x^2 = 15$, what is the one-feature ridge weight (no intercept) when $\lambda = 15$?

(A) 1
(B) 2
(C) 0.5
(D) 1.5

18. In a logistic regression API where $C = 1 \div \lambda$, what does increasing C do?

(A) Strengthens regularisation
(B) Changes the loss to MSE
(C) Weakens regularisation
(D) Removes the intercept

19. Why must features be scaled before applying ridge or lasso?

(A) Penalties only work on integers
(B) The penalty treats all weights alike, so feature units would decide which are shrunk
(C) Scaling removes outliers automatically
(D) Unscaled features make the penalised loss non-convex, so the optimiser cannot find the minimum

20. How should the regularisation strength be chosen?

(A) By maximising training accuracy
(B) By testing every candidate value on the held-out test set
(C) By always using the library default
(D) By cross-validation on the training data

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 3 · FIT · LESSON 5 · 24 MIN

Other Classic Learners: k-NN, Naive Bayes and SVMs

Three more supervised methods every practitioner should recognise

- Explain how k-nearest neighbours predicts and how k affects it
- Compute a naive Bayes posterior for a simple text example
- Describe the margin, hinge loss and kernels in support vector machines
- Choose a sensible model family for a given problem

Linear and logistic regression are not the only classic tools. **k-nearest neighbours** predicts from similar past examples, **naive Bayes** applies Bayes' rule with a strong simplifying assumption, and **support vector machines** find the widest possible gap between classes. Each is still useful as a baseline or in niche settings.

Three classic learners at a glance

kNN

k-nearest neighbours

Find the k closest training examples and vote (classification) or average (regression). No training phase; prediction can be slow.

NB

Naive Bayes

Assume features are independent given the class, multiply simple probabilities, pick the most probable class. Fast and strong for text.

SVM

Support vector machine

Find the boundary with the widest margin between classes; kernels allow curved boundaries.

Worked calculations

MEASURE	FORMULA	WHAT IT TELLS YOU
k-NN vote	`k = 5 neighbours: 3 'default', 2 'repay'`	Predict 'default' with an estimated probability of 3 ÷ 5 = 0.6.
Naive Bayes score	$P(\text{class} \mid x) \propto P(\text{class}) \times \prod P(x_j \mid \text{class})$	Multiply the prior by each feature's likelihood, then normalise.
Spam example	`Spam: 0.4 × 0.5 × 0.1 = 0.020; Ham: 0.6 × 0.05 × 0.3 = 0.009`	For an email with 'offer' and 'meeting': P(spam) = 0.020 ÷ 0.029 ≈ 0.69.
SVM margin	$\text{margin width} = 2 \div \lVert w \rVert$	With w = (3, 4), ‖w‖ = 5, so the margin is 0.4. Smaller weights mean a wider margin.
Hinge loss	$L = \max(0, 1 - y\, f(x))$ with $y \in \{-1, +1\}$	y = +1, f(x) = 0.3 gives 0.7; f(x) = 1.5 gives 0, because it is beyond the margin.

Choosing k in k-NN

Small k (e.g. 1)

- Very flexible, follows noise
- Low bias, high variance
- Sensitive to mislabelled points

Large k (e.g. 50)

- Smooth, stable predictions
- Higher bias, lower variance
- Can blur genuine local patterns

Copyright © ZyeemZia

Comparing three learners with cross-validation

Python notebook

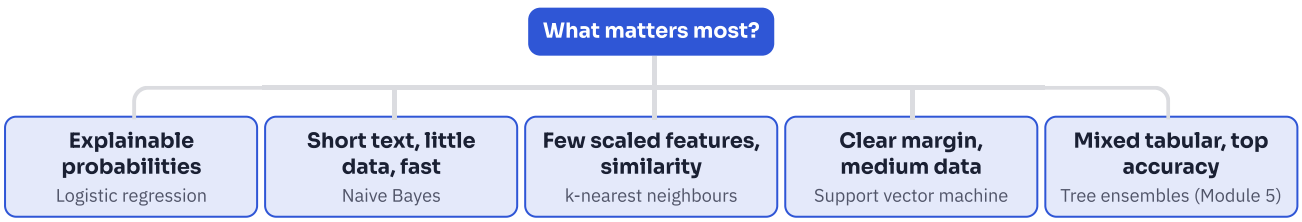
```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
X, y = load_breast_cancer(return_X_y=True)
models = {'kNN': make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=7)),
          'NB': GaussianNB(), 'SVM': make_pipeline(StandardScaler(), SVC(C=1.0))}
for name, m in models.items():
    print(name, round(cross_val_score(m, X, y, cv=5).mean(), 3))
# output: kNN 0.97
# output: NB 0.939
# output: SVM 0.974
```

k-NN and SVM need scaled features; naive Bayes does not. On this small, clean dataset all three are strong, so simplicity, speed and explainability may decide.

Which classic model when?

MODEL	STRENGTHS	WEAKNESSES	TYPICAL USE
Linear / logistic regression	Fast, explainable, calibrated probabilities	Needs engineered non-linearities	Credit scoring, baselines, regulated settings
k-NN	No training, adapts to local patterns	Slow at prediction, needs scaling, struggles in high dimensions	Small datasets, similarity search, baselines
Naive Bayes	Very fast, works with little data, good for text	Independence assumption; probabilities often over-confident	Spam and complaint routing, quick text baselines
SVM	Strong on medium-sized, high-dimensional data	Slow on very large data, probabilities need extra calibration	Text classification, bioinformatics, small image tasks
Tree ensembles (next module)	Top accuracy on tabular data, little preprocessing	Less transparent, larger models	Most tabular business problems

A quick guide to classic learners



A starting point, not a rule: whichever family you pick, compare it with a baseline on the same folds before committing.

Copyright © ZyeemZia

INSIGHT · THE CURSE OF DIMENSIONALITY

In high dimensions, almost all points are far from each other and distances become similar, so 'nearest' neighbours are not very near. Distance-based methods such as k-NN weaken as the number of features grows unless the data truly lies on a lower-dimensional structure. Feature selection or dimensionality reduction helps.

SCENARIO · DECIDE · ROUTING BANGLA AND ENGLISH COMPLAINTS

A telecom operator wants a quick first model to route customer complaints (in Bangla and English) into eight categories. There are 3,000 labelled messages and results are needed within a week. What is the most sensible first model?

- (A) k-NN on raw character counts without scaling
- (B) Wait until 100,000 messages are labelled
- (C) TF-IDF features with a naive Bayes or linear classifier as a fast baseline
- (D) Train a transformer from scratch on the 3,000 messages

Circle your choice. Answer and feedback: module answer key.

TEMPLATE **Model comparison sheet**

Compare candidate model families on the same folds and record the non-accuracy factors that often decide the choice.

MODEL	CV METRIC (MEAN ± SD)	TRAIN TIME	PREDICT TIME PER 1,000 ROWS	EXPLAINABILITY	DECISION
Logistic regression (L2)	*AUC 0.861 ± 0.012*	*4 s*	*2 ms*	*High: coefficients and reason codes*	*Baseline and fallback*
SVM (RBF kernel)	*AUC 0.868 ± 0.015*	*9 min*	*180 ms*	*Low*	*Rejected: small gain, slow, opaque*

REFLECT

For a classification problem you know, which of k-NN, naive Bayes, SVM and logistic regression would you try first, and what would make you switch?

Copyright © ZyeemZia

Key terms

k-nearest neighbours
A method that predicts from the labels or values of the k most similar training examples.

Support vector machine
A classifier that finds the boundary maximising the margin between classes.

Hinge loss
The SVM loss max(0, 1 − y f(x)), zero for points correctly classified beyond the margin.

Curse of dimensionality
The set of problems, such as distances losing meaning, that arise as the number of features grows.

Naive Bayes
A probabilistic classifier that assumes features are conditionally independent given the class.

Margin
The distance between the decision boundary and the closest training points of each class.

Kernel trick
Computing similarities as if data were mapped into a higher-dimensional space, without building that space explicitly.

Key takeaways

- k-NN predicts from similar past cases; it needs scaled features and struggles in high dimensions.
- Naive Bayes is a fast, strong baseline for text despite its independence assumption.
- SVMs maximise the margin; kernels let them draw non-linear boundaries on medium-sized data.
- Compare candidate families on identical folds and record cost and explainability, not only accuracy.

KNOWLEDGE CHECK

21. With k = 7, the neighbours are 4 'fraud' and 3 'legitimate'. What does k-NN predict?

(A) Legitimate, because it is the majority class overall
(B) Fraud, with probability 1.0
(C) No prediction, because k must be even
(D) Fraud, with estimated probability about 0.57

22. An SVM has weight vector w = (6, 8). What is the margin width 2 ÷ ||w||?

(A) 0.4
(B) 0.2
(C) 2
(D) 0.1

23. What is the hinge loss for y = −1 and f(x) = 0.5?

(A) 0.5
(B) 0
(C) 1.5
(D) −0.5

24. What is the 'naive' assumption in naive Bayes?

(A) Features are independent of each other given the class
(B) All classes have equal prior probability
(C) Every feature is normally distributed
(D) The data contains no missing values

25. Why does k-NN often weaken with hundreds of raw features?

(A) k-NN cannot store that many columns
(B) Distances become less informative in high dimensions
(C) It needs labels for every feature
(D) It requires a GPU once there are more than 100 features to compare

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 3 · FIT

Answer key

Knowledge checks

1. A $\hat{y} = 14$, residual = $y - \hat{y} = 15 - 14 = 1$.

2. C $R^2 = 1 - 20 \div 80 = 0.75$.

3. B A fan shape shows the error spread grows with the level, violating constant variance; a log target often helps.

4. D With collinearity, many weight combinations fit nearly equally well, so small data changes swing the individual coefficients.

5. D R^2 compares with the mean predictor; on test data it can even be negative if the model is worse than the mean.

6. B $2 \times (\hat{y} - y) \times x = 2 \times (-2) \times 2 = -8$.

7. C Mini-batches balance gradient noise and hardware efficiency.

8. A Diverging training and validation curves signal overfitting.

9. B Unequal scales create elongated contours that cause zig-zagging.

10. D If a model cannot memorise a handful of examples, there is probably a bug in the data or code.

11. A $1 \div (1 + e^0) = 1 \div 2 = 0.5$.

12. C Odds ratio = e^1.099 = 3.

13. C Loss = $-\ln(1 - p)$; for $p = 0.9$ it is $-\ln 0.1 \approx 2.30$, the largest.

14. A Equal scores give equal probabilities.

15. D The threshold should reflect error costs and capacity, not a default value.

16. B The L1 penalty's sharp corner at zero lets optimisation land exactly on zero.

17. A $w = 30 \div (15 + 15) = 1$; without the penalty it would be 2.

18. C C is the inverse of the penalty strength.

19. B A feature in large units needs a small weight and is barely penalised; scaling puts features on equal footing.

20. D Cross-validation estimates generalisation for each value without touching the test set.

21. D 4 of 7 votes give 'fraud' and a probability estimate of $4 \div 7 \approx 0.57$.

22. B $\|w\| = 10$, so $2 \div 10 = 0.2$.

23. C $\max(0, 1 - (-1)(0.5)) = \max(0, 1.5) = 1.5$.

24. A The model multiplies per-feature likelihoods, which is valid only under conditional independence.

25. B The curse of dimensionality makes points roughly equidistant, so neighbours stop being meaningful.

Scenarios & case studies

Scenario · A puzzling coefficient
D is best. Correct. Holding floor area fixed, more rooms means smaller rooms, which may genuinely use less. Collinearity also makes the estimate unstable, so regularisation or feature redesign helps.

Scenario · Slow convergence
A is best. Correct. Wildly different scales stretch the loss surface so one learning rate is too big for some directions and too small for others. Standardising usually fixes this immediately.

Case study · A transparent credit scorecard
Logistic regression gives coefficients that convert directly into points per feature range, simple reason codes for declines, stable behaviour and easy validation by the regulator; a 0.03 AUC gap may be worth less than explainability and lower model risk. The threshold should come from expected profit and loss: compare the margin earned on good loans with the loss on defaults and choose the cut-off that maximises value within the lender's risk appetite, checking approval rates across groups. The boosted model can be used as a challenger in shadow mode, to suggest new features or interactions to add to the scorecard, or to review borderline cases, and can be adopted later if it proves stable and explainable.

Scenario · Interpreting a coefficient
D is best. Correct. Coefficients act on the log-odds; exponentiating gives an odds ratio of about 2.01.

Scenario · Many correlated features
A is best. Correct. Lasso tends to pick one feature from a correlated group somewhat arbitrarily; adding an L2 component stabilises the selection.

Copyright © ZyeemZia

Scenario · Routing Bangla and English complaints

C is best. Correct. With little labelled text and a tight deadline, a sparse text representation with naive Bayes or logistic regression is quick, cheap and often surprisingly strong.

Copyright © ZyeemZia

04

MODULE 4 · EVALUATE

Evaluation: Measuring Models Honestly

Split, cross-validate and measure with the right metrics, then guard against overfitting and leakage

BY THE END OF THIS MODULE YOU WILL BE ABLE TO

- Design train, validation and test splits and cross-validation that mirror deployment
- Choose and compute regression metrics that match the cost of errors
- Evaluate classifiers with confusion matrices, ROC and PR curves, thresholds and calibration
- Diagnose bias and variance and tune hyperparameters efficiently
- Detect and prevent data leakage and run reproducible, fair experiments

LESSONS

1. Splits and Cross-Validation · 24 min
2. Regression Metrics · 20 min
3. Classification Metrics and Thresholds · 26 min
4. Bias, Variance and Tuning · 24 min
5. Leakage and Experiment Hygiene · 26 min

MODULE 4 · EVALUATE · LESSON 1 · 24 MIN

Splits and Cross-Validation

Estimate real-world performance honestly before a model meets real users

- Explain the separate roles of training, validation and test sets
- Apply k-fold, stratified, grouped and time-based cross-validation
- Report a cross-validated score with its spread
- Choose a split that mirrors how the model will be used

A model's score on the data it was trained on says almost nothing about how it will perform tomorrow. Evaluation is the craft of **simulating the future** with data the model has not seen, in a way that matches how predictions will actually be made.

Three datasets, three jobs

SET	USED FOR	TYPICAL SHARE	GOLDEN RULE
Training	Fitting model parameters	60–80%	The model may see it as often as it likes
Validation	Choosing features, models and hyperparameters	10–20% (or cross-validation folds)	Used repeatedly, so it slowly becomes optimistic
Test	One final, unbiased estimate	10–20%	Touched once, at the end; never used to make choices

Cross-validation schemes

k-fold and stratified k-fold

- Split data into k folds; each fold is validation once
- Stratified keeps class proportions in every fold
- Default for independent rows

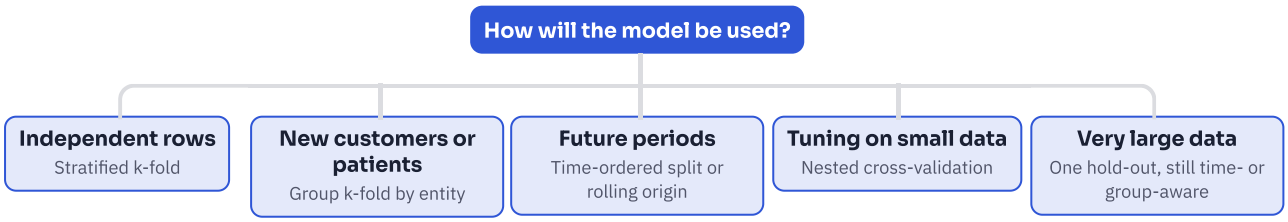
Group k-fold

- All rows of a group (patient, customer, factory) stay in one fold
- Prevents the model from recognising the same entity
- Essential when entities repeat

Time-series split

- Train on the past, validate on the next period
- Expanding or rolling windows
- Mandatory for forecasting and drifting data

Choosing a validation scheme



The split must mimic how predictions will be made. When several apply, combine them, for example group-aware folds inside a time-ordered split.

Copyright © ZyeemZia

Summarising cross-validation

MEASURE	FORMULA	WHAT IT TELLS YOU
CV estimate	`score_CV = (1/k) Σ score_i`	Fold scores 0.80, 0.82, 0.78, 0.84, 0.81 give a mean of 0.81.
Spread	`s = √[Σ(score_i − mean)² ÷ (k − 1)]`	Deviations −0.01, 0.01, −0.03, 0.03, 0 give s = √(0.002 ÷ 4) ≈ 0.022.
Standard error	`SE ≈ s ÷ √k`	0.022 ÷ √5 ≈ 0.010. Report 0.81 ± 0.01, and treat smaller differences between models with caution.

Seeing how splitters behave

Python notebook

```
import numpy as np
from sklearn.model_selection import GroupKFold, TimeSeriesSplit
X = np.arange(8).reshape(-1, 1)
groups = ['p1', 'p1', 'p2', 'p2', 'p3', 'p3', 'p4', 'p4']
for tr, va in GroupKFold(n_splits=2).split(X, groups=groups):
    print(sorted({groups[i] for i in va}))
# output: ['p2', 'p4']
# output: ['p1', 'p3']
for tr, va in TimeSeriesSplit(n_splits=3).split(X):
    print(tr.tolist(), va.tolist())
# output: [0, 1] [2, 3]
# output: [0, 1, 2, 3] [4, 5]
# output: [0, 1, 2, 3, 4, 5] [6, 7]
```

GroupKFold never splits a patient across folds. TimeSeriesSplit always validates on rows that come after the training rows.

WATCH OUT · MATCH THE SPLIT TO DEPLOYMENT

If the model will score **new customers**, validate on customers it has never seen (group split). If it will predict **next month**, validate on a later period (time split). A random split in these cases measures a skill the model will never be asked to use, and the score will be too high.

A time-based evaluation plan



JAN 2024 – JUN 2025
Training
Fit candidate models and pipelines.

JUL – DEC 2025
Validation
Choose features, hyperparameters and the threshold.

JAN – MAR 2026
Test
Score the final choice once.

APR 2026 ONWARD
Deployment
Monitor live performance against the test result.

Each period is later than the one before, so evaluation imitates predicting the future. Touch the test window only once, after every choice has been made.

Copyright © ZyeemZia

Practical rules

- Use stratified folds for classification, especially with rare classes.
- Use 5 or 10 folds for small and medium data; a single hold-out is fine for very large data.
- Put every preprocessing step inside the pipeline that is cross-validated.
- For tuning plus an unbiased estimate on small data, use **nested cross-validation**: an inner loop tunes, an outer loop scores.
- Lock the test set away early and record when it was used.

SCENARIO · DECIDE · PATIENTS WITH MANY VISITS

A Chattogram hospital builds a model to predict complications from visit records. Most patients have 5–20 visits. A random 5-fold split gives AUC 0.93. The model will be used for new patients. What should the team do?

(A) Evaluate on the training set to double-check

(B) Re-evaluate with group k-fold so each patient's visits are in only one fold

(C) Keep the random split, since 5 folds already average out noise

(D) Use 20 folds instead of 5 for a more precise estimate

Circle your choice. Answer and feedback: module answer key.

TEMPLATE ## Evaluation plan

Agree how the model will be judged before any experiments, so results cannot be cherry-picked.

ITEM	DECISION	REASON
Split strategy	*Time-based: train on Jan 2024–Jun 2025, validate Jul–Dec 2025, test Jan–Mar 2026*	*Model will predict future months*
Primary metric	*Recall at the top 5% of scores*	*Matches review team capacity*

Copyright © ZyeemZia

REFLECT

How will your model be used: for new entities, for future periods, or both? Write down the split you would use and what a random split would get wrong.

Key terms

Training set
The data used to fit a model's parameters.

Validation set
Data used to compare models and tune hyperparameters during development.

Test set
Held-out data used once for a final, unbiased estimate of performance.

k-fold cross-validation
Splitting data into k parts and training k times, each time validating on a different part.

Stratified sampling
Splitting so that each subset keeps the overall class proportions.

Group k-fold
Cross-validation that keeps all rows of the same group together in one fold.

Nested cross-validation
An outer cross-validation loop for scoring wrapped around an inner loop for tuning.

Key takeaways

- Training fits, validation chooses, and the test set reports once.
- Match the split to deployment: group splits for new entities, time splits for future periods.
- Report the cross-validation mean and spread, not only the best fold.
- Keep every preprocessing step inside the cross-validated pipeline; use nested CV to report a tuned procedure on small data.

KNOWLEDGE CHECK

1. Fold scores are 0.70, 0.74, 0.72, 0.76, 0.68. What is the CV mean?
   - (A) 0.72
   - (B) 0.74
   - (C) 0.70
   - (D) 0.76
2. Which split suits a model that forecasts next quarter's sales?
   - (A) Random 80/20 split of all rows
   - (B) Stratified k-fold by store
   - (C) Train on earlier periods and validate on later ones
   - (D) Leave-one-out cross-validation on rows shuffled at random
3. Why should the test set be used only once?
   - (A) It is usually too small to give a stable estimate more than once
   - (B) Each use to make a choice makes its estimate optimistic
   - (C) Libraries delete it after one use
   - (D) It contains no labels

Copyright © ZyeemZia

4. A model scores customers it has never seen before. Rows include several transactions per customer. Which scheme fits?

   (A) Plain k-fold on transactions
   (B) Time-series split by row number only
   (C) No validation, just a test set
   (D) Group k-fold by customer

5. What does nested cross-validation provide?

   (A) Faster training than a single split
   (B) A larger training set
   (C) Automatic feature engineering
   (D) An unbiased score for a whole procedure that includes tuning

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 4 · EVALUATE · LESSON 2 · 20 MIN

Regression Metrics

MAE, RMSE, MAPE and R^2: what each one rewards and when it misleads

- Compute MAE, MSE, RMSE, MAPE and R^2 by hand
- Choose a regression metric that matches the cost of errors
- Recognise the pitfalls of percentage errors
- Use quantile loss when over- and under-prediction cost differently

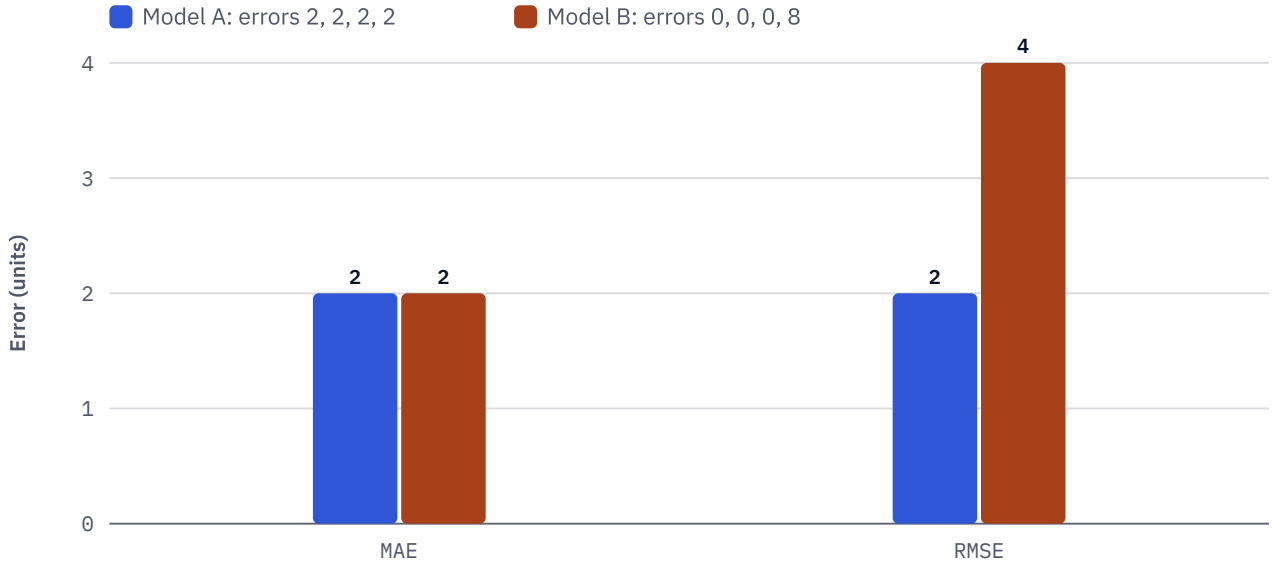
Every regression metric summarises the errors differently, and the choice changes which model looks best. Choose the metric that matches **how errors cost money, time or safety** in the real decision, then use the same metric for tuning and reporting.

Regression metrics with a worked example

MEASURE	FORMULA	WHAT IT TELLS YOU
Data	`actual = (10, 20, 30, 40); predicted = (12, 19, 33, 36)`	Errors are +2, −1, +3, −4.
Mean absolute error	$MAE = (1/n)\ \Sigma\ \lvert y_i - \hat{y}_i \rvert$	(2 + 1 + 3 + 4) ÷ 4 = 2.5 units.
Mean squared error	$MSE = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2$	(4 + 1 + 9 + 16) ÷ 4 = 7.5 squared units.
Root mean squared error	$RMSE = \sqrt{MSE}$	$\sqrt{7.5} \approx 2.74$ units; always ≥ MAE, more so when errors vary in size.
Mean absolute percentage error	$MAPE = (100/n)\ \Sigma\ \lvert y_i - \hat{y}_i \rvert \div \lvert y_i \rvert$	(20% + 5% + 10% + 10%) ÷ 4 = 11.25%. Undefined when any actual is 0.
Quantile (pinball) loss	$L_\tau = \tau(y - \hat{y})$ if $y \geq \hat{y}$, else $(1 - \tau)(\hat{y} - y)$	With $\tau = 0.9$, under-predicting by 10 costs 9 while over-predicting by 10 costs 1.

Copyright © ZyeemZia

Same MAE, different RMSE



Computed values. Both models miss by 8 units in total, but RMSE doubles for model B because squaring punishes its single large miss. Choose the metric whose penalty matches the real cost of errors.

Checking the arithmetic

Python notebook

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, mean_absolute_percentage_error
y, p = [10, 20, 30, 40], [12, 19, 33, 36]
mse = mean_squared_error(y, p)
print(mean_absolute_error(y, p), mse, round(mse ** 0.5, 3))
# output: 2.5 7.5 2.739
print(round(mean_absolute_percentage_error(y, p), 4))
# output: 0.1125
```

The library returns MAPE as a fraction (0.1125), not a percentage. Always check the units a function reports.

Which metric when?

METRIC	REWARDS	GOOD FOR	WATCH OUT
MAE	Being close in typical cases	Delivery times, staffing	Optimised by the median, not the mean
RMSE	Avoiding large errors	Energy load, where big misses are costly	Dominated by a few outliers
MAPE	Small relative errors	Comparing series of different sizes	Explodes near zero; punishes over-forecasts more than under-forecasts
R^2	Explaining variance vs the mean	Quick comparison on one dataset	Not comparable across datasets; says nothing about error size
Quantile loss	Correct upper or lower bound	Safety stock, capacity planning	Need one model or output per quantile

Copyright © ZyeemZia

EXAMPLE · ASYMMETRIC COSTS IN A POWER PLANT

For a power plant forecasting next-day load in Chattogram, under-forecasting forces expensive emergency purchases while over-forecasting wastes some fuel. Optimising the 0.8 or 0.9 quantile, rather than the mean, builds that asymmetry directly into the model.

Mean versus median prediction

Minimising squared error

- Best constant prediction is the mean
- Pulled by extreme values
- Natural for symmetric noise

Minimising absolute error

- Best constant prediction is the median
- Robust to extreme values
- Natural for skewed targets such as amounts

SCENARIO · DECIDE · FORECASTING PHARMACY DEMAND

A pharmacy chain forecasts daily sales for 3,000 products. Many slow-moving medicines sell 0 units on most days. The team proposes MAPE as the single headline metric. What is the main problem?

(A) MAPE is too lenient on large errors

(B) MAPE can only be used for classification

(C) MAPE requires the data to be normally distributed

(D) MAPE is undefined or explodes on days with zero actual sales

Circle your choice. Answer and feedback: module answer key.

Before reporting a regression result

☐ The metric matches the business cost of errors

☐ A naive baseline (mean, median or last value) is reported alongside

☐ Errors are broken down by important segments, not just averaged

☐ Units are clear: absolute, squared or percentage

☐ Large individual errors have been inspected, not just summarised

REFLECT

For a number your team forecasts, is under-prediction or over-prediction more costly? Which metric or quantile would reflect that?

Key terms

MAE

Mean absolute error: the average size of errors, in the target's units.

MSE

Mean squared error: the average squared error, sensitive to large mistakes.

Copyright © ZyeemZia

RMSE
Root mean squared error: the square root of MSE, back in the target's units.

Quantile loss
An asymmetric loss that trains a model to predict a chosen percentile of the target.

Intermittent demand
A demand series with many zero periods, common for slow-moving products.

MAPE
Mean absolute percentage error: average error as a percentage of the actual value.

Prediction interval
A range expected to contain the actual value with a stated probability.

Key takeaways

- MAE measures typical error and is minimised by the median; RMSE punishes large misses and is minimised by the mean.
- MAPE breaks down near zero actuals; prefer scaled or weighted alternatives for intermittent demand.
- Quantile loss builds asymmetric costs into a forecast.
- Always report a naive baseline and errors by segment.

KNOWLEDGE CHECK

6. Errors are 3, −3, 3, −3. What is the MAE?
   - (A) 0
   - (B) 3
   - (C) 9
   - (D) 6
7. Errors are 0, 0, 0, 8. What is the RMSE?
   - (A) 2
   - (B) 8
   - (C) 4
   - (D) 16
8. Which constant prediction minimises MAE on a skewed target?
   - (A) The median
   - (B) The mean
   - (C) The maximum
   - (D) Zero
9. Why is MAPE unsuitable when some actual values are zero?
   - (A) It ignores negative errors, so over-forecasts and under-forecasts look the same
   - (B) It divides by the actual value, which makes the error undefined
   - (C) It always equals RMSE
   - (D) It requires classification labels
10. A team wants a forecast that is rarely too low, because shortages are dangerous. What should they optimise?
    - (A) The mean using MSE
    - (B) The median of the target, trained with MAE
    - (C) R^2 on the training set
    - (D) A high quantile such as the 0.9 quantile

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 4 · EVALUATE · LESSON 3 · 26 MIN

Classification Metrics and Thresholds

Confusion matrices, precision, recall, ROC and PR curves, and calibration

- Build a confusion matrix and compute precision, recall, F1 and specificity
- Interpret ROC-AUC and PR-AUC, and know when each is more informative
- Choose a threshold from costs or capacity
- Check whether predicted probabilities are calibrated

A classifier's errors come in two kinds that rarely cost the same. Missing a fraudulent transfer and blocking a genuine one are different failures with different owners. Good evaluation keeps the two kinds apart and then chooses a **threshold** that balances them for the business.

The confusion matrix (positive = fraud, 1,000 transfers)

ACTUALLY FRAUD (NO → YES) →

False negative (FN) = 40
- Fraud missed
- Loss to the customer or the firm

True positive (TP) = 80
- Fraud caught

True negative (TN) = 860
- Genuine transfer passed

False positive (FP) = 20
- Genuine transfer blocked
- Review cost and customer friction

PREDICTED FRAUD (NO → YES) →

Recall uses the top row (80 ÷ 120) and precision uses the right column (80 ÷ 100). The large bottom-left cell is why accuracy looks flattering on rare-event problems.

Metrics from the matrix above

MEASURE	FORMULA	WHAT IT TELLS YOU
Accuracy	`(TP + TN) ÷ total`	(80 + 860) ÷ 1,000 = 0.94, flattering because most cases are legitimate.
Precision	`TP ÷ (TP + FP)`	80 ÷ 100 = 0.80: of the flagged transfers, 80% are fraud.
Recall (sensitivity)	`TP ÷ (TP + FN)`	80 ÷ 120 ≈ 0.667: two-thirds of fraud is caught.
Specificity	`TN ÷ (TN + FP)`	860 ÷ 880 ≈ 0.977 of legitimate transfers pass untouched.
F1 score	`2 × precision × recall ÷ (precision + recall)`	2 × 0.8 × 0.667 ÷ 1.467 ≈ 0.727, the harmonic mean.
Brier score	$(1/n)\ \Sigma\ (p_i - y_i)^2$	Mean squared error of probabilities; 0 is perfect. Rewards calibrated, sharp predictions.

Copyright © ZyeemZia

ROC curve versus precision–recall curve

ROC and ROC-AUC

- Plots recall against false positive rate at every threshold
- AUC = probability a random positive scores above a random negative
- 0.5 is random, 1.0 is perfect
- Can look excellent when positives are very rare

Precision–recall and PR-AUC

- Plots precision against recall
- Baseline equals the positive rate (e.g. 0.01)
- Shows the false-alarm burden directly
- Preferred for rare-event problems such as fraud and disease

Metrics and threshold choice in code

Python notebook

```
import numpy as np
from sklearn.metrics import confusion_matrix, precision_score, recall_score, roc_auc_score
y = np.array([0, 0, 0, 0, 0, 0, 1, 1, 1, 1])
p = np.array([0.1, 0.2, 0.15, 0.3, 0.45, 0.6, 0.4, 0.7, 0.8, 0.9])
for t in (0.5, 0.35):
    pred = (p >= t).astype(int)
    print(t, confusion_matrix(y, pred).ravel().tolist(), round(precision_score(y, pred), 3),
recall_score(y, pred))
# output: 0.5 [5, 1, 1, 3] 0.75 0.75
# output: 0.35 [4, 2, 0, 4] 0.667 1.0
print(round(roc_auc_score(y, p), 3))
# output: 0.917
```

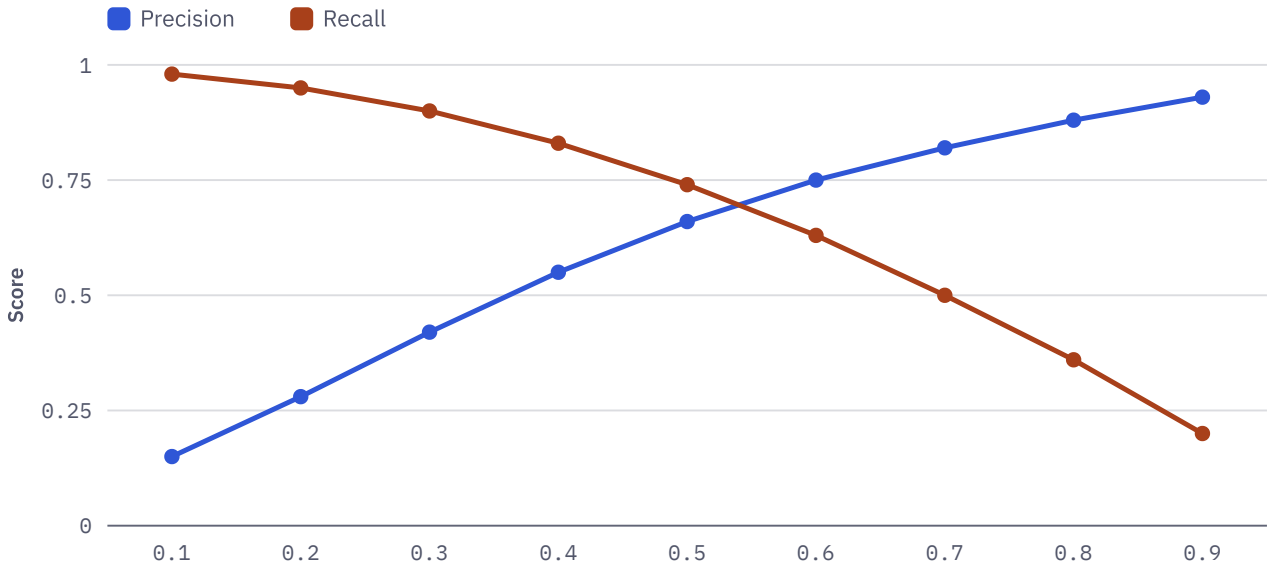
ravel() returns TN, FP, FN, TP. Lowering the threshold from 0.5 to 0.35 catches every positive but doubles the false alarms. AUC does not depend on the threshold.

Choosing a threshold

1. **List the costs** Cost of a false positive (e.g. a 50 BDT manual review) and of a false negative (e.g. an average 8,000 BDT fraud loss).
2. **Note capacity** The review team can handle 400 alerts a day.
3. **Sweep thresholds** On validation data, compute TP, FP and FN at many thresholds.
4. **Score each threshold** Expected cost = FP × 50 + FN × 8,000, subject to alerts ≤ 400 a day.
5. **Pick, then confirm** Choose the lowest-cost feasible threshold, confirm on the test set and monitor in production.

Copyright © ZyeemZia

Precision and recall as the threshold moves (illustrative)



Illustrative curves (x-axis: decision threshold). Raising the threshold trades recall for precision; the right point comes from costs and review capacity, not from where the lines cross.

INSIGHT · CALIBRATION MATTERS FOR DECISIONS

If a model says 0.2 for a group of patients, about 20% of them should have the outcome. Boosted trees, SVMs and over-trained networks are often mis-calibrated. Check with a reliability plot (predicted versus observed rate per bin) and fix with Platt scaling or isotonic regression fitted on validation data.

SCENARIO · DECIDE · RARE DISEASE SCREENING

A screening model for a rare condition (0.5% prevalence) has ROC-AUC 0.95. The clinical lead wants to know whether it is good enough. Which additional evidence is MOST useful?

- (A) The training-set ROC-AUC
- (B) The number of features in the model
- (C) The precision–recall curve and the precision at the recall level the clinic needs
- (D) Accuracy at the 0.5 threshold

Circle your choice. Answer and feedback: module answer key.

TEMPLATE **Threshold decision record**

Document how and why a probability threshold was chosen, so it can be reviewed and revisited when costs or capacity change.

MODEL AND VERSION	COST OF FP	COST OF FN	CAPACITY LIMIT	CHOSEN THRESHOLD	EXPECTED PRECISION / RECALL	APPROVED BY
Fraud scorer v3.2	*50 BDT review*	*8,000 BDT average loss*	*400 alerts per day*	*0.62*	*0.41 / 0.78 on validation*	*Head of fraud operations*

Copyright © ZyeemZia

MODEL AND VERSION	COST OF FP	COST OF FN	CAPACITY LIMIT	CHOSEN THRESHOLD	EXPECTED PRECISION / RECALL	APPROVED BY

REFLECT

For a classifier you know, estimate the cost of a false positive and of a false negative. What would a threshold based on those costs look like compared with 0.5?

Key terms

Confusion matrix
A table counting true positives, false positives, false negatives and true negatives.

Precision
The share of predicted positives that are truly positive.

Recall
The share of actual positives that the model correctly identifies; also called sensitivity.

F1 score
The harmonic mean of precision and recall.

ROC-AUC
The area under the curve of true positive rate against false positive rate; the probability a random positive outranks a random negative.

PR-AUC
The area under the precision–recall curve; informative for rare positive classes.

Reliability diagram
A plot of predicted probability against observed frequency used to check calibration.

Key takeaways

- Keep the two error types apart: precision asks how many flags are right, recall asks how many positives are caught.
- ROC-AUC measures ranking; with rare positives, PR curves and precision at the operating point are more informative.
- Choose the threshold from error costs and capacity, and record why.
- Check calibration with a reliability plot when probabilities drive pricing, triage or expected-cost decisions.

KNOWLEDGE CHECK

11. TP = 30, FP = 10, FN = 20. What is recall?

(A) 0.6
(B) 0.75
(C) 0.4
(D) 0.67

Copyright © ZyeemZia

12. Precision = 0.5 and recall = 1.0. What is F1?

(A) About 0.750
(B) About 0.500
(C) About 0.667
(D) Exactly 1.0

13. What does a ROC-AUC of 0.5 indicate?

(A) Perfect ranking
(B) 50% accuracy at every threshold
(C) Ranking no better than random
(D) Perfect calibration

14. Lowering the decision threshold usually...

(A) Increases recall and reduces precision
(B) Increases both precision and recall
(C) Reduces recall and increases precision
(D) Has no effect on the confusion matrix

15. A model predicts 0.8 for 100 loans and 40 of them default. What does this show?

(A) The model is perfectly calibrated
(B) The model is under-confident
(C) Calibration cannot be judged from this, because it needs the whole score range
(D) The model is over-confident in that range and poorly calibrated

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 4 · EVALUATE · LESSON 4 · 24 MIN

Bias, Variance and Tuning

Diagnose underfitting and overfitting, read learning curves and search hyperparameters efficiently

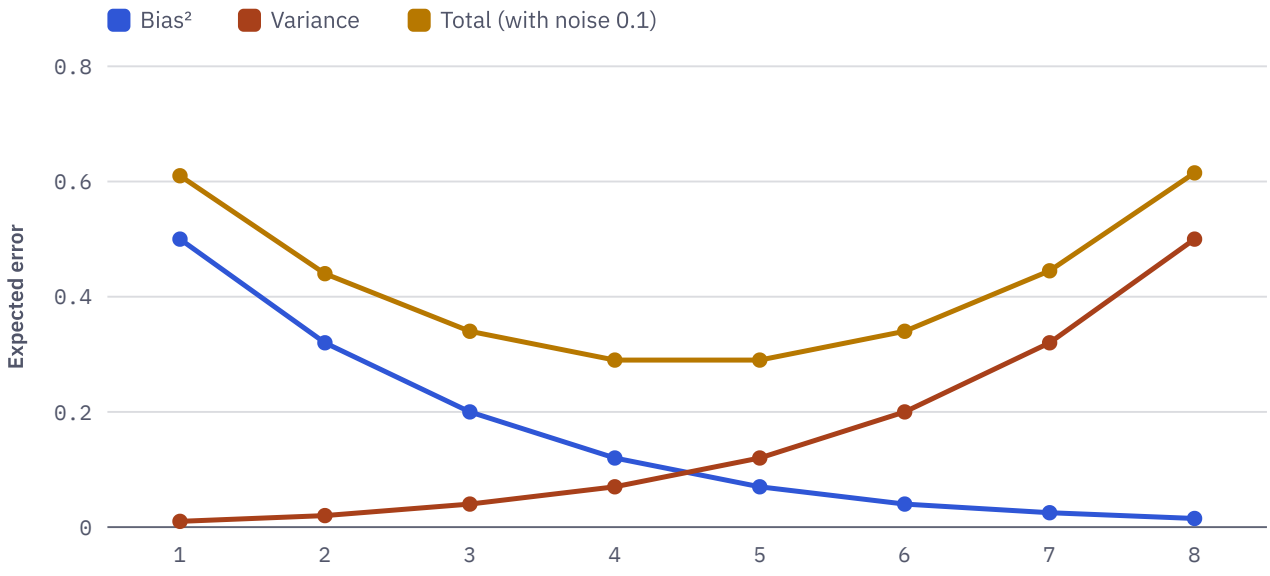
- Explain the bias–variance trade-off and its decomposition
- Diagnose underfitting and overfitting from training and validation scores
- Use learning and validation curves to decide what to do next
- Choose between grid, random and Bayesian hyperparameter search

Every model error has three sources: **bias** from assumptions too simple to capture the pattern, **variance** from sensitivity to the particular training sample, and **noise** that no model can remove. Knowing which one dominates tells you whether to add complexity, add data or add regularisation.

The bias–variance decomposition (squared error)

MEASURE	FORMULA	WHAT IT TELLS YOU
Expected error	`E[(y − ŷ)²] = Bias² + Variance + Irreducible noise`	Averaged over many possible training sets.
Bias	`Bias = E[ŷ] − f(x)`	How far the average model is from the truth. High for over-simple models.
Variance	`Variance = E[(ŷ − E[ŷ])²]`	How much predictions change between training sets. High for over-flexible models.
Worked example	`Bias² = 4, variance = 9, noise = 2`	Expected MSE = 15. Halving variance (to 4.5) gives 10.5; removing all bias gives only 11.

The bias–variance trade-off (illustrative)



Illustrative numbers (x-axis: model complexity, low to high). Bias falls and variance rises as complexity grows; total error is lowest in between and never drops below the irreducible noise.

Copyright © ZyeemZia

Diagnosing from training and validation scores

PATTERN	DIAGNOSIS	WHAT HELPS
Training poor, validation poor (similar)	High bias: underfitting	More features, more flexible model, less regularisation, train longer
Training excellent, validation much worse	High variance: overfitting	More data, stronger regularisation, simpler model, early stopping, bagging
Both good, small gap	Good fit	Confirm on test set; check segments
Validation better than training	Often regularisation such as dropout, or a leak or easy validation set	Check the split before celebrating

Learning curves: what more data will do

High-bias learning curve

- Training and validation scores converge quickly
- Both plateau at a poor level
- More data will not help much
- Change the model or features

High-variance learning curve

- Large gap between training and validation
- Validation still rising as data grows
- More data is likely to help
- So will regularisation

A validation curve for tree depth

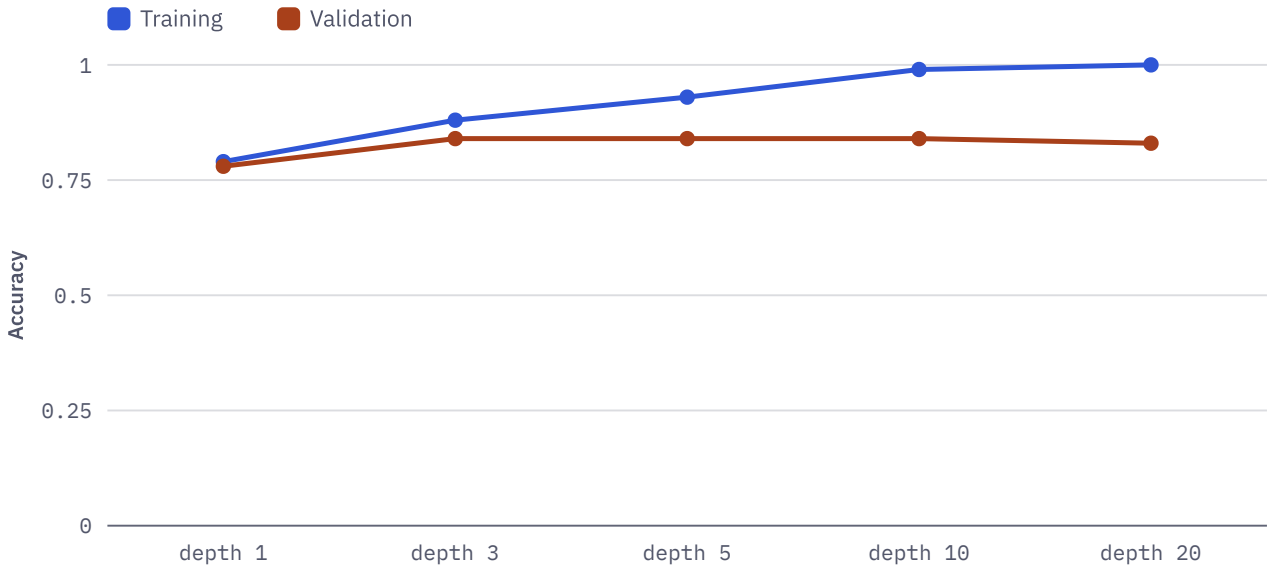
Python notebook

```
from sklearn.datasets import make_classification
from sklearn.model_selection import validation_curve
from sklearn.tree import DecisionTreeClassifier
X, y = make_classification(n_samples=800, n_features=20, n_informative=6, flip_y=0.1,
random_state=1)
depths = [1, 3, 5, 10, 20]
tr, va = validation_curve(DecisionTreeClassifier(random_state=0), X, y, param_name='max_depth',
param_range=depths, cv=5)
for d, a, b in zip(depths, tr.mean(1), va.mean(1)):
    print(d, round(a, 2), round(b, 2))
# output: 1 0.79 0.78
# output: 3 0.88 0.84
# output: 5 0.93 0.84
# output: 10 0.99 0.84
# output: 20 1.0 0.83
```

A depth-1 tree underfits: both scores are low. From depth 3 the validation score stops improving while the training score climbs to a perfect 1.0, a widening gap that signals memorisation. Prefer the simplest depth near the validation peak, here about 3.

Copyright © ZyeemZia

Validation curve for tree depth (from the code above)



Validation accuracy stops improving at depth 3 while training accuracy climbs to 1.0. The growing gap is variance; the simplest depth near the validation peak is the better choice.

Hyperparameter search strategies

STRATEGY	HOW IT WORKS	STRENGTH	WEAKNESS
Grid search	Try every combination of listed values	Simple, exhaustive on small grids	Explodes combinatorially; wastes trials on unimportant parameters
Random search	Sample combinations from ranges	Covers important parameters better for the same budget	Can miss narrow optima
Bayesian optimisation	Builds a model of score versus settings and picks promising trials	Efficient when each trial is expensive	More complex; sequential by nature
Successive halving / early stopping of trials	Start many trials cheaply, keep the best, give them more budget	Very efficient for neural networks and boosting	Early results may mislead

PRO TIP · SEARCH SMART, NOT WIDE

Tune the few hyperparameters that matter most (learning rate, regularisation strength, tree depth, number of estimators), use log-scaled ranges for rates and penalties, fix a budget before starting, and record every trial. Most gains come from the first 20–50 well-chosen trials.

Copyright © ZyeemZia

SCENARIO · DECIDE · NEXT STEP AFTER A LEARNING CURVE

A model for predicting machine failures in a textile mill shows training AUC 0.99 and validation AUC 0.74. The learning curve shows validation AUC still rising steadily as more data is added. Labelling more failure events is feasible over the next quarter. What is the BEST plan?

- (A) Collect more labelled data and add regularisation in the meantime
- (B) Stop work because 0.74 is the ceiling
- (C) Use a more complex model to raise training AUC to 1.0
- (D) Tune on the test set to find better settings

Circle your choice. Answer and feedback: module answer key.

REFLECT

Think of the last model you tuned. Which hyperparameters did you search, how many trials did you run, and how did you know when to stop?

Key terms

Bias
Error from overly simple assumptions that make a model systematically miss the true pattern.

Irreducible error
Noise in the target that no model can remove.

Learning curve
A plot of training and validation performance against training set size.

Random search
Hyperparameter tuning by sampling settings at random from defined ranges.

Model variance
Error from a model's sensitivity to the specific training sample it saw.

Underfitting
When a model is too simple to capture the pattern, so training and validation scores are both poor.

Validation curve
A plot of training and validation performance against one hyperparameter's value.

Key takeaways

- Error = bias² + variance + irreducible noise; diagnose which dominates before acting.
- High bias: both scores poor. High variance: a large training–validation gap.
- Learning curves tell you whether more data is likely to help.
- Random search, Bayesian optimisation or successive halving beat large grids for the same budget; log every trial.

KNOWLEDGE CHECK

Copyright © ZyeemZia

16. Bias² = 1, variance = 6 and noise = 3. What is the expected squared error?

(A) 7
(B) 10
(C) 9
(D) 16

17. Training accuracy 0.62 and validation accuracy 0.60 on a problem where experts reach 0.90. What is the likely issue?

(A) High bias (underfitting)
(B) High variance (overfitting)
(C) Data leakage
(D) Perfect fit

18. Which change typically reduces variance?

(A) Deeper trees with no limits
(B) Removing the validation set
(C) Stronger regularisation
(D) Adding many random features

19. Why does random search often beat grid search for the same number of trials?

(A) It always finds the global optimum
(B) It tries more distinct values of the parameters that really matter
(C) It needs no validation data
(D) It trains each model faster

20. A learning curve shows training and validation scores already converged at a low level. Will more data help much?

(A) Yes, more data always fixes low scores
(B) Yes, but only if the test set is enlarged at the same time
(C) No, the model must be deleted
(D) Probably not; change the model or features

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 4 · EVALUATE · LESSON 5 · 26 MIN

Leakage and Experiment Hygiene

Find the information that should not be there, and make results reproducible

- Recognise the main types of data leakage
- Detect leakage with practical checks
- Run experiments that are reproducible and fairly compared
- Decide whether a difference between models is real

Leakage happens when information that will not be available at prediction time sneaks into training or evaluation. Leaky models look brilliant in the notebook and fail in production. It is the most common and most expensive mistake in applied ML, and it is entirely preventable.

Types of leakage

TYPE	WHAT HAPPENS	EXAMPLE
Target leakage	A feature is derived from or recorded after the outcome	'Collections call made' used to predict loan default
Temporal leakage	Training uses data from after the prediction time	Random split on a time series; future prices in features
Group leakage	The same entity appears in training and validation	Images of the same patient in both sets
Preprocessing leakage	Statistics or selections learned on all data	Scaling, imputation or feature selection before the split
Proxy leakage	An artefact correlates with the label by accident	Scans from the intensive care unit carry a scanner marker that the model learns

Why 'reminder SMS sent' leaks: one loan's timeline



DAY 0 — Application scored
Prediction time: only data known now may be used.

DAY 30 — First instalment due

DAY 31 — Payment missed

DAY 32 — Reminder SMS sent
Created by the outcome process; it cannot exist at Day 0.

DAY 120 — Default recorded
The label.

Any feature created between the prediction time and the label is leakage, however predictive it looks. A timestamp audit places every feature on this line.

Copyright © ZyeemZia

Leakage you can measure: feature selection on pure noise

Python notebook

```
import numpy as np
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
rng = np.random.default_rng(0)
X = rng.normal(size=(100, 5000)); y = rng.integers(0, 2, 100)
X_sel = SelectKBest(f_classif, k=20).fit_transform(X, y)
print('leaky', round(cross_val_score(LogisticRegression(), X_sel, y, cv=5).mean(), 2))
# output: leaky 0.85
pipe = make_pipeline(SelectKBest(f_classif, k=20), LogisticRegression())
print('honest', round(cross_val_score(pipe, X, y, cv=5).mean(), 2))
# output: honest 0.48
```

The features and labels are random, so the true accuracy is 50%. Selecting features on all rows first produces an impressive but meaningless 85%. Putting selection inside the pipeline gives the honest answer.

Practical leakage checks

- **Too good to be true**: a score far above published or expert benchmarks deserves suspicion, not celebration.
- **Feature importance review**: if one feature dominates, ask exactly when and how it is recorded.
- **Timestamp audit**: for every feature, confirm it is available before the prediction time.
- **Entity overlap**: count customers, patients or devices that appear in both training and validation.
- **Shadow test**: score live data for a few weeks before go-live and compare with offline estimates.

WATCH OUT · BENCHMARK CONTAMINATION IN PRE-TRAINED MODELS

Leakage also affects large pre-trained models: public benchmark questions may have appeared in their training data, so headline scores overstate ability on your task. Evaluate on a private, recent set built from your own cases, keep it out of fine-tuning data and prompt examples, and refresh it over time.

CASE STUDY · THE MODEL THAT READ THE SCANNER

A team built a pneumonia detector from 40,000 chest X-rays across three hospitals and reached AUC 0.96. On images from a fourth hospital, AUC dropped to 0.71. Investigation showed that one hospital had a much higher pneumonia rate, used a different portable scanner and printed a distinctive text marker in the corner of each image. The training split was random across images, and some patients had several images.

1. What forms of leakage or shortcut learning are present?
2. How should the evaluation have been designed?
3. What changes would make the model more trustworthy?

Copyright © ZyeemZia

Model answer: module answer key.

Experiment hygiene essentials

E1
Version everything
Data snapshot, code commit, environment and configuration for every run.

E2
Fix seeds
Make splits and training reproducible so differences come from changes, not luck.

E3
Same folds
Compare models on identical folds so paired differences are meaningful.

E4
Log all trials
Record failures too; selective reporting hides how many attempts were made.

E5
One test
Use the test set once, at the end, for the chosen model only.

Is the improvement real?

MEASURE	FORMULA	WHAT IT TELLS YOU
Paired fold differences	$d_i = score_B_i - score_A_i$ for each fold i	Use the same folds for both models so fold difficulty cancels out.
Mean and spread	$\bar{d} = mean(d_i)$; $SE = s_d \div \sqrt{k}$	Differences 0.010, 0.006, 0.012, 0.004, 0.008 give $\bar{d} = 0.008$ and $s_d \approx 0.0032$, so $SE \approx 0.0014$.
Rule of thumb	Treat $\bar{d}$ as real if it is well above $2 \times SE$ and worth its cost	Here $0.008 > 0.0028$, so B is consistently better; whether 0.008 matters is a business question.

SCENARIO · DECIDE · A COLLECTIONS FEATURE

A default model's most important feature is 'number of reminder SMS sent'. Reminders are sent only after a payment is missed. The model will score applications at approval time. What should you do?

(A) Keep it, because it is genuinely predictive

(B) Remove the feature, because it is recorded after the outcome process has started and is unavailable at approval

(C) Cap the feature at 5 reminders to reduce its influence

(D) Keep it only in the test set

Circle your choice. Answer and feedback: module answer key.

Leakage review before sign-off

☐ Every feature has a documented source and availability time

☐ No feature is created by, or after, the outcome process

☐ Splits respect time order and entity groups

☐ All preprocessing and selection steps are inside the cross-validated pipeline

☐ The top features have been reviewed with a domain expert

☐ Offline scores are compared with a shadow run on live data

Copyright © ZyeemZia

REFLECT

Review the top five features of a model you know. For each, can you state exactly when the value is recorded relative to the moment of prediction?

Key terms

Data leakage
Information unavailable at prediction time that enters training or evaluation and inflates results.

Temporal leakage
Using data from after the prediction time in features or training.

Shadow deployment
Running a model on live data without acting on its predictions, to compare with expectations.

Paired comparison
Comparing two models on exactly the same folds or examples so shared difficulty cancels out.

Target leakage
A feature that is derived from, or recorded after, the outcome being predicted.

Shortcut learning
A model relying on a spurious cue, such as a scanner marker, instead of the real signal.

Reproducibility
The ability to rerun an experiment and obtain the same results from the same inputs.

Key takeaways

- Leakage is information unavailable at prediction time reaching training or evaluation; it inflates offline scores.
- Audit every feature's timestamp, split by time and entity, and keep preprocessing inside the pipeline.
- Treat results that look too good to be true as a bug report.
- Version data, code, environment and seeds, and compare models on identical folds with paired differences.
- For pre-trained models, use private, recent evaluation sets to avoid benchmark contamination.

KNOWLEDGE CHECK

21. Which is an example of target leakage in a churn model?

(A) Using the customer's tenure in months
(B) Using the number of logins in the 30 days before scoring
(C) Using the customer's plan type
(D) Using 'account closure form submitted' as a feature

22. Scaling features on the full dataset before cross-validation is an example of...

(A) Target leakage
(B) Preprocessing leakage
(C) Group leakage
(D) Label noise in validation

23. Paired fold differences have mean 0.002 and standard error 0.004. What should you conclude?

(A) Model B is clearly better
(B) Model A is clearly better
(C) The difference is not reliably distinguishable from zero
(D) The folds must have been different

Copyright © ZyeemZia

24. Why should two candidate models be evaluated on identical folds?

(A) So that differences in fold difficulty cancel out in the comparison

(B) So that both models train faster

(C) Because libraries require it

(D) So that the test set can be reused for tuning without becoming optimistic

25. A model's offline AUC is 0.99, far above anything published for the problem. What is the BEST first reaction?

(A) Deploy immediately

(B) Audit the top features and the split for leakage

(C) Publish the result

(D) Add more features and train for longer to try to reach 1.0

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 4 · EVALUATE

Answer key

Knowledge checks

1. A Sum 3.60 ÷ 5 = 0.72.

2. C Forecasts must be validated on the future relative to training data.

3. B Choosing based on test results turns it into another validation set.

4. D Grouping keeps each customer's rows together so validation mimics new customers.

5. D The outer loop scores the complete tune-then-fit process on data the tuning never saw.

6. B Absolute errors are all 3, so the mean is 3.

7. C MSE = 64 ÷ 4 = 16; RMSE = 4. MAE would be only 2, showing RMSE's sensitivity to large errors.

8. A Absolute loss is minimised by the median.

9. B Division by zero breaks the metric.

10. D A high quantile places the prediction above most outcomes, so it is rarely too low.

11. A Recall = 30 ÷ (30 + 20) = 0.6. Precision would be 0.75.

12. C 2 × 0.5 × 1 ÷ 1.5 ≈ 0.667.

13. C AUC 0.5 means a positive is as likely as not to outrank a negative.

14. A More cases are flagged, catching more positives but adding false positives.

15. D Calibrated predictions of 0.8 should see about 80 defaults, not 40.

16. B 1 + 6 + 3 = 10.

17. A Both scores are low and close together, so the model is too simple.

18. C Regularisation constrains the model, making it less sensitive to the training sample.

19. B A grid repeats the same few values of each parameter; random sampling explores each dimension more widely.

20. D Converged low curves signal high bias, which more data alone rarely fixes.

21. D A closure form is part of the outcome itself and is recorded after the churn decision.

22. B Statistics from validation rows influenced training.

23. C A mean smaller than its standard error is well within noise.

24. A Paired comparison removes shared variation and makes small differences detectable.

25. B Results that are too good to be true usually are; check for leakage first.

Scenarios & case studies

Scenario · Patients with many visits
B is best. Correct. With random splits, the model sees other visits of the same patient during training and partly recognises the person, which will not happen with new patients.

Scenario · Forecasting pharmacy demand
D is best. Correct. Division by zero makes MAPE unusable for intermittent demand. Use MAE, a scaled error, or a weighted absolute percentage error over totals instead.

Scenario · Rare disease screening
C is best. Correct. With 0.5% prevalence, even a high AUC can mean most positives are false alarms. PR analysis shows the real workload and yield.

Scenario · Next step after a learning curve
A is best. Correct. A large gap with a rising validation curve indicates high variance that more data and regularisation will both reduce.

Case study · The model that read the scanner
The model likely learned the hospital and scanner marker as a proxy for pneumonia (proxy leakage or shortcut learning), and patients with several images appeared in both training and validation (group leakage). Evaluation should group by patient and hold out whole hospitals, testing on a site never seen in training. Fixes include cropping or masking text markers, normalising image appearance across scanners, balancing or stratifying by site, checking saliency maps for attention on non-clinical regions, and requiring acceptable performance on each external site before deployment.

Copyright © ZyeemZia

Scenario · A collections feature

B is best. Correct. This is target leakage: reminders are a consequence of missed payments and do not exist when the loan is approved.

Copyright © ZyeemZia

05

MODULE 5 · ENSEMBLE

Trees and Ensembles: The Workhorses of Tabular Data

Grow decision trees, combine them in random forests and gradient boosting, and explain what they learn

BY THE END OF THIS MODULE YOU WILL BE ABLE TO

- Build and control decision trees using impurity measures and complexity limits
- Apply bagging and random forests and use out-of-bag estimates
- Train and tune gradient boosting models with early stopping
- Interpret ensembles with permutation importance, Shapley attributions and dependence plots
- Choose a tabular model family based on accuracy, explainability, cost and risk

LESSONS

1. Decision Trees · 24 min
2. Bagging and Random Forests · 24 min
3. Gradient Boosting · 26 min
4. Interpreting and Choosing Tabular Models · 26 min

MODULE 5 · ENSEMBLE · LESSON 1 · 24 MIN

Decision Trees

Learn a flowchart of questions from data, and know why single trees overfit

- Explain how a tree chooses splits using Gini impurity, entropy or variance reduction
- Compute impurity and information gain for a split by hand
- Control tree complexity with depth and leaf-size limits
- Describe the strengths and weaknesses of single trees

A **decision tree** predicts by asking a sequence of yes/no questions about the features: is the petal width at most 0.8 cm? Is the customer's tenure under six months? Each answer leads down a branch until a **leaf** gives the prediction. Trees capture non-linear effects and interactions automatically and are easy to read, which is why they underpin the most successful tabular methods.

How a tree scores a split

MEASURE	FORMULA	WHAT IT TELLS YOU
Gini impurity	$G = 1 - \Sigma_k\ p_k^2$	Node with 6 positives and 4 negatives: $1 - (0.6^2 + 0.4^2) = 1 - 0.52 = 0.48$.
Entropy	$H = -\Sigma_k\ p_k\ \log_2\ p_k$	Same node: $-0.6 \log_2 0.6 - 0.4 \log_2 0.4 \approx 0.442 + 0.529 = 0.971$ bits.
Weighted child impurity	Σ (n_child ÷ n_parent) × impurity_child	Split into left (4+, 0−) and right (2+, 4−): Gini = 0.4 × 0 + 0.6 × 0.444 ≈ 0.267.
Gain	Gain = impurity_parent − weighted impurity_children	Gini decrease = 0.48 − 0.267 ≈ 0.213. Information gain with entropy = 0.971 − 0.6 × 0.918 ≈ 0.420.
Regression trees	Split to minimise Σ $(y - \bar{y}_leaf)^2$ in the children	Each leaf predicts the mean of its training targets.

How a tree is grown

01 Start — All training rows in the root → **02 Search** — Try every feature and threshold → **03 Split** — Choose the largest impurity decrease → **04 Recurse** — Repeat in each child → **05 Stop** — Depth, leaf size or no gain →

06 Predict — Majority class or mean in the leaf

Copyright © ZyeemZia

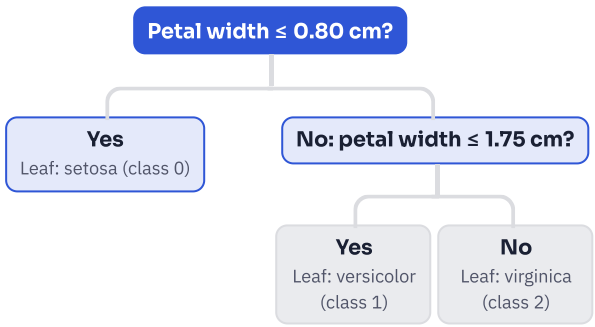
A readable two-level tree

Python notebook

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, export_text
X, y = load_iris(return_X_y=True)
tree = DecisionTreeClassifier(max_depth=2, random_state=0).fit(X, y)
print(export_text(tree, feature_names=['sepal_len', 'sepal_wid', 'petal_len', 'petal_wid']))
# output: |--- petal_wid <= 0.80
# output: |   |--- class: 0
# output: |--- petal_wid >  0.80
# output: |   |--- petal_wid <= 1.75
# output: |   |   |--- class: 1
# output: |   |--- petal_wid >  1.75
# output: |   |   |--- class: 2
```

Two questions about petal width separate the three species almost perfectly. A domain expert can read and challenge every rule.

The fitted iris tree, drawn



Each internal node is one learned question and each leaf a prediction. Reading the tree from the root shows exactly why a flower received its class.

Strengths and weaknesses of a single tree

Strengths

- Readable rules
- Handles non-linearity and interactions
- No scaling needed; mixed feature types
- Fast to train and predict

Weaknesses

- High variance: small data changes give a different tree
- Overfits easily if grown deep
- Step-shaped predictions; poor extrapolation
- Greedy splits may miss the best overall tree

Key hyperparameters

SETTING	EFFECT OF INCREASING IT	TYPICAL STARTING RANGE
max_depth	More complex tree, higher variance	3–10
min_samples_leaf	Smoother predictions, lower variance	5–100 rows, or 1% of data
min_impurity_decrease	Fewer, more meaningful splits	Small values such as 0.001
ccp_alpha (cost-complexity pruning)	Prunes weak branches after growing	Choose by cross-validation
class_weight	More attention to rare classes	'balanced' for imbalanced data

Copyright © ZyeemZia

INSIGHT · TREES DO NOT EXTRAPOLATE

A regression tree predicts the average of training rows in a leaf, so it can never predict beyond the range it has seen. If house prices in a growing Dhaka neighbourhood rise above any training price, a tree will cap its predictions at the old maximum. Linear models, or trees on detrended targets, behave better when extrapolation is unavoidable.

SCENARIO · DECIDE · AN UNSTABLE TREE

A bank trains a decision tree for small business loan approvals. Retraining on a new month of data produces a completely different top split and very different rules, which confuses loan officers. Accuracy is similar both times. What is the best response?

- (A) Grow the tree deeper so it captures every pattern
- (B) Retrain daily so that the rules change more smoothly
- (C) Constrain the tree (depth, minimum leaf size) or move to an ensemble while keeping explanations
- (D) Remove the validation set so that more data is used

Circle your choice. Answer and feedback: module answer key.

REFLECT

Sketch a three-level decision tree for a decision you make often, such as approving a leave request. Which question goes at the top, and why would a tree learned from data perhaps choose differently?

Key terms

Decision tree
A model that predicts by applying a sequence of feature-based splits ending in leaves.

Entropy
$-\Sigma\, p_k \log_2 p_k$; a measure of uncertainty in a node's class distribution.

Leaf
A terminal node of a tree that holds the prediction.

Greedy algorithm
An algorithm that makes the best local choice at each step without revisiting it.

Gini impurity
$1 - \Sigma\, p_k^2$; the probability of mislabelling a random item if labelled by the node's class distribution.

Information gain
The reduction in entropy achieved by a split.

Pruning
Removing branches that add little predictive value to reduce overfitting.

Copyright © ZyeemZia

Key takeaways

- A tree asks a sequence of yes/no questions chosen greedily to reduce impurity (Gini or entropy).
- Trees handle non-linearity, interactions and mixed feature types without scaling.
- Single trees have high variance; limit depth and leaf size or prune.
- Regression trees cannot predict outside the range of their training targets.

KNOWLEDGE CHECK

1. What is the Gini impurity of a node with 5 positives and 5 negatives?
   - (A) 0.5
   - (B) 0
   - (C) 1
   - (D) 0.25
2. What is the entropy of a pure node?
   - (A) 1
   - (B) 0.5
   - (C) 0
   - (D) Undefined
3. A parent has Gini 0.5. The weighted Gini of its children is 0.3. What is the Gini decrease?
   - (A) 0.8
   - (B) 0.2
   - (C) 0.15
   - (D) 0.3
4. Which change makes a decision tree less likely to overfit?
   - (A) Increasing the maximum depth
   - (B) Removing all stopping rules
   - (C) Using a smaller training set so each leaf sees fewer rows
   - (D) Increasing the minimum number of samples per leaf
5. Why can a regression tree not predict values above its training maximum?
   - (A) Trees always use a sigmoid output
   - (B) Trees require standardised targets
   - (C) Trees can only perform classification
   - (D) Each leaf predicts an average of training targets

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 5 · ENSEMBLE · LESSON 2 · 24 MIN

Bagging and Random Forests

Average many decorrelated trees to cut variance

- Explain bootstrap sampling and why averaging reduces variance
- Describe how random forests decorrelate trees with feature sampling
- Use out-of-bag error as a built-in validation estimate
- Tune the few random forest settings that matter

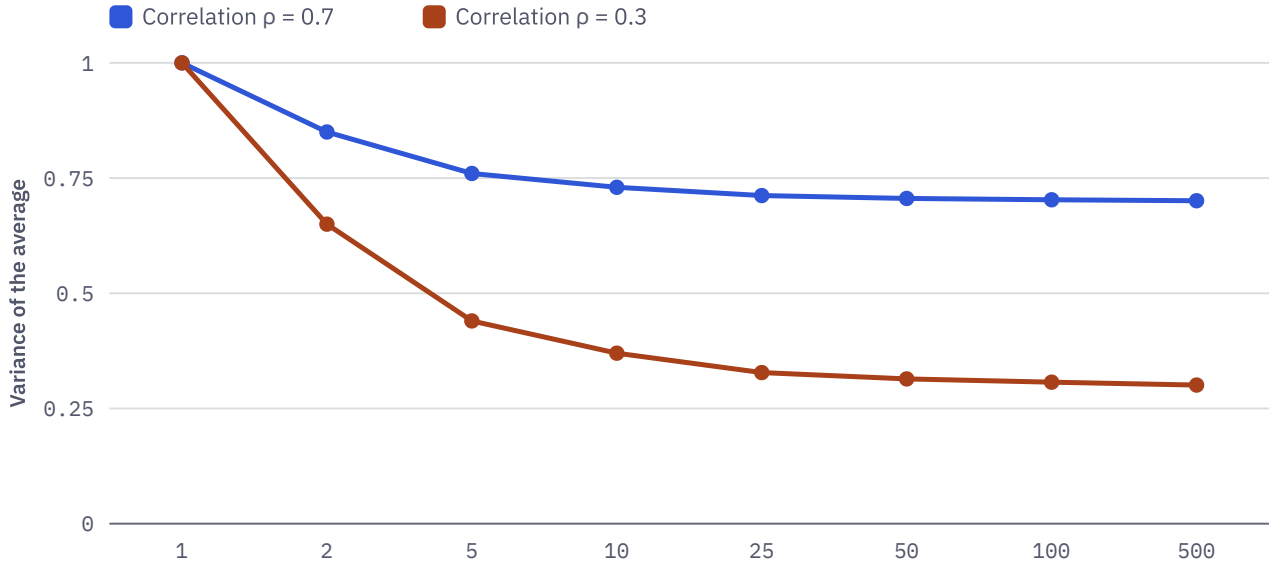
A single deep tree is accurate on average but erratic. **Bagging** (bootstrap aggregating) trains many trees on different random resamples of the data and averages them; their individual errors partly cancel. A **random forest** adds one more twist: each split considers only a random subset of features, which makes the trees less alike and the average even more stable.

Why averaging works

MEASURE	FORMULA	WHAT IT TELLS YOU
Variance of an average	`Var(avg) = ρσ² + (1 − ρ)σ² ÷ B`	σ² is one tree's variance, ρ the correlation between trees, B the number of trees.
Worked example	`σ² = 1, ρ = 0.3, B = 100`	0.3 + 0.7 ÷ 100 = 0.307: averaging cuts variance by about two-thirds.
The lesson	`As B grows, Var → ρσ²`	More trees cannot remove correlated error; lowering ρ through feature sampling is what makes forests strong.
Bootstrap coverage	`P(row included) = 1 − (1 − 1/n)ⁿ ≈ 0.632`	Each bootstrap sample contains about 63% of distinct rows; the other 37% are 'out of bag' for that tree.

Copyright © ZyeemZia

Variance of a forest's average as trees are added ($\sigma^2 = 1$)



Computed from Var = $\rho\sigma^2 + (1 - \rho)\sigma^2 \div B$ (x-axis: number of trees). Adding trees quickly reaches a floor equal to ρ; only decorrelating the trees, for example with a smaller max_features, lowers that floor.

How a random forest is built

1. **Bootstrap** For each of B trees, draw n rows with replacement.
2. **Random features per split** At each split, consider only a random subset of features, often √d for classification.
3. **Grow deep trees** Trees are usually grown deep with little pruning; averaging controls variance.
4. **Aggregate** Classification: average predicted probabilities (or majority vote). Regression: average predictions.
5. **Out-of-bag check** Score each row using only trees that did not see it, giving a free validation estimate.

Random forest with out-of-bag scoring

Python notebook

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
tree = DecisionTreeClassifier(random_state=0).fit(X_tr, y_tr)
rf = RandomForestClassifier(n_estimators=300, oob_score=True, random_state=0).fit(X_tr, y_tr)
print(round(tree.score(X_te, y_te), 3), round(rf.oob_score_, 3), round(rf.score(X_te, y_te), 3))
# output: 0.902 0.962 0.944
```

The forest beats the single tree on the test set, and its out-of-bag estimate (0.962) is close to the test score without using any separate validation data.

Random forest settings that matter

SETTING	WHAT IT DOES	GUIDANCE
n_estimators	Number of trees	More is never worse for accuracy, only slower; 200–1,000 is common

Copyright © ZyeemZia

SETTING	WHAT IT DOES	GUIDANCE
max_features	Features tried at each split	Lower values decorrelate trees; √d (classification) or d/3 (regression) are good defaults
min_samples_leaf	Minimum rows per leaf	Raise for noisy data or to calibrate probabilities better
max_depth	Tree depth limit	Often left unlimited; limit for speed or memory
class_weight	Weights for rare classes	'balanced' or 'balanced_subsample' for imbalanced problems

Random forest versus single tree

Single tree

- One readable set of rules
- High variance
- Fast, tiny model

Random forest

- Hundreds of trees, not readable directly
- Much lower variance, strong accuracy
- Robust defaults; little tuning needed
- Larger memory and slower prediction

EXAMPLE · CROP DISEASE RISK ACROSS DISTRICTS

An agricultural extension programme in northern Bangladesh used a random forest to predict which villages were at high risk of rice blast in the coming fortnight, using weather, humidity, crop stage and past outbreaks. With little tuning, the forest outperformed the earlier rule of thumb, and out-of-bag estimates allowed the small team to validate without sacrificing scarce labelled seasons.

SCENARIO · DECIDE · MORE TREES OR DIFFERENT TREES?

A forest with 500 trees and max_features set to all features performs only slightly better than a single tree. A colleague suggests raising the number of trees to 5,000. What is the more promising change?

(A) Reduce max_features so that trees consider different features and become less correlated

(B) Increase to 5,000 trees

(C) Turn off bootstrapping

(D) Prune every tree to depth 1

Circle your choice. Answer and feedback: module answer key.

REFLECT

Where in your work would a model that is accurate but not directly readable be acceptable? What explanation would stakeholders need instead?

Copyright © ZyeemZia

Key terms

Bootstrap sample
A sample of n rows drawn with replacement from a dataset of n rows.

Random forest
A bagged ensemble of decision trees where each split considers a random subset of features.

max_features
The number of features randomly considered at each split in a random forest.

Decorrelation
Making ensemble members' errors less similar so that averaging cancels more of them.

Bagging
Training models on bootstrap samples and averaging their predictions to reduce variance.

Out-of-bag (OOB) score
Accuracy estimated for each row using only trees that did not include it in their bootstrap sample.

Ensemble
A model that combines the predictions of several models.

Key takeaways

- Bagging averages trees trained on bootstrap samples, cutting variance.
- Random feature subsets at each split decorrelate trees, which lowers the error floor.
- More trees stabilise predictions but cannot remove error shared by correlated trees.
- The out-of-bag score gives a built-in validation estimate.

KNOWLEDGE CHECK

6. Roughly what share of distinct rows appears in one bootstrap sample of a large dataset?
   - (A) About 50%
   - (B) About 63%
   - (C) 100%
   - (D) About 37%

7. With $\sigma^2 = 4$, $\rho = 0.5$ and $B = 50$ trees, what is the variance of the average?
   - (A) 4
   - (B) 0.08
   - (C) 2.04
   - (D) 2.5

8. What is the main purpose of random feature selection at each split?
   - (A) To make trees less correlated
   - (B) To speed up prediction only
   - (C) To guarantee interpretability
   - (D) To remove the need for bootstrapping

9. How is the out-of-bag estimate computed?
   - (A) By evaluating on the test set
   - (B) Each row is scored by trees whose bootstrap samples did not include it
   - (C) By scoring each row with every tree
   - (D) By cross-validating each tree separately on its own bootstrap sample and averaging

10. Which statement about adding more trees to a random forest is correct?
    - (A) It always causes overfitting
    - (B) It turns the forest into a single tree
    - (C) It increases the variance of the average, because each extra tree adds its own noise
    - (D) It stabilises predictions but cannot remove error shared by correlated trees

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 5 · ENSEMBLE · LESSON 3 · 26 MIN

Gradient Boosting

Build trees in sequence, each fixing the errors of the ones before

- Explain boosting as stage-wise fitting of residuals or negative gradients
- Carry out a boosting update by hand
- Tune learning rate, number of trees, depth and early stopping
- Compare boosting with random forests and know when each fits

Random forests build independent trees in parallel and average them. **Gradient boosting** builds small trees one after another; each new tree is trained to correct what the current ensemble still gets wrong. With careful tuning it is usually the most accurate method for tabular data, and modern histogram-based implementations train quickly on millions of rows.

The boosting update

MEASURE	FORMULA	WHAT IT TELLS YOU
Additive model	$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$	F_m is the ensemble after m trees, h_m the new tree, η the learning rate (shrinkage).
What the tree fits (squared error)	$r_i = y_i - F_{m-1}(x_i)$	For squared error, the negative gradient is simply the residual.
General losses	$r_i = -\partial L(y_i, F)/\partial F$	For log loss, the negative gradient is y − p, the gap between the label and the predicted probability.
Worked example	F_0 = mean = 10; a row has y = 14; the tree predicts 4 for its leaf; η = 0.1	F_1 = 10 + 0.1 × 4 = 10.4. The new residual is 3.6, and the next tree keeps chipping away.

Boosting, round by round

01 Start — Constant prediction, e.g. the mean

→

02 Residuals — Compute what is still wrong

→

03 Fit a small tree — Depth 3–8 on the residuals

→

04 Shrink and add — Add η × tree to the ensemble

→

05 Check validation — Stop when it stops improving

Hyperparameters that matter

SETTING	EFFECT	TYPICAL GUIDANCE
learning_rate (η)	Size of each tree's contribution	0.01–0.1; smaller needs more trees but often generalises better
n_estimators / max_iter	Number of boosting rounds	Set high and use early stopping on validation data
max_depth or max_leaf_nodes	Complexity of each tree and interactions captured	Depth 3–8 or 15–63 leaves

Copyright © ZyeemZia

SETTING	EFFECT	TYPICAL GUIDANCE
subsample / colsample	Row and feature sampling per tree	0.5–0.9 adds randomness and reduces overfitting
L1/L2 leaf regularisation, min child weight	Limits extreme leaf values	Raise for noisy data
early_stopping rounds	Stop when validation stalls	10–100 rounds of patience

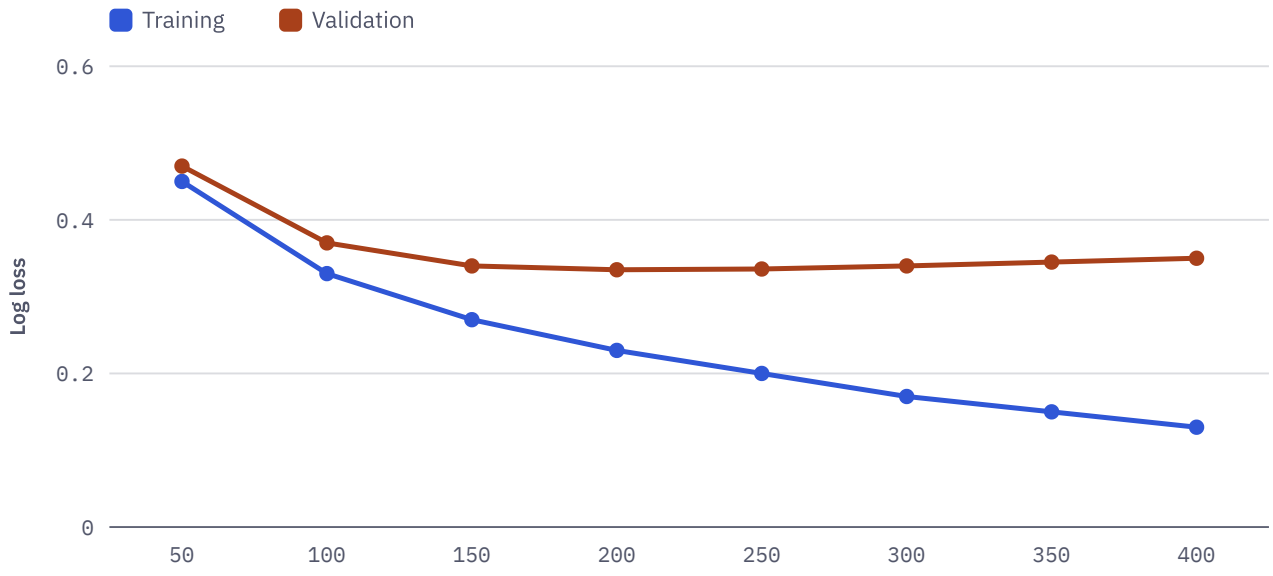
Histogram-based boosting with early stopping

Python notebook

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import HistGradientBoostingClassifier
X, y = make_classification(n_samples=5000, n_features=20, n_informative=8, random_state=0)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0)
gb = HistGradientBoostingClassifier(learning_rate=0.1, max_iter=1000, early_stopping=True,
                                    validation_fraction=0.1, n_iter_no_change=20,
random_state=0).fit(X_tr, y_tr)
print(gb.n_iter_, round(gb.score(X_te, y_te), 3))
# output: 104 0.952
```

Although up to 1,000 rounds were allowed, training stopped after 104 because the internal validation score stopped improving. Early stopping chooses the number of trees for you.

Training and validation loss over boosting rounds (illustrative)



Illustrative numbers (x-axis: boosting round). Training loss keeps falling, but validation loss flattens and then creeps up; early stopping keeps the model from about round 200.

Copyright © ZyeemZia

Random forest versus gradient boosting

Random forest

- Trees built independently, in parallel
- Reduces variance by averaging deep trees
- Strong with almost no tuning
- Hard to overfit by adding trees

Gradient boosting

- Trees built sequentially
- Reduces bias by correcting errors step by step
- Usually more accurate, but needs tuning
- Can overfit if run too long; use early stopping

INSIGHT · STILL THE TABULAR DEFAULT

For small and medium tabular data, gradient-boosted trees remain the strong default baseline. Current libraries handle missing values and categorical features natively and can train on GPUs. Newer options, including pre-trained tabular models that predict from training examples supplied in context, can be competitive on small datasets: treat them as extra candidates compared on the same folds, not as automatic replacements.

PRO TIP · A DEPENDABLE TUNING ORDER

Fix a moderate learning rate (0.05–0.1) with early stopping, tune tree complexity (depth or leaves) and minimum leaf size, then row and feature sampling, then regularisation. Finally, lower the learning rate and let early stopping add more trees for a last small gain. Keep the validation scheme fixed throughout.

CASE STUDY · FORECASTING LATE SHIPMENTS FOR A GARMENT EXPORTER

A Narayanganj knitwear exporter wants to predict at order acceptance whether an order will ship late. There are 18,000 historical orders with 60 features (buyer, product type, fabric, quantity, lead time, factory load, season, supplier history). A logistic regression baseline scores AUC 0.74. A random forest scores 0.80. A first boosting model with default settings scores 0.79 on validation but 0.99 on training.

1. What does the gap in the boosting results suggest, and how would you address it?
2. Which validation design suits this problem?
3. How would you decide between the forest and a tuned boosting model?

Model answer: module answer key.

Copyright © ZyeemZia

SCENARIO · DECIDE · LEARNING RATE AND TREES

A team halves the learning rate of a boosting model from 0.1 to 0.05 but keeps the number of trees fixed at 300, and validation performance drops. What is the most likely explanation?

(A) The trees became deeper automatically

(B) With smaller steps, 300 trees are no longer enough; allow more rounds with early stopping

(C) The validation set must have leaked

(D) A smaller learning rate always hurts boosting

Circle your choice. Answer and feedback: module answer key.

REFLECT

Think of a tabular prediction problem in your organisation. Would you start with a random forest or gradient boosting, and what would make you switch?

Key terms

Gradient boosting
An ensemble method that adds trees sequentially, each fitted to the negative gradient of the loss of the current model.

Pseudo-residual
The negative gradient of the loss for each row, which the next boosting tree is trained to predict; for squared error it equals the ordinary residual.

Histogram-based boosting
Boosting that bins feature values into histograms to find splits quickly on large data.

Subsampling
Training each tree on a random fraction of rows or features to add diversity and reduce overfitting.

Shrinkage
Scaling each new tree's contribution by a learning rate below 1 to improve generalisation.

Early stopping
Halting training when a validation metric stops improving for a set number of rounds.

Weak learner
A simple model, such as a shallow tree, that is only slightly better than chance on its own.

Key takeaways

- Boosting adds shallow trees in sequence, each fitting what the ensemble still gets wrong.
- The learning rate and the number of rounds trade off; set rounds high and use early stopping.
- Tune tree complexity and sampling before fine-tuning the learning rate.
- Gradient-boosted trees remain the strong default for tabular data; compare any newcomer fairly.

KNOWLEDGE CHECK

Copyright © ZyeemZia

11. Current prediction is 50, target 60, the new tree predicts the residual exactly, and η = 0.2. What is the updated prediction?

(A) 52
(B) 60
(C) 50.2
(D) 62

12. For squared-error loss, what does each new tree in gradient boosting fit?

(A) The original target values
(B) Random noise
(C) The residuals of the current ensemble
(D) The predictions made by the previous tree

13. Which pairing is MOST typical of boosting?

(A) A few very deep trees built in parallel
(B) One large tree pruned heavily
(C) Many shallow trees built in sequence
(D) Independent trees on bootstrap samples only

14. Why is early stopping important in gradient boosting?

(A) Boosting can keep fitting noise if run for too many rounds
(B) It makes each tree deeper
(C) It removes the need for a learning rate
(D) It guarantees a calibrated model

15. For log loss, what is the negative gradient for a row with label 1 and predicted probability 0.7?

(A) 0.7
(B) −0.3
(C) 1.7
(D) 0.3

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 5 · ENSEMBLE · LESSON 4 · 26 MIN

Interpreting and Choosing Tabular Models

Explain what drives predictions and pick the right model for the job

- Compare impurity, permutation and Shapley-based importance
- Use partial dependence and individual conditional expectation plots
- Apply monotonic constraints where domain logic demands them
- Choose a model family using accuracy, explainability, cost and risk

Ensembles are accurate but not readable as a whole. **Interpretation tools** reveal which features matter, how they affect predictions and why a single case received its score. These are essential for debugging, for winning stakeholder trust and for regulators who expect reasons for decisions such as loan refusals.

Global and local explanation tools

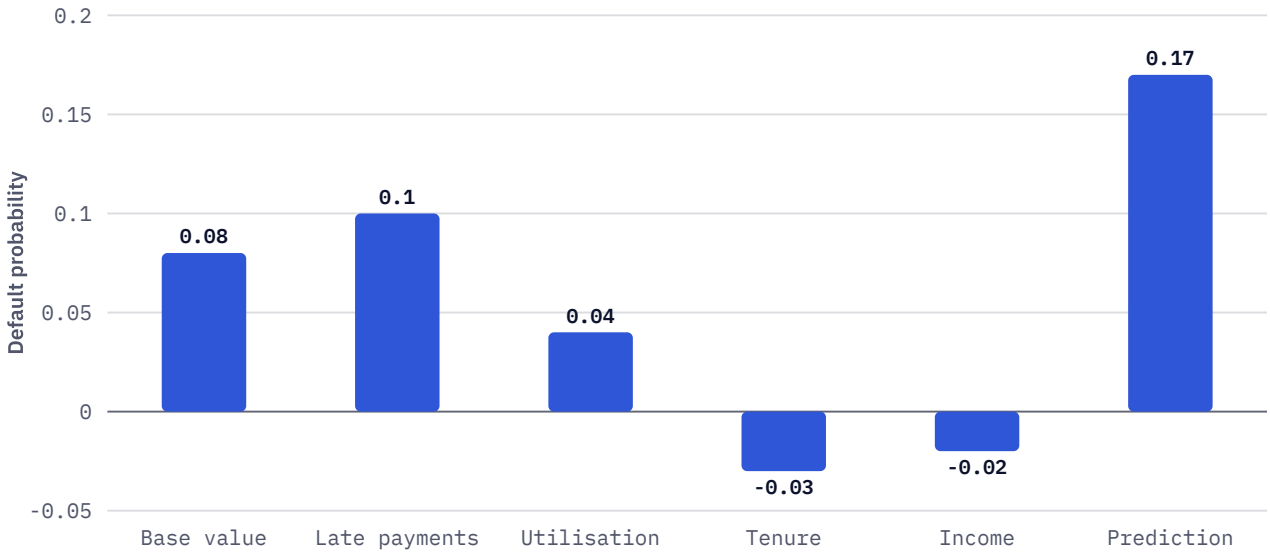
TOOL	QUESTION IT ANSWERS	CAVEATS
Impurity (gain) importance	Which features did the trees split on most?	Biased towards high-cardinality and continuous features; computed on training data
Permutation importance	How much does the validation score drop if this feature is shuffled?	Correlated features share and hide importance
Shapley-value attributions	How much did each feature push this prediction up or down?	Can be slow; explains the model, not the real world
Partial dependence (PD)	How does the average prediction change as one feature varies?	Averages can hide opposite effects in subgroups
Individual conditional expectation (ICE)	How does one case's prediction change as one feature varies?	Many lines; read alongside PD

Attribution arithmetic

MEASURE	FORMULA	WHAT IT TELLS YOU
Permutation importance	`PI_j = score(original) − score(feature j shuffled)`	If AUC falls from 0.86 to 0.79 when income is shuffled, its importance is 0.07.
Additive explanation	$f(x) = \phi_0 + \Sigma_j \phi_j$	φ_0 is the average prediction; each φ_j is a feature's contribution for this case.
Worked example	$\phi_0 = 0.10$ default rate; $\phi_{late} = +0.12$, $\phi_{income} = -0.04$, $\phi_{tenure} = -0.03$	Prediction = 0.10 + 0.12 − 0.04 − 0.03 = 0.15. Late payments are the main reason this applicant scores high.

Copyright © ZyeemZia

Why one applicant scored 0.17 (illustrative Shapley contributions)



Illustrative values. The base value plus the signed contributions (0.08 + 0.10 + 0.04 − 0.03 − 0.02) equals the prediction, so each bar is a reason that can be reported to the applicant or a reviewer.

Permutation importance on validation data

Python notebook

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(data.data, data.target, test_size=0.25, random_state=0,
stratify=data.target)
rf = RandomForestClassifier(n_estimators=300, random_state=0).fit(X_tr, y_tr)
r = permutation_importance(rf, X_te, y_te, n_repeats=10, random_state=0)
top = r.importances_mean.argsort()[::-1][:3]
print([data.feature_names[i] for i in top])
# output: ['worst perimeter', 'worst area', 'worst concave points']
```

Importance is measured on held-out data, so it reflects what the model relies on to generalise. Perimeter, area and radius are strongly correlated size measurements, so they share importance and each may look smaller than the group really is.

WATCH OUT · EXPLANATIONS DESCRIBE THE MODEL, NOT THE WORLD

A Shapley plot showing that 'district' raises predicted default does not prove that living there causes default; it shows how the model uses the feature. Explanations are excellent for debugging and accountability, but causal claims still need experiments or causal analysis.

Copyright © ZyeemZia

Monotonic constraints

- Many domains have logic everyone agrees on: more late payments should never lower predicted default risk.
- Boosting libraries can enforce that a prediction only rises (or only falls) with a feature.
- Constraints prevent odd, unfair-looking reversals caused by noise in sparse regions.
- They usually cost little accuracy and greatly increase stakeholder trust.
- Document each constraint and the reason for it.

Choosing a model family for tabular data

NEED	GOOD FIRST CHOICE	WHY
Strict explainability, regulated decision	Logistic regression or scorecard; constrained boosting with explanations	Reasons for every decision, stable behaviour
Best accuracy on medium-to-large tabular data	Gradient boosting with early stopping	Captures non-linearity and interactions efficiently
Strong result with minimal tuning	Random forest	Robust defaults, OOB validation
Tiny dataset	Regularised linear model	Low variance
Extrapolation beyond training range	Linear model or trees on detrended target	Trees cannot extrapolate
Very low latency on edge devices	Small tree ensemble or linear model	Fast and compact

SCENARIO · DECIDE · EXPLAINING A DECLINED LOAN

A microfinance institution uses gradient boosting to score loan applications. A borrower in Bogura who was declined asks why. What is the most appropriate response mechanism?

(A) Tell the borrower the model is too complex to explain

(B) Retrain the model without the borrower's data and compare

(C) Show the global feature importance chart for the whole model

(D) Generate reason codes from the case's Shapley-style attributions, reviewed for plain-language clarity

Circle your choice. Answer and feedback: module answer key.

TEMPLATE **Model explanation card**

Summarise how a deployed tabular model behaves, for reviewers, auditors and support staff.

ITEM	CONTENT
Top drivers (permutation importance, validation data)	*Late payments 12m, utilisation ratio, tenure, income band*
Constraints applied	*Monotonic increasing in late payments and utilisation*

Copyright © ZyeemZia

ITEM	CONTENT

REFLECT

Who would ask for explanations of a model in your organisation: customers, managers, auditors or regulators? What form of explanation would each of them actually understand?

Key terms

Feature importance
A score of how much a model relies on a feature for its predictions.

Shapley values
Attributions from cooperative game theory that fairly divide a prediction among features.

ICE plot
Individual conditional expectation: one line per case showing how its prediction changes with a feature.

Reason code
A plain-language statement of a top factor behind an individual model decision.

Permutation importance
The drop in a model's score when one feature's values are randomly shuffled.

Partial dependence plot
A plot of the average model prediction as one feature varies, others averaged out.

Monotonic constraint
A rule that forces a model's prediction to move in only one direction as a feature increases.

Key takeaways

- Use permutation importance on validation data for global importance; gain importance is biased.
- Shapley attributions explain single predictions additively; partial dependence and ICE plots show shape.
- Explanations describe how the model uses features, not what causes the outcome.
- Monotonic constraints encode agreed domain logic and prevent odd reversals.
- Choose a model family by accuracy, explainability, cost and risk together.

KNOWLEDGE CHECK

16. Baseline AUC is 0.82; after shuffling 'tenure' it is 0.76. What is the permutation importance?

(A) 0.76
(B) 0.06
(C) 0.82
(D) 1.58

Copyright © ZyeemZia

17. $\varphi_0 = 0.2$ and contributions are +0.15, −0.05 and +0.02. What is the prediction?

(A) 0.32
(B) 0.12
(C) 0.2
(D) 0.42

18. Why can impurity-based importance be misleading?

(A) It is always computed on the test set
(B) It only works for linear models, so tree ensembles must use coefficients instead
(C) It favours high-cardinality and continuous features and uses training data
(D) It cannot rank more than five features

19. What does a monotonic constraint guarantee?

(A) The model is causal
(B) Predictions move in only one direction as the constrained feature increases
(C) The model's probabilities are perfectly calibrated for every value of the feature
(D) The feature has the highest importance

20. Two correlated features each show low permutation importance. What is a likely explanation?

(A) Both features are useless
(B) The model is underfitting
(C) Permutation importance only works for categorical features, not numeric ones
(D) Shuffling one leaves the other to carry the same information

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 5 · ENSEMBLE

Answer key

Knowledge checks

1. A $1 - (0.5^2 + 0.5^2) = 0.5$, the maximum for two classes.

2. C All items belong to one class, so there is no uncertainty.

3. B $0.5 - 0.3 = 0.2$.

4. D Larger leaves average more rows, giving smoother, lower-variance predictions.

5. D Averages of training targets cannot exceed the largest training target.

6. B $1 - (1 - 1/n)^n$ approaches $1 - 1/e \approx 0.632$.

7. C $0.5 \times 4 + 0.5 \times 4 \div 50 = 2 + 0.04 = 2.04$.

8. A Lower correlation means averaging cancels more error.

9. B OOB uses each tree's left-out rows as a built-in validation set.

10. D Variance approaches $\rho\sigma^2$; more trees do not cause overfitting in the usual sense.

11. A $50 + 0.2 \times 10 = 52$.

12. C The negative gradient of squared error is the residual.

13. C Boosting adds many small trees, each correcting the last.

14. A Validation-based stopping chooses the round count before overfitting sets in.

15. D $y - p = 1 - 0.7 = 0.3$.

16. B $0.82 - 0.76 = 0.06$.

17. A $0.2 + 0.15 - 0.05 + 0.02 = 0.32$.

18. C Features with many possible split points get chosen more often, even if they generalise poorly.

19. B It enforces shape, not causality or calibration.

20. D Correlated features mask each other; group them and permute together.

Scenarios & case studies

Scenario · An unstable tree
C is best. Correct. Single deep trees have high variance. Limits make them more stable; ensembles average many trees for stable predictions, explained with tools such as permutation importance.

Scenario · More trees or different trees?
A is best. Correct. With all features available at every split, trees choose similar splits and are highly correlated; averaging then removes little variance.

Case study · Forecasting late shipments for a garment exporter
The 0.99 versus 0.79 gap suggests the boosted model is overfitting: too many rounds or too complex trees. Use early stopping, a smaller learning rate, shallower trees or fewer leaves, a larger minimum leaf size and row and feature subsampling. Because orders are predicted in time and buyers repeat, validate with a time-based split (train on older orders, validate on later ones), and check buyer overlap. Compare the tuned boosting model and the forest on identical folds with the business metric (for example recall of late orders at the capacity of the expediting team), then weigh accuracy gains against stability, training time and the explanations the planning team needs.

Scenario · Learning rate and trees
B is best. Correct. Learning rate and number of trees trade off: halving η roughly doubles the rounds needed to reach the same fit.

Scenario · Explaining a declined loan
D is best. Correct. Local attributions identify the factors that pushed this applicant's score up most, which can be translated into clear reasons and checked by staff.

Copyright © ZyeemZia

06

MODULE 6 · DISCOVER

Beyond Labels: Unsupervised Learning, Recommendation and Reinforcement

Find structure without labels, recommend what people need and learn from rewards

BY THE END OF THIS MODULE YOU WILL BE ABLE TO

- Cluster data with k-means and alternatives, choose k and turn clusters into segments
- Reduce dimensionality with PCA and use non-linear maps appropriately
- Detect anomalies and build feedback loops that improve detection
- Design and evaluate recommender systems, including cold start
- Explain reinforcement learning, bandits and when RL is practical and safe

LESSONS

1. Clustering · 26 min
2. PCA and Dimensionality Reduction · 24 min
3. Anomaly Detection · 24 min
4. Recommender Systems · 26 min
5. Reinforcement Learning: An Introduction · 26 min

MODULE 6 · DISCOVER · LESSON 1 · 26 MIN

Clustering

Find natural groups in unlabelled data and judge whether they are useful

- Run the k-means algorithm by hand and explain its objective
- Choose the number of clusters with inertia and silhouette scores
- Compare k-means, hierarchical clustering, DBSCAN and Gaussian mixtures
- Turn clusters into actionable, validated segments

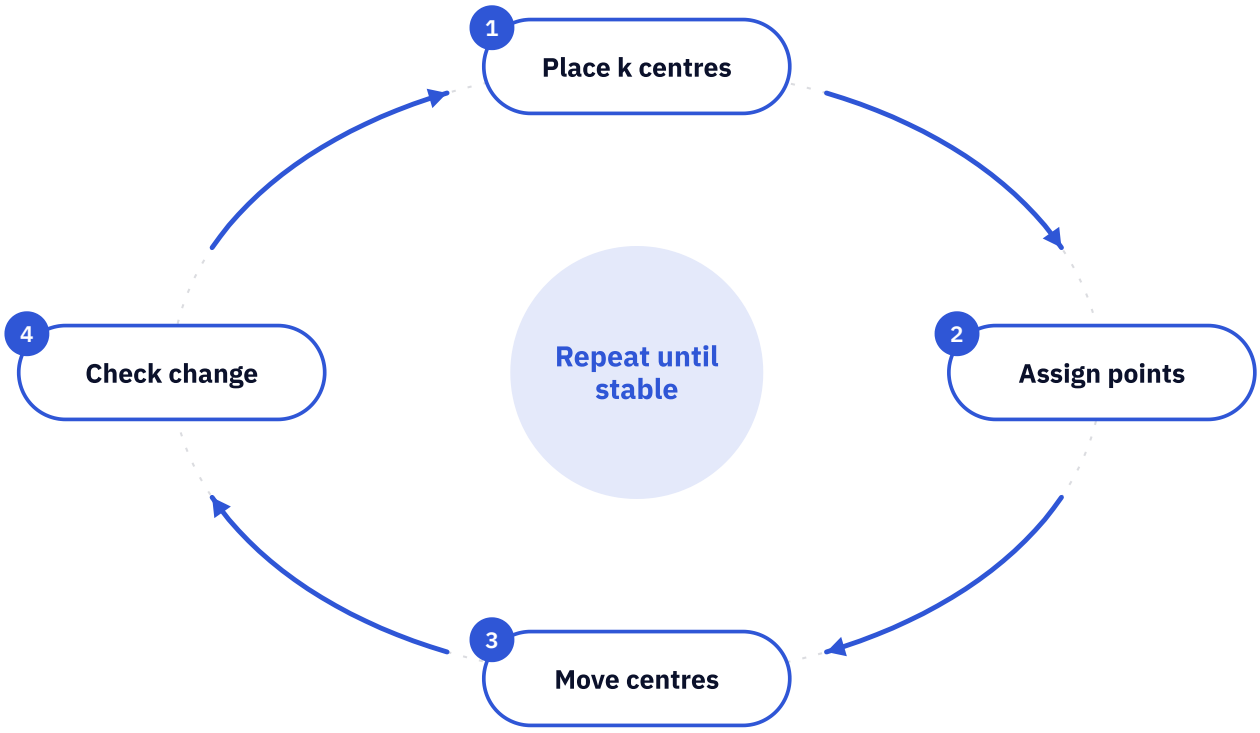
Much of the world's data has no labels. **Clustering** groups similar records together so that people can see structure: types of customers, patterns of machine behaviour, kinds of patients. Clusters are not ground truth; they are a lens. Their value depends on whether they lead to better decisions.

k-means in numbers

MEASURE	FORMULA	WHAT IT TELLS YOU
Objective (inertia)	$J = \Sigma_i \lVert x_i - \mu_{c(i)} \rVert^2$	Sum of squared distances from each point to its assigned cluster centre. k-means tries to minimise it.
Assignment step	$c(i) = \arg\min_k \lVert x_i - \mu_k \rVert$	Each point joins its nearest centre.
Update step	μ_k = mean of points assigned to cluster k	Centres move to the average of their members; repeat until assignments stop changing.
Worked iteration	Points 1, 2, 6, 8; starting centres 1 and 8	Assign {1, 2} and {6, 8}; new centres 1.5 and 7; inertia = 0.25 + 0.25 + 1 + 1 = 2.5.
Silhouette of a point	$s = (b - a) \div \max(a, b)$	a = mean distance within its cluster, b = mean distance to the nearest other cluster. For point 1: a = 1, b = (5 + 7) ÷ 2 = 6, so s = 5 ÷ 6 ≈ 0.83.

Copyright © ZyeemZia

The k-means loop



1. **Place k centres** — Smart random start (k-means++), several restarts
2. **Assign points** — Each point joins its nearest centre
3. **Move centres** — Each centre moves to the mean of its points
4. **Check change** — Stop when assignments no longer change

Each turn can only lower inertia, so the loop always settles, but possibly in a poor local minimum. That is why libraries run several restarts and keep the best.

Choosing k with silhouette scores

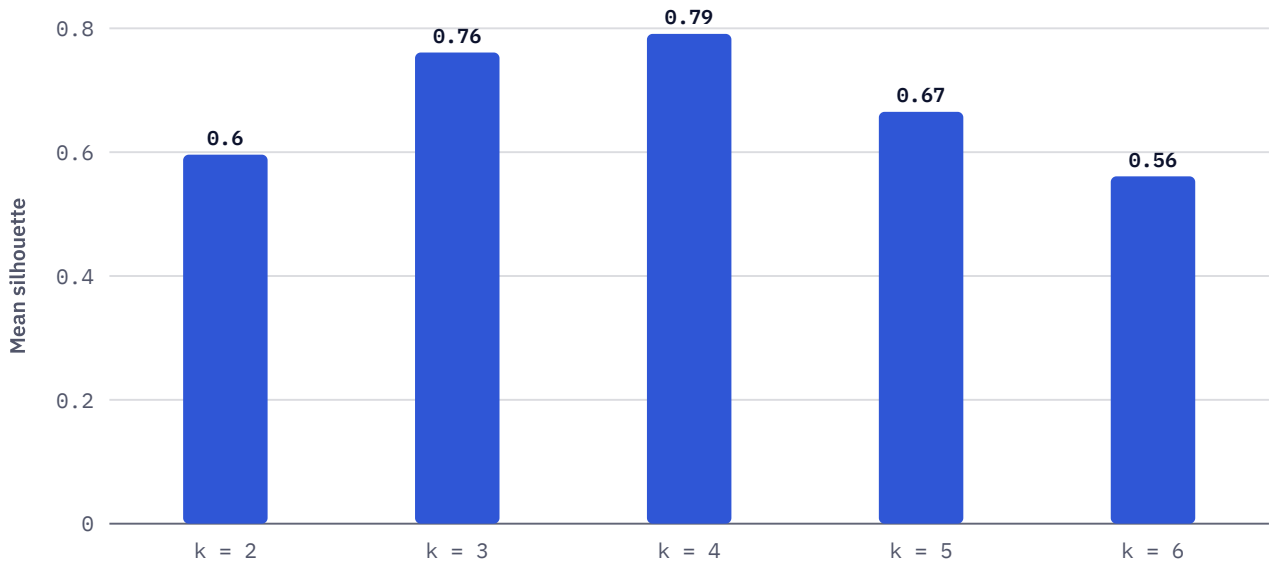
Python notebook

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
X, _ = make_blobs(n_samples=500, centers=4, cluster_std=1.0, random_state=42)
for k in [2, 3, 4, 5, 6]:
    km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)
    print(k, round(km.inertia_), round(silhouette_score(X, km.labels_), 3))
# output: 2 15737 0.596
# output: 3 3426 0.761
# output: 4 949 0.791
# output: 5 856 0.665
# output: 6 769 0.561
```

Inertia always falls as k rises, so look for the 'elbow' where gains flatten (here after k = 4). The silhouette score peaks at k = 4, matching the four groups that generated the data.

Copyright © ZyeemZia

Silhouette score by number of clusters (from the code above)



The score peaks at k = 4, the number of groups that generated the data. On real data the peak is rarely this clear, so combine it with whether the segments are useful.

Clustering methods compared

METHOD	FINDS	NEEDS	WATCH OUT
k-means	Round, similar-sized clusters	k in advance; scaled numeric features	Sensitive to outliers and initialisation; use several restarts
Hierarchical (agglomerative)	A tree of nested clusters (dendrogram)	A linkage rule (Ward, average, complete)	Slow on large data; cut the tree to choose clusters
DBSCAN	Dense regions of any shape; labels sparse points as noise	Radius ε and minimum points	Struggles when densities vary; ε is hard to set in high dimensions
Gaussian mixture	Overlapping, elliptical clusters with soft membership	Number of components	Can overfit; check with information criteria such as BIC

WATCH OUT · SCALE AND CHOOSE FEATURES FIRST

Clustering uses distances, so feature scaling and feature choice decide the result. Clustering customers on raw 'annual spend in taka' and 'number of children' will simply sort them by spend. Decide what similarity should mean for the business, select features that express it, then standardise.

From clusters to segments people can use

1. **Define the purpose** For example: tailor training for mobile money agents.
2. **Select and scale features** Transaction volume, cash-in/cash-out ratio, active hours, customer count, float shortages.
3. **Cluster and compare** Try several k and methods; check silhouette and stability across resamples.
4. **Profile** Describe each cluster with averages and typical examples; give it a plain name.
5. **Validate with experts** Field managers confirm the groups make sense and suggest actions.

Copyright © ZyeemZia

6. **Act and measure** Pilot a different intervention per segment and track outcomes.

CASE STUDY · SEGMENTING MOBILE MONEY AGENTS

A mobile financial service has 180,000 agents across Bangladesh. Head office wants to stop sending the same training and float-management messages to everyone. The analytics team has 12 months of agent data: daily transaction counts, cash-in and cash-out values, float top-ups, shortages, location type (urban, peri-urban, rural) and agent tenure.

1. Which features would you use, and how would you prepare them?

2. How would you decide the number of segments?

3. How would you show the segments add business value?

Model answer: module answer key.

SCENARIO · DECIDE · INTERPRETING A CLUSTER

A hospital clusters patients by length of stay, number of procedures and age. One cluster contains 8% of patients with very long stays. A manager wants to label it 'high-risk patients' in the electronic record. What is the best response?

- (A) Rerun k-means until the cluster disappears
- (B) Profile and validate the cluster with clinicians before naming it or using it in care decisions
- (C) Use the cluster as a training label for a supervised risk model
- (D) Label it immediately, since the data clearly separated it

Circle your choice. Answer and feedback: module answer key.

REFLECT

What groups exist in your organisation's customers, assets or users? Which features would define 'similar' for the decision you care about?

Key terms

Clustering
Grouping unlabelled records so that items in the same group are more similar than items in different groups.

k-means
A clustering algorithm that alternates between assigning points to the nearest centre and moving centres to their members' mean.

Copyright © ZyeemZia

Inertia
The sum of squared distances from points to their cluster centres; the k-means objective.

DBSCAN
A density-based clustering method that finds arbitrarily shaped clusters and marks sparse points as noise.

Gaussian mixture model
A probabilistic model that represents data as a blend of several normal distributions with soft memberships.

Silhouette score
A measure from −1 to 1 of how well each point fits its own cluster compared with the nearest other cluster.

Dendrogram
A tree diagram showing how hierarchical clustering merges groups.

Key takeaways

- Clusters are a lens, not ground truth; judge them by the decisions they improve.
- k-means suits round, similar-sized clusters on scaled features; DBSCAN finds odd shapes and noise.
- Inertia always falls with k; use the elbow, silhouette and business usefulness together.
- Feature choice and scaling define what "similar" means.

KNOWLEDGE CHECK

1. Points 2, 4, 10, 12 with starting centres 2 and 12. What are the centres after one k-means iteration?
   - (A) 3 and 11
   - (B) 2 and 12
   - (C) 4 and 10
   - (D) 7 and 7
2. A point has a = 2 and b = 8. What is its silhouette?
   - (A) 0.25
   - (B) 6
   - (C) 0.75
   - (D) −0.75
3. Why should you not pick k purely by minimising inertia?
   - (A) Inertia is undefined for any k above 3 clusters
   - (B) Inertia always decreases as k increases
   - (C) Inertia increases with k
   - (D) Inertia ignores distances
4. Which method can find crescent-shaped clusters and flag outliers as noise?
   - (A) k-means
   - (B) Linear regression
   - (C) PCA
   - (D) DBSCAN
5. What is the best evidence that customer segments are valuable?
   - (A) The silhouette score is above 0.5
   - (B) There are exactly five segments
   - (C) The clusters have similar sizes
   - (D) Tailored actions for each segment improve outcomes compared with a control group

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 6 · DISCOVER · LESSON 2 · 24 MIN

PCA and Dimensionality Reduction

Compress many correlated features into a few informative directions

- Explain what principal components are and how variance is explained
- Compute explained variance ratios from eigenvalues
- Use PCA for compression, noise reduction and visualisation
- Know when to use t-SNE or UMAP, and their limits

Datasets often have many features that move together: dozens of sensor readings on one turbine, hundreds of pixels in an image, survey questions that measure the same attitude. **Principal component analysis (PCA)** finds new axes, ordered by how much variation they capture, so that a few components can summarise many correlated features.

PCA essentials

MEASURE	FORMULA	WHAT IT TELLS YOU
Centre the data	$X_c = X - \mu$	Subtract each feature's mean (and usually standardise).
Covariance matrix	$C = X_c^\top X_c \div (n - 1)$	A d × d matrix of how features vary together.
Principal components	$C\ v = \lambda\ v$	Eigenvectors v are the new directions; eigenvalues λ are the variance along each one.
Explained variance ratio	$EVR_k = \lambda_k \div \Sigma\ \lambda$	Eigenvalues 4, 1 and 0.5 give 4 ÷ 5.5 ≈ 72.7% for the first component and (4 + 1) ÷ 5.5 ≈ 90.9% for the first two.
Projection	$z = W^\top (x - \mu)$	W holds the top k eigenvectors as columns; z is the compressed k-dimensional representation.

How many components keep 95% of the variance?

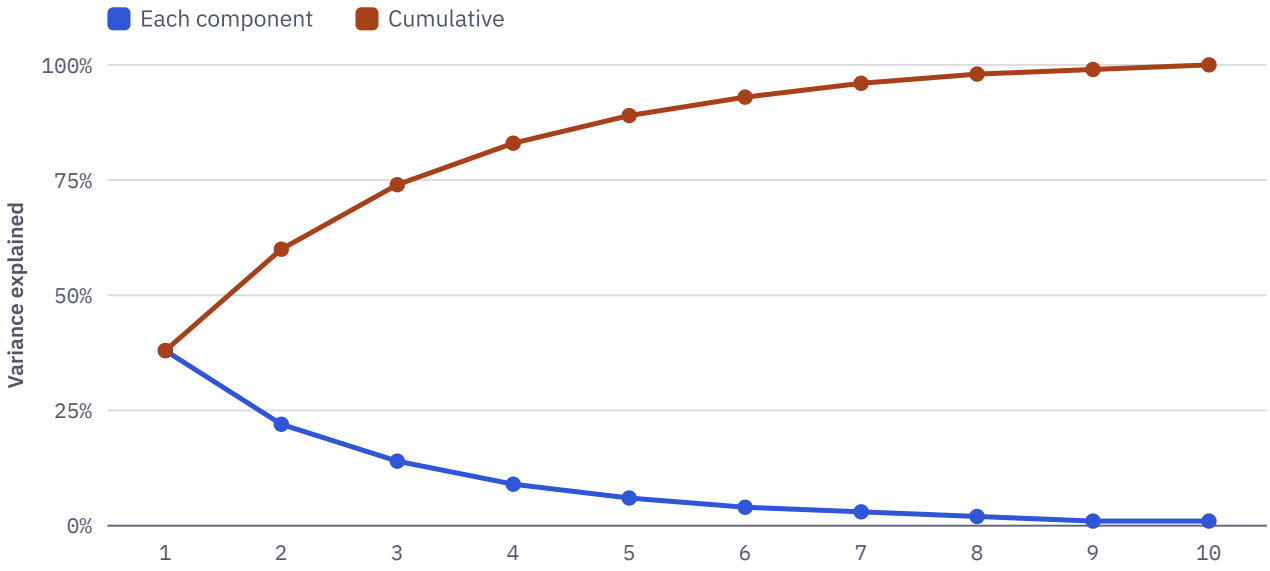
Python notebook

```
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
X, _ = load_digits(return_X_y=True)
pca = PCA(n_components=0.95).fit(X)
print(X.shape[1], pca.n_components_, round(pca.explained_variance_ratio_[:2].sum(), 3))
# output: 64 29 0.285
```

Images of handwritten digits have 64 pixel features, but 29 components retain 95% of the variance. The first two components alone keep only 28.5%, so a 2-D plot hides a lot of structure.

Copyright © ZyeemZia

Explained variance by component (illustrative)



Illustrative numbers (x-axis: component number). The first few components carry most of the variance; here 5 components keep 89% and 6 keep 93%, so a 90–95% rule suggests keeping 6.

Uses of dimensionality reduction

PCA (linear)

- Compression and faster training
- Removing correlated, redundant features
- Noise reduction by dropping small components
- Can be inverted to reconstruct data

t-SNE and UMAP (non-linear)

- 2-D or 3-D maps for visual exploration
- Preserve local neighbourhoods well
- Distances between far-apart clusters are not meaningful
- Not for use as model features without care

Practical tips

- Standardise features first, unless they share units and scale on purpose.
- Fit PCA on training data only, inside the pipeline, like any other preprocessing step.
- Choose the number of components by cumulative explained variance (for example 90–95%) or by downstream validation score.
- Components are combinations of all features, so they are harder to explain than original features.
- PCA captures linear structure only; curved structure needs non-linear methods or autoencoders.

EXAMPLE · CONDITION MONITORING IN A POWER PLANT

A combined-cycle plant in Chattogram records 140 sensor channels per turbine, many of them strongly correlated temperatures and pressures. Engineers used PCA on a period of healthy operation: about 12 components captured 97% of the variance. In live operation, a rising reconstruction error (the part of a reading that the 12 components cannot explain) gave early warning of unusual behaviour, which leads naturally into anomaly detection.

Copyright © ZyeemZia

SCENARIO · DECIDE · PCA BEFORE A CLASSIFIER

A data scientist applies PCA to the full dataset, keeps 20 components and then cross-validates a classifier on them. Results look good. What is the main methodological issue?

(A) PCA cannot be used before a classifier

(B) PCA must be applied to the labels as well

(C) PCA was fitted on all rows, so validation data influenced the components; fit it inside the cross-validated pipeline

(D) 20 components is always too many

Circle your choice. Answer and feedback: module answer key.

Using PCA responsibly

☐ Features are standardised unless they share meaningful units

☐ PCA is fitted on training data only, inside the pipeline

☐ The number of components is chosen by explained variance or validation score

☐ The loss of interpretability is acceptable for the use case

☐ 2-D maps from t-SNE or UMAP are used for exploration, not as evidence of distances

☐ Component loadings are inspected to understand what each component captures

REFLECT

Which of your datasets has many correlated features? What would you lose and gain by replacing them with a handful of principal components?

Key terms

Dimensionality reduction
Representing data with fewer features while preserving as much useful information as possible.

Principal component
A new axis formed as a weighted combination of the original features.

Explained variance ratio
The share of total variance captured by a component.

t-SNE
A non-linear technique that maps high-dimensional data to two or three dimensions for visualisation, preserving local neighbourhoods.

Principal component analysis
A linear method that finds orthogonal directions of maximum variance in the data.

Eigenvalue
In PCA, the amount of variance captured along a principal component.

Reconstruction error
The difference between original data and its reconstruction from a reduced representation.

Copyright © ZyeemZia

Key takeaways

- PCA finds orthogonal directions of maximum variance and keeps the few that matter.
- Standardise first and fit PCA inside the pipeline on training data only.
- Choose components by cumulative explained variance or by downstream validation score.
- t-SNE and UMAP are for visual exploration; distances and gap sizes in them are not reliable.

KNOWLEDGE CHECK

6. Eigenvalues are 6, 3 and 1. What share of variance do the first two components explain?

   (A) 60%
   (B) 90%
   (C) 30%
   (D) 100%

7. Why standardise before PCA on features with different units?

   (A) PCA cannot handle negative numbers
   (B) It doubles the number of components available, so more variance can be explained
   (C) Otherwise features with large numeric ranges dominate the components
   (D) It removes the need to centre the data

8. What does a principal component represent?

   (A) A direction of maximum variance, orthogonal to earlier components
   (B) A cluster of similar rows
   (C) A single original feature, chosen because it has the highest correlation with the target
   (D) A decision boundary between classes

9. A t-SNE map shows two clusters far apart. What can you safely conclude?

   (A) The clusters are exactly as far apart in the original data as the map shows
   (B) Points in each cluster are locally similar; the gap size is not reliable
   (C) The data has exactly two classes
   (D) PCA would show the same picture

10. Which use of PCA is most appropriate?

    (A) Making a model's features easier to explain to auditors and regulators
    (B) Creating labels for supervised learning
    (C) Encoding categorical variables
    (D) Compressing many correlated sensor readings before modelling

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 6 · DISCOVER · LESSON 3 · 24 MIN

Anomaly Detection

Find the rare, the strange and the dangerous when labels are scarce

- Distinguish point, contextual and collective anomalies
- Apply statistical, isolation-based, density-based and reconstruction methods
- Evaluate anomaly detectors with precision at k and analyst feedback
- Design a detection workflow that learns from investigated alerts

Fraud, equipment faults, network intrusions and data-entry errors share two traits: they are **rare**, and they are **different**. Labels are usually scarce, so anomaly detection often starts unsupervised, scoring how unusual each record is, and becomes supervised as investigators confirm cases.

Kinds of anomaly

TYPE	MEANING	EXAMPLE
Point anomaly	A single record is unusual on its own	A 5,00,000 BDT wallet transfer from a student account
Contextual anomaly	Unusual given its context, normal elsewhere	35 °C in a hospital cold store, normal outdoors in summer
Collective anomaly	A sequence or group is unusual, each item may not be	Fifty small transfers to new recipients within ten minutes

Detection methods

METHOD	IDEA	GOOD FOR
Statistical rules (z-score, IQR, control charts)	Flag values far from typical	Single metrics, fast baselines
Isolation forest	Anomalies are isolated by fewer random splits	Tabular data, many features, fast
Local outlier factor	Compares a point's local density with its neighbours'	Clusters of different density
Reconstruction error (PCA, autoencoders)	Normal data reconstructs well; anomalies do not	Sensor and image data
Supervised classifier	Learns from confirmed anomalies	When enough labelled cases exist

Scoring and evaluating anomalies

MEASURE	FORMULA	WHAT IT TELLS YOU
Isolation idea	`score ↑ as average path length E[h(x)] ↓`	Points that random trees isolate in few splits get high anomaly scores.
Reconstruction error	$e = \lVert x - \hat{x} \rVert^2$	$\hat{x}$ is the reconstruction from PCA or an autoencoder trained on normal data.

Copyright © ZyeemZia

MEASURE	FORMULA	WHAT IT TELLS YOU
Precision at k	`P@k = confirmed anomalies in top k ÷ k`	If analysts review the top 50 alerts and confirm 12, P@50 = 0.24.
Recall at k	`R@k = confirmed anomalies in top k ÷ all known anomalies`	With 20 known frauds in the period, R@50 = 12 ÷ 20 = 0.6.

Isolation forest on planted anomalies

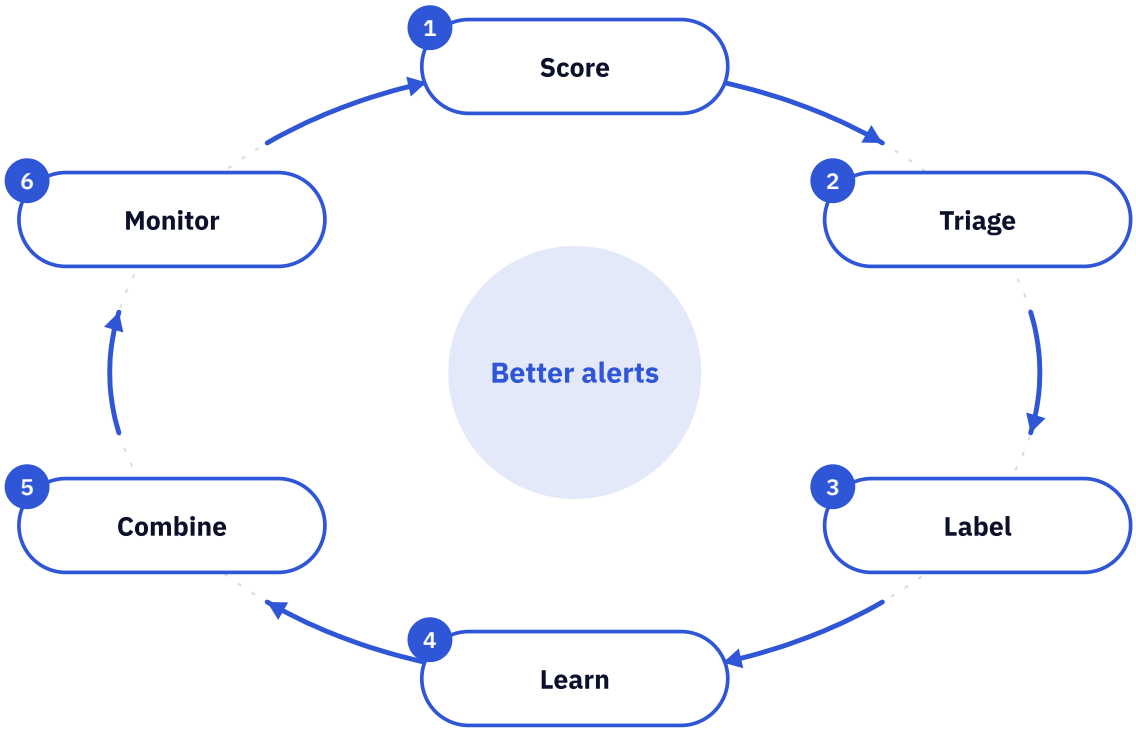
Python notebook

```
import numpy as np
from sklearn.ensemble import IsolationForest
rng = np.random.default_rng(1)
X = np.vstack([rng.normal(0, 1, size=(990, 2)), rng.uniform(-6, 6, size=(10, 2))])
y = np.r_[np.zeros(990), np.ones(10)]
iso = IsolationForest(n_estimators=200, random_state=0).fit(X)
score = -iso.score_samples(X)
top20 = np.argsort(score)[::-1][:20]
print('anomalies in top 20:', int(y[top20].sum()))
# output: anomalies in top 20: 5
```

Ten anomalies were drawn uniformly over a wide square; some happened to land inside the normal cloud and are genuinely indistinguishable. Precision at 20 is 0.25, recall 0.5, which is typical: detectors rank cases for human review rather than give final answers.

Copyright © ZyeemZia

A detection loop that improves over time



1. **Score** — Unsupervised detector ranks records
2. **Triage** — Analysts review the top alerts
3. **Label** — Record confirmed and false alarms
4. **Learn** — Train a supervised model on labels
5. **Combine** — Blend both scores; tune to capacity
6. **Monitor** — Track precision and new patterns

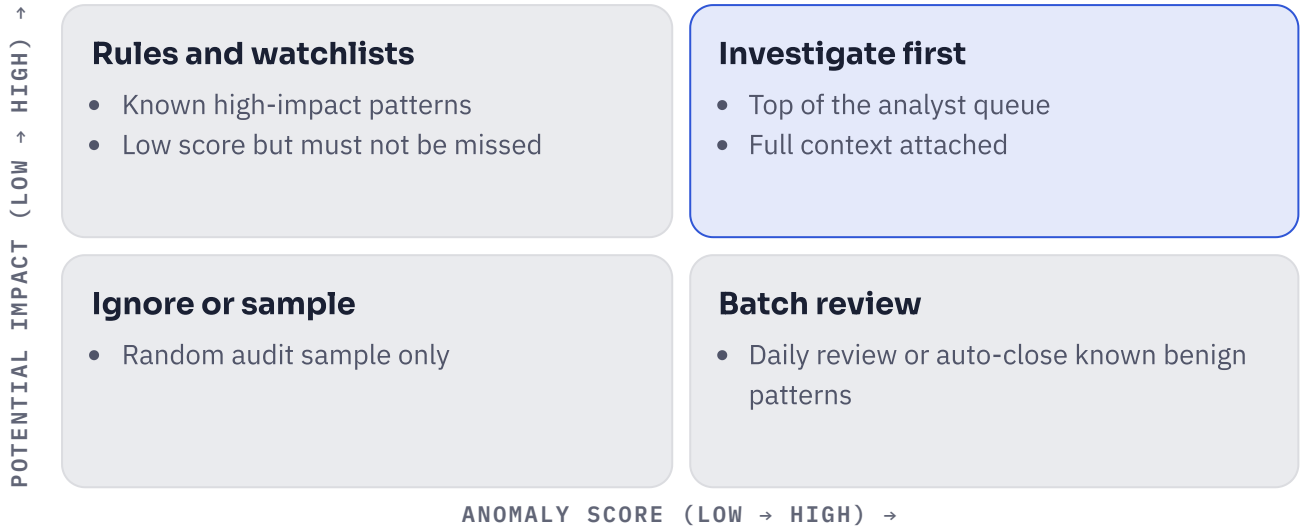
Each pass turns analyst decisions into labels, so the system moves from purely unsupervised scoring towards a supervised model tuned to review capacity.

PRO TIP · CONTEXT BEATS CLEVERNESS

A transaction that is unusual for the whole population may be normal for that customer. Features that compare a record with the entity's own history (amount ÷ customer's 90-day median, new device, new recipient, unusual hour for this customer) often matter more than the choice of algorithm.

Copyright © ZyeemZia

Triage: where each alert should go



Ranking by score alone mixes trivial oddities with costly ones. Combining score with impact keeps scarce analyst time on the top-right cell.

SCENARIO · DECIDE · SENSOR ALERTS AT A WATER TREATMENT PLANT

An anomaly detector on pump sensors produces 300 alerts a day. Operators can review 20 and have started ignoring all alerts. What is the best improvement?

- (A) Rank alerts by score, show only the top 20 with context, and use operator feedback to retrain and tune
- (B) Lower the threshold so that even more anomalies are caught
- (C) Send all 300 alerts to senior management instead
- (D) Switch the detector off until a perfect model is available

Circle your choice. Answer and feedback: module answer key.

REFLECT

Where in your organisation could rare events be caught earlier? How many alerts per day could people realistically review, and how would their feedback be captured?

Key terms

Anomaly detection
Identifying records that differ markedly from the majority of the data.

Collective anomaly
A group or sequence of records that is unusual together.

Contextual anomaly
A value that is unusual only given its context, such as time, place or entity.

Isolation forest
An ensemble that scores anomalies by how few random splits are needed to isolate a point.

Copyright © ZyeemZia

Local outlier factor
A score comparing a point's local density with that of its neighbours.

Alert fatigue
The tendency of people to ignore alerts when too many are false or irrelevant.

Autoencoder
A neural network trained to reconstruct its input through a narrow bottleneck; poor reconstruction suggests an anomaly.

Key takeaways

- Anomalies are rare and different: point, contextual or collective.
- Start unsupervised (isolation forest, reconstruction error), then learn from analyst labels.
- Compare each record with its own entity history, not only with the population.
- Rank alerts to fit review capacity, and measure precision at that capacity.

KNOWLEDGE CHECK

11. Analysts review the top 40 alerts and confirm 10. What is precision at 40?

(A) 0.25
(B) 0.40
(C) 0.10
(D) 4.00

12. Why does an isolation forest give anomalies high scores?

(A) They have the largest feature values
(B) They always belong to the minority class label, which the forest is told in advance
(C) They are separated from other points with fewer random splits
(D) They have the most neighbours

13. A cold-store temperature of 30 °C is normal outdoors but alarming in the store. What kind of anomaly is it?

(A) Point
(B) Collective
(C) Contextual
(D) Not an anomaly

14. An autoencoder is trained on normal machine vibration. How does it flag anomalies?

(A) Abnormal inputs reconstruct poorly, giving high reconstruction error
(B) It predicts the failure date directly
(C) It clusters inputs into exactly two groups
(D) It outputs class labels from supervised training

15. What is the most valuable use of analyst decisions on alerts?

(A) Deleting them once the alert is closed
(B) Using them only for staff performance reviews
(C) Ignoring them, because an unsupervised detector cannot learn from labels
(D) Turning them into labels to train and tune detection models

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 6 · DISCOVER · LESSON 4 · 26 MIN

Recommender Systems

Suggest the right item to the right person, and measure whether it helps

- Compare popularity, content-based and collaborative filtering approaches
- Explain matrix factorisation and compute a predicted rating
- Evaluate recommendations with precision@k, recall@k and NDCG
- Handle cold start, implicit feedback and feedback loops

Recommenders decide what millions of people see: products in an online shop, videos, news, courses, even loan offers. They work with a large, sparse matrix of **users × items**, where only a tiny fraction of entries (purchases, clicks, ratings) are known, and they try to fill in the rest.

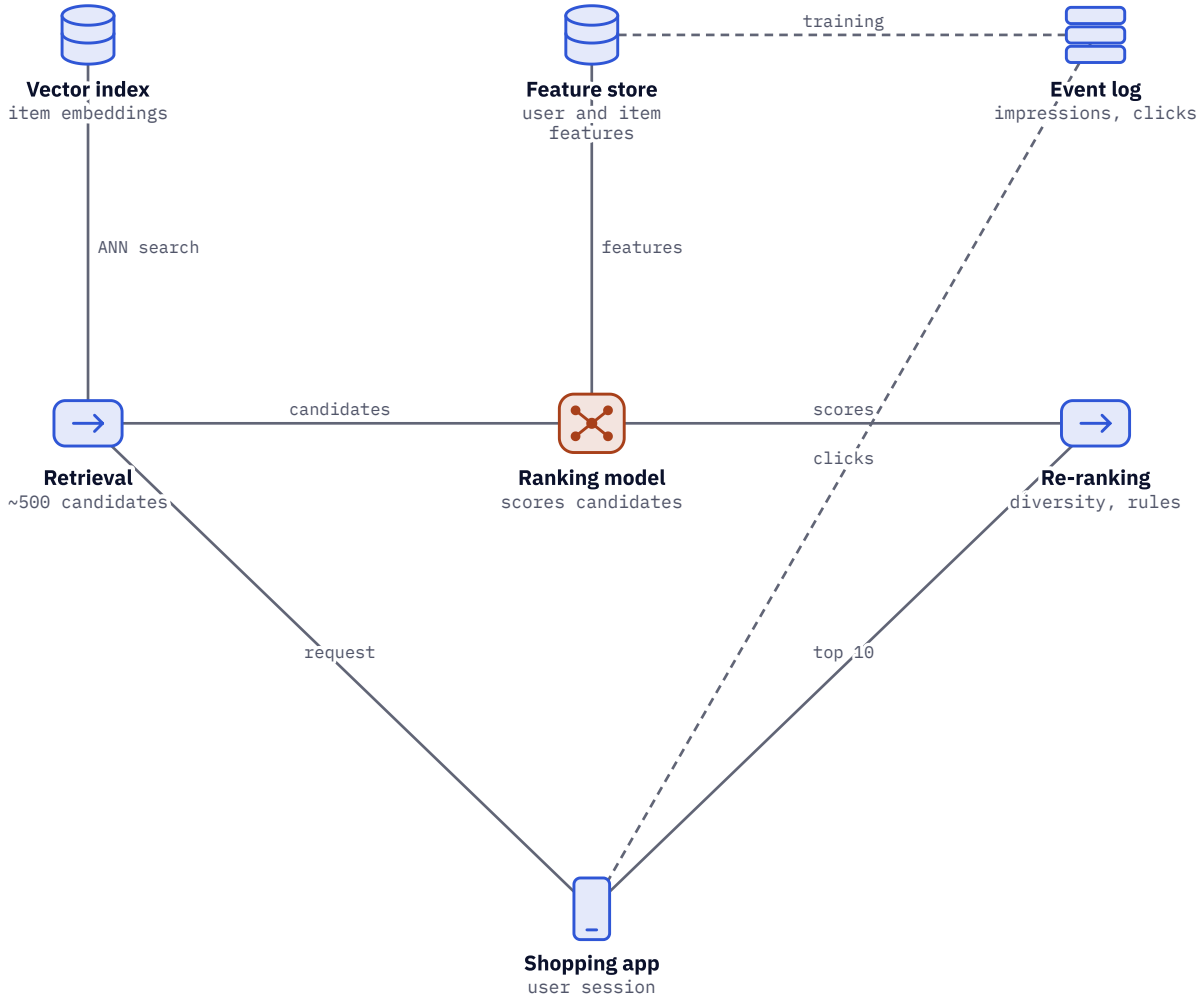
Main approaches

APPROACH	HOW IT WORKS	STRENGTH	WEAKNESS
Popularity	Recommend the most popular items, perhaps by segment	Strong baseline, no cold-start problem for users	Not personal; reinforces popular items
Content-based	Recommend items similar to what the user liked, using item attributes or embeddings	Works for new items with descriptions	Narrow, 'more of the same' suggestions
Collaborative filtering	Use patterns of many users: people who liked X also liked Y	Discovers non-obvious links	Cold start for new users and items
Hybrid and two-stage	Retrieve candidates cheaply, then rank them with a richer model	Scales to huge catalogues, uses all signals	More engineering complexity

Large catalogues use **two stages**. **Retrieval** narrows millions of items to a few hundred candidates, often with a two-tower model that embeds users and items in the same vector space and an **approximate nearest-neighbour (ANN)** index that finds the closest item vectors in milliseconds. **Ranking** then scores those candidates with a richer model and many features, and a final re-ranking step adds diversity, freshness and business rules.

Copyright © ZyeemZia

A two-stage recommender in production



Cheap retrieval keeps latency low, the ranking model spends compute only on candidates, and logged impressions and clicks feed the next training run. Logging what was shown, not just what was clicked, is what makes later evaluation possible.

Collaborative filtering arithmetic

MEASURE	FORMULA	WHAT IT TELLS YOU
Matrix factorisation	$\hat{r}_{ui} = \mu + b_u + b_i + p_u \cdot q_i$	μ global mean, b_u user bias, b_i item bias, p_u and q_i learned factor vectors.
Worked prediction	$\mu = 3.5,\ b_u = 0.3,\ b_i = -0.2,\ p_u = (1.0, 0.5),\ q_i = (0.8, -0.4)$	p_u · q_i = 0.8 − 0.2 = 0.6, so $\hat{r}$ = 3.5 + 0.3 − 0.2 + 0.6 = 4.2.
Item–item cosine	$\text{sim}(i, j) = (r_i \cdot r_j) \div (\lVert r_i \rVert \lVert r_j \rVert)$	Compares two items by the users who interacted with them.
Precision@k	relevant items in top k ÷ k	3 relevant in the top 5 gives 0.6.
DCG@k and NDCG	$\text{DCG} = \Sigma\ \text{rel}_i \div \log_2(i + 1)$; NDCG = DCG ÷ ideal DCG	Rewards placing relevant items near the top of the list.

Copyright © ZyeemZia

Item–item similarity on a tiny interaction matrix

Python notebook

```
import numpy as np
# rows = users, columns = courses: Python, Statistics, Excel, Design
R = np.array([[1, 1, 0, 0], [1, 1, 1, 0], [0, 1, 1, 0], [0, 0, 0, 1]])
norms = np.linalg.norm(R, axis=0)
S = (R.T @ R) / np.outer(norms, norms)
print(S[0].round(2).tolist())
# output: [1.0, 0.82, 0.5, 0.0]
user = np.array([1, 0, 0, 0])
scores = S @ user; scores[user == 1] = -1
print(int(scores.argmax()))
# output: 1
```

A learner who took Python is recommended Statistics (column 1), the course most often taken by the same people. Design shares no learners with Python, so its similarity is 0.

Explicit versus implicit feedback

Explicit

- Ratings, likes, reviews
- Clear preference signal
- Sparse: few people rate
- Self-selection bias

Implicit

- Clicks, views, purchases, watch time
- Plentiful
- Absence is ambiguous: not seen or not liked?
- Needs weighting and negative sampling

Problems every recommender meets

- **Cold start**: new users and items have no history; use content features, popularity by segment, or onboarding questions.
- **Feedback loops**: items shown more get clicked more and are shown even more; add exploration and diversity.
- **Popularity bias**: niche items starve; re-rank for diversity and coverage.
- **Offline–online gap**: offline metrics on past logs reflect what the old system showed; confirm with A/B tests.
- **Business rules**: stock, margins, fairness to sellers and safety filters sit on top of the model.

SCENARIO · DECIDE · RECOMMENDING COURSES TO NEW STUDENTS

A university e-learning platform in Dhaka wants recommendations for first-year students who have just enrolled and have no activity yet. Collaborative filtering is planned. What is the best approach for these new users?

(A) Show random courses until each student has 50 interactions

(B) Use collaborative filtering only; it will work from the first day

(C) Wait until the end of the first semester before recommending

(D) Start with content and context: programme, chosen modules and popular courses among similar students, then blend in collaborative filtering as activity grows

Circle your choice. Answer and feedback: module answer key.

TEMPLATE **Recommender evaluation plan**

Agree offline and online measures, guardrails and baselines before launching a recommender.

Copyright © ZyeemZia

MEASURE	TYPE	TARGET OR GUARDRAIL	BASELINE	HOW MEASURED
Recall@20 of next-week purchases	*Offline*	*≥ 0.25*	*Popularity by category: 0.17*	*Time-split holdout of the last 4 weeks*
Revenue per session	*Online*	*+3% vs control*	*Current homepage*	*Two-week A/B test, 50/50 split*

REFLECT

Where does your organisation decide what to show or offer to people? Which feedback signals do you have, and what feedback loop could a recommender create?

Key terms

Recommender system
A system that predicts which items a user is likely to find relevant and ranks them.

Content-based filtering
Recommending items whose attributes resemble those a user liked before.

Cold start
The difficulty of recommending for new users or items that have little or no interaction history.

NDCG
Normalised discounted cumulative gain: a ranking metric that rewards relevant items placed near the top.

Collaborative filtering
Recommending from patterns in many users' interactions, without needing item content.

Matrix factorisation
Representing users and items as low-dimensional vectors whose dot product predicts interaction.

Implicit feedback
Behavioural signals such as clicks or purchases used as indirect evidence of preference.

Copyright © ZyeemZia

Key takeaways

- Recommenders fill in a sparse user × item matrix from explicit or implicit feedback.
- Popularity is a strong baseline; content-based methods solve item cold start; collaborative filtering finds non-obvious links.
- At scale, embedding-based retrieval with ANN search feeds a richer ranking model.
- Watch feedback loops and popularity bias; confirm offline gains with online experiments.

KNOWLEDGE CHECK

16. $\mu = 4.0$, $b_u = -0.5$, $b_i = 0.2$ and $p_u \cdot q_i = 0.3$. What is the predicted rating?

(A) 4.5 (B) 4.0
(C) 3.5 (D) 4.3

17. Two relevant items appear in a top-10 list. What is precision@10?

(A) 0.2 (B) 0.1
(C) 2 (D) 0.5

18. Which approach handles brand-new items with good descriptions best?

(A) Pure collaborative filtering (B) Popularity of past items only
(C) Content-based filtering (D) User–user similarity

19. Why is a missing click ambiguous in implicit feedback?

(A) Clicks are always random (B) The user may not have seen the item rather than disliked it
(C) Missing values usually mean that the item was out of stock at the time (D) It proves the user disliked the item

20. A recommender keeps showing the same popular products, and their sales grow while others shrink. What is this?

(A) Cold start caused by new products joining (B) Data leakage
(C) Underfitting (D) A feedback loop with popularity bias

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 6 · DISCOVER · LESSON 5 · 26 MIN

Reinforcement Learning: An Introduction

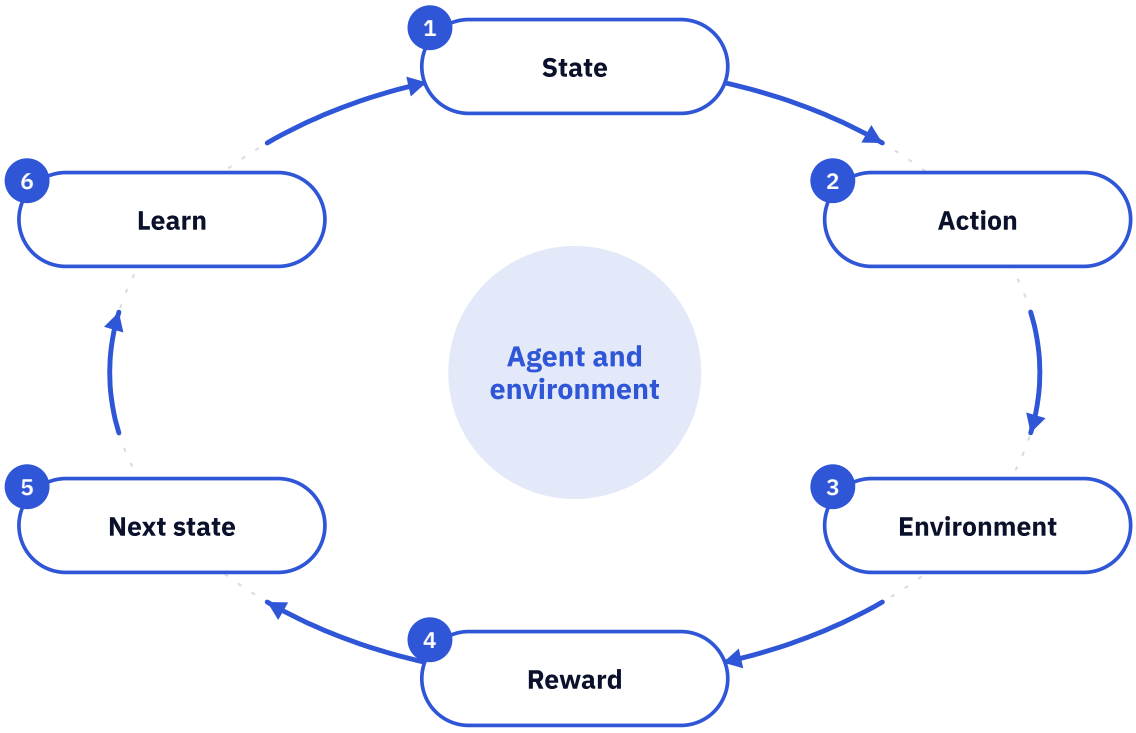
Learning to act by trial, error and reward

- Describe agents, environments, states, actions, rewards and policies
- Compute discounted returns and a Q-learning update
- Explain the exploration–exploitation trade-off and multi-armed bandits
- Judge when reinforcement learning is practical and safe

Supervised learning is told the right answer. **Reinforcement learning (RL)** is not: an agent takes actions, sees what happens and receives rewards, and must discover which actions lead to the most reward over time. RL powers game-playing systems, robotics, some recommendation and pricing systems, and the fine-tuning of language models from human preferences.

Copyright © ZyeemZia

The reinforcement learning loop



1. **State** — What the agent observes now
2. **Action** — What it chooses to do
3. **Environment** — The world responds
4. **Reward** — A number saying how good that was
5. **Next state** — The loop repeats
6. **Learn** — Update the policy to get more reward

The agent never sees the right answer, only rewards that may arrive late. Learning means adjusting the policy so that, over many turns of this loop, total discounted reward rises.

Core RL quantities

MEASURE	FORMULA	WHAT IT TELLS YOU
Discounted return	$G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2\, r_{t+3} + \dots$	γ between 0 and 1 discounts future rewards. Rewards 1, 0, 2 with $\gamma = 0.9$ give $1 + 0 + 2 \times 0.81 = 2.62$.
Action value	$Q(s, a)$ = expected return after taking a in s and following the policy	Tells the agent how good each action is in each state.
Q-learning update	$Q(s, a) \leftarrow Q(s, a) + \alpha\,[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$	Move the estimate towards the reward plus the best estimated future value.

Copyright © ZyeemZia

MEASURE	FORMULA	WHAT IT TELLS YOU
Worked update	`Q = 2, r = 1, γ = 0.9, max Q(s′, ·) = 3, α = 0.5`	Target = 1 + 2.7 = 3.7; new Q = 2 + 0.5 × 1.7 = 2.85.
ε-greedy policy	`With probability ε explore at random, otherwise take the best-known action`	ε = 0.1 means one action in ten is exploratory.

An ε-greedy bandit learning which offer works best

Python notebook

```
import numpy as np
rng = np.random.default_rng(0)
true_rates = [0.05, 0.11, 0.08]          # unknown conversion rate of three offers
counts, values = np.zeros(3), np.zeros(3)
for t in range(5000):
    a = rng.integers(3) if rng.random() < 0.1 else int(values.argmax())
    reward = float(rng.random() < true_rates[a])
    counts[a] += 1
    values[a] += (reward - values[a]) / counts[a]
print(counts.astype(int).tolist(), values.round(3).tolist())
# output: [712, 4049, 239] [0.049, 0.105, 0.075]
```

The agent spends most of its trials on the offer with the highest estimated conversion while still exploring the others occasionally. This is a multi-armed bandit, the simplest RL setting, with no states, only actions and rewards.

Bandits versus full reinforcement learning

Multi-armed bandit

- No state, or simple context
- Each action's reward comes immediately
- Used for adaptive A/B tests, offers, headlines
- Practical and widely deployed

Full RL (sequential decisions)

- Actions change future states
- Rewards may be delayed many steps
- Used in robotics, control, games, operations
- Needs simulators or very safe exploration

WATCH OUT · REWARD DESIGN IS EVERYTHING

An agent optimises exactly the reward it is given. Reward a call-centre routing agent for 'short calls' and it may learn to transfer difficult customers away. Specify rewards that reflect the true goal, add constraints for safety, and test in simulation before any real-world exploration.

INSIGHT · REWARD LEARNING IN LANGUAGE MODELS

Reinforcement learning ideas also shape modern chat models. After pre-training, models are tuned on human or model-generated preference comparisons, either by training a reward model and optimising against it with RL (RLHF) or with direct preference methods such as DPO that learn from the preference pairs without a separate reward model. The same warning applies: the model optimises the measured preference, which can reward confident or flattering answers over correct ones.

When is RL a sensible choice?

QUESTION	FAVOURABLE ANSWER	IF NOT
Are decisions sequential, with actions affecting the future?	Yes	Supervised learning or bandits are simpler

Copyright © ZyeemZia

QUESTION	FAVOURABLE ANSWER	IF NOT
Is there a simulator or a safe way to explore?	Yes, e.g. a building energy simulator	Exploration may be too costly or dangerous
Can the goal be expressed as a reward?	Clearly and measurably	The agent may game a poor proxy
Is there enough interaction data or compute?	Yes	Classical control or heuristics may win

SCENARIO · DECIDE · AIR-CONDITIONING CONTROL IN A HOSPITAL

A hospital in Sylhet wants to reduce chiller energy use with RL that adjusts set points every 15 minutes. Operating theatres must stay within strict temperature and humidity limits. What is the most responsible plan?

(A) Reward the agent only for lower energy use

(B) Train and test in a calibrated building simulator with hard safety limits, then deploy gradually with human override

(C) Let the agent explore freely on the live system so that it learns faster

(D) Use supervised learning to copy the current manual settings

Circle your choice. Answer and feedback: module answer key.

TEMPLATE **Reward design worksheet**

Make the goal, reward, constraints and failure modes of an RL or bandit system explicit before training.

ELEMENT	DEFINITION	HOW IT COULD BE GAMED	SAFEGUARD
Reward	*Energy saved per hour (kWh)*	*Let theatres warm up to save energy*	*Large penalty for any breach of temperature or humidity limits*
Action space	*Chiller set point 6–10 °C in 0.5 °C steps*	*Rapid oscillation wears equipment*	*Limit changes to one step per 15 minutes*

Copyright © ZyeemZia

REFLECT

Name one sequential decision in your field, such as inventory reordering or treatment scheduling. What would the state, actions and reward be, and what could go wrong if the reward were slightly mis-specified?

Key terms

Agent
The learner and decision-maker in reinforcement learning.

Environment
Everything the agent interacts with, which responds to actions with new states and rewards.

Policy
The agent's rule for choosing actions given a state.

Discount factor
The parameter γ that sets how much future rewards count relative to immediate ones.

Q-learning
An algorithm that learns action values by bootstrapping from the best estimated next-state value.

Exploration–exploitation trade-off
Balancing trying new actions to learn against using the best-known action to earn reward.

Multi-armed bandit
A simplified RL problem with actions and immediate rewards but no state transitions.

Key takeaways

- RL learns a policy from rewards through trial and error; rewards may be delayed.
- Bandits handle immediate-reward choices and are widely practical; full RL suits sequential decisions with a simulator.
- Balance exploration and exploitation, for example with ε-greedy.
- An agent optimises exactly the reward given: design it carefully, add constraints and test in simulation.
- Preference optimisation (RLHF, DPO) applies the same ideas, and the same risks, to language models.

KNOWLEDGE CHECK

21. Rewards 2, 2, 2 with γ = 0.5. What is the discounted return from the first step?

(A) 6
(B) 3
(C) 2.5
(D) 3.5

22. Q = 5, r = 0, γ = 1, max Q(s′, ·) = 4, α = 0.5. What is the new Q?

(A) 5
(B) 4.5
(C) 4
(D) 2.5

23. What does ε = 0.2 in an ε-greedy policy mean?

(A) The agent is 20% accurate
(B) Rewards are discounted by 20%
(C) About 20% of actions are chosen at random to explore
(D) The learning rate is 0.2

Copyright © ZyeemZia

24. Which problem suits a multi-armed bandit best?

(A) Choosing which of four promotional messages to show to maximise clicks

(B) Controlling a robot arm through a long sequence of moves to pick up parcels

(C) Clustering customers into segments

(D) Predicting house prices from features

25. Why is a simulator so valuable in RL?

(A) It removes the need for a reward

(B) The agent can explore and fail safely and cheaply before acting in the real world

(C) It guarantees that the learned policy will be optimal when transferred to reality

(D) It turns RL into supervised learning

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 6 · DISCOVER

Answer key

Knowledge checks

1. A Assign {2, 4} and {10, 12}; means are 3 and 11.

2. C (8 − 2) ÷ 8 = 0.75.

3. B With k = n, inertia is zero; use the elbow, silhouette, stability and business usefulness.

4. D Density-based clustering is not limited to round clusters.

5. D Business value is shown by better decisions, ideally tested against a control.

6. B (6 + 3) ÷ 10 = 0.9.

7. C Variance depends on units, so unscaled large-range features take over.

8. A Components are ordered directions of maximum variance.

9. B t-SNE preserves local neighbourhoods but distorts global distances.

10. D PCA excels at compressing correlated numeric features; it reduces interpretability.

11. A 10 ÷ 40 = 0.25.

12. C Short average path lengths in random trees indicate easy isolation.

13. C It is unusual only in its context.

14. A Reconstruction error rises for patterns it has not learned.

15. D Feedback converts an unsupervised start into an improving supervised system.

16. B 4.0 − 0.5 + 0.2 + 0.3 = 4.0.

17. A 2 ÷ 10 = 0.2.

18. C Content features exist before any interactions do.

19. B Absence of interaction mixes 'not exposed' with 'not interested'.

20. D Exposure drives clicks, which drive more exposure; exploration and diversity help.

21. D 2 + 2 × 0.5 + 2 × 0.25 = 3.5.

22. B Target = 0 + 4 = 4; new Q = 5 + 0.5 × (4 − 5) = 4.5.

23. C ε is the exploration probability.

24. A Actions with immediate rewards and no state transitions fit a bandit.

25. B Exploration in the real world can be costly or dangerous.

Scenarios & case studies

Case study · Segmenting mobile money agents

Use behavioural features that relate to the decision, such as average daily transactions, cash-in/cash-out ratio, float shortage frequency, peak-hour share and tenure; log-transform skewed volumes and standardise everything, keeping location type for profiling rather than clustering if the aim is behaviour-based groups. Compare k from about 3 to 8 using silhouette, inertia and stability across bootstrap samples, then choose a number field teams can act on (often 4–6). Profile and name each segment, review with regional managers, and run a pilot in which each segment receives tailored training or float alerts while a control group receives the standard messages, measuring shortages, transaction growth and agent churn.

Scenario · Interpreting a cluster

B is best. Correct. Clusters describe similarity in the chosen features, not clinical risk. Experts must confirm what the group represents before it influences care.

Scenario · PCA before a classifier

C is best. Correct. Like scaling or feature selection, PCA learns from data and must be fitted on training folds only.

Scenario · Sensor alerts at a water treatment plant

A is best. Correct. Matching alert volume to capacity and learning from feedback restores trust and improves precision over time.

Scenario · Recommending courses to new students

D is best. Correct. New users have no interaction history, so collaborative filtering alone cannot help. Programme-based popularity and content features bridge the cold start.

Copyright © ZyeemZia

Scenario · Air-conditioning control in a hospital

B is best. Correct. A simulator allows exploration without risk, constraints protect patients, and staged deployment with override builds confidence.

Copyright © ZyeemZia

07

MODULE 7 · DEEP

Neural Networks and Deep Learning

Build intuition for neural networks, CNNs, sequence models and transformers, and train them well

BY THE END OF THIS MODULE YOU WILL BE ABLE TO

- Compute forward passes and parameter counts for neural networks
- Train deep networks stably with backpropagation, normalisation and regularisation
- Apply convolutional networks and transfer learning to image problems
- Model sequences with embeddings, recurrent networks and lag features
- Explain attention and transformers and choose how to adapt foundation models

LESSONS

1. Neurons, Layers and Activations · 24 min
2. Training Deep Networks · 26 min
3. Convolutional Neural Networks · 24 min
4. Sequence Models and Embeddings · 24 min
5. Transformers, Attention and Foundation Models · 34 min

MODULE 7 · DEEP · LESSON 1 · 24 MIN

Neurons, Layers and Activations

How a neural network turns inputs into predictions, one layer at a time

- Describe a neuron as a weighted sum followed by an activation
- Compute a forward pass through a small network by hand
- Compare common activation functions and output layers
- Count the parameters of a fully connected network

A **neural network** is a stack of simple steps. Each layer multiplies its inputs by a weight matrix, adds a bias and applies a non-linear **activation**. Stack enough layers and the network can represent remarkably complex functions, from recognising a crack in a fabric image to translating a Bangla sentence.

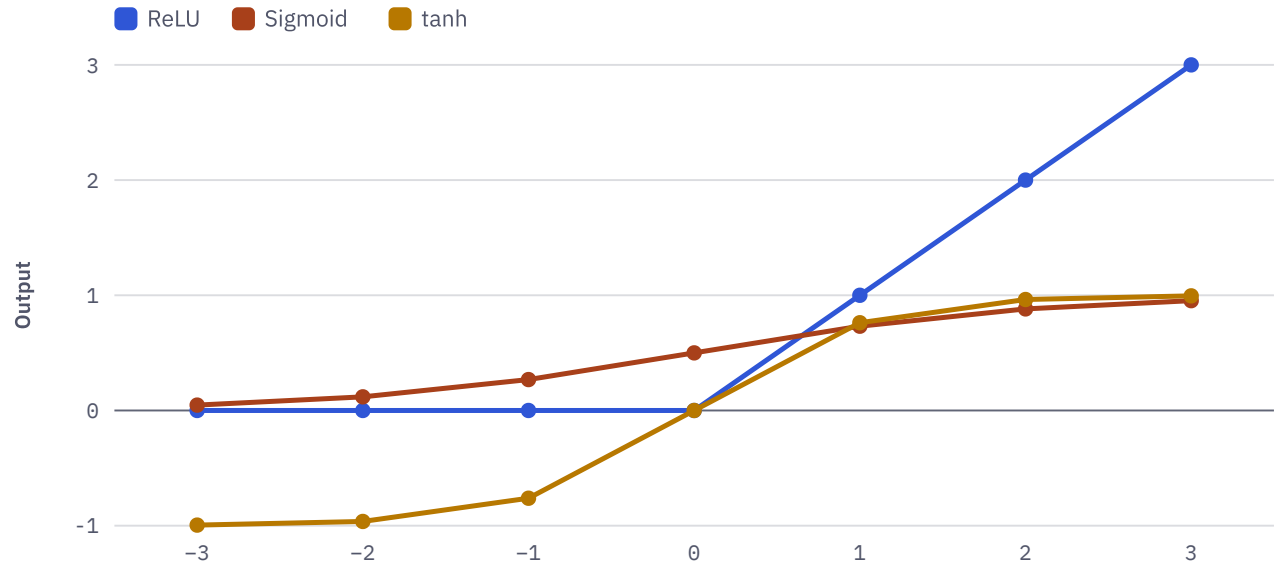
> DEFINITION · ARTIFICIAL NEURON
> A neuron computes $z = w \cdot x + b$ and outputs $a = g(z)$, where g is an activation function. A **layer** applies many neurons to the same input at once, which is a single matrix multiplication: $a = g(xW + b)$.

Building blocks

MEASURE	FORMULA	WHAT IT TELLS YOU
Layer	$h = g(x\,W + b)$	x has d_in values, W is d_in × d_out, b has d_out values.
ReLU	$\mathrm{ReLU}(z) = \max(0, z)$	Simple, fast and the default in hidden layers; gradients do not shrink for positive z.
Sigmoid and tanh	$\sigma(z) = 1 \div (1 + e^{-z})$; $\tanh(z) \in (-1, 1)$	Used for gates and binary outputs; saturate for large |z|, which slows learning in deep stacks.
Softmax output	$p_k = e^{z_k} \div \Sigma\, e^{z_j}$	For multi-class outputs; pairs with cross-entropy loss.
Parameter count (dense layer)	$d_{in} \times d_{out} + d_{out}$	A 784 → 128 → 10 network has 784 × 128 + 128 = 100,480 plus 128 × 10 + 10 = 1,290, so 101,770 parameters.

Copyright © ZyeemZia

Three activation functions side by side



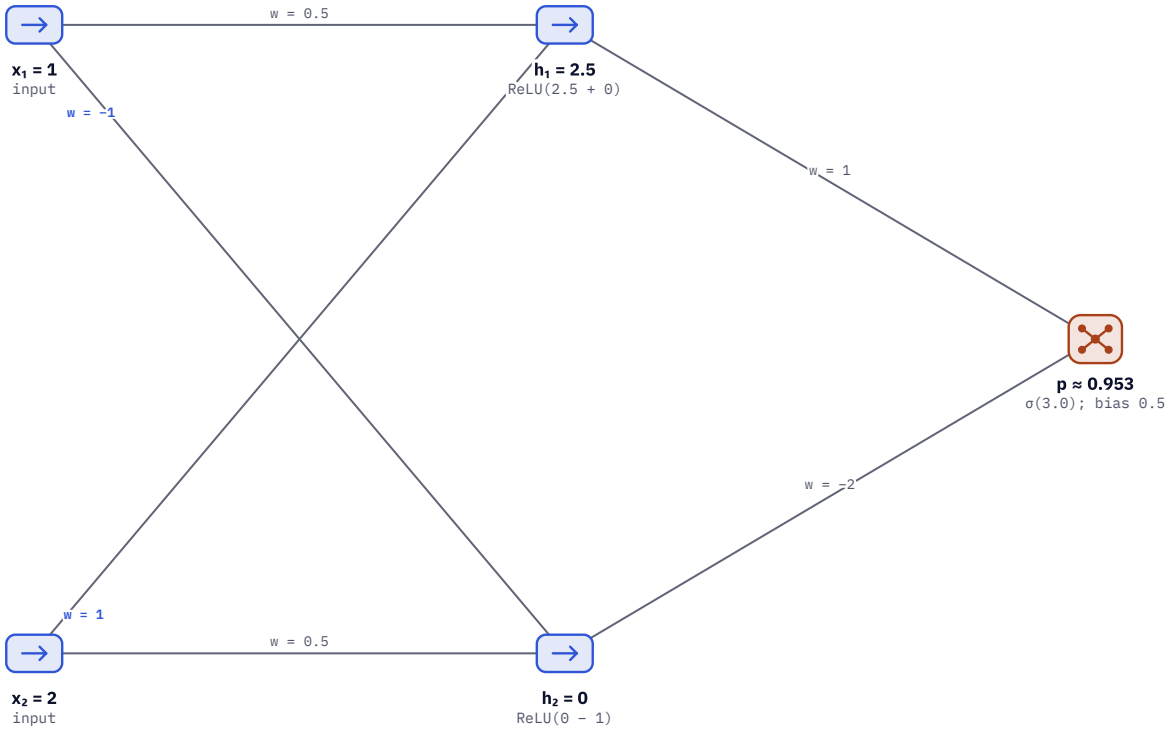
Computed values (x-axis: input z). Sigmoid and tanh flatten at both ends, so their gradients vanish for large |z|; ReLU keeps a constant slope for positive inputs, which is why it trains deep stacks more easily.

A forward pass by hand

1. **Input** x = (1, 2).
2. **Hidden layer** W_1 = [[0.5, −1], [1, 0.5]], b_1 = (0, −1). xW_1 = (1 × 0.5 + 2 × 1, 1 × (−1) + 2 × 0.5) = (2.5, 0). Adding b_1 gives (2.5, −1).
3. **ReLU** h = (max(0, 2.5), max(0, −1)) = (2.5, 0).
4. **Output layer** W_2 = (1, −2), b_2 = 0.5: z = 2.5 × 1 + 0 × (−2) + 0.5 = 3.0.
5. **Sigmoid** p = σ(3.0) ≈ 0.953, the predicted probability of the positive class.

Copyright © ZyeemZia

The worked forward pass as a network



Each arrow is one weight from the matrices W_1 and W_2, and each hidden node adds its bias before ReLU. Because h_2 is zero, its −2 weight contributes nothing to this prediction.

The same forward pass in NumPy

Python notebook

```
import numpy as np
x = np.array([1.0, 2.0])
W1 = np.array([[0.5, -1.0], [1.0, 0.5]]); b1 = np.array([0.0, -1.0])
h = np.maximum(0, x @ W1 + b1)
W2 = np.array([1.0, -2.0]); b2 = 0.5
p = 1 / (1 + np.exp(-(h @ W2 + b2)))
print(h.tolist(), round(float(p), 3))
# output: [2.5, 0.0] 0.953
```

Deep learning frameworks do exactly this, at scale and on GPUs, and also compute the gradients automatically.

Why non-linear activations are essential

Without activations

- Stacking layers gives $xW_1W_2 = xW$
- Any depth collapses into one linear model
- Cannot represent curves or interactions

With activations

- Each layer bends and folds the space
- Depth builds features of increasing abstraction
- Can approximate very complex functions

Matching the output layer to the task

TASK	OUTPUT LAYER	LOSS
Regression	One linear unit (no activation)	MSE or MAE

Copyright © ZyeemZia

TASK	OUTPUT LAYER	LOSS
Binary classification	One sigmoid unit	Binary cross-entropy
Multi-class (one label)	K units with softmax	Categorical cross-entropy
Multi-label (several labels)	K sigmoid units	Binary cross-entropy per label

INSIGHT · WHEN TO REACH FOR DEEP LEARNING

Neural networks shine on **unstructured data** (images, audio, text) and very large datasets, where they learn features themselves. On ordinary tabular business data of modest size, gradient-boosted trees usually match or beat them with less tuning. Choose by data type and evidence, not fashion.

SCENARIO · DECIDE · CHOOSING AN OUTPUT LAYER

A hospital wants one model that tags each chest X-ray with any of 14 findings; an image can show several findings at once. Which output design is correct?

- (A) One softmax over 14 classes
- (B) 14 ReLU outputs trained with MSE
- (C) One linear output predicting the number of findings
- (D) 14 sigmoid outputs trained with binary cross-entropy for each finding

Circle your choice. Answer and feedback: module answer key.

REFLECT

Explain, as if to a manager, why a neural network needs non-linear activations. Use the idea of bending or folding a sheet of paper.

Key terms

Neural network
A model built from layers of weighted sums and non-linear activations, trained by gradient descent.

Activation function
A non-linear function, such as ReLU or sigmoid, applied to a neuron's weighted sum.

Hidden layer
A layer between the input and output whose values are learned intermediate features.

Multi-label classification
A task where each example can belong to several classes at once.

Neuron
A unit that computes a weighted sum of inputs plus a bias and applies an activation function.

ReLU
Rectified linear unit: max(0, z); the standard hidden-layer activation.

Forward pass
Computing a network's output from its input by applying each layer in turn.

Copyright © ZyeemZia

Key takeaways

- A layer is a matrix multiplication plus bias followed by a non-linear activation.
- Without activations, any depth collapses into one linear model.
- ReLU-family activations are the default in hidden layers; match the output layer and loss to the task.
- A dense layer has d_in × d_out + d_out parameters.
- On modest tabular data, gradient-boosted trees usually beat neural networks.

KNOWLEDGE CHECK

1. What is ReLU(−3)?
   - (A) 0
   - (B) −3
   - (C) 3
   - (D) 1
2. How many parameters does a dense layer from 100 inputs to 20 outputs have, including biases?
   - (A) 2,000
   - (B) 120
   - (C) 2,020
   - (D) 2,100
3. Why does a network of linear layers without activations add no power over one linear layer?
   - (A) Linear layers cannot be trained
   - (B) The composition of linear maps is itself a linear map
   - (C) Linear layers have no parameters
   - (D) Linear layers can only be trained on image data, not on tabular data
4. Which output is right for predicting one of five mutually exclusive product categories?
   - (A) Five sigmoid units, one per category
   - (B) One ReLU unit
   - (C) One linear unit
   - (D) Five units with softmax
5. On a 20,000-row tabular credit dataset, which model family is usually a strong first choice?
   - (A) A very deep convolutional network
   - (B) A transformer trained from scratch
   - (C) A recurrent network
   - (D) Gradient-boosted trees

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 7 · DEEP · LESSON 2 · 26 MIN

Training Deep Networks

Backpropagation, initialisation, normalisation and the regularisers that make deep learning work

- Explain backpropagation and compute a gradient for a single neuron
- Recognise vanishing and exploding gradients and their remedies
- Apply dropout, weight decay, data augmentation and early stopping
- Set up a sensible training run and read its curves

Training a deep network means running gradient descent on millions of parameters. **Backpropagation** computes all the gradients efficiently by applying the chain rule from the output back to the input. Around it sits a toolbox of practical techniques that keep training stable and stop the network from memorising its data.

Backpropagation for one sigmoid neuron with log loss

MEASURE	FORMULA	WHAT IT TELLS YOU
Forward	`z = w x + b; p = σ(z); L = −[y ln p + (1 − y) ln(1 − p)]`	One input, one weight, one example.
Key result	`∂L/∂z = p − y`	The sigmoid and log-loss derivatives combine neatly.
Chain rule	`∂L/∂w = (p − y) x; ∂L/∂b = p − y`	Multiply by the derivative of z with respect to each parameter.
Worked example	`x = 2, w = 0.5, b = 0, y = 1`	z = 1, p ≈ 0.731, ∂L/∂w = (0.731 − 1) × 2 ≈ −0.538. A step with η = 0.1 raises w to about 0.554.

Keeping gradients healthy

PROBLEM	SYMPTOM	REMEDIES
Vanishing gradients	Early layers barely learn; loss plateaus	ReLU-family activations, careful initialisation (He or Xavier), residual connections, normalisation layers
Exploding gradients	Loss jumps to NaN	Lower learning rate, gradient clipping, normalisation, check inputs
Dead ReLUs	Many units always output 0	Lower learning rate, leaky ReLU variants, better initialisation
Internal scale drift	Training is slow and sensitive to the learning rate	Batch normalisation (vision) or layer normalisation (transformers)

The regularisation toolbox

R1

Weight decay

Penalise large weights (L2); AdamW applies it cleanly.

R2

Dropout

Randomly zero a fraction of units during training, e.g. 20–50%; surviving activations are scaled by 1 ÷ keep probability.

R3

Data augmentation

Create realistic variants: flips, crops, lighting changes for images; noise or time shifts for audio and sensors.

Copyright © ZyeemZia

R4

Early stopping

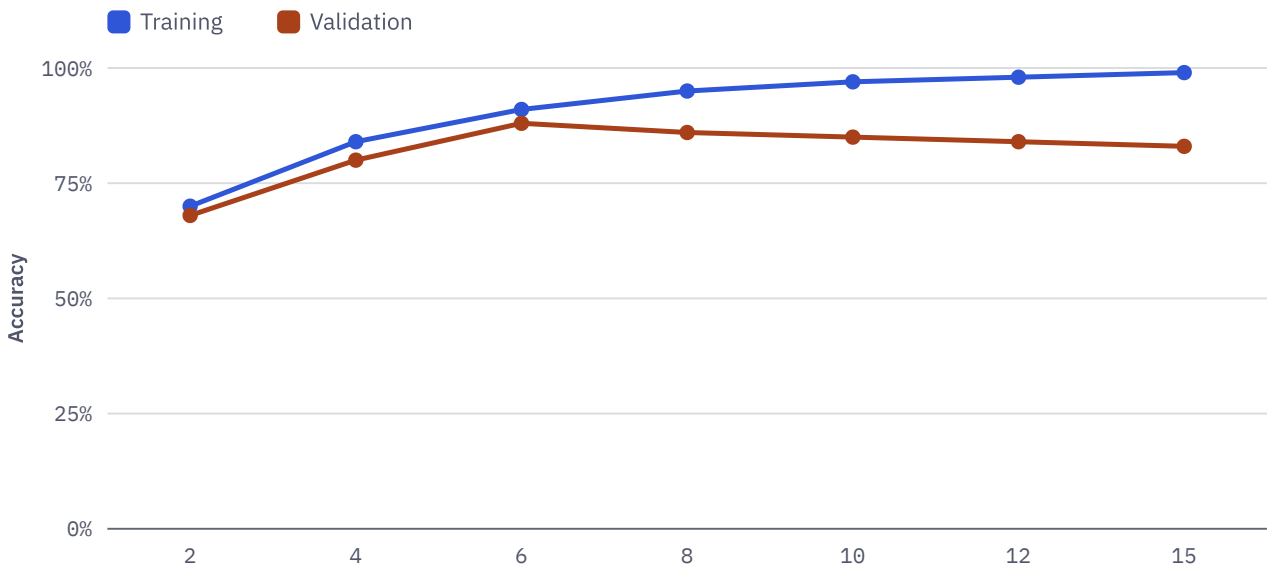
Keep the checkpoint with the best validation score and stop when it stops improving.

R5

Transfer learning

Start from weights pre-trained on large data so that far less task data is needed.

Overfitting in accuracy curves (illustrative)



Illustrative numbers (x-axis: epoch). Validation accuracy peaks early while training accuracy keeps rising: keep the best checkpoint, then use augmentation, dropout, weight decay or a pre-trained start to close the gap.

A small neural network with early stopping

Python notebook

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neural_network import MLPClassifier
X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
net = make_pipeline(StandardScaler(), MLPClassifier(hidden_layer_sizes=(64,), early_stopping=True,
                                                    max_iter=500, random_state=0)).fit(X_tr, y_tr)
print(net[-1].n_iter_, round(net.score(X_te, y_te), 3))
# output: 42 0.962
```

early_stopping holds back 10% of the training data internally and stops when its score stops improving, here after 42 epochs. Larger image and text models use dedicated deep learning frameworks, but the training ideas are the same.

A sensible first training run

1. **Baseline** Train a simple model first so you know what 'good' means.
2. **Overfit a small batch** Confirm the network can drive training loss near zero on a few examples.

Copyright © ZyeemZia

3. **Standard recipe** AdamW, learning rate around 1e-3 (lower for fine-tuning), mini-batches of 32–256, ReLU or GELU activations.
4. **Regularise** Add augmentation, dropout or weight decay once the model overfits.
5. **Track** Log training and validation loss and metrics every epoch, plus learning rate and gradient norms.
6. **Select** Keep the best validation checkpoint; evaluate once on the test set.

PRO TIP · COMPUTE AND CARBON ARE REAL COSTS

Training large networks costs money, time and energy. Start small, use pre-trained models, train in mixed precision (16-bit formats where the hardware supports them), stop runs early when they are clearly worse, and record accelerator-hours and, where available, energy per experiment. A model that is 0.2% better but needs ten times the compute may not be worth it.

SCENARIO · DECIDE · VALIDATION LOSS RISING

A garment defect classifier trained on 3,000 images reaches 99% training accuracy by epoch 15, while validation accuracy peaks at 88% at epoch 6 and then declines. What is the best combination of next steps?

- (A) Train for 50 more epochs to let validation recover
- (B) Remove dropout to increase capacity
- (C) Use the epoch-6 checkpoint, add image augmentation and consider starting from a pre-trained backbone
- (D) Raise the learning rate tenfold

Circle your choice. Answer and feedback: module answer key.

REFLECT

Recall a model whose training you watched. Which signs (loss curves, gradient norms, validation scores) told you it was going well or badly, and which of the techniques above would you try first next time?

Key terms

Backpropagation
The algorithm that computes gradients of the loss for every weight by applying the chain rule backwards through the network.

Gradient clipping
Limiting the size of gradients during training to prevent unstable, exploding updates.

Batch normalisation
Normalising layer activations across a mini-batch to stabilise and speed up training.

Checkpoint
A saved copy of model weights at a point in training.

Vanishing gradient
Gradients shrinking towards zero as they pass back through many layers, so early layers learn slowly.

Dropout
A regulariser that randomly disables a fraction of units during each training step.

Data augmentation
Creating modified copies of training examples to improve robustness and reduce overfitting.

Copyright © ZyeemZia

Key takeaways

- Backpropagation applies the chain rule to compute every gradient efficiently.
- Healthy gradients need good initialisation, ReLU-family activations, normalisation and residual connections.
- Regularise with weight decay, dropout, augmentation, early stopping and transfer learning.
- Start from a standard recipe (AdamW, sensible learning rate) and track curves every epoch.
- Record compute and energy; small gains may not justify large costs.

KNOWLEDGE CHECK

6. For a sigmoid neuron with log loss, p = 0.2, y = 1 and x = 3. What is ∂L/∂w?

   (A) 2.4 (B) −2.4
   (C) −0.8 (D) 0.6

7. With dropout keep probability 0.8, by what factor are surviving activations scaled during training?

   (A) 0.80 (B) 0.20
   (C) 1.25 (D) 5.00

8. Training loss suddenly becomes NaN. Which remedy is MOST relevant?

   (A) Lower the learning rate and apply gradient clipping (B) Add more layers
   (C) Remove the validation set (D) Increase dropout to 90% so fewer weights are updated at each step

9. What do residual (skip) connections mainly help with?

   (A) Reducing the dataset size (B) Letting gradients flow through very deep networks
   (C) Converting regression to classification (D) Removing the need for activations

10. A model trained on 1,500 labelled X-rays overfits badly. Which approach usually helps most?

    (A) Train a much larger network from random weights so it can learn every detail (B) Remove augmentation to keep images realistic
    (C) Use the test set for early stopping (D) Fine-tune a network pre-trained on a large image dataset, with augmentation

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 7 · DEEP · LESSON 3 · 24 MIN

Convolutional Neural Networks

Learn visual features with filters that slide across images

- Explain convolution, filters, stride, padding and pooling
- Compute convolution output sizes and parameter counts
- Describe how CNNs build features from edges to objects
- Apply transfer learning and augmentation to small image datasets

Images have structure: nearby pixels are related, and the same pattern (an edge, a stitch, a crack) can appear anywhere. A **convolutional neural network (CNN)** uses small filters that slide across the image, sharing the same weights at every position. That makes CNNs efficient and naturally good at finding patterns wherever they occur.

Convolution arithmetic

MEASURE	FORMULA	WHAT IT TELLS YOU
Output size	`out = ⌊(n + 2p − k) ÷ s⌋ + 1`	Input 32, kernel 3, padding 1, stride 1: (32 + 2 − 3) ÷ 1 + 1 = 32 ('same' size).
With stride 2	`(32 + 2 − 3) ÷ 2 + 1 = 16 (rounded down)`	Stride 2 halves the spatial size.
Max pooling 2 × 2	`out = n ÷ 2`	Keeps the strongest response in each 2 × 2 block: 32 → 16.
Conv layer parameters	`(k × k × C_in + 1) × C_out`	3 × 3 filters on an RGB input with 64 filters: (27 + 1) × 64 = 1,792.
Dense comparison	`A dense layer from a 32 × 32 × 3 image to 64 units`	3,072 × 64 + 64 = 196,672 parameters: weight sharing makes convolution about 110 times cheaper here.

A vertical-edge filter in NumPy

Python notebook

```
import numpy as np
img = np.array([[0, 0, 9, 9]] * 4, dtype=float)   # dark left, bright right
k = np.array([[-1, 1], [-1, 1]], dtype=float)     # responds to left-to-right brightening
out = np.array([[(img[i:i+2, j:j+2] * k).sum() for j in range(3)] for i in range(3)])
print(out.tolist())
# output: [[0.0, 18.0, 0.0], [0.0, 18.0, 0.0], [0.0, 18.0, 0.0]]
```

The filter responds strongly only where brightness changes from left to right, marking the edge. A CNN learns thousands of such filters from data instead of having them designed by hand.

Copyright © ZyeemZia

How a CNN builds understanding (top layer first)

Classifier head	`Dense or pooling layers` `Class probabilities: 'hole', 'stain', 'OK'`
Deep layers	`Object parts and whole patterns` `Large receptive field`
Middle layers	`Textures and motifs` `Combinations of edges`
Early layers	`Edges, colours, simple gradients` `Small receptive field`
Input	`Pixels: height × width × channels`

Training from scratch versus transfer learning

From scratch

- Random initial weights
- Needs tens of thousands to millions of images
- Long training, more compute
- Only for very different image types with huge data

Transfer learning

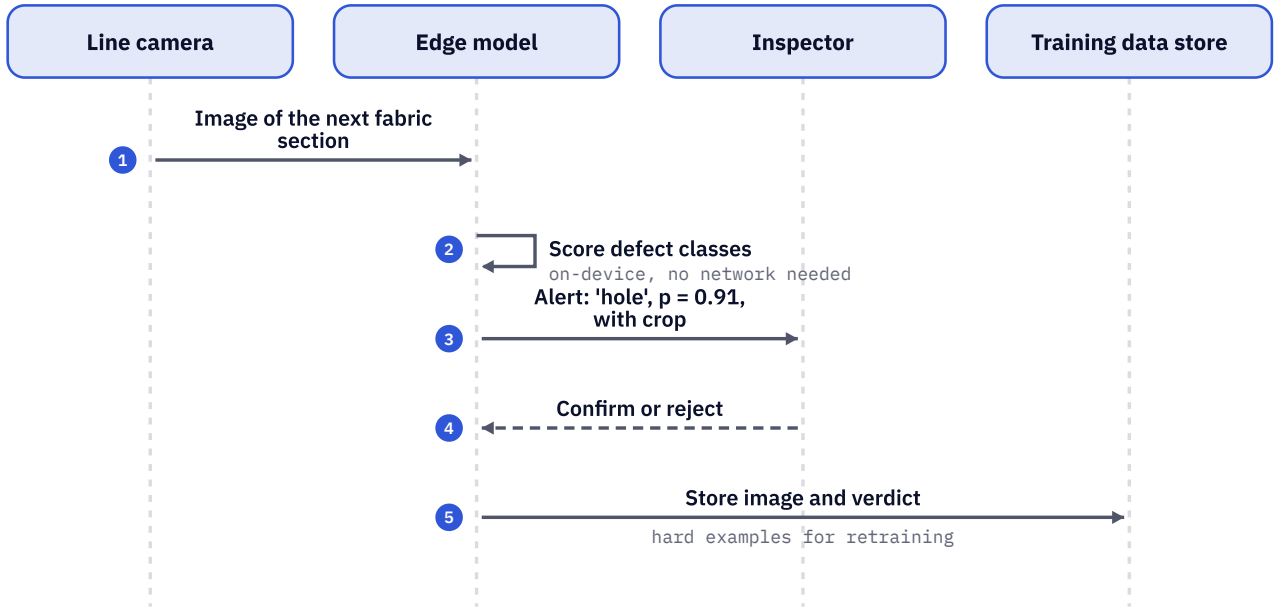
- Start from a network pre-trained on a large image collection
- Replace the head; fine-tune some or all layers
- Works with hundreds to a few thousand images
- The default for most business image tasks

Image data practice

- Augment with transformations that preserve the label: flips, small rotations, crops, brightness changes.
- Do not augment in ways that change meaning (flipping text, or rotating images where orientation matters).
- Split by source (camera, factory line, hospital, patient) to avoid leakage between near-identical images.
- Check model attention with saliency maps to catch shortcuts such as watermarks or rulers.
- Collect hard examples from production and add them to the training set.

Copyright © ZyeemZia

A defect model on the factory line



The model assists the inspector rather than replacing them, and every confirmed or rejected alert becomes a new label. Rejected alerts are the most valuable examples for the next training run.

CASE STUDY · FABRIC DEFECT DETECTION ON A FACTORY LINE

A knitwear factory in Gazipur inspects fabric rolls by eye. Inspectors miss about 15% of defects late in shifts. The factory has a camera over the line and 2,400 labelled images (1,900 'OK', 500 with holes, stains or broken yarn), all from one machine, taken over two weeks.

1. How would you build the first model?

2. What are the main risks in the data, and how would you reduce them?

3. How should the model be used on the line?

Model answer: module answer key.

Copyright © ZyeemZia

SCENARIO · DECIDE · LEAKAGE IN AN IMAGE SPLIT

A team randomly splits 10,000 frames extracted from 50 inspection videos into training and test sets. Test accuracy is 99.5%, but the model fails on new videos. What went wrong?

(A) Consecutive frames from the same video were in both sets; split by video instead

(B) 99.5% shows the model is ready, and the new videos must be faulty

(C) Images should have been converted to grayscale

(D) The model needed more convolution layers

Circle your choice. Answer and feedback: module answer key.

REFLECT

Where could image models help in your organisation, for example in inspection, documents or medical imaging? How many labelled images exist, and how would you split them to avoid near-duplicates across sets?

Key terms

Convolutional neural network
A neural network that uses learned filters slid across the input to detect local patterns.

Stride
The step size by which a filter moves across the input.

Pooling
Downsampling a feature map, for example by taking the maximum in each small block.

Transfer learning
Reusing a model trained on one task as the starting point for another.

Filter (kernel)
A small matrix of weights convolved with the input to produce a feature map.

Padding
Adding border values around the input so filters can cover the edges and control output size.

Receptive field
The region of the input image that influences a given unit in a network.

Key takeaways

- Convolutions share small filters across positions, making CNNs efficient for images.
- Output size = $\lfloor (n + 2p - k) \div s \rfloor + 1$; parameters = $(k \times k \times C_{in} + 1) \times C_{out}$.
- Transfer learning from a pre-trained backbone is the default for business image tasks.
- Augment only in label-preserving ways, and split by source to avoid near-duplicate leakage.
- Check saliency maps for shortcuts and feed hard production examples back into training.

KNOWLEDGE CHECK

Copyright © ZyeemZia

11. Input 28, kernel 5, padding 0, stride 1. What is the output size?

(A) 24 (B) 28
(C) 23 (D) 14

12. How many parameters does a conv layer with 32 filters of size 3 × 3 on a 16-channel input have?

(A) 4,608 (B) 512
(C) 4,640 (D) 9,248

13. Why are CNNs efficient for images?

(A) They ignore most pixels and only look at the centre of each image (B) They require no training
(C) The same filter weights are shared across all positions (D) They use only one layer

14. Which augmentation is inappropriate for classifying handwritten digits?

(A) Horizontal flips that turn some digits into different shapes (B) Small rotations of a few degrees
(C) Slight changes in stroke thickness (D) Small shifts in position

15. A company has 900 labelled product photos. What is usually the best starting point?

(A) Train a large CNN from random weights (B) Use k-means on raw pixels
(C) Use linear regression on pixel values (D) Fine-tune a pre-trained CNN

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 7 · DEEP · LESSON 4 · 24 MIN

Sequence Models and Embeddings

Model text, time series and other ordered data

- Explain tokens and embeddings as learned representations
- Describe how recurrent networks carry state through a sequence
- Explain why LSTMs and GRUs were introduced
- Frame time-series forecasting as a supervised learning problem

Text, speech, sensor streams, transaction histories and clinical notes are **sequences**: order matters. Sequence models read one element at a time, or all at once with attention, and carry information from earlier positions to later ones. Before any of that, discrete items such as words must become vectors.

DEFINITION · TOKENS AND EMBEDDINGS

A **token** is a unit of text, often a word piece: 'unhappiness' might become 'un', 'happi', 'ness'. An **embedding** maps each token (or user, product or merchant) to a dense vector of, say, 256 numbers. The vectors are learned so that items used in similar contexts end up close together.

Recurrent networks

MEASURE	FORMULA	WHAT IT TELLS YOU
Simple RNN step	`h_t = tanh(W_x x_t + W_h h_{t-1} + b)`	The hidden state h_t summarises everything seen so far.
Worked example	`W_x = 0.5, W_h = 0.8, b = 0, inputs 1, 0, 0`	$h_1 = \tanh(0.5) \approx 0.462$; $h_2 = \tanh(0.8 \times 0.462) \approx 0.354$; $h_3 \approx 0.276$. The memory of the first input fades step by step.
Why memory fades	`∂h_t/∂h_{t-k} involves k repeated factors of W_h and tanh'`	Products of many factors below 1 vanish, so plain RNNs forget long-range information.
Gated units (LSTM, GRU)	`Gates in (0, 1) decide what to keep, forget and output`	A protected memory path lets information travel across hundreds of steps.

A recurrent state fading over time

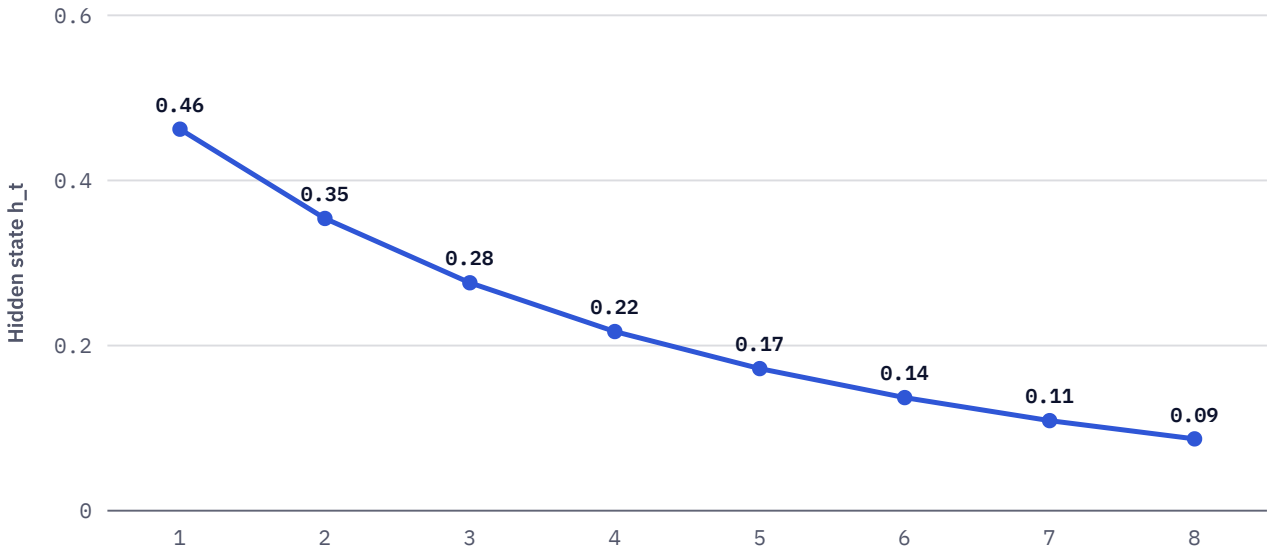
Python notebook

```
import numpy as np
Wx, Wh, h = 0.5, 0.8, 0.0
for x in [1.0, 0.0, 0.0]:
    h = np.tanh(Wx * x + Wh * h)
    print(round(float(h), 3))
# output: 0.462
# output: 0.354
# output: 0.276
```

With no new input, the state shrinks at each step. Over long sequences this fading, and the matching vanishing gradients, is what gated units and attention were designed to overcome.

Copyright © ZyeemZia

A simple RNN forgetting its first input



Computed with W_x = 0.5, W_h = 0.8 and inputs 1, 0, 0, … (x-axis: time step). The memory of the first input shrinks at every step; gated units and attention were designed to keep it.

Sequence model families

FAMILY	HOW IT READS	STRENGTHS	LIMITS
Bag of words / TF-IDF + linear model	Ignores order	Fast, strong baseline for classification	Misses word order and context
RNN, LSTM, GRU	One step at a time, carrying state	Natural for streams; small models	Hard to parallelise; long-range limits
1-D convolutions	Sliding filters over time	Fast; good for local patterns in signals	Limited long-range context unless deep
Transformers	All positions at once with attention	Long context, highly parallel, scale well	Compute grows with sequence length; data-hungry from scratch

Forecasting as supervised learning

1. **Choose horizon and grain** For example, daily sales per store, predicted 7 days ahead.
2. **Build lag features** Sales 7, 14 and 28 days ago; rolling means and trends, computed only from the past.
3. **Add known-future features** Calendar, holidays such as Eid, planned promotions, weather forecasts.
4. **Train and validate in time order** Rolling-origin evaluation: train up to a date, forecast the next window, move forward.
5. **Compare with baselines** Seasonal naive and simple exponential smoothing must be beaten.

Copyright © ZyeemZia

Rolling-origin evaluation for a 7-day forecast

FOLD 1
Train to 31 Jan
Forecast 1–7 Feb and score it.

FOLD 2
Train to 7 Feb
Forecast 8–14 Feb.

FOLD 3
Train to 14 Feb
Forecast 15–21 Feb.

FOLD 4
Train to 21 Feb
Forecast 22–28 Feb.

The training origin moves forward and the model is always scored on the window just after it, exactly as it will be used. Average the fold errors and compare them with a seasonal naive baseline.

EXAMPLE · ELECTRICITY LOAD IN DHAKA

A distribution company forecasting hourly feeder load found that gradient-boosted trees on lag, calendar and temperature features beat an LSTM trained on raw sequences, at a fraction of the cost. The LSTM helped only after the team added the same calendar and weather inputs. Engineering the right inputs mattered more than the architecture.

SCENARIO · DECIDE · REPRESENTING MERCHANTS

A payments company has 400,000 merchants and wants a fraud model to capture which merchants behave similarly. One-hot encoding would create 400,000 sparse columns. What is a better representation?

- (A) Use the merchant's name length as a feature
- (B) Number the merchants 1 to 400,000 and use that number as a feature
- (C) Learn merchant embeddings from transaction sequences or co-occurrence and use them as features
- (D) Drop merchant information entirely

Circle your choice. Answer and feedback: module answer key.

TEMPLATE **Forecast feature register**

List every input to a forecasting model with its availability at prediction time, so that no future information slips in.

FEATURE	TYPE	COMPUTED FROM	KNOWN AT FORECAST TIME?	NOTES
sales_lag_7	*Lag*	*Actual sales 7 days before the target date*	*Yes, if horizon ≤ 7 days*	*Use lag ≥ horizon for longer forecasts*
is_eid_week	*Known future*	*Published holiday calendar*	*Yes*	*Check moon-sighting date changes*

Copyright © ZyeemZia

REFLECT

Which sequences does your organisation record (texts, logs, transactions, readings)? Would a model need long-range context, or would well-chosen lag features be enough?

Key terms

Token
A unit of text, such as a word or word piece, processed by a language model.

Recurrent neural network
A network that processes a sequence step by step while maintaining a hidden state.

GRU
Gated recurrent unit: a simpler gated recurrent cell similar in purpose to an LSTM.

Lag feature
A feature equal to the value of a series a fixed number of steps in the past.

Tokeniser
The component that splits raw text into tokens and maps them to integer IDs.

LSTM
Long short-term memory: a recurrent unit with gates that control what is remembered and forgotten.

Hidden state
The vector a recurrent network carries forward that summarises the sequence so far.

Key takeaways

- Tokens and other discrete items become dense, learned embedding vectors.
- Plain RNNs forget long-range information; LSTMs, GRUs and attention address this.
- Forecasting works well as supervised learning with lag and known-future features.
- Validate forecasts in time order and always beat a seasonal naive baseline.

KNOWLEDGE CHECK

16. With W_x = 1, W_h = 0.5, b = 0 and $h_0 = 0$, what is h_1 for input $x_1 = 0$? (tanh(0) = 0)

(A) 0.5
(B) 0
(C) 1
(D) 0.462

17. Why were LSTMs and GRUs introduced?

(A) To preserve information and gradients over long sequences
(B) To process images faster
(C) To remove the need for large amounts of labelled training data
(D) To replace embeddings

18. What property do well-trained word embeddings have?

(A) Every word has a vector of all zeros except for a single one
(B) Vectors are sorted alphabetically
(C) Words used in similar contexts have similar vectors
(D) Vectors equal the word's length

Copyright © ZyeemZia

19. For a daily forecast made on 1 March, which feature is NOT allowed?

(A) Sales on 22 February
(B) Actual sales on 3 March
(C) Whether 5 March is a public holiday
(D) The 7-day average up to 28 February

20. Which evaluation suits a time-series forecasting model?

(A) Random k-fold over all days
(B) Training-set accuracy
(C) Leave-one-out cross-validation on shuffled days
(D) Rolling-origin evaluation in time order

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 7 · DEEP · LESSON 5 · 34 MIN

Transformers, Attention and Foundation Models

The architecture behind modern language and vision models, and how to adapt, ground, align and shrink them

- Explain self-attention and compute it for a tiny example
- Describe encoder, decoder and encoder–decoder transformers and their uses
- Explain pre-training, fine-tuning and parameter-efficient adaptation
- Decide between a classic model, a fine-tuned transformer and a large foundation model

The **transformer** replaced recurrence with **attention**: every position in a sequence looks at every other position and decides how much each matters. Because all positions are processed in parallel, transformers train efficiently on enormous datasets. Pre-trained transformers, often called **foundation models**, now underpin translation, search, chat assistants and many vision systems.

Scaled dot-product attention

MEASURE	FORMULA	WHAT IT TELLS YOU
Queries, keys, values	`Q = XW_Q; K = XW_K; V = XW_V`	Each token produces a query (what it looks for), a key (what it offers) and a value (what it passes on).
Attention	`Attention(Q, K, V) = softmax(QKᵀ ÷ √d_k) V`	Scores compare each query with every key; softmax turns scores into weights; the output is a weighted mix of values.
Worked example	`Tokens (1, 0), (0, 1), (1, 1) with Q = K = V = X, d_k = 2`	Token 3's scores are (1, 1, 2) ÷ √2 ≈ (0.71, 0.71, 1.41); softmax gives (0.248, 0.248, 0.503); output ≈ (0.752, 0.752).
Why divide by √d_k	`Keeps dot products in a range where softmax is not saturated`	Large scores would make attention almost one-hot and slow learning.
Cost	`Attention compute and memory grow with n² for n tokens`	Doubling the context length roughly quadruples attention cost.

Self-attention for three tokens

Python notebook

```
import numpy as np
X = np.array([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])
Q = K = V = X
scores = Q @ K.T / np.sqrt(K.shape[1])
weights = np.exp(scores) / np.exp(scores).sum(axis=1, keepdims=True)
print(weights[2].round(3).tolist(), (weights @ V)[2].round(3).tolist())
# output: [0.248, 0.248, 0.503] [0.752, 0.752]
```

Real transformers learn W_Q, W_K and W_V, run many attention heads in parallel, add position information and stack dozens of layers, but each head performs exactly this calculation.

Copyright © ZyeemZia

Inside a transformer block (top first)

Output	`Next-token probabilities or task head`
Feed-forward network	`Two dense layers per position` `Residual connection and normalisation`
Multi-head self-attention	`Several heads attend to different relations` `Causal mask in decoders: no peeking at future tokens`
Position information	`Learned, fixed or rotary (relative) position encodings` `Gives the model word order`
Token embeddings	`Token IDs → dense vectors`

Transformer families

FAMILY	SEES	TYPICAL USE
Encoder-only	Whole input in both directions	Classification, entity extraction, embeddings for search
Decoder-only	Only earlier tokens (causal)	Text generation, chat assistants, code; the design of most large language models
Encoder–decoder	Encoder reads input; decoder generates output	Translation, summarisation
Vision transformer	Image patches as tokens	Image classification and detection at scale
Mixture-of-experts (variant)	Routes each token to a few expert feed-forward blocks	Very large models with lower compute per token

Modern chat models are built in stages. Self-supervised **pre-training** on very large text, code and often image collections teaches next-token prediction. **Supervised fine-tuning** on instruction–response examples teaches the model to follow requests. **Preference optimisation** then learns from comparisons of better and worse answers, either through a reward model and reinforcement learning (RLHF) or with direct methods such as DPO. Evaluation and red-teaming come before release, and monitoring continues after it.

Ways to adapt a pre-trained model

Prompting

- No training; instructions and examples in the input
- Fast to try
- Limited control; cost per call

Fine-tuning

- Update all or most weights on task data
- Strong task performance
- Needs labelled data and compute

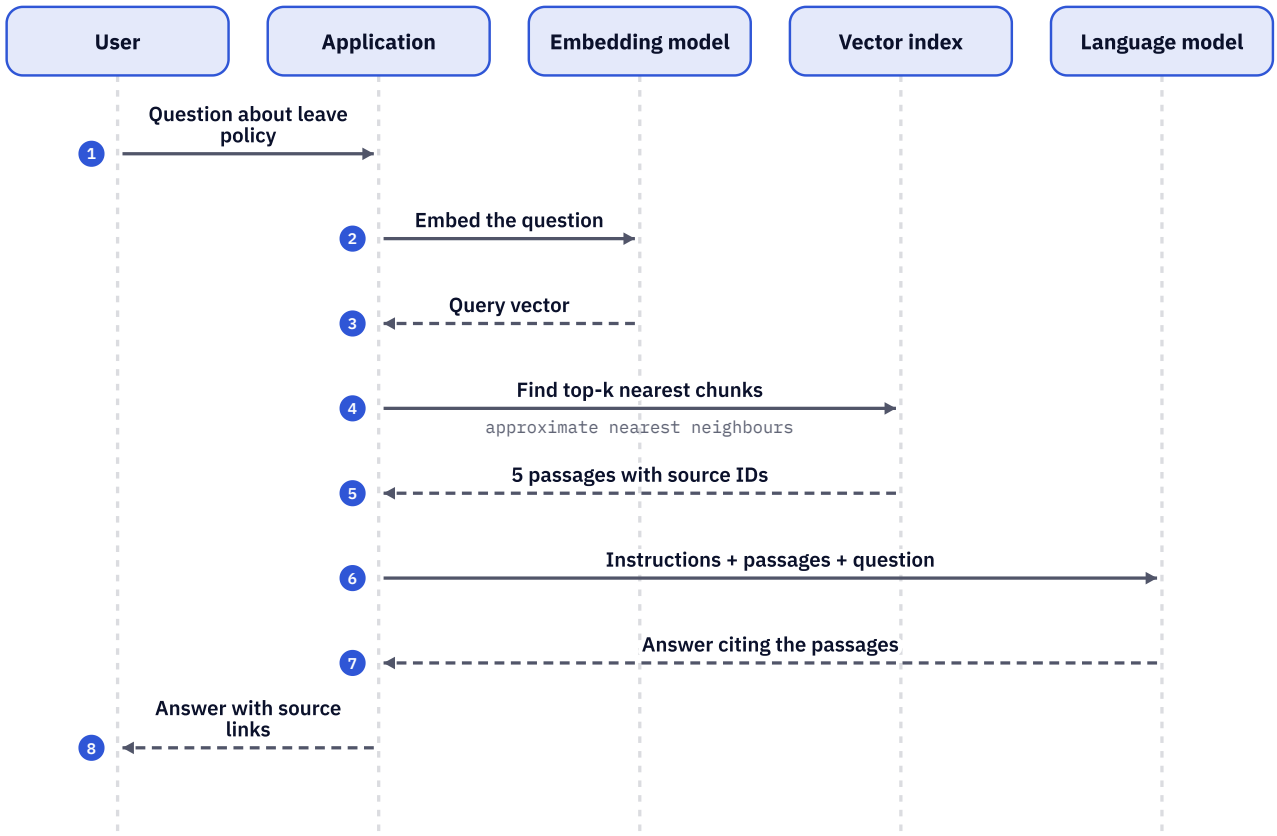
Parameter-efficient fine-tuning

- Train small added matrices (adapters, low-rank updates such as LoRA)
- Far less memory and storage
- Close to full fine-tuning quality on many tasks

Retrieval-augmented generation (RAG) grounds answers in your own documents without changing the model. Documents are split into chunks, each chunk is embedded, and the vectors go into an index built for fast approximate nearest-neighbour search. At question time the closest chunks are retrieved and placed in the prompt. RAG suits knowledge that changes often or answers that must cite sources; fine-tuning suits a consistent format, style or task behaviour.

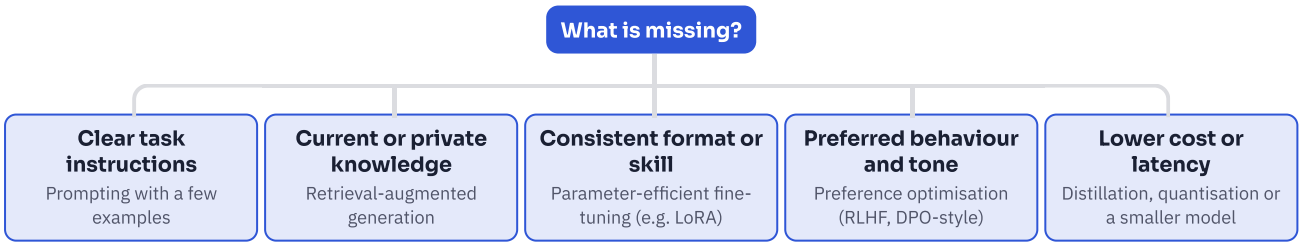
Copyright © ZyeemZia

Retrieval-augmented generation, step by step



Two separate things can fail: retrieval can miss the right passage, and generation can ignore or misread it. Evaluate them separately, and treat retrieved text as untrusted input that may contain injected instructions.

Choosing how to adapt a foundation model



Try the cheapest branch that addresses the actual gap first, and measure each step against the same evaluation set. The branches combine: a quantised, LoRA-tuned model can still use retrieval.

Efficient adaptation and inference arithmetic

MEASURE	FORMULA	WHAT IT TELLS YOU
LoRA update	`W' = W + BA, with B (d × r) and A (r × k), r ≪ d, k`	The pre-trained W stays frozen; only the small matrices B and A are trained.
LoRA parameters	`d = k = 4,096, r = 8: r(d + k) = 65,536 versus d × k = 16,777,216`	About 0.4% of the weights in that matrix are trained, so each task adapter is small to store and swap.

Copyright © ZyeemZia

MEASURE	FORMULA	WHAT IT TELLS YOU
Weight memory	`memory ≈ parameters × bytes per parameter`	7 billion parameters: about 14 GB at 16-bit, 7 GB at 8-bit, 3.5 GB at 4-bit, before activations and the attention cache.
Distillation loss	`L = α · CE(y, p_student) + (1 − α) · T² · KL(p_teacher,T ‖ p_student,T)`	A small student learns from the labels and from the teacher's softened probabilities at temperature T.

WATCH OUT · LARGE MODELS STILL NEED EVALUATION

Pre-trained models can produce fluent but wrong output, reflect biases in their training text and perform unevenly across languages such as Bangla. Public benchmark scores may be inflated by contamination and rarely match your task, so build a versioned evaluation set from real, recent cases. Score retrieval and generation separately; if another model acts as judge, check it against human ratings, because judges can favour longer answers, their own style or the first option shown. Rerun the set after every prompt, model or data change, check results by language and group, protect personal data in prompts and logs, and keep a simpler baseline for comparison.

SCENARIO · DECIDE · CLASSIFYING BANGLA COMPLAINTS

A bank receives 20,000 customer complaints a month in Bangla, English and mixed script. It has 6,000 labelled complaints across 12 categories. A TF-IDF linear model reaches macro-F1 0.71. What is the most sensible next step?

(A) Train a transformer from scratch on the 6,000 complaints

(B) Replace the classifier with a large generative model and skip evaluation

(C) Keep the baseline because transformers do not support Bangla

(D) Fine-tune a pre-trained multilingual encoder on the labelled data and compare it with the baseline on the same split

Circle your choice. Answer and feedback: module answer key.

Before adopting a pre-trained or foundation model

☐ A simple baseline has been measured on the same evaluation set

☐ Performance is checked separately for each language, region and user group

☐ Licence terms allow the intended commercial use

☐ Personal and confidential data handling in prompts, logs and fine-tuning is approved

☐ Latency, cost and energy per request fit the budget at expected volume; a smaller, distilled or quantised model has been tried

☐ Failure modes such as fluent but wrong output have mitigations and human review

☐ User inputs and retrieved documents are treated as untrusted, with prompt-injection tests

☐ A rollback plan exists if quality drops after an update of the underlying model

Copyright © ZyeemZia

REFLECT

For one text or image task in your organisation, would you use a classic model, a fine-tuned pre-trained model or a prompted large model? List the deciding factors: data, cost, latency, privacy and the need for explanations.

Key terms

Transformer
A neural architecture built from self-attention and feed-forward layers that processes all sequence positions in parallel.

Self-attention
A mechanism in which each position computes weighted combinations of all positions based on query–key similarity.

Attention head
One of several parallel attention computations, each able to focus on different relationships.

Positional encoding
Information added to token embeddings so that a transformer knows the order of tokens.

Foundation model
A large model pre-trained on broad data that can be adapted to many downstream tasks.

Fine-tuning
Continuing to train a pre-trained model on task-specific data.

Parameter-efficient fine-tuning
Adapting a large model by training a small number of added or selected parameters.

Retrieval-augmented generation (RAG)
Retrieving relevant passages from an index of your documents and adding them to the prompt so a language model answers from them.

LoRA (low-rank adaptation)
A parameter-efficient fine-tuning method that freezes pre-trained weights and trains small low-rank matrices added to them.

Preference optimisation
Tuning a model on comparisons of better and worse outputs, via a reward model and RL (RLHF) or directly (for example DPO).

Quantisation
Storing and computing with lower-precision numbers, such as 8-bit or 4-bit weights, to cut memory and speed up inference.

Knowledge distillation
Training a smaller student model to imitate a larger teacher model's outputs.

Key takeaways

- Attention lets every token weigh every other token; cost grows with the square of the context length.
- Large language models are pre-trained, instruction-tuned and preference-optimised before release.
- Adapt with the cheapest method that fixes the real gap: prompting, RAG, LoRA-style fine-tuning or preference tuning.
- Distillation and quantisation cut cost and latency; measure the quality they give up.
- Evaluate on private, versioned, task-specific sets, separately for retrieval and generation, and by language and group.

KNOWLEDGE CHECK

Copyright © ZyeemZia

21. In scaled dot-product attention, why are scores divided by √d_k?

(A) To make the weights sum to √d_k
(B) To remove the need for values
(C) To reverse the token order
(D) To keep scores from growing so large that softmax saturates

22. Context length doubles from 4,000 to 8,000 tokens. Roughly how does self-attention cost change?

(A) It stays roughly the same
(B) It roughly quadruples
(C) It doubles
(D) It halves

23. Which family is typically used for generating text one token at a time?

(A) Encoder-only
(B) Convolutional
(C) Decoder-only
(D) k-means

24. Attention weights for a token are (0.5, 0.5) over values (2, 4) and (6, 0). What is the output?

(A) (4, 2)
(B) (8, 4)
(C) (2, 4)
(D) (3, 3)

25. What is the main advantage of parameter-efficient fine-tuning?

(A) It guarantees better accuracy than full fine-tuning on every downstream task
(B) It adapts a large model while training and storing far fewer parameters
(C) It requires no labelled data at all
(D) It removes the need for evaluation

26. A team moves a 7-billion-parameter model from 16-bit to 4-bit weights. Roughly how much memory do the weights alone now need?

(A) About 14 GB
(B) About 3.5 GB
(C) About 28 GB
(D) About 0.9 GB

☐ Lesson complete · Date _____ · Knowledge check score ____ / 6

Copyright © ZyeemZia

MODULE 7 · DEEP

Answer key

Knowledge checks

1. A ReLU returns max(0, z).

2. C 100 × 20 + 20 = 2,020.

3. B xW_1W_2 equals xW for $W = W_1W_2$.

4. D Mutually exclusive classes need probabilities that sum to one.

5. D Boosted trees usually lead on modest tabular data; deep learning excels on unstructured data.

6. B (p − y) × x = (0.2 − 1) × 3 = −2.4.

7. C 1 ÷ 0.8 = 1.25 keeps the expected activation unchanged.

8. A NaN losses typically come from exploding updates.

9. B Skip paths give gradients a short route back to early layers.

10. D Transfer learning plus augmentation reduces the data needed.

11. A (28 − 5) ÷ 1 + 1 = 24.

12. C (3 × 3 × 16 + 1) × 32 = 145 × 32 = 4,640.

13. C Weight sharing cuts parameters and detects patterns anywhere.

14. A Flips can change meaning, for example making digits unreadable or ambiguous.

15. D Pre-trained features make small datasets workable.

16. B tanh(1 × 0 + 0.5 × 0) = tanh(0) = 0.

17. A Gates protect a memory path against vanishing gradients.

18. C Embeddings capture distributional similarity.

19. B Future actuals are unknown at prediction time; known-future calendar facts are fine.

20. D Forecasts must be tested on periods after the training data.

21. D Large dot products push softmax towards one-hot outputs with tiny gradients.

22. B Attention cost scales with n^2.

23. C Decoders use causal masking to predict the next token.

24. A 0.5 × (2, 4) + 0.5 × (6, 0) = (1 + 3, 2 + 0) = (4, 2).

25. B Small trainable additions save memory and storage with near full quality.

26. B Weight memory is roughly parameters × bytes per parameter: 7 billion × 0.5 bytes ≈ 3.5 GB. At 16-bit it was about 14 GB. Activations and the attention cache need extra memory on top.

Scenarios & case studies

Scenario · Choosing an output layer
D is best. Correct. Findings are not mutually exclusive, so each needs its own independent probability.

Scenario · Validation loss rising
C is best. Correct. The model is overfitting a small dataset; early stopping, augmentation and transfer learning are the standard remedies.

Case study · Fabric defect detection on a factory line
Fine-tune a pre-trained CNN with a new head for the defect classes (or a detection model if defect location matters), using augmentation that matches real variation (brightness, small shifts, slight blur), class weights for the rarer defect types, and a validation split by date or roll to avoid near-duplicate images. Risks: one machine and two weeks mean limited variation in lighting, fabric colour and yarn, so collect images from other machines, shifts and fabric types and check performance per fabric. Use the model to flag likely defects for inspectors rather than replace them, set the threshold for high recall, and feed confirmed misses and false alarms back into training.

Scenario · Leakage in an image split
A is best. Correct. Neighbouring frames are almost identical, so the test set measured memorisation rather than generalisation to new footage.

Scenario · Representing merchants
C is best. Correct. Dense embeddings place similar merchants close together in a compact vector that downstream models can use.

Copyright © ZyeemZia

Scenario · Classifying Bangla complaints

D is best. Correct. A multilingual encoder brings language understanding, including mixed script, and 6,000 labels are enough for fine-tuning. Comparing on the same split shows whether the gain is real.

Copyright © ZyeemZia

08

MODULE 8 · DEPLOY

MLOps, Monitoring and Responsible ML

Ship models safely, keep them healthy in production and govern them responsibly

BY THE END OF THIS MODULE YOU WILL BE ABLE TO

- Choose serving patterns and release models with shadow, canary and A/B strategies
- Monitor data, predictions and performance and respond to drift
- Build reproducible, tested and automated ML pipelines
- Measure and mitigate unfairness and apply privacy, security and governance controls

LESSONS

1. From Notebook to Production · 26 min
2. Monitoring and Drift · 26 min
3. MLOps: Versioning, Pipelines and Automation · 26 min
4. Responsible Machine Learning · 28 min

MODULE 8 · DEPLOY · LESSON 1 · 26 MIN

From Notebook to Production

Package, serve and release models safely

- Choose between batch, online, streaming and edge serving
- Package a full preprocessing-and-model pipeline for reliable reuse
- Release models with shadow, canary, blue–green and A/B strategies
- Size serving capacity and experiments with simple arithmetic

A model in a notebook creates no value. Value starts when predictions reach the people and systems that act on them, reliably, every day. Deployment turns a trained pipeline into a **service** with owners, interfaces, performance targets and a safe way to release new versions.

Serving patterns

PATTERN	HOW IT WORKS	TYPICAL LATENCY	EXAMPLE
Batch	Score many records on a schedule and store results	Minutes to hours	Nightly churn scores for the retention team
Online (request–response)	An API scores one request on demand	Milliseconds	Fraud check during a wallet transfer
Streaming	Score events continuously from a message stream	Seconds	Sensor anomaly alerts in a power plant
Edge / on-device	Model runs on a phone, camera or machine	Milliseconds, works offline	Defect detection on a factory-line camera

Save and reload the whole pipeline, not just the model

Python notebook

```
import joblib
from sklearn.datasets import make_classification
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
X, y = make_classification(n_samples=500, random_state=0)
pipe = make_pipeline(StandardScaler(), LogisticRegression()).fit(X, y)
joblib.dump(pipe, 'churn-model-v1.joblib')
served = joblib.load('churn-model-v1.joblib')
print(bool((served.predict_proba(X[:5]) == pipe.predict_proba(X[:5])).all()),
round(float(served.predict_proba(X[:1])[0, 1]), 3))
# output: True 0.693
```

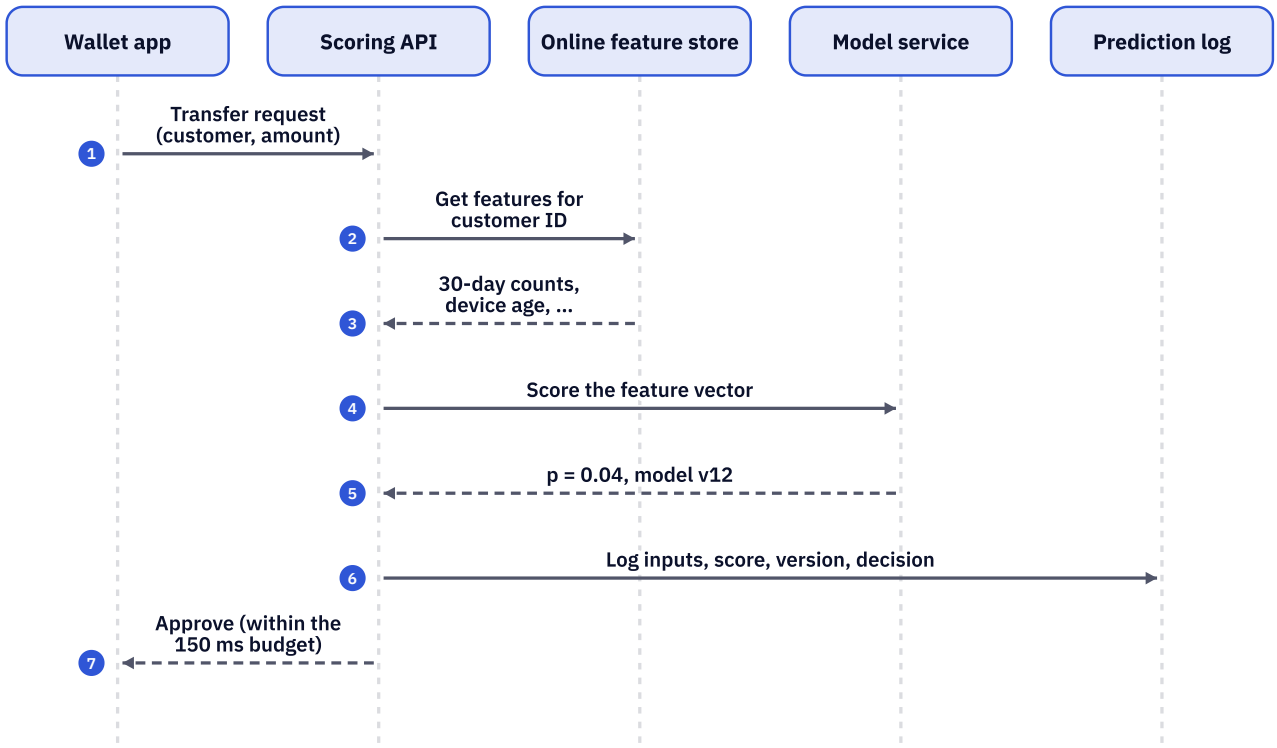
Saving the pipeline keeps scaling and the model together, so production applies exactly the training transformations. Record the library versions too: serialised objects should be loaded with the same versions that created them.

WATCH OUT · TRAINING-SERVING SKEW

If features are computed one way in the training notebook (SQL over a warehouse) and another way in the live service (application code), small differences in rounding, time zones or missing-value handling silently degrade the model. Share one feature definition between training and serving, for example through a feature store or a shared library, and test that both paths give identical values.

Copyright © ZyeemZia

Online scoring of a wallet transfer



The online store serves the same feature definitions used to build training data, which prevents skew. Logging inputs, score and model version with each decision makes any single outcome explainable later.

Release strategies

Shadow

- New model scores live traffic, results not used
- Zero customer risk
- Compares predictions and latency with the current model

Canary and blue–green

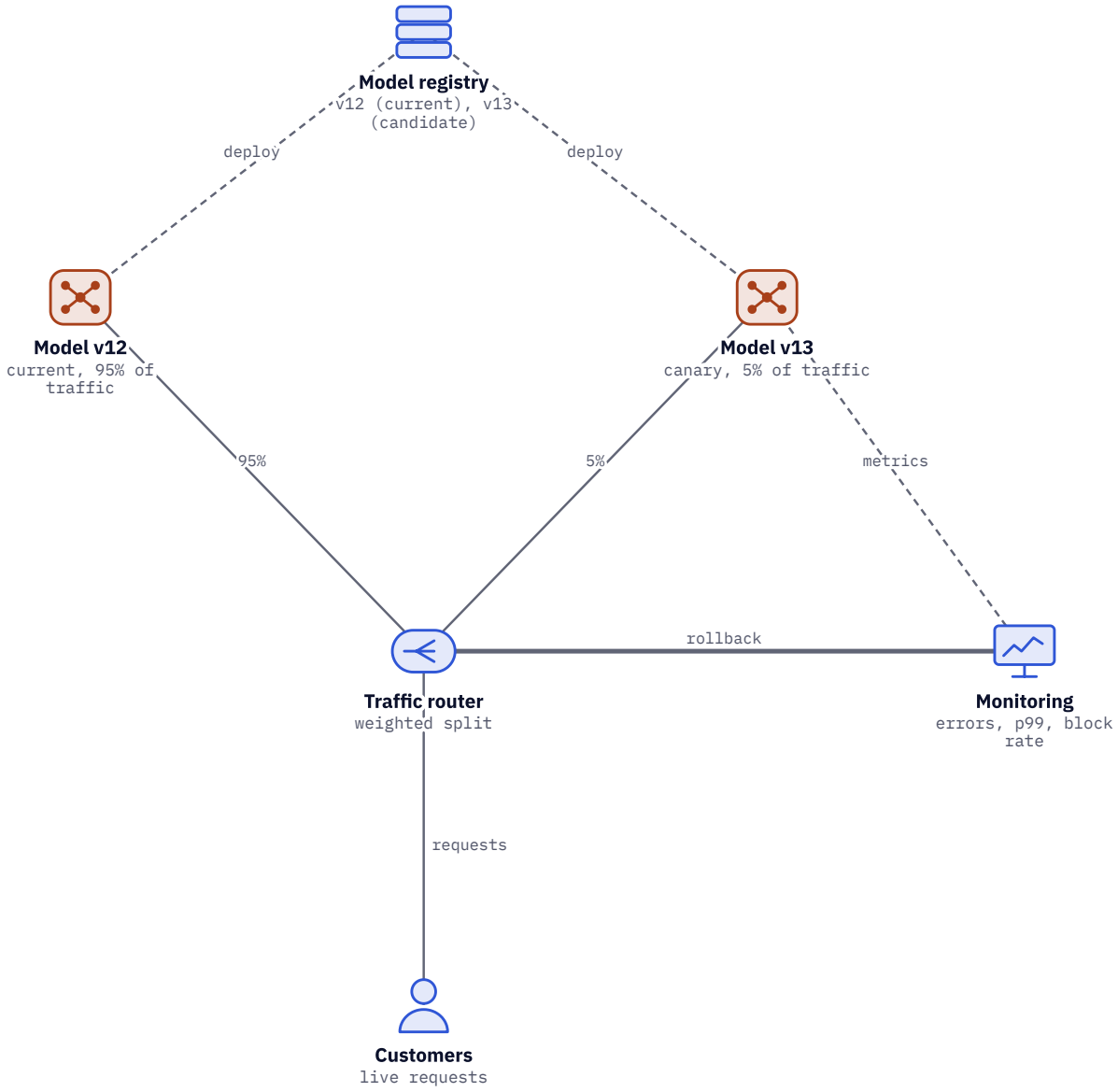
- Canary: send a small share of traffic (e.g. 5%) first, then increase
- Blue–green: switch all traffic between two environments, instant rollback
- Limits the blast radius of a bad release

A/B test

- Random split of users between models
- Measures causal effect on business metrics
- Needs enough traffic and time for statistical power

Copyright © ZyeemZia

A canary release with automatic rollback



Only 5% of customers meet the new version at first. If errors, latency or block rate cross agreed thresholds, monitoring tells the router to send all traffic back to v12 within minutes.

Serving and experiment arithmetic

MEASURE	FORMULA	WHAT IT TELLS YOU
Little's law (concurrency)	`concurrent requests = arrival rate × latency`	200 requests/s × 0.05 s = 10 requests in flight on average; plan for peaks with headroom.
Latency percentiles	`p95 = latency below which 95% of requests complete`	Report p95 and p99, not just the average; slow tails hurt users and upstream timeouts.
A/B sample size (rule of thumb)	`n per arm ≈ 16 × p(1 − p) ÷ δ²`	About 80% power at 5% significance. Base conversion 10%, detecting a 1-point lift: 16 × 0.09 ÷ 0.0001 = 14,400 users per arm.

Copyright © ZyeemZia

A production-readiness path

1. **Package** Version the pipeline, dependencies and a model card.
2. **Interface** Define the input schema, output format, error handling and fallbacks (for example a rule-based score if the model is unavailable).
3. **Test** Unit tests for features, schema validation, reproducibility test, load test against the latency target.
4. **Shadow** Run on live traffic without acting; compare with offline expectations.
5. **Canary** Expose to a small share of traffic with automatic rollback triggers.
6. **Full release and review** Roll out fully, then review business impact after an agreed period.

TEMPLATE

Release plan

Agree how a model version moves from registry to full production, with clear go/no-go criteria and rollback triggers.

STAGE	TRAFFIC SHARE	DURATION	GO CRITERIA	ROLLBACK TRIGGER	APPROVER
Shadow	*100% scored, 0% acted on*	*7 days*	*Score correlation with current model ≥ 0.9; p99 < 150 ms*	*Error rate > 0.5%*	*ML lead*
Canary	*5%*	*3 days*	*Block rate within ±10% of control; no complaint spike*	*Block rate +25% or p99 > 200 ms*	*Fraud operations head*

SCENARIO · DECIDE · RELEASING A NEW FRAUD MODEL

A mobile financial service has a new fraud model with better offline results. Transfers must be checked within 150 ms, and a bad model could block thousands of genuine customers within minutes. What is the safest release plan?

(A) Shadow it on live traffic, then canary to a small share with automatic rollback on error, latency or block-rate thresholds

(B) Run a six-month A/B test before any deployment

(C) Switch all traffic at midnight when volumes are low

(D) Release to all users and monitor complaints on social media

Circle your choice. Answer and feedback: module answer key.

Copyright © ZyeemZia

REFLECT

For a model you know, which serving pattern fits, what latency is acceptable, and what should happen when the model service is unavailable?

Key terms

Batch inference
Scoring many records together on a schedule and storing the predictions.

Online inference
Scoring individual requests on demand, usually through an API with tight latency limits.

Training–serving skew
Differences between how features or preprocessing work in training and in production.

Feature store
A system that stores and serves consistent feature definitions and values for training and inference.

Rollback
Returning production to a previous known-good model version when a release causes problems.

Canary release
Exposing a new version to a small share of traffic before rolling it out widely.

Blue–green deployment
Running two production environments and switching traffic between them for instant rollback.

Key takeaways

- Pick the serving pattern from latency needs: batch, online, streaming or edge.
- Serialise and version the whole pipeline, and share feature definitions between training and serving.
- Release in stages: shadow, then canary with automatic rollback, then A/B tests for business impact.
- Plan capacity with arrival rate × latency and report p95/p99 latency, not averages.

KNOWLEDGE CHECK

1. A service receives 50 requests per second with an average latency of 0.2 s. How many requests are in flight on average?
   - (A) 10
   - (B) 250
   - (C) 0.25
   - (D) 4
2. Base conversion is 20% and you want to detect a 2-point change. Using $n \approx 16p(1-p) \div \delta^2$, roughly how many users per arm?
   - (A) 640
   - (B) 1,600
   - (C) 6,400
   - (D) 16,000
3. Why serialise the full pipeline instead of the model alone?
   - (A) Pipelines are smaller files
   - (B) So that production applies exactly the same preprocessing as training
   - (C) Models cannot be saved on their own without the preprocessing steps attached
   - (D) It removes the need for testing

Copyright © ZyeemZia

4. Which release exposes a model to live inputs with no effect on customers?

   (A) Canary release
   (B) Blue–green switch
   (C) A/B test
   (D) Shadow deployment

5. Nightly scores for a call-centre's next-day call list are best served by...

   (A) Online inference
   (B) Edge inference
   (C) Streaming inference
   (D) Batch inference

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 8 · DEPLOY · LESSON 2 · 26 MIN

Monitoring and Drift

Know when a model is going wrong before the business feels it

- List what to monitor across data, predictions, performance and business outcomes
- Distinguish covariate drift, label drift and concept drift
- Compute and interpret the population stability index
- Monitor when labels arrive late and decide when to retrain

Models are trained on the past but used in the future, and the world keeps changing: customer behaviour shifts after a festival, a competitor launches a new product, a sensor is recalibrated, a pandemic rewrites every pattern. **Monitoring** watches the model in production and raises the alarm when its inputs, outputs or results move.

What to monitor (top layer first)

Business outcomes	`Fraud losses, retention, revenue, patient outcomes` `The reason the model exists`
Model performance	`Precision, recall, MAE when labels arrive` `By segment, not just overall`
Prediction behaviour	`Score distribution, share of positive decisions` `Sudden shifts are early warnings`
Input data	`Missing values, schema changes, out-of-range values` `Feature distribution drift`
System health	`Latency p95/p99, error rates, throughput` `Resource usage and cost`

Kinds of drift

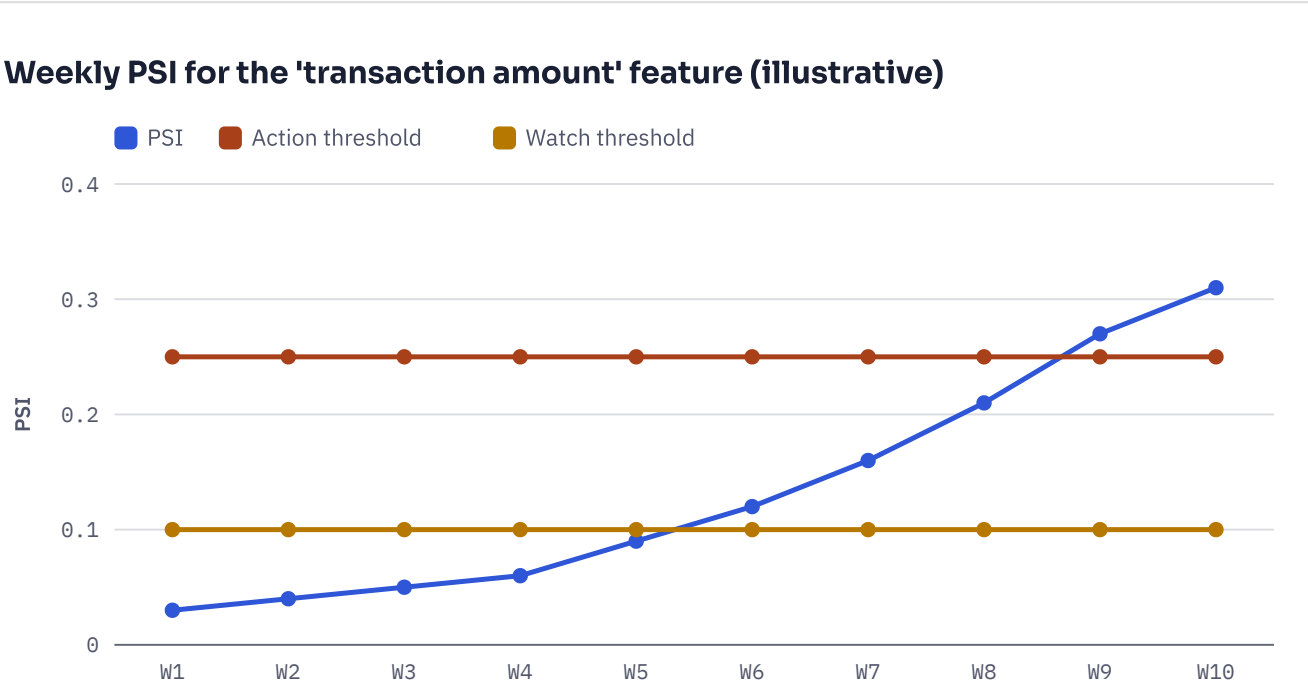
DRIFT	WHAT CHANGES	EXAMPLE	DETECTABLE WITHOUT LABELS?
Covariate (data) drift	Distribution of inputs P(x)	More young users join after a campus campaign	Yes, compare feature distributions
Label (prior) drift	Share of each outcome P(y)	Fraud rate doubles during a festival season	Partly, through prediction distribution
Concept drift	Relationship between inputs and outcome P(y | x)	Fraudsters change tactics, so old signals stop working	Usually only once labels arrive
Upstream data change	Meaning or format of a field	Amounts switched from taka to paisa in one system	Yes, with data validation rules

Population stability index (PSI)

MEASURE	FORMULA	WHAT IT TELLS YOU
Definition	$PSI = \Sigma_{bins} (a_i - e_i) \times \ln(a_i \div e_i)$	e_i = share of reference (training) data in bin i; a_i = share of current data in bin i.

Copyright © ZyeemZia

MEASURE	FORMULA	WHAT IT TELLS YOU
Worked example	`e = (0.25, 0.25, 0.25, 0.25); a = (0.10, 0.20, 0.30, 0.40)`	0.137 + 0.011 + 0.009 + 0.071 ≈ 0.228.
Common reading	`< 0.1 stable; 0.1–0.25 moderate shift; > 0.25 significant shift`	0.228 is a moderate shift close to the action threshold: investigate.
Practical notes	`Use reference-data deciles as bins; add a small constant to empty bins`	Zero shares make the logarithm undefined.



Illustrative numbers (x-axis: week). A slow climb through the watch zone gives weeks of warning before the action threshold; a sudden jump in a single week more often signals an upstream data change.

Computing PSI for a shifted feature

Python notebook

```
import numpy as np
rng = np.random.default_rng(0)
ref = rng.normal(50, 10, 5000)      # e.g. customer age at training time
cur = rng.normal(55, 10, 5000)      # this month's traffic
edges = np.quantile(ref, np.linspace(0, 1, 11)); edges[0], edges[-1] = -np.inf, np.inf
e = np.histogram(ref, edges)[0] / len(ref)
a = np.histogram(cur, edges)[0] / len(cur)
print(round(float(np.sum((a - e) * np.log(a / e))), 3))
# output: 0.266
```

A shift of half a standard deviation in the mean gives a PSI of about 0.27, above the usual 0.25 action threshold. PSI flags change; whether the change hurts performance still needs investigation.

Copyright © ZyeemZia

When labels arrive late

Leading indicators (now)

- Input drift and data quality
- Prediction distribution and decision rates
- Proxy outcomes such as early repayment signals
- Feedback from operations staff

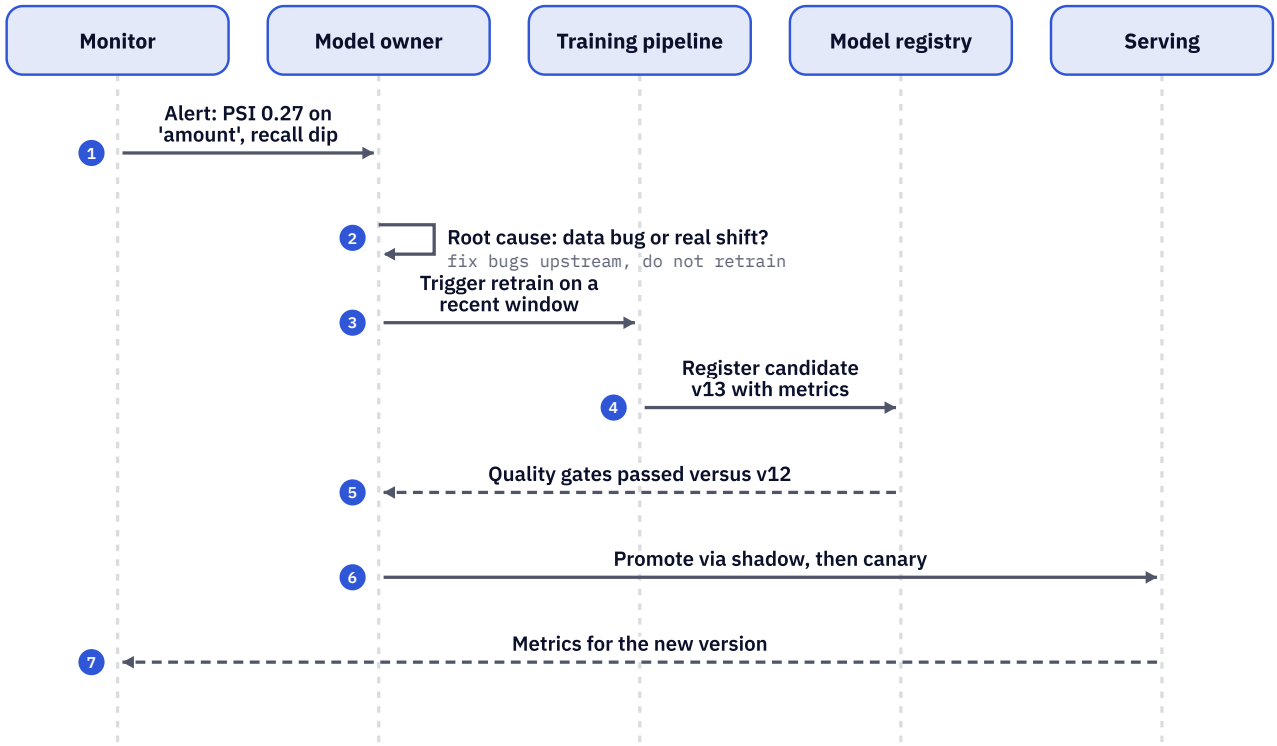
Lagging indicators (later)

- True performance once outcomes are known
- Loan default after 6–12 months
- Churn after 30 days
- Confirmed fraud after investigation

PRO TIP · RETRAINING POLICY

Decide in advance when to retrain: on a schedule (monthly), on a trigger (PSI above 0.25 on key features, or recall below an agreed floor), or both. Always validate a retrained model against the current one on recent data before promoting it; retraining on corrupted data can make things worse.

From drift alert to a safely promoted model



A trigger starts an investigation, not an automatic swap. Retraining is used only for genuine shifts, and the new model must beat the current one on recent data before it is staged.

SCENARIO · DECIDE · A SUDDEN SCORE SHIFT

On Monday the average fraud score jumps from 0.03 to 0.11 and alerts triple. No major fraud event is known. The 'transaction amount' feature's distribution has also shifted sharply. What should the team check FIRST?

- (A) Lower the threshold to reduce alerts
- (B) Whether an upstream change altered how the amount field is recorded, such as units or currency
- (C) Retrain the model immediately on this week's data
- (D) Assume a new fraud wave and block high-value transfers

Circle your choice. Answer and feedback: module answer key.

Copyright © ZyeemZia

TEMPLATE

Model monitoring plan

Define what is watched, the thresholds that trigger action and who responds, before go-live.

SIGNAL	METRIC	THRESHOLD	CHECK FREQUENCY	OWNER	ACTION
Input drift	*PSI on top 10 features*	*> 0.25 on any feature*	*Daily*	*ML engineer*	*Investigate source; consider retrain*
Performance	*Recall on confirmed fraud*	*< 0.70 over 2 weeks*	*Weekly*	*Fraud analytics lead*	*Root-cause review; retrain or roll back*

REFLECT

For a deployed or planned model, which signal would warn you first that something had gone wrong, and how many days or months would pass before true labels confirmed it?

Key terms

Model monitoring
Ongoing measurement of a deployed model's inputs, outputs, performance and system health.

Concept drift
A change in the relationship between inputs and the target, so the learned mapping becomes outdated.

Population stability index
A measure of how much a variable's distribution has shifted between two samples.

Retraining trigger
A predefined condition, such as drift or performance decline, that starts model retraining.

Data drift
A change in the distribution of model inputs compared with the training data.

Label drift
A change in the overall frequency of target classes.

Delayed labels
Outcomes that become known only some time after a prediction is made.

Copyright © ZyeemZia

Key takeaways

- Monitor in layers: system health, input data, predictions, performance and business outcomes.
- Covariate, label and concept drift differ; only some are visible before labels arrive.
- PSI flags distribution change; investigate the cause before retraining.
- Set retraining triggers and owners in advance, and validate every retrained model before promotion.
- Track performance by segment, since averages hide local failures.

KNOWLEDGE CHECK

6. Reference shares are (0.5, 0.5) and current shares are (0.6, 0.4). What is the PSI? (ln 1.2 ≈ 0.182, ln 0.8 ≈ −0.223)

   (A) About 0.2
   (B) About 0.04
   (C) About 0.4
   (D) Exactly 0.0

7. Fraudsters change tactics so that previously strong signals stop predicting fraud. Which drift is this?

   (A) Covariate drift
   (B) Label drift
   (C) Concept drift
   (D) System drift

8. Which signal is available immediately for a loan model whose defaults appear after a year?

   (A) Input and score distribution drift
   (B) Default rate of this month's loans
   (C) Final recall on this month's applicants
   (D) Twelve-month profit

9. Why validate a retrained model before promoting it?

   (A) Retrained models are always identical to the old model
   (B) Retraining on recent data can make a model worse, for example if that data is corrupted
   (C) Validation is only needed for the first version, since later versions reuse its settings
   (D) Promotion happens automatically anyway

10. Why monitor performance by segment rather than overall?

    (A) Segments always have more data than the whole
    (B) Overall metrics cannot be computed
    (C) Segments remove the need for labels
    (D) A decline in one group can be hidden by stable overall averages

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 8 · DEPLOY · LESSON 3 · 26 MIN

MLOps: Versioning, Pipelines and Automation

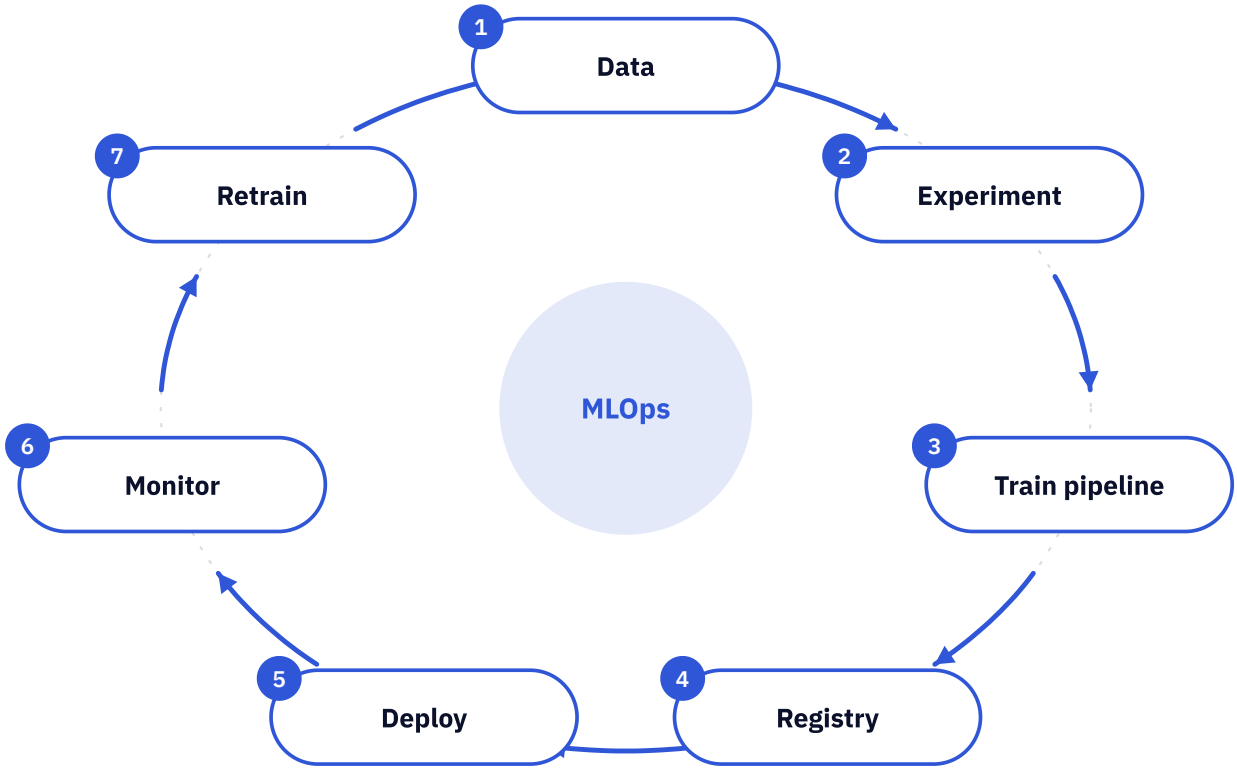
Make the whole ML life cycle repeatable, testable and auditable

- Describe the MLOps life cycle and its main components
- Version data, code, models and configurations together
- Test ML systems at the data, model and behaviour levels
- Plan continuous training and deployment at a suitable maturity level

MLOps applies the discipline of software engineering and operations to machine learning. The goal is simple to state: any model in production can be traced to exactly the data, code and settings that produced it; it can be rebuilt automatically; it is tested before release; and it can be rolled back in minutes.

Copyright © ZyeemZia

The MLOps life cycle



1. **Data** — Ingest, validate, version
2. **Experiment** — Track runs, metrics, artefacts
3. **Train pipeline** — Automated, reproducible
4. **Registry** — Approve and stage models
5. **Deploy** — CI/CD with staged release
6. **Monitor** — Drift, performance, cost
7. **Retrain** — Scheduled or triggered

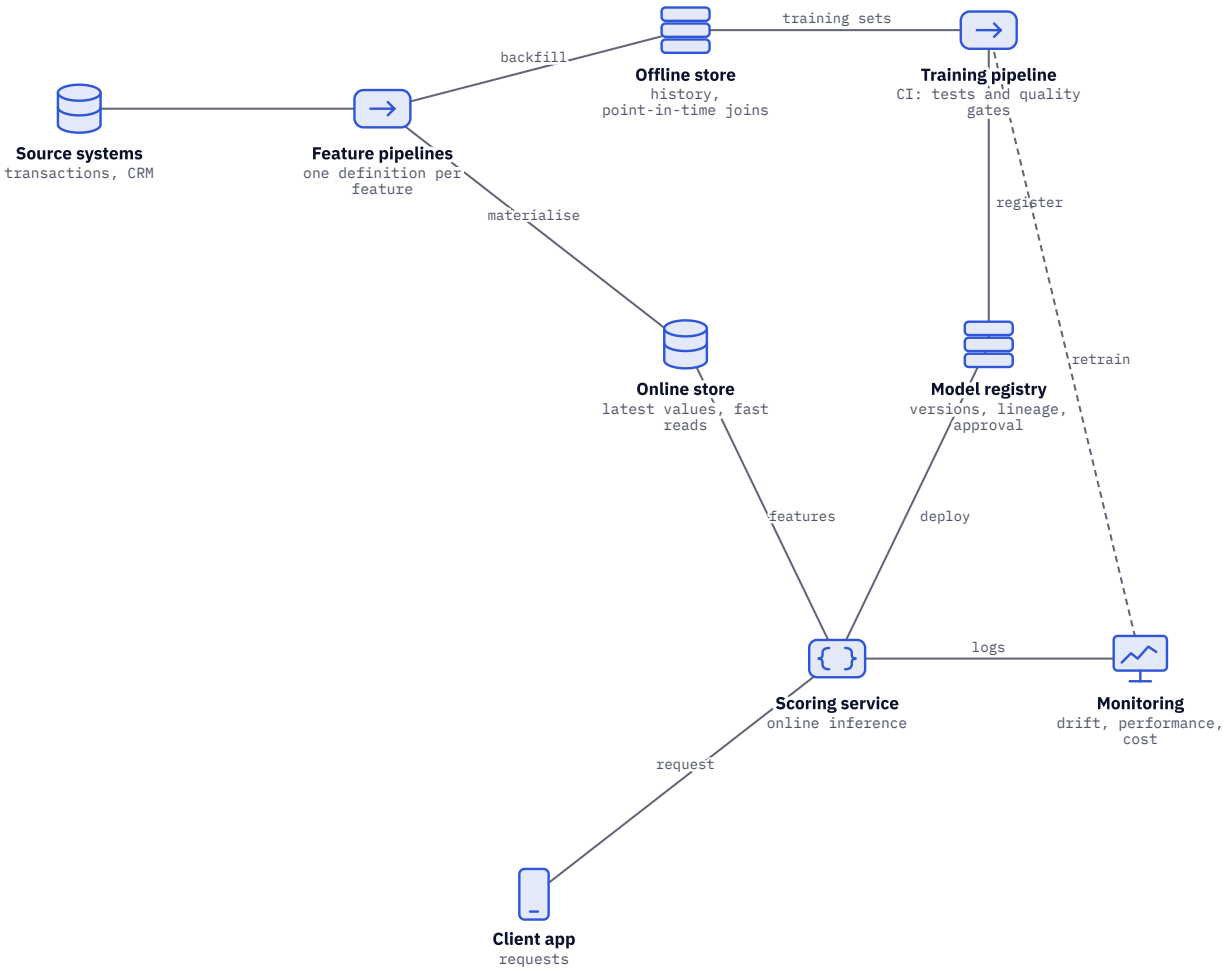
Retraining feeds straight back into data and experiments, so the life cycle is a loop. Automation is valuable exactly where the loop turns often.

Copyright © ZyeemZia

An MLOps platform (top layer first)

Governance	`Model cards, approvals, audit logs` `Access control`
Serving and monitoring	`APIs, batch jobs, dashboards, alerts`
Model registry	`Versions, stages (staging, production), lineage`
Pipelines and orchestration	`Training, evaluation and deployment workflows` `Schedules and triggers`
Experiment tracking	`Parameters, metrics, artefacts per run`
Data and feature management	`Versioned datasets, feature store, validation rules`

Training and serving around a feature store and registry



One feature definition fills both stores, so training and serving see identical values. The registry is the only route from training to serving, which gives every deployed model a traceable lineage.

Copyright © ZyeemZia

Testing ML systems

LEVEL	WHAT IS TESTED	EXAMPLES
Data tests	Schema, ranges, nulls, freshness, distribution	age between 0 and 110; no more than 2% missing income; data less than 24 hours old
Feature tests	Feature code gives correct, identical values in training and serving	Unit tests for a 30-day aggregate; parity test between batch and online paths
Model quality gates	Minimum metrics versus baseline and current model	AUC not lower than production by more than 0.005; recall ≥ 0.7 in every region
Behavioural tests	Expected directions and invariances	More late payments never lowers risk; changing a name does not change the score
Operational tests	Latency, memory, load, failure handling	p99 < 150 ms at 300 requests/s; fallback works when the model times out
Language-model application tests	Prompts, retrieval and outputs against a versioned evaluation set	Answer accuracy and citation support on 200 reference questions after every prompt or model change; prompt-injection test cases

A behavioural (directional) test

Python notebook

```
import numpy as np
from sklearn.linear_model import LogisticRegression
rng = np.random.default_rng(0)
late = rng.integers(0, 6, 1000); income = rng.normal(50, 15, 1000)
y = (rng.random(1000) < 1 / (1 + np.exp(-(0.6 * late - 0.03 * income)))).astype(int)
model = LogisticRegression().fit(np.c_[late, income], y)
p_base = model.predict_proba([[1, 50.0]])[0, 1]
p_worse = model.predict_proba([[4, 50.0]])[0, 1]
assert p_worse > p_base, 'more late payments must not lower predicted risk'
print('directional test passed', round(p_base, 2), round(p_worse, 2))
# output: directional test passed 0.32 0.75
```

Tests like this run automatically in the training pipeline. If a retrained model ever violated agreed domain logic, the pipeline would fail and the model would not be promoted.

MLOps maturity

LEVEL	CHARACTERISTICS	TYPICAL FOR
0: Manual	Notebooks, manual hand-over, no monitoring	First proof of concept
1: Automated training	Reproducible pipeline, experiment tracking, registry, scheduled retraining	First production models
2: Full CI/CD/CT	Tested code and pipelines deployed automatically; triggered retraining; automated quality gates and rollback	Many models, frequent change, high stakes

CASE STUDY · MLOPS FOR A FRAUD PLATFORM

A mobile financial service runs four fraud models built by different analysts. Each was trained in a personal notebook; two can no longer be reproduced because the original data extracts were overwritten. Retraining takes

Copyright © ZyeemZia

three weeks of manual work, so models are refreshed twice a year while fraud patterns change monthly. The regulator has asked how a specific customer's transfer was blocked last March.

1. What are the main risks in the current set-up?

2. Which MLOps capabilities would you introduce first, and why?

3. How would you answer the regulator's question in future?

Model answer: module answer key.

SCENARIO · DECIDE · IRREPRODUCIBLE RESULTS

A data scientist cannot reproduce last quarter's model: rerunning the notebook gives AUC 0.79 instead of 0.82. The data extract was refreshed in place, library versions have changed and no random seed was set. What is the most important fix for the future?

- (A) Save only the final model file
- (B) Retrain until AUC reaches 0.82 again
- (C) Avoid updating libraries ever again
- (D) Version data snapshots, code, environment and configuration together for every training run

Circle your choice. Answer and feedback: module answer key.

Production-readiness checklist

- ☐ Data, code, environment and configuration are versioned for the model
- ☐ The training pipeline runs end to end without manual steps
- ☐ Data tests and model quality gates block bad releases
- ☐ Behavioural tests encode agreed domain rules
- ☐ The model is registered with an owner, model card and approval
- ☐ Monitoring dashboards and alerts are live with named responders
- ☐ Rollback to the previous version has been tested

REFLECT

Where does your team sit on the MLOps maturity scale today? Name the single capability that would remove the most manual risk, and what it would take to introduce it.

Copyright © ZyeemZia

Key terms

MLOps
Practices and tools for reliably building, deploying, monitoring and maintaining ML systems.

Model registry
A central store of model versions with metadata, lineage and deployment stage.

Continuous training
Automatically retraining models on new data on a schedule or trigger.

Behavioural test
A test of whether a model's outputs respond to inputs in expected, agreed ways.

Experiment tracking
Recording parameters, metrics, code versions and artefacts for each training run.

Lineage
The traceable chain from data and code through training to a deployed model and its predictions.

Quality gate
An automated check that a model must pass before it can be promoted.

Key takeaways

- Any production model should trace to exact data, code, environment and settings.
- Automate the pipeline and block bad releases with data tests, quality gates and behavioural tests.
- A feature store and a model registry connect training and serving consistently.
- For language-model applications, version prompts, retrieval indexes and evaluation sets like code.
- Mature teams retrain on triggers and can roll back in minutes.

KNOWLEDGE CHECK

11. Which item is NOT needed to reproduce a training run?

(A) The colour scheme of the dashboard
(B) The exact data snapshot
(C) The code version
(D) The library versions and random seeds

12. What does a model registry mainly provide?

(A) Raw data storage for all systems, replacing the data warehouse and lake
(B) Automatic labelling of data
(C) Versioned models with metadata, lineage and deployment stages
(D) A replacement for monitoring

13. Which is a behavioural test?

(A) Checking that a column has no nulls
(B) Measuring p99 latency
(C) Checking that changing an applicant's name does not change the score
(D) Counting rows in the training table

14. A team retrains manually twice a year, but patterns change monthly. Which capability helps most?

(A) An automated training pipeline with drift-based retraining triggers
(B) Larger GPUs
(C) A bigger test set
(D) More dashboards for managers, without alerts or any retraining triggers

15. Why log the model version with every prediction?

(A) To make predictions faster
(B) To reduce storage costs
(C) Because logging versions improves the model's accuracy over time
(D) To trace and explain any individual decision later

Copyright © ZyeemZia

☐ Lesson complete · Date _____ · Knowledge check score ____ / 5

Copyright © ZyeemZia

MODULE 8 · DEPLOY · LESSON 4 · 28 MIN

Responsible Machine Learning

Fairness, transparency, privacy, security and governance in practice

- Measure group fairness with selection rates, disparate impact and error-rate gaps
- Choose transparency and documentation suited to the risk
- Apply privacy and security safeguards across the ML life cycle
- Set up proportionate governance with human oversight

Machine learning now decides who gets a loan, which patient is seen first and which job applicant is interviewed. Responsible ML means making those systems **fair, transparent, private, secure and accountable**, in proportion to the harm they could cause. It is not a final checklist: it runs through framing, data, modelling, deployment and monitoring.

Five pillars of responsible ML

F

Fairness

Outcomes and errors do not unjustifiably differ across groups.

T

Transparency

People can understand what the system does and why a decision was made.

P

Privacy

Personal data is minimised, protected and used only as agreed.

S

Security and robustness

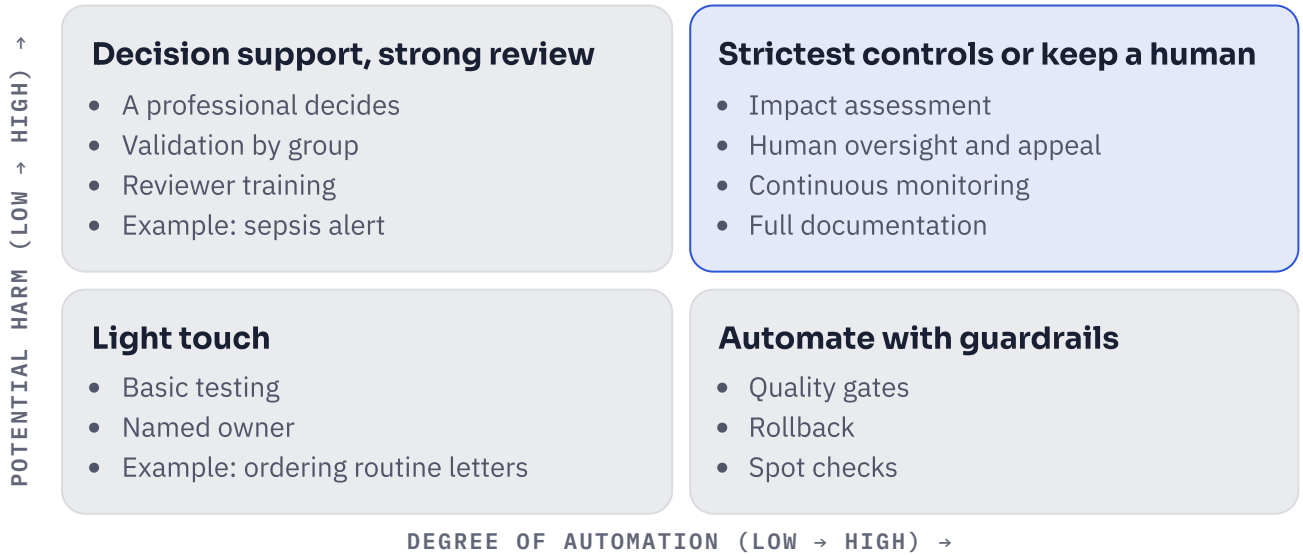
The system resists manipulation, poisoning and misuse, and fails safely.

A

Accountability

Named owners, documentation, human oversight and routes to challenge decisions.

Proportionate oversight



Controls should grow with the harm a system can cause and with how little a human checks it. The top-right cell is where regulation is strictest and where automation most needs justifying.

Copyright © ZyeemZia

Group fairness measures

MEASURE	FORMULA	WHAT IT TELLS YOU
Selection rate	`SR_g = positive decisions in group g ÷ members of group g`	Group A: 60 of 100 approved = 0.60. Group B: 36 of 80 approved = 0.45.
Disparate impact ratio	`DI = SR_lower ÷ SR_higher`	0.45 ÷ 0.60 = 0.75, below the common 0.8 screening rule, so investigate.
Equal opportunity gap	`TPR_A − TPR_B, where TPR = approved among those who would repay`	Compares how often qualified people in each group are approved.
Calibration within groups	`P(y = 1 | score = s, group = g) similar across groups`	A score of 0.2 should mean about 20% risk in every group.
The trade-off	`With different base rates, these criteria generally cannot all hold at once`	Choose the fairness definition that fits the decision and document why.

Checking selection rates and error gaps by group

Python notebook

```
import pandas as pd
df = pd.DataFrame({'group': ['A'] * 5 + ['B'] * 5,
                   'approved': [1, 1, 1, 0, 1, 1, 0, 0, 1, 0],
                   'repaid':   [1, 1, 0, 0, 1, 1, 1, 0, 1, 1]})
rates = df.groupby('group').approved.mean()
print(rates.to_dict(), round(rates['B'] / rates['A'], 2))
# output: {'A': 0.8, 'B': 0.4} 0.5
tpr = df[df.repaid == 1].groupby('group').approved.mean()
print(tpr.round(2).to_dict())
# output: {'A': 1.0, 'B': 0.5}
```

Group B is approved half as often (disparate impact 0.5), and among people who would repay, only half of group B are approved versus all of group A. Real audits use far larger samples and confidence intervals.

Where unfairness enters, and what helps

STAGE	SOURCE	MITIGATION
Framing	Proxy target encodes past inequality	Choose targets carefully; consult affected groups
Data	Under-representation, historical bias, label bias	Collect more representative data; audit labels
Features	Proxies for protected attributes (postcode, name)	Review features; test the impact of removing them
Model and threshold	Different error rates across groups	Measure per group; adjust thresholds or retrain with constraints where lawful and justified
Deployment	Feedback loops; uneven human override	Monitor outcomes per group; review overrides

Copyright © ZyeemZia

Privacy and security safeguards

Privacy

- Collect only what the purpose needs
- Pseudonymise and restrict access
- Retention limits and deletion
- Techniques such as differential privacy or federated learning where risk is high
- Follow applicable data protection law and consent terms

Security

- Data poisoning: validate and track training data sources
- Adversarial inputs: test robustness, rate-limit, monitor
- Model extraction and inversion: limit output detail, authenticate APIs
- Secrets and supply chain: scan dependencies, protect model files
- Language-model apps: prompt injection and data leakage through outputs; treat inputs and retrieved text as untrusted

TEMPLATE **Model card**

A short, standard record that travels with a model so users, reviewers and auditors understand it.

SECTION	CONTENT
Intended use and users	*Rank small-business loan applications for officer review; not for automatic rejection*
Performance by group	*AUC 0.79 overall; 0.77–0.81 across regions; approval-rate ratio by gender 0.91*

Documentation travels with the system. A **datasheet for a dataset** records why and how the data was collected, its composition, labelling process, known gaps and permitted uses; a **model card** records intended use, performance by group, limitations and owners. Risk-based regulation expects similar evidence: the EU AI Act, for example, treats uses such as credit scoring, recruitment and access to essential services as high-risk, with duties for risk management, data governance, technical documentation, logging and human oversight.

Copyright © ZyeemZia

AI governance milestones a practitioner should know

JAN 2023
NIST AI Risk Management Framework 1.0
Voluntary framework: govern, map, measure, manage.

DEC 2023
ISO/IEC 42001
International standard for an AI management system.

JUL 2024
NIST generative AI profile
Companion to the framework for generative-model risks.

AUG 2024
EU AI Act enters into force
Risk-based rules that phase in over several years.

FEB 2025
EU AI Act: prohibitions apply
Banned practices, plus AI-literacy duties.

AUG 2025
EU AI Act: general-purpose models
Documentation and transparency duties for providers of general-purpose AI models.

FROM AUG 2026
EU AI Act: high-risk systems
Phased obligations; changes to these dates were proposed in late 2025, so check the current official timetable.

Even outside the EU, these documents shape procurement, audits and partner expectations. Use the dates for orientation and always confirm the current official text and any local law that applies.

CASE STUDY · A FAIRNESS REVIEW FOR A LENDING MODEL

A digital lender in Bangladesh uses a gradient-boosted model to approve small loans through a mobile app. An internal review finds that approval rates for women applicants are 40% versus 55% for men (ratio 0.73). Gender is not a model input, but 'type of mobile handset', 'top-up pattern' and 'time of app use' are, and repayment rates among approved women are higher than among approved men.

1. What do these facts suggest about the model?

2. What analyses would you run next?

3. What actions could the lender take, and who should decide?

Copyright © ZyeemZia

Model answer: module answer key.

SCENARIO · DECIDE · PROPORTIONATE OVERSIGHT

A hospital plans two ML systems: one suggests the order in which non-urgent outpatient letters are typed, the other flags emergency patients for possible sepsis. How should oversight differ?

(A) Avoid ML in hospitals entirely

(B) Let both run automatically with no human review

(C) Apply light controls to the letter-ordering tool and strict validation, clinician oversight and monitoring to the sepsis tool

(D) Apply identical heavy review to both, so that nothing is missed

Circle your choice. Answer and feedback: module answer key.

REFLECT

Pick one model that affects people in your organisation. Which groups could be treated differently by it, which fairness measure would matter most, and who is accountable if it causes harm?

Key terms

Responsible ML
Designing, building and operating ML systems that are fair, transparent, private, secure and accountable.

Equal opportunity
A fairness criterion requiring similar true positive rates across groups.

Model card
A short document describing a model's purpose, data, performance, limitations and ownership.

Human oversight
Arrangements that let people monitor, interpret, override or stop an automated system.

Disparate impact ratio
The ratio of selection rates between a less-favoured and a more-favoured group.

Proxy variable
A feature that indirectly reveals a sensitive attribute, such as postcode standing in for ethnicity.

Data poisoning
An attack that corrupts training data to change a model's behaviour.

Datasheet for a dataset
A document recording a dataset's purpose, collection, composition, labelling, known gaps and permitted uses.

Key takeaways

- Responsible ML covers fairness, transparency, privacy, security and accountability, in proportion to harm.
- Measure fairness by group (selection rates, error rates, calibration); the criteria can conflict, so choose and document.
- Unfairness enters through targets, data, proxy features, thresholds and feedback loops.
- Datasheets and model cards travel with the system; regulation increasingly expects them.
- Scale oversight with potential harm and the degree of automation.

Copyright © ZyeemZia

KNOWLEDGE CHECK

16. Group X has a selection rate of 0.30 and group Y 0.50. What is the disparate impact ratio?

(A) 0.2
(B) 0.6
(C) 1.67
(D) 0.8

17. A model does not use gender, yet approval rates differ strongly by gender. What is a likely cause?

(A) Other features act as proxies for gender
(B) The model is perfectly fair by definition
(C) Gender must have leaked from the test set
(D) Disparate impact can only arise from explicit inputs

18. What does equal opportunity compare across groups?

(A) Overall accuracy
(B) Number of features used
(C) True positive rates among those who truly qualify
(D) Average age of applicants

19. Which is a data poisoning risk?

(A) A user forgets a password
(B) An attacker submits crafted records that are later used for training
(C) A model runs slowly on old hardware and keeps timing out during peak traffic
(D) A dashboard uses the wrong colours

20. What is the main purpose of a model card?

(A) To store the model's weights
(B) To replace monitoring
(C) To list every training row and label so that auditors can recheck the data by hand
(D) To document a model's intended use, performance, limitations and ownership

21. An auditor asks how a training dataset was collected, how it was labelled and which groups it under-represents. Which document should answer this?

(A) The model card
(B) The release plan
(C) The datasheet for the dataset
(D) The monitoring plan

☐ Lesson complete · Date _____ · Knowledge check score ____ / 6

Copyright © ZyeemZia

MODULE 8 · DEPLOY

Answer key

Knowledge checks

1. A 50 × 0.2 = 10 by Little's law.

2. C 16 × 0.16 ÷ 0.0004 = 6,400.

3. B Separating them invites training–serving skew.

4. D In shadow mode outputs are logged but not acted on.

5. D There is no need for real-time scoring; batch is simpler and cheaper.

6. B (0.1 × 0.182) + (−0.1 × −0.223) ≈ 0.018 + 0.022 = 0.040.

7. C The relationship between features and outcome has changed.

8. A Leading indicators come from inputs and predictions.

9. B New is not automatically better.

10. D Averages mask localised failures.

11. A Reproducibility depends on data, code, environment and configuration.

12. C The registry is the source of truth for model versions.

13. C Behavioural tests check invariances and expected directions of model outputs.

14. A Automation makes frequent, safe retraining practical.

15. D Prediction-level lineage supports audits and debugging.

16. B 0.30 ÷ 0.50 = 0.6.

17. A Removing a sensitive attribute does not remove correlated proxies.

18. C It asks whether qualified people are treated alike.

19. B Poisoning corrupts training data to change model behaviour.

20. D It communicates essential facts to users and reviewers.

21. C A datasheet documents a dataset's purpose, collection, labelling, composition and known gaps. A model card covers the model's intended use and performance, while release and monitoring plans cover deployment.

Scenarios & case studies

Scenario · Releasing a new fraud model
A is best. Correct. Shadow mode exposes latency and prediction differences without risk; a monitored canary limits harm if something is wrong.

Scenario · A sudden score shift
B is best. Correct. A sharp, simultaneous shift in one feature and the scores usually points to a pipeline or data change rather than real-world behaviour.

Case study · MLOps for a fraud platform
Risks include irreproducible models, stale models facing concept drift, key-person dependency, no consistent testing and no audit trail. Start with versioned data snapshots and code, experiment tracking and a model registry, then an automated training pipeline with data tests, quality gates and behavioural tests, followed by monitoring with drift triggers so retraining can run monthly. To answer the regulator, log every prediction with model version, input features, score, threshold and reason codes, linked through the registry to the training data version and code commit; then the March decision can be reconstructed and explained exactly.

Scenario · Irreproducible results
D is best. Correct. Reproducibility needs all inputs pinned: data, code, dependencies, parameters and seeds.

Case study · A fairness review for a lending model
Gender is not used directly, but other features may act as proxies, and the higher repayment of approved women suggests that qualified women are being rejected more often (an equal-opportunity gap), so the threshold or features are disadvantaging them. Next, measure approval rates, true positive rates and calibration by gender with confidence intervals, test how removing or modifying proxy-like features changes the gaps and accuracy, and review historical labels for bias. Options include removing or reworking proxy features, recalibrating or setting thresholds to equalise opportunity where lawful, adding human review for borderline cases and collecting better repayment-capacity data. The decision belongs to an accountable owner and governance committee with legal, risk and business input, documented in the model card and monitored after the change.

Copyright © ZyeemZia

Scenario · Proportionate oversight

C is best. Correct. Governance should be proportionate to potential harm; the sepsis tool affects life-critical decisions.

Copyright © ZyeemZia

REFERENCE

Glossary

A

Activation function A non-linear function, such as ReLU or sigmoid, applied to a neuron's weighted sum. (Deep)

Adam An adaptive optimiser combining momentum with per-parameter step sizes based on recent gradient magnitudes. (Fit)

Agent The learner and decision-maker in reinforcement learning. (Discover)

Aggregation window The time span, such as the last 30 days, over which a feature is summarised. (Prepare)

Alert fatigue The tendency of people to ignore alerts when too many are false or irrelevant. (Discover)

Annotation guidelines Written rules and examples that tell labellers how to assign each label consistently. (Prepare)

Anomaly detection Identifying records that differ markedly from the majority of the data. (Discover)

Attention head One of several parallel attention computations, each able to focus on different relationships. (Deep)

Autoencoder A neural network trained to reconstruct its input through a narrow bottleneck; poor reconstruction suggests an anomaly. (Discover)

B

Backpropagation The algorithm that computes gradients of the loss for every weight by applying the chain rule backwards through the network. (Deep)

Bagging Training models on bootstrap samples and averaging their predictions to reduce variance. (Ensemble)

Base rate The overall proportion of positive cases in the population. (Ground)

Baseline A simple reference result, such as the majority class or the current rule, that a model must beat. (Ground)

Batch inference Scoring many records together on a schedule and storing the predictions. (Deploy)

Batch normalisation Normalising layer activations across a mini-batch to stabilise and speed up training. (Deep)

Bayes' rule A formula that updates a prior probability using evidence: P(A | B) = P(B | A)P(A) ÷ P(B). (Ground)

Behavioural test A test of whether a model's outputs respond to inputs in expected, agreed ways. (Deploy)

Bias Error from overly simple assumptions that make a model systematically miss the true pattern. (Evaluate)

Blue–green deployment Running two production environments and switching traffic between them for instant rollback. (Deploy)

Bootstrap sample A sample of n rows drawn with replacement from a dataset of n rows. (Ensemble)

C

Calibration How well predicted probabilities match observed frequencies, for example 30% predicted means about 30% happen. (Fit)

Canary release Exposing a new version to a small share of traffic before rolling it out widely. (Deploy)

Chain rule The rule for differentiating composed functions: multiply the outer derivative by the inner derivative. (Ground)

Checkpoint A saved copy of model weights at a point in training. (Deep)

Class imbalance A situation where one class is much rarer than another in the target. (Prepare)

Class weight A multiplier on the loss for examples of a class, used to make rare classes count more. (Prepare)

Clustering Grouping unlabelled records so that items in the same group are more similar than items in different groups. (Discover)

Cohen's kappa An agreement statistic for two labellers that corrects for agreement expected by chance. (Prepare)

Cold start The difficulty of recommending for new users or items that have little or no interaction history. (Discover)

Collaborative filtering Recommending from patterns in many users' interactions, without needing item content. (Discover)

Collective anomaly A group or sequence of records that is unusual together. (Discover)

Concept drift A change in the relationship between inputs and the target, so the learned mapping becomes outdated. (Deploy)

Conditional probability The probability of an event given that another event has occurred, written P(A | B). (Ground)

Confusion matrix A table counting true positives, false positives, false negatives and true negatives. (Evaluate)

Content-based filtering Recommending items whose attributes resemble those a user liked before. (Discover)

Contextual anomaly A value that is unusual only given its context, such as time, place or entity. (Discover)

Continuous training Automatically retraining models on new data on a schedule or trigger. (Deploy)

Copyright © ZyeemZia

Convolutional neural network A neural network that uses learned filters slid across the input to detect local patterns. (Deep)

Cosine similarity The dot product of two vectors divided by the product of their norms; measures directional similarity. (Ground)

Curse of dimensionality The set of problems, such as distances losing meaning, that arise as the number of features grows. (Fit)

Cyclical encoding Representing periodic values such as hour or month with sine and cosine so the ends of the cycle are close. (Prepare)

D

Data augmentation Creating modified copies of training examples to improve robustness and reduce overfitting. (Deep)

Data drift A change in the distribution of model inputs compared with the training data. (Deploy)

Data leakage Information unavailable at prediction time that enters training or evaluation and inflates results. (Evaluate)

Data poisoning An attack that corrupts training data to change a model's behaviour. (Deploy)

Data profiling Systematic summary of a dataset's columns, types, ranges, missing values and duplicates. (Prepare)

Data provenance A record of where data came from and how it was transformed on the way to the model. (Prepare)

Datasheet for a dataset A document recording a dataset's purpose, collection, composition, labelling, known gaps and permitted uses. (Deploy)

DBSCAN A density-based clustering method that finds arbitrarily shaped clusters and marks sparse points as noise. (Discover)

Decision threshold The probability cut-off above which a case is assigned to the positive class. (Fit)

Decision tree A model that predicts by applying a sequence of feature-based splits ending in leaves. (Ensemble)

Decorrelation Making ensemble members' errors less similar so that averaging cancels more of them. (Ensemble)

Delayed labels Outcomes that become known only some time after a prediction is made. (Deploy)

Dendrogram A tree diagram showing how hierarchical clustering merges groups. (Discover)

Derivative The instantaneous rate of change of a function with respect to its input; the slope of the curve. (Ground)

Dimensionality reduction Representing data with fewer features while preserving as much useful information as possible. (Discover)

Discount factor The parameter γ that sets how much future rewards count relative to immediate ones. (Discover)

Disparate impact ratio The ratio of selection rates between a less-favoured and a more-favoured group. (Deploy)

Dot product The sum of products of matching entries of two vectors; the core of linear model scores. (Ground)

Dropout A regulariser that randomly disables a fraction of units during each training step. (Deep)

E

Early stopping Halting training when a validation metric stops improving for a set number of rounds. (Ensemble)

Eigenvalue In PCA, the amount of variance captured along a principal component. (Discover)

Elastic net A penalty combining L1 and L2 terms. (Fit)

Embedding A learned dense vector that represents an item such as a word, product or user so that similar items are close. (Ground)

Ensemble A model that combines the predictions of several models. (Ensemble)

Entropy $-\Sigma\, p_k \log_2 p_k$; a measure of uncertainty in a node's class distribution. (Ensemble)

Environment Everything the agent interacts with, which responds to actions with new states and rewards. (Discover)

Epoch One full pass through the training data. (Fit)

Equal opportunity A fairness criterion requiring similar true positive rates across groups. (Deploy)

Euclidean distance The straight-line distance between two points: the norm of their difference. (Ground)

Experiment tracking Recording parameters, metrics, code versions and artefacts for each training run. (Deploy)

Explained variance ratio The share of total variance captured by a component. (Discover)

Exploration–exploitation trade-off Balancing trying new actions to learn against using the best-known action to earn reward. (Discover)

F

F1 score The harmonic mean of precision and recall. (Evaluate)

Feature An input variable used by a model, such as age, transaction amount or pixel intensity. (Ground)

Feature engineering Creating new input variables from raw data to make patterns easier for a model to learn. (Prepare)

Feature importance A score of how much a model relies on a feature for its predictions. (Ensemble)

Feature selection Choosing a subset of features to improve generalisation, speed or interpretability. (Prepare)

Copyright © ZyeemZia

Feature store A system that stores and serves consistent feature definitions and values for training and inference. (Deploy)

Filter (kernel) A small matrix of weights convolved with the input to produce a feature map. (Deep)

Fine-tuning Continuing to train a pre-trained model on task-specific data. (Deep)

Forward pass Computing a network's output from its input by applying each layer in turn. (Deep)

Foundation model A large model pre-trained on broad data that can be adapted to many downstream tasks. (Deep)

G

Gaussian mixture model A probabilistic model that represents data as a blend of several normal distributions with soft memberships. (Discover)

Gini impurity $1 - \Sigma\, p_k^2$; the probability of mislabelling a random item if labelled by the node's class distribution. (Ensemble)

Gradient The vector of partial derivatives of a function; it points in the direction of steepest increase. (Ground)

Gradient boosting An ensemble method that adds trees sequentially, each fitted to the negative gradient of the loss of the current model. (Ensemble)

Gradient check Comparing an analytic gradient with a numerical finite-difference estimate to catch bugs. (Ground)

Gradient clipping Limiting the size of gradients during training to prevent unstable, exploding updates. (Deep)

Gradient descent An iterative method that updates parameters in the direction opposite to the gradient of the loss. (Fit)

Greedy algorithm An algorithm that makes the best local choice at each step without revisiting it. (Ensemble)

Group k-fold Cross-validation that keeps all rows of the same group together in one fold. (Evaluate)

GRU Gated recurrent unit: a simpler gated recurrent cell similar in purpose to an LSTM. (Deep)

H

Hidden layer A layer between the input and output whose values are learned intermediate features. (Deep)

Hidden state The vector a recurrent network carries forward that summarises the sequence so far. (Deep)

High cardinality A categorical feature with a very large number of distinct values. (Prepare)

Hinge loss The SVM loss $\max(0, 1 - y\, f(x))$, zero for points correctly classified beyond the margin. (Fit)

Histogram-based boosting Boosting that bins feature values into histograms to find splits quickly on large data. (Ensemble)

Human oversight Arrangements that let people monitor, interpret, override or stop an automated system. (Deploy)

Hyperparameter A setting chosen before training, such as the regularisation strength, rather than learned from data. (Fit)

I

ICE plot Individual conditional expectation: one line per case showing how its prediction changes with a feature. (Ensemble)

Implicit feedback Behavioural signals such as clicks or purchases used as indirect evidence of preference. (Discover)

Imputation Filling missing values with estimates such as the median, the most frequent value or a model prediction. (Prepare)

Inertia The sum of squared distances from points to their cluster centres; the k-means objective. (Discover)

Information gain The reduction in entropy achieved by a split. (Ensemble)

Inter-annotator agreement The degree to which independent labellers assign the same labels to the same items. (Prepare)

Interaction feature A feature that combines two or more features, such as their product, to capture joint effects. (Prepare)

Intercept The model's prediction when all features are zero; the bias term b. (Fit)

Intermittent demand A demand series with many zero periods, common for slow-moving products. (Evaluate)

Interquartile range (IQR) The difference between the 75th and 25th percentiles; a robust measure of spread. (Prepare)

Irreducible error Noise in the target that no model can remove. (Evaluate)

Isolation forest An ensemble that scores anomalies by how few random splits are needed to isolate a point. (Discover)

K

k-fold cross-validation Splitting data into k parts and training k times, each time validating on a different part. (Evaluate)

k-means A clustering algorithm that alternates between assigning points to the nearest centre and moving centres to their members' mean. (Discover)

k-nearest neighbours A method that predicts from the labels or values of the k most similar training examples. (Fit)

Copyright © ZyeemZia

Kernel trick Computing similarities as if data were mapped into a higher-dimensional space, without building that space explicitly. (Fit)

Knowledge distillation Training a smaller student model to imitate a larger teacher model's outputs. (Deep)

L

Label The known answer attached to a training example in supervised learning, also called the target. (Ground)

Label drift A change in the overall frequency of target classes. (Deploy)

Label noise Errors or inconsistencies in the target labels of a training set. (Prepare)

Lag feature A feature equal to the value of a series a fixed number of steps in the past. (Deep)

Lasso Linear regression with an L1 penalty on the sum of absolute weights; can zero out features. (Fit)

Leaf A terminal node of a tree that holds the prediction. (Ensemble)

Learning curve A plot of training and validation performance against training set size. (Evaluate)

Learning rate The step size used when updating parameters in the direction opposite to the gradient. (Ground)

Learning-rate schedule A planned change of the learning rate during training, such as warm-up then decay. (Fit)

Least squares Fitting parameters by minimising the sum of squared differences between predictions and targets. (Fit)

Lift How much more often the target occurs in a model-selected group than in the population as a whole. (Ground)

Lineage The traceable chain from data and code through training to a deployed model and its predictions. (Deploy)

Linear regression A model that predicts a numeric target as a weighted sum of features plus an intercept. (Fit)

Local outlier factor A score comparing a point's local density with that of its neighbours. (Discover)

Log loss Binary cross-entropy; the average negative log probability assigned to the true class. (Fit)

Log transform Applying a logarithm to compress large values and reduce right skew. (Prepare)

Log-odds (logit) The natural log of p ÷ (1 − p); logistic regression models it as a linear function of features. (Fit)

Logistic regression A linear classifier that maps a weighted sum of features through the sigmoid to give a probability. (Fit)

LoRA (low-rank adaptation) A parameter-efficient fine-tuning method that freezes pre-trained weights and trains small low-rank matrices added to them. (Deep)

Loss function A function that measures how wrong a model's predictions are; training tries to minimise it. (Ground)

LSTM Long short-term memory: a recurrent unit with gates that control what is remembered and forgotten. (Deep)

M

Machine learning Building systems that improve at a task by learning patterns from data instead of following hand-written rules. (Ground)

MAE Mean absolute error: the average size of errors, in the target's units. (Evaluate)

MAPE Mean absolute percentage error: average error as a percentage of the actual value. (Evaluate)

Margin The distance between the decision boundary and the closest training points of each class. (Fit)

Matrix A rectangular array of numbers; a dataset is commonly an n × d matrix. (Ground)

Matrix factorisation Representing users and items as low-dimensional vectors whose dot product predicts interaction. (Discover)

max_features The number of features randomly considered at each split in a random forest. (Ensemble)

MCAR Missing completely at random: missingness unrelated to any data, observed or not. (Prepare)

Min–max scaling Rescaling a feature linearly into a fixed range, usually 0 to 1. (Prepare)

Mini-batch A small subset of training examples used to compute one gradient update. (Fit)

Missing indicator A binary feature recording whether a value was originally missing. (Prepare)

MLOps Practices and tools for reliably building, deploying, monitoring and maintaining ML systems. (Deploy)

MNAR Missing not at random: missingness depends on the unobserved value itself. (Prepare)

Model A function with learned parameters that maps input features to a prediction. (Ground)

Model card A short document describing a model's purpose, data, performance, limitations and ownership. (Deploy)

Model monitoring Ongoing measurement of a deployed model's inputs, outputs, performance and system health. (Deploy)

Model registry A central store of model versions with metadata, lineage and deployment stage. (Deploy)

Model variance Error from a model's sensitivity to the specific training sample it saw. (Evaluate)

Momentum An optimiser technique that accumulates past gradients to smooth and accelerate updates. (Fit)

Monotonic constraint A rule that forces a model's prediction to move in only one direction as a feature increases. (Ensemble)

Copyright © ZyeemZia

MSE Mean squared error: the average squared error, sensitive to large mistakes. (Evaluate)

Multi-armed bandit A simplified RL problem with actions and immediate rewards but no state transitions. (Discover)

Multi-label classification A task where each example can belong to several classes at once. (Deep)

Multicollinearity Strong correlation among features that makes individual coefficients unstable and hard to interpret. (Fit)

N

Naive Bayes A probabilistic classifier that assumes features are conditionally independent given the class. (Fit)

NDCG Normalised discounted cumulative gain: a ranking metric that rewards relevant items placed near the top. (Discover)

Nested cross-validation An outer cross-validation loop for scoring wrapped around an inner loop for tuning. (Evaluate)

Neural network A model built from layers of weighted sums and non-linear activations, trained by gradient descent. (Deep)

Neuron A unit that computes a weighted sum of inputs plus a bias and applies an activation function. (Deep)

Norm The length of a vector; the Euclidean (L2) norm is the square root of the sum of squares. (Ground)

Normal distribution The symmetric bell-shaped distribution defined by a mean and a standard deviation. (Ground)

Normal equation The closed-form least-squares solution $w = (X^TX)^{-1}X^Ty$. (Fit)

O

Odds ratio The factor by which the odds change for a one-unit increase in a feature: e raised to its coefficient. (Fit)

One-hot encoding Representing a categorical value as a set of binary columns, one per category. (Prepare)

Online inference Scoring individual requests on demand, usually through an API with tight latency limits. (Deploy)

Ordinal encoding Mapping ordered categories to integers that preserve their order. (Prepare)

Out-of-bag (OOB) score Accuracy estimated for each row using only trees that did not include it in their bootstrap sample. (Ensemble)

Outlier An observation far from most others; it may be an error or a genuine rare event. (Prepare)

Overfitting When a model learns noise in the training data and performs worse on new data. (Fit)

Oversampling Duplicating or synthesising minority-class examples in the training data. (Prepare)

P

Padding Adding border values around the input so filters can cover the edges and control output size. (Deep)

Paired comparison Comparing two models on exactly the same folds or examples so shared difficulty cancels out. (Evaluate)

Parameter-efficient fine-tuning Adapting a large model by training a small number of added or selected parameters. (Deep)

Partial dependence plot A plot of the average model prediction as one feature varies, others averaged out. (Ensemble)

Partial derivative The derivative of a multi-variable function with respect to one variable, holding the others fixed. (Ground)

Percentile The value below which a given percentage of observations fall; the 50th percentile is the median. (Ground)

Permutation importance The drop in a model's score when one feature's values are randomly shuffled. (Ensemble)

Pipeline A chained sequence of preprocessing steps and a model that is fitted and applied as one object. (Prepare)

Point-in-time correctness Computing features only from data that existed at the moment of prediction. (Prepare)

Policy The agent's rule for choosing actions given a state. (Discover)

Pooling Downsampling a feature map, for example by taking the maximum in each small block. (Deep)

Population stability index A measure of how much a variable's distribution has shifted between two samples. (Deploy)

Positional encoding Information added to token embeddings so that a transformer knows the order of tokens. (Deep)

Posterior The updated probability of a hypothesis after taking the evidence into account. (Ground)

PR-AUC The area under the precision–recall curve; informative for rare positive classes. (Evaluate)

Precision The share of predicted positives that are truly positive. (Evaluate)

Prediction interval A range expected to contain the actual value with a stated probability. (Evaluate)

Prediction time The moment a prediction is made; only data available at that moment may be used as features. (Ground)

Preference optimisation Tuning a model on comparisons of better and worse outputs, via a reward model and RL (RLHF) or directly (for example DPO). (Deep)

Copyright © ZyeemZia

Principal component A new axis formed as a weighted combination of the original features. (Discover)

Principal component analysis A linear method that finds orthogonal directions of maximum variance in the data. (Discover)

Prior The probability of a hypothesis before seeing the current evidence; in classification, often the base rate. (Ground)

Problem framing Defining the decision, target, unit, timing, metric and baseline before building a model. (Ground)

Proxy target A measurable stand-in for the real goal; it can carry bias or diverge from what truly matters. (Ground)

Proxy variable A feature that indirectly reveals a sensitive attribute, such as postcode standing in for ethnicity. (Deploy)

Pruning Removing branches that add little predictive value to reduce overfitting. (Ensemble)

Pseudo-residual The negative gradient of the loss for each row, which the next boosting tree is trained to predict; for squared error it equals the ordinary residual. (Ensemble)

Q

Q-learning An algorithm that learns action values by bootstrapping from the best estimated next-state value. (Discover)

Quality gate An automated check that a model must pass before it can be promoted. (Deploy)

Quantile loss An asymmetric loss that trains a model to predict a chosen percentile of the target. (Evaluate)

Quantisation Storing and computing with lower-precision numbers, such as 8-bit or 4-bit weights, to cut memory and speed up inference. (Deep)

R

R^2 (coefficient of determination) The proportion of variance in the target explained by the model relative to predicting the mean. (Fit)

Random forest A bagged ensemble of decision trees where each split considers a random subset of features. (Ensemble)

Random search Hyperparameter tuning by sampling settings at random from defined ranges. (Evaluate)

Reason code A plain-language statement of a top factor behind an individual model decision. (Ensemble)

Recall The share of actual positives that the model correctly identifies; also called sensitivity. (Evaluate)

Receptive field The region of the input image that influences a given unit in a network. (Deep)

Recommender system A system that predicts which items a user is likely to find relevant and ranks them. (Discover)

Reconstruction error The difference between original data and its reconstruction from a reduced representation. (Discover)

Recurrent neural network A network that processes a sequence step by step while maintaining a hidden state. (Deep)

Regularisation Adding a penalty or constraint during training to reduce model complexity and overfitting. (Fit)

Reinforcement learning Learning a policy of actions by trial and error to maximise long-term reward. (Ground)

Reliability diagram A plot of predicted probability against observed frequency used to check calibration. (Evaluate)

ReLU Rectified linear unit: max(0, z); the standard hidden-layer activation. (Deep)

Representativeness How well a dataset reflects the population and conditions where the model will be used. (Prepare)

Reproducibility The ability to rerun an experiment and obtain the same results from the same inputs. (Evaluate)

Residual The difference between an observed value and the model's prediction for it. (Fit)

Responsible ML Designing, building and operating ML systems that are fair, transparent, private, secure and accountable. (Deploy)

Retraining trigger A predefined condition, such as drift or performance decline, that starts model retraining. (Deploy)

Retrieval-augmented generation (RAG) Retrieving relevant passages from an index of your documents and adding them to the prompt so a language model answers from them. (Deep)

RFM features Recency, frequency and monetary features summarising customer behaviour over a window. (Prepare)

Ridge regression Linear regression with an L2 penalty on the sum of squared weights. (Fit)

RMSE Root mean squared error: the square root of MSE, back in the target's units. (Evaluate)

ROC-AUC The area under the curve of true positive rate against false positive rate; the probability a random positive outranks a random negative. (Evaluate)

Rollback Returning production to a previous known-good model version when a release causes problems. (Deploy)

S

Self-attention A mechanism in which each position computes weighted combinations of all positions based on query–key similarity. (Deep)

Self-supervised learning Learning from labels generated automatically from the data itself, such as predicting a hidden word. (Ground)

Copyright © ZyeemZia

Shadow deployment Running a model on live data without acting on its predictions, to compare with expectations. (Evaluate)

Shapley values Attributions from cooperative game theory that fairly divide a prediction among features. (Ensemble)

Shortcut learning A model relying on a spurious cue, such as a scanner marker, instead of the real signal. (Evaluate)

Shrinkage Scaling each new tree's contribution by a learning rate below 1 to improve generalisation. (Ensemble)

Sigmoid The S-shaped function $1 \div (1 + e^{-z})$ that maps any real number into the interval (0, 1). (Ground)

Silhouette score A measure from −1 to 1 of how well each point fits its own cluster compared with the nearest other cluster. (Discover)

SMOTE A technique that creates synthetic minority examples by interpolating between neighbouring minority points. (Prepare)

Softmax A function that converts a vector of scores into probabilities that sum to one. (Fit)

Standard deviation The square root of the variance, in the original units of the data. (Ground)

Standardisation Rescaling a feature to mean 0 and standard deviation 1. (Prepare)

Stochastic gradient descent Gradient descent using one or a few examples per update, giving noisy but cheap steps. (Fit)

Stratified sampling Splitting so that each subset keeps the overall class proportions. (Evaluate)

Stride The step size by which a filter moves across the input. (Deep)

Subsampling Training each tree on a random fraction of rows or features to add diversity and reduce overfitting. (Ensemble)

Supervised learning Learning a mapping from inputs to known outputs using labelled examples. (Ground)

Support vector machine A classifier that finds the boundary maximising the margin between classes. (Fit)

T

t-SNE A non-linear technique that maps high-dimensional data to two or three dimensions for visualisation, preserving local neighbourhoods. (Discover)

Target encoding Replacing a category with a statistic of the target, such as its smoothed mean, for that category. (Prepare)

Target leakage A feature that is derived from, or recorded after, the outcome being predicted. (Evaluate)

Target variable The quantity or category a supervised model is trained to predict. (Ground)

Temporal leakage Using data from after the prediction time in features or training. (Evaluate)

Test set Held-out data used once for a final, unbiased estimate of performance. (Evaluate)

TF-IDF A text weighting that increases with a word's frequency in a document and decreases with its frequency across documents. (Prepare)

Token A unit of text, such as a word or word piece, processed by a language model. (Deep)

Tokeniser The component that splits raw text into tokens and maps them to integer IDs. (Deep)

Training set The data used to fit a model's parameters. (Evaluate)

Training–serving skew Differences between how features or preprocessing work in training and in production. (Deploy)

Transfer learning Reusing a model trained on one task as the starting point for another. (Deep)

Transformer A neural architecture built from self-attention and feed-forward layers that processes all sequence positions in parallel. (Deep)

U

Underfitting When a model is too simple to capture the pattern, so training and validation scores are both poor. (Evaluate)

Undersampling Removing majority-class examples from the training data to balance classes. (Prepare)

Unit of prediction The entity and grain for which one prediction is made, such as customer-week or transaction. (Ground)

Unsupervised learning Finding structure, groups or unusual points in data that has no labels. (Ground)

V

Validation curve A plot of training and validation performance against one hyperparameter's value. (Evaluate)

Validation set Data used to compare models and tune hyperparameters during development. (Evaluate)

Vanishing gradient Gradients shrinking towards zero as they pass back through many layers, so early layers learn slowly. (Deep)

Variance The average squared deviation from the mean; a measure of spread. (Ground)

Vector An ordered list of numbers representing one example, a set of weights or an embedding. (Ground)

Vectorisation Writing computations as whole-array operations rather than loops, for speed and clarity. (Ground)

Copyright © ZyeemZia

W

Weak learner A simple model, such as a shallow tree, that is only slightly better than chance on its own. (Ensemble)

Weight decay Shrinking weights a little at every update; equivalent to L2 regularisation for plain gradient descent. (Fit)

Winsorising Capping extreme values at chosen percentiles instead of removing them. (Prepare)

Copyright © ZyeemZia

CAPSTONE

Capstone: Take one model along the ZMLS Model Pathway

Choose a real prediction problem from your work or a realistic public dataset (for example: churn for a mobile financial service in Dhaka, late-shipment risk for a garment exporter, readmission risk for a hospital, or load forecasting for a power utility). Take it through all eight stages of the ZMLS Model Pathway and produce a reproducible project that a reviewer can rerun and an operations team could run.

Deliverables

☐ Problem framing canvas: decision, target, unit, timing, metric and baseline

☐ Data report: sources, label quality, profiling, cleaning log and leakage review

☐ Feature pipeline with point-in-time features, encoding and scaling inside one fitted pipeline

☐ Baseline and linear or logistic model with regularisation tuned by cross-validation

☐ Evaluation plan and results: split design, metrics, threshold choice, calibration and error analysis by segment

☐ Tree-ensemble model (random forest or gradient boosting) with explanation of key drivers

☐ One unsupervised, recommendation, deep learning or foundation-model component (for example embeddings for search) justified for the problem

☐ Model comparison sheet with a reasoned final choice

☐ Deployment and monitoring plan: serving pattern, release strategy, drift checks and retraining triggers

☐ Model card and datasheet with fairness checks, limitations, privacy and governance notes

Self-assessment rubric

CRITERION	DEVELOPING	PROFICIENT	EXCELLENT
Framing	○ Target or metric unclear	○ Clear target, metric and baseline	○ Decision-linked metric with value estimate and stakeholder sign-off
Data & features	○ Raw data used as is	○ Cleaned data and sensible features	○ Point-in-time features, documented cleaning and leakage review
Modelling	○ Single untuned model	○ Baseline plus tuned candidates	○ Well-justified models with regularisation and clear trade-offs
Evaluation	○ Training score or random split only	○ Appropriate split, metrics and threshold	○ Deployment-matched validation, calibration and segment analysis
Ensembles & beyond	○ Not attempted	○ Working ensemble or advanced component	○ Tuned, explained and compared fairly with simpler models
Reproducibility	○ Results cannot be rerun	○ Code and data versions recorded	○ One-command pipeline with fixed seeds and tests
Deployment & monitoring	○ Not considered	○ Serving and monitoring plan	○ Release strategy, drift thresholds, owners and rollback
Responsible ML	○ Not addressed	○ Model card with basic checks	○ Measured fairness, privacy and risk controls proportionate to impact

Copyright © ZyeemZia

CAPSTONE

Planning pages

Sketch your design, timeline or architecture here.

Copyright © ZyeemZia

EXAM PREPARATION

Practice exam · 60 questions

The official online exam draws 40 questions at random from a separate, secret bank that tests the same skills, with a 60-minute limit and a 80% pass mark. Practise with these questions first: time yourself, mark your answers, then check the key and read every explanation, including for the questions you got right.

Ground — Foundations: Framing and the Maths of Machine Learning

1. A bank in Dhaka asks for an ML model to decide whether customers qualify for a statutory fee waiver. The eligibility rules are fixed in regulation and every input is in the core banking system. What should the ML lead recommend?

   (A) Train a gradient-boosted classifier on past waiver decisions
   (B) Implement the regulation directly as deterministic business rules
   (C) Collect a year of new data before deciding on an approach
   (D) Build a model on past decisions and use the regulation's rules only to explain its output

2. A telecom churn model reaches 98% accuracy. Monthly churn is 2%. The sponsor wants to deploy it immediately. What should you check FIRST?

   (A) Whether it beats the majority-class baseline at finding churners
   (B) Whether a deeper model with more features can push overall accuracy from 98% to 99%
   (C) Whether the model can be retrained daily
   (D) Whether the dashboard can display accuracy in real time

3. A garment exporter wants to predict late shipments at the time an order is accepted. A candidate feature is 'final inspection duration'. How should the team treat it?

   (A) Keep it because it has the highest correlation with lateness
   (B) Exclude it, since it is only known near the end of production
   (C) Impute it with the historical average at prediction time
   (D) Keep it but apply a log transform to reduce skew

4. A team compares customer embedding vectors whose lengths vary widely with account age, but the direction encodes behaviour. Which similarity measure suits finding customers with similar behaviour?

   (A) Raw dot product between the embedding vectors
   (B) Cosine similarity between the embedding vectors
   (C) Euclidean distance on the unscaled vectors
   (D) Difference between the vectors' norms

5. During training, a model's loss falls slowly but steadily and is still decreasing after many epochs, far above the expected minimum. Data and gradients have been checked. What is the most reasonable adjustment?

   (A) Reduce the learning rate by a factor of ten so that each update is more precise
   (B) Try a larger learning rate, such as three or ten times the current one
   (C) Remove half of the training data
   (D) Stop training and declare the model converged

6. A fraud model flags transactions with 90% sensitivity and a 1% false positive rate. Fraud occurs in 0.2% of transactions. Approximately what fraction of flagged transactions are fraudulent?

   (A) About 90%
   (B) About 99%
   (C) About 50%
   (D) About 15%

7. An analyst shows that districts with more mobile money agents have higher school enrolment and proposes adding agents to raise enrolment. What is the main flaw?

   (A) The sample of districts is too large to be reliable
   (B) The association may be driven by a common factor such as income or urbanisation
   (C) Enrolment cannot be predicted with regression
   (D) Agent counts must be standardised before any analysis of enrolment can be trusted

Copyright © ZyeemZia

8. A hospital wants to predict bed demand per ward per day for the next week. Which baseline should a model be required to beat?

   (A) The same weekday's demand from the previous week for each ward
   (B) A constant prediction of zero beds
   (C) The maximum demand ever observed in the hospital
   (D) A random number between the minimum and maximum demand

Prepare — Data Preparation and Feature Engineering

9. A hospital is labelling chest X-rays as 'pneumonia' or 'normal'. Two radiologists agree on 92% of images, but kappa is 0.41. What does this MOST likely indicate?

   (A) Their labels are excellent, because 92% agreement shows labelling can scale safely
   (B) Kappa is invalid for medical images
   (C) They agree on the common normal images but often disagree on pneumonia
   (D) One radiologist must be labelling at random

10. In a loan dataset, self-employed applicants leave the 'monthly income' field blank far more often than salaried applicants. Which handling is MOST appropriate for a gradient-boosted model?

    (A) Impute with a training-set statistic and add a 'was missing' indicator feature
    (B) Drop all rows with missing income before training
    (C) Fill blanks with zero income for every applicant
    (D) Fill blanks using the median income of the test set so that test rows look realistic

11. A data engineer standardises all features using the mean and standard deviation of the complete dataset, then performs cross-validation. What is the problem?

    (A) Standardisation is unnecessary for any model
    (B) Cross-validation cannot be used with standardised data, so a single split is required
    (C) Validation folds influenced the scaling statistics, a form of leakage
    (D) The mean should be replaced by the maximum value

12. A team wants to add 'merchant ID' (35,000 distinct values) to a logistic regression fraud model. Which encoding is MOST suitable?

    (A) One-hot encoding with a separate binary column for every one of the 35,000 merchants
    (B) Ordinal encoding using merchant ID numbers
    (C) Dropping the column because IDs are never useful
    (D) Out-of-fold smoothed target encoding or frequency encoding

13. A customer's 'transactions in the last 30 days' feature is computed from a table that already includes transactions after the scoring date. Offline AUC is 0.97, production AUC is 0.71. What is the root cause?

    (A) The production data has more customers than training
    (B) AUC is not a valid metric for churn
    (C) The feature was not point-in-time correct and leaked future behaviour
    (D) The model was trained for too many epochs

14. A fraud team oversampled the minority class to 50% and then split into training and test sets. Test precision is 0.97. What should they do to obtain a trustworthy estimate?

    (A) Increase oversampling to 70% fraud
    (B) Report recall only, since precision is unreliable whenever classes are imbalanced
    (C) Evaluate on the training set to confirm
    (D) Split first, resample only training data, and test at the real fraud rate

15. Which feature would MOST help a linear model capture that risk rises when debt is large relative to income?

    (A) A debt-to-income ratio feature
    (B) Debt and income standardised separately
    (C) The logarithm of income only
    (D) A one-hot encoding of the applicant's district

Copyright © ZyeemZia

16. A team ran feature selection using correlation with the target on the full dataset, then cross-validated a model on the selected features. Why might the reported score be optimistic?

(A) Selection used validation rows, so the chosen features fit those rows by chance
(B) Correlation cannot be computed on numeric features
(C) Cross-validation always underestimates performance, so the true score is even higher
(D) Fewer features always lower accuracy

Fit — Supervised Learning: Linear Models and Optimisation

17. A regression model for garment order lead time has training R^2 of 0.93 and test R^2 of 0.41. The model uses 220 features and 1,500 orders. What is the MOST appropriate next step?

(A) Add polynomial terms for every feature so the model can raise its test R^2 further
(B) Apply regularisation and tune its strength with cross-validation
(C) Report the training R^2 because it uses more data
(D) Remove the test set and train on all orders

18. A logistic regression for hospital readmission has a coefficient of 0.69 on 'prior admissions in the last year'. How should a clinician interpret it?

(A) Each extra prior admission roughly doubles the odds, other features held fixed
(B) Each extra admission adds 69 percentage points to the patient's readmission probability
(C) The model is 69% confident that frequent patients return
(D) Readmission probability is 0.69 for anyone with an admission

19. Training a model with gradient descent, the loss goes up after each epoch and turns into NaN. Inputs are standardised and contain no missing values. What should be changed FIRST?

(A) Add more hidden layers
(B) Lower the learning rate
(C) Increase the number of epochs
(D) Remove the intercept term

20. A churn dataset has 150 candidate features, and the business wants a model that uses as few as possible for easy monitoring. Which approach fits BEST?

(A) Ridge regression with a very large penalty
(B) L1-regularised logistic regression with strength tuned by cross-validation
(C) Unregularised logistic regression with all 150 features, then remove those with small coefficients
(D) k-nearest neighbours with k = 1

21. A data scientist uses gradient descent on features with very different units, and training zig-zags slowly. Which change MOST directly addresses the cause?

(A) Use a deeper network with more parameters
(B) Shuffle the rows in a different order
(C) Standardise the features to comparable scales
(D) Switch the loss from MSE to log loss

22. A model outputs softmax scores z = (1, 1, 1, 1) for four product categories. What does this indicate?

(A) The model is certain of the first class
(B) The softmax is broken, because scores for different classes must always differ
(C) The model is completely uncertain, assigning 0.25 to each class
(D) Each class has probability 1

23. A team needs a quick spam filter for SMS messages with only 2,000 labelled examples and must deploy on low-cost hardware. Which first model is MOST sensible?

(A) Multinomial naive Bayes on word count features
(B) A large transformer trained from scratch
(C) k-NN with k = 1 on raw message text
(D) A deep convolutional network on pixel images of messages

Copyright © ZyeemZia

Evaluate — Evaluation: Measuring Models Honestly

24. A model predicting monthly electricity demand for Dhaka feeders is validated with a random 80/20 split across 2019–2025 and reports MAPE of 3%. It will forecast the next six months. What is the MAIN flaw in this evaluation?

(A) Random splitting lets the model learn from periods after those it is validated on
(B) MAPE cannot be used for electricity demand
(C) Six years of data are too few for any forecast
(D) An 80/20 split is too small for time series; a 70/30 random split would be accurate

25. A clinic's model detects a disease with 1% prevalence. At the chosen threshold, recall is 0.90 and the false positive rate is 0.05. The director asks what share of flagged patients actually have the disease. What is the BEST estimate?

(A) About 90%
(B) About 95%
(C) About 50%
(D) About 15%

26. Two fraud models have ROC-AUC 0.97 and 0.96. Fraud is 0.1% of transactions and the review team can examine 500 alerts a day. Which comparison is MOST decision-relevant?

(A) The difference in ROC-AUC alone, since it summarises performance across all thresholds
(B) Overall accuracy at a 0.5 threshold
(C) Precision and recall among the top 500 scores per day for each model
(D) The training loss of each model

27. A team tuned 300 hyperparameter combinations using the test set to pick the best and reports its test score. What is wrong?

(A) 300 combinations are too few for a reliable search, so at least 1,000 should be tried
(B) The test set was used to make choices, so its score is optimistically biased
(C) Hyperparameters should never be tuned at all
(D) The test set should be larger than the training set

28. A model shows training RMSE 2.1 and validation RMSE 2.3, while a simple expert rule achieves 1.4. What does this suggest?

(A) The model overfits, so it needs stronger regularisation and fewer features
(B) The validation set is leaking into training
(C) The model should be deployed as it is
(D) The model underfits; it needs better features or more flexibility

29. A churn model's most important feature is 'retention offer accepted'. Offers are made only to customers who have already asked to leave. The model scores customers monthly. What should the team conclude?

(A) It is a strong driver and should be kept
(B) It should be log-transformed to reduce dominance
(C) It should be used only in the validation set
(D) The feature leaks the outcome and must be removed

30. Over five identical folds, model B beats model A by 0.004, 0.006, 0.005, 0.003 and 0.007 AUC. What is the MOST accurate conclusion?

(A) B is consistently better; whether the gain is worth its cost is a business call
(B) The difference is random noise, because any gain smaller than 0.01 AUC is not meaningful
(C) A is better because its results are more stable
(D) No conclusion is possible without the test set

31. A pneumonia model trained on images from three hospitals performs much worse at a fourth hospital. Which evaluation design would have revealed this risk earlier?

(A) Leave-one-hospital-out validation
(B) Random k-fold across all images
(C) Stratified k-fold by label
(D) Evaluating on the training images

Copyright © ZyeemZia

Ensemble — Trees and Ensembles: The Workhorses of Tabular Data

32. A node contains 8 'on time' and 2 'late' orders. A candidate split produces a left child with 6 on time and 0 late, and a right child with 2 on time and 2 late. What is the Gini decrease?

(A) 0.32
(B) 0.12
(C) 0.20
(D) 0.08

33. A random forest with max_features equal to the total number of features barely beats a single deep tree. What is the MOST likely reason?

(A) The forest has too few rows in each bootstrap sample to learn the patterns well
(B) Random forests cannot model interactions
(C) The trees are highly correlated, so averaging cancels little error
(D) The trees are too shallow to be useful

34. A small NGO has only 1,200 labelled survey responses and cannot afford a separate validation set. Which approach gives a reasonable built-in performance estimate for a random forest?

(A) Use training accuracy
(B) Use the out-of-bag score
(C) Use the number of trees as a proxy
(D) Use impurity-based feature importance

35. A gradient boosting model reaches training AUC 0.995 and validation AUC 0.78 after 3,000 rounds at learning rate 0.3. Which change is MOST likely to help?

(A) Increase rounds to 10,000 at the same learning rate so the model can converge fully
(B) Remove row and feature subsampling
(C) Increase tree depth to 16
(D) Lower the learning rate and use early stopping on validation data

36. A lender's boosted model sometimes lowers predicted risk as the number of missed payments rises, in sparse regions of the data. Risk officers find this unacceptable. What should the data scientist do?

(A) Remove missed payments from the model
(B) Explain that the model must be correct because it learned the pattern from real data
(C) Increase the number of trees until the effect disappears
(D) Apply a monotonic increasing constraint on missed payments and re-validate

37. A real-estate firm in Dhaka uses a regression tree ensemble to value flats. Prices in a new development will exceed anything in the training data. What risk should they flag?

(A) Tree ensembles always over-predict brand-new properties because they average many deep trees
(B) Tree ensembles require standardised prices
(C) Tree ensembles cannot predict above the range of training targets, so values will be capped
(D) Tree ensembles cannot use location features

38. A risk committee asks why one specific applicant received a high default score from a gradient-boosted model. Which tool answers this question MOST directly?

(A) Global impurity-based feature importance
(B) The model's overall ROC curve
(C) Local Shapley-value attributions for that applicant
(D) A partial dependence plot averaged over all applicants

Discover — Beyond Labels: Unsupervised Learning, Recommendation and Reinforcement

39. A retailer clusters customers using raw 'annual spend in taka' and 'number of store visits'. The resulting clusters are simply bands of spending. What is the MOST likely cause?

(A) k-means cannot handle two features
(B) The number of clusters was too large
(C) Visits should have been removed
(D) The features were not scaled, so spend dominated the distance calculation

Copyright © ZyeemZia

40. An analyst runs k-means for k from 2 to 10 and picks k = 10 because inertia is lowest. Why is this a poor choice?

(A) Inertia rises as k grows, so k = 10 is actually the worst choice rather than the best one
(B) Inertia always falls as k grows, so its minimum does not identify natural structure
(C) k-means cannot use more than eight clusters
(D) Inertia measures classification accuracy

41. PCA on standardised survey data gives eigenvalues 5, 2, 1.5, 1 and 0.5. How many components are needed to retain at least 85% of the variance?

(A) 2
(B) 4
(C) 5
(D) 3

42. A bank's fraud team can investigate 100 alerts a day. An unsupervised detector produces scores for 2 million daily transactions. How should alerts be generated?

(A) Send the 100 highest-scored transactions, enriched with context, and record analyst outcomes
(B) Send every transaction above a fixed anomaly score each day, however many alerts that creates
(C) Send a random 100 transactions for fairness
(D) Send only transactions above the maximum amount ever seen

43. An online bookshop in Dhaka launches 500 new titles weekly. Its collaborative filtering recommender never suggests them. What is the BEST fix?

(A) Increase the number of latent factors so that the collaborative model can represent new titles
(B) Retrain the collaborative model daily
(C) Remove older books from the catalogue
(D) Add content-based signals such as genre, author and description embeddings in a hybrid model

44. A recommender's offline precision@10 improved after retraining on click logs, but an A/B test shows no change in purchases. What is a likely explanation?

(A) Offline logs reflect what the old system chose to show, so offline gains may not transfer
(B) Precision@10 cannot be computed offline
(C) A/B tests always show smaller effects than reality
(D) The model must have more parameters than the old one, so it needs more traffic to show gains

45. A team wants RL to set daily prices for a new product with no simulator, and each price change affects revenue immediately and visibly. Which approach is MOST practical to start with?

(A) Full deep reinforcement learning exploring any price on live customers from day one
(B) A contextual bandit with a constrained set of approved price points
(C) Unsupervised clustering of competitor prices
(D) A fixed price chosen once and never revisited

46. In Q-learning, Q(s, a) = 10, reward r = 2, $\gamma = 0.9$, the best next-state value is 10 and $\alpha = 0.1$. What is the updated Q(s, a)?

(A) 11.0
(B) 10.0
(C) 12.0
(D) 10.1

Deep — Neural Networks and Deep Learning

47. A team adds LoRA adapters of rank r = 16 to a frozen 2,048 × 2,048 weight matrix. How many trainable parameters does that matrix gain?

(A) 4,194,304
(B) 32,768
(C) 65,536
(D) 2,048

48. A model must flag which of 20 possible defects appear on each garment photo; a garment can have several defects. Which output layer and loss are correct?

(A) A 20-way softmax with categorical cross-entropy
(B) One linear output with MSE
(C) 20 sigmoid outputs with binary cross-entropy
(D) 20 ReLU outputs with hinge loss

Copyright © ZyeemZia

49. Training a deep network, the loss stays flat and gradient norms in the first layers are near zero, while later layers learn. The network uses sigmoid activations throughout. What is the MOST effective change?

(A) Use ReLU-family activations with suitable initialisation or residual connections
(B) Add more sigmoid layers so that the network has more capacity to learn early features
(C) Increase dropout to 0.9
(D) Remove the validation set

50. A conv layer receives a 64 × 64 × 3 image and applies 16 filters of size 5 × 5 with stride 1 and no padding. What is the output shape?

(A) 64 × 64 × 16
(B) 60 × 60 × 3
(C) 60 × 60 × 16
(D) 32 × 32 × 16

51. A hospital has 1,200 labelled skin-lesion images and wants a classifier. Which approach is MOST appropriate?

(A) Train a large CNN from random initialisation on the 1,200 images, using all of them for training
(B) Cluster the images with k-means and call the clusters diagnoses
(C) Use a decoder-only text model on file names
(D) Fine-tune a pre-trained image model with augmentation, splitting by patient

52. A forecasting team uses an LSTM on raw daily sales and gets worse results than gradient-boosted trees with lag, holiday and promotion features. What is the MOST reasonable conclusion?

(A) LSTMs can never forecast sales
(B) Boosted trees always beat neural networks on sales data, so the LSTM should be abandoned
(C) The LSTM must be leaking future data
(D) The features drove the gain; give the LSTM the same inputs before judging it

53. A legal team must classify contracts of up to 30,000 tokens. Their transformer handles 4,000 tokens. Which issue should they plan for?

(A) Transformers cannot read legal language
(B) Attention cost falls as documents get longer
(C) Attention cost grows with the square of length, so chunking or long-context methods are needed
(D) Tokenisation compresses long documents enough that context length no longer matters in practice

Deploy — MLOps, Monitoring and Responsible ML

54. A garment buyer portal needs delivery-risk scores for 40,000 open orders, refreshed each morning before planners start work. Which serving pattern fits BEST?

(A) An online API called for every page view
(B) An on-device model in each planner's phone
(C) A nightly batch job that writes scores to a table the portal reads
(D) A streaming pipeline scoring every keystroke

55. After deployment, a credit model's live scores differ from offline scores for the same applicants. Investigation shows income is rounded to thousands in the app but not in the training data. What is this problem called?

(A) Covariate drift over time
(B) Label drift
(C) Training–serving skew
(D) Overfitting

56. A team computes PSI for a key feature and gets 0.31. No labels are available yet. What is the MOST appropriate response?

(A) Ignore it until labels arrive in six months
(B) Investigate the cause of the shift and assess its likely impact before deciding on retraining
(C) Retrain automatically on this month's data without review, since PSI above 0.25 always means retrain
(D) Delete the feature from the live model immediately

Copyright © ZyeemZia

57. Fraud patterns change monthly, but models are retrained manually twice a year, and the last two retrains could not be reproduced. Which investment gives the GREATEST improvement?

(A) A larger cloud GPU for faster manual training

(B) An automated, versioned training pipeline with quality gates and a model registry

(C) Hiring an additional analyst to rerun the existing notebooks more carefully each month

(D) Switching to a deep learning model

58. A loan model approves 48% of applicants from group A and 30% from group B. What is the disparate impact ratio and what does it suggest?

(A) 0.625; below the 0.8 screening rule, so investigate

(B) 0.18; within acceptable limits

(C) 1.6; group B is favoured, so the model should be adjusted in group A's favour

(D) 0.8; exactly at the threshold

59. A support assistant built on a language model with retrieval was fine last month. After the vendor updated the underlying model, complaints about wrong answers rose. Which practice would have caught this before users did?

(A) A versioned set of reference questions rerun as a release gate after every change

(B) Raising the temperature so that answers become more varied and natural

(C) Doubling the number of retrieved passages in every prompt

(D) Checking the new model version's position on public leaderboards before switching over

60. A regulator asks why a specific customer's transfer was blocked eight months ago. Which practice makes this question answerable?

(A) Keeping only monthly aggregate dashboards

(B) Retraining the model on the customer's data and comparing the new score with the old one

(C) Storing the final model file without metadata

(D) Logging each prediction with model version, inputs, score, threshold and reason codes

Copyright © ZyeemZia

EXAM PREPARATION

Practice exam answers & explanations

1. B When rules are known, complete and stable, ML only approximates them and adds opacity and error. A trained classifier would copy past mistakes; waiting or layering rules on a model adds cost with no benefit.

2. A Predicting 'no churn' for all customers already scores 98%. The model's value is in finding churners, so compare recall, precision or lift against the baseline before anything else.

3. B Features must be available at prediction time. Inspection duration leaks future information; imputing or transforming it does not change that.

4. B Cosine similarity compares direction and removes the effect of length. Dot products and raw Euclidean distance are distorted by norm differences, and norms alone ignore direction.

5. B Steady but very slow progress suggests steps are too small. Shrinking the learning rate slows it further; cutting data does not help; the model has clearly not converged.

6. D Per 100,000: 180 frauds caught, 998 false alarms. Precision ≈ 180 ÷ 1,178 ≈ 15%. Low base rates make false alarms dominate.

7. B Correlation does not establish causation. Wealthier or more urban districts may have both more agents and higher enrolment; an experiment or causal analysis is needed.

8. A A seasonal naive forecast is a strong, honest baseline for daily data with weekly patterns. The other options are straw men that any model would beat.

9. C High raw agreement with modest kappa is typical when one class dominates: agreement on easy majority cases hides disagreement on the minority class that matters. Guidelines need refining before scaling.

10. A Missingness here is informative, so an indicator preserves the signal while imputation keeps the rows. Dropping rows biases the sample, zero is a false value, and test-set statistics leak.

11. C Statistics must be learned on the training part of each fold. Putting the scaler inside a pipeline fixes this automatically.

12. D Very high cardinality makes one-hot sparse and unwieldy, and ordinal IDs impose a meaningless order. Careful target or frequency encoding captures merchant risk compactly.

13. C Using events after the prediction moment gives the model information it will not have in production, inflating offline results.

14. D Resampling before the split leaks near-duplicates into the test set and distorts the class balance, inflating precision.

15. A A linear model adds weighted inputs and cannot form a ratio itself; the ratio must be engineered.

16. A Any data-driven step, including feature selection, must happen within each training fold or it leaks information from the validation data.

17. B A large gap with many features relative to rows signals overfitting. Regularisation tuned by cross-validation reduces variance; adding terms makes it worse and the other options hide the problem.

18. A Coefficients act on log-odds; e^0.69 ≈ 2, so the odds roughly double. They are not probability increments or confidence levels.

19. B A loss that grows and overflows is the signature of steps that are too large. Capacity and epochs do not fix an unstable optimiser.

20. B L1 drives many weights to exactly zero, giving a sparse model. Ridge shrinks but rarely removes features, unregularised models keep everything, and k-NN uses every feature in its distances.

21. C Unequal scales distort the loss surface so a single learning rate cannot suit all directions; standardising fixes the geometry.

22. C Equal scores produce equal probabilities that sum to one: 0.25 each.

23. A Naive Bayes on counts is fast, needs little data and runs cheaply. The alternatives need far more data or compute, or rely on weak distance measures.

24. A Forecasts must be validated on later periods than training. A random split leaks future patterns and overstates accuracy; the metric choice and ratio are secondary.

25. D Per 10,000 patients: 90 true positives and 495 false positives, so precision ≈ 90 ÷ 585 ≈ 15%.

26. C With extreme imbalance and fixed capacity, what matters is how many frauds each model finds within the alerts the team can handle.

27. B Selecting on the test set turns it into a validation set. A separate untouched test set, or nested cross-validation, is needed for an unbiased estimate.

28. D Close training and validation errors that are both worse than a simple rule indicate high bias.

29. D Offers follow a request to leave, so the feature encodes the outcome process. It is target leakage regardless of transformation.

30. A Every paired difference is positive with low spread, so the gain is consistent; its practical value depends on costs.

31. A Holding out whole hospitals tests generalisation to new sites and exposes reliance on site-specific cues.

Copyright © ZyeemZia

32. B Parent Gini = 1 − (0.64 + 0.04) = 0.32. Left = 0; right = 0.5; weighted = 0.4 × 0.5 = 0.20. Decrease = 0.32 − 0.20 = 0.12.

33. C Considering all features at every split makes trees similar; lowering max_features decorrelates them.

34. B Out-of-bag scoring evaluates each row using trees that did not see it, providing validation without a separate split.

35. D The model is overfitting. Shrinkage plus early stopping limits the effective number of rounds; the other changes add complexity.

36. D Monotonic constraints encode agreed domain logic, prevent noisy reversals and usually cost little accuracy.

37. C Leaves average training targets, so predictions are bounded by the training range; extrapolation needs other approaches.

38. C Local attributions decompose this applicant's score into feature contributions; global tools describe the model on average.

39. D Spend has a far larger numeric range, so unscaled Euclidean distance is driven almost entirely by it.

40. B Use the elbow, silhouette, stability and usefulness; the lowest inertia trivially favours more clusters.

41. D Total = 10. First two give 70%; three give 85%.

42. A Matching volume to capacity and capturing feedback maximises value and enables a supervised model later.

43. D New items have no interactions; content features let the system recommend them from day one.

44. A Logged data is biased by past exposure; online experiments measure the true effect.

45. B Bandits handle immediate rewards with controlled exploration; unconstrained deep RL on live prices is risky.

46. D Target = 2 + 0.9 × 10 = 11; new Q = 10 + 0.1 × (11 − 10) = 10.1.

47. C LoRA adds B (2,048 × 16) and A (16 × 2,048): r(d + k) = 16 × 4,096 = 65,536 parameters, about 1.6% of the 4,194,304 in the frozen matrix. 32,768 counts only one of the two matrices.

48. C Multi-label tasks need independent probabilities per label; softmax assumes exactly one class.

49. A This is the vanishing-gradient pattern; non-saturating activations, proper initialisation and skip connections restore gradient flow.

50. C (64 − 5) ÷ 1 + 1 = 60 in each spatial dimension; 16 filters give 16 channels.

51. D Transfer learning makes small image datasets workable, and patient-level splits avoid leakage.

52. D The comparison mixed architecture with information; the LSTM lacked the calendar and promotion inputs.

53. C Self-attention scales quadratically; long documents need chunking, hierarchical models or efficient attention variants.

54. C Scores are needed once a day for a known set of orders; batch is simplest, cheapest and reliable.

55. C Different feature computation in training and serving causes skew; the fix is a shared feature definition and parity tests.

56. B A PSI above 0.25 signals significant shift; the cause may be a data error or real change, so investigate before acting.

57. B Automation and versioning make frequent, reproducible retraining possible; compute or headcount alone do not.

58. A 0.30 ÷ 0.48 = 0.625, below 0.8, which warrants a fairness review.

59. A A regression suite on your own reference questions detects quality drops caused by any upstream change. Leaderboards measure other tasks, and changing temperature or passage count does not test anything.

60. D Prediction-level logs linked to registered model versions allow exact reconstruction and explanation.

Copyright © ZyeemZia

ZYEEMZIA™ ACADEMY

Your certification path

Each ZyeemZia™ track stands on its own. Together they form a networking ladder from first network to expert design, plus paths in leadership, artificial intelligence, cybersecurity, IT service management and IT support. Take the next step at the sites below.

Networking ladder

ASSOCIATE
ZCNA
ZyeemZia™ Certified Network Associate
zcna.zyeemzia.com

→

PROFESSIONAL
ZCNP
ZyeemZia™ Certified Network Professional
zcnp.zyeemzia.com

→

EXPERT
ZCIE
ZyeemZia™ Certified Internetwork Expert
zcie.zyeemzia.com

→

EXPERT
ZCIE Security
ZyeemZia™ Certified Internetwork Expert — Security
zcie-security.zyeemzia.com

→

DESIGN EXPERT
ZCDE
ZyeemZia™ Certified Design Expert
zcde.zyeemzia.com

Leadership & management

PROFESSIONAL
ZPMP
ZyeemZia™ Project Management Professional
zpmp.zyeemzia.com

PROFESSIONAL
ZEPMS
ZyeemZia™ Engineering Project Management Specialization
zepms.zyeemzia.com

PROFESSIONAL
ZHRLS
ZyeemZia™ Human Resource Management and Leadership Specialization
zhrls.zyeemzia.com

Artificial intelligence

PROFESSIONAL
ZAIG
ZyeemZia™ AI Governance
zaig.zyeemzia.com

PROFESSIONAL
ZMLS
ZyeemZia™ Machine Learning Specialization
zmls.zyeemzia.com

ARCHITECT
ZAIA
ZyeemZia™ Artificial Intelligence Architecture
zaia.zyeemzia.com

Cybersecurity

PROFESSIONAL
ZCSS
ZyeemZia™ Cybersecurity Specialization
zcss.zyeemzia.com

EXPERT
ZCSTG
ZyeemZia™ Cyber Security – Technology and Governance
zcstg.zyeemzia.com

Copyright © ZyeemZia

IT service management

MASTER

ZITIL Master

ZyeemZia™ Information Technology Infrastructure Library Master

zitil.zyeemzia.com

IT operations & support

ASSOCIATE

ZITSS

ZyeemZia™ IT Support Specialization

zitss.zyeemzia.com

Copyright © ZyeemZia

ZyeemZia™

ZMLS

Machine Learning Specialization

CERTIFICATE OF ACHIEVEMENT

This certifies that

_____

has successfully completed all modules of the ZyeemZia™ Machine Learning Specialization and passed the ZMLS certification exam

- Problem framing & ML maths
- Data preparation & features
- Supervised models & evaluation
- Ensembles, unsupervised & deep learning
- MLOps & responsible ML

ISSUED

_____

CREDENTIAL ID

_____

Earn the official certificate online at zmls.zyeemzia.com: complete every lesson and pass the official exam. Anyone can verify your credential ID at zmls.zyeemzia.com/#verify. This page is for your printed record.

Copyright © ZyeemZia

NOTES

My notes

Copyright © ZyeemZia